

UI FULL STACK

BY

SUDHAKAR SHARMA

NARESH TECHNOLOGIES

FRIENDS XEROX

ALL SOFTWARE INSTITUTES MATERIAL AVAILABLE

{ADD: GAYATHRI NAGAR,SAP STREET,BEHIND HUDA MYTHRIVANAM.AMEERPET}

CELL : 9908909131

[GOODS ONCE SOLD WILL NOT BE TAKEN BACK]

UI-FullStack Web Development

1. Who can learn UI?

A. Any Graduate - Technical / Non-Tech

2. Course Content - Overview

a) Front End Web Development 100%

HTML

CSS

Bootstrap

Sass

JavaScript

jQuery

b) Back End

NodeJS

MongoDB

Express

3. What is difference between JavaFullStack, .NETFullStack, PythonFullStack and UIFullStack?

A.

Front End is same

HTML, CSS, JavaScript [50%]

Java

Java, JSP, Spring, Oracle, MySql

.NET

C#, ASP.NET, MVC

4. What is UI Full Stack Web Development?

A. UI - User Interface [Front End]

HTML, CSS, Bootstrap, Sass, JavaScript, jQuery

Full Stack

Both Front-End and Backend

Software Application

1. Desktop

2. Web Application 100%

- 3. Distributed Application 50% [50% => React, Angular, Vue]
- 4. Mobile Application
- 5. AI Application
- 6. IOT Application

1. Web Applications

2. Distributed Applications

Web Applications

1. Front End

HTML
CSS
JavaScript
Bootstrap
Sass
jQuery

2. Back End

NodeJS
Express
MongoDB

Role of Front-End Technologies

HTML	It is used for presentation.
CSS	It is used to make the presentation attractive and responsive.
Sass	It simplifies your CSS.
Bootstrap	It provides templates and components for web design.
JavaScript	It is a language used for client side interactions.
jQuery	It is a library for building UI. It uses JavaScript as language. [AJAX] [React, Angular, Vue, Backbone, KnockoutJS...]

Web Development

FAQ: What is Network?

Ans: A group of computers connected with each other for sharing of information and devices.

Computer Networks are classified into 3 major

- a) LAN
- b) MAN
- c) WAN

Introduced in early 1960's By US-DOD "ARPANET"

FAQ: What is Internet?

Ans: It is a WAN that connects computers all over the world
Introduced by CERN Labs.

FAQ: What is Web?

Ans: Tim Berners Lee introduced the concept of web.
Web is a portion of Internet with restricted access.
Web is Maintained by "W3C" community
[World Wide Web Consortium]
Web latest version is "3.0".

Setup Environment for Web Development

- PC 2GB RAM OS - Window-8+, MAC, Linux 10GB
- Download and Install "NodeJS"
<https://nodejs.org/en/>

Run => CMD

C:\>node -v

- Download and Install "Visual Studio Code"

Basic Tool Chain for Web Development

1. Package Manager

- It is a software tool used by developers to download and install various libraries required for project.

- **There are various package manager tools**

- a) **NPM** **
- b) Yarn
- c) RubyGems
- d) Bower
- e) NuGet etc..

You download Node JS on your PC so that you can get NPM package manager

www.nodejs.org

[Node Package Manager]

```
C:\>npm install packageName
```

```
C:\> node -v
```

```
C:\> npm -v
```

2. Editor

- Editor is a software tool that provides an IDE [Integrated Development Environment] for developers, so that they can build, debug, test & and deploy.
- There are various editors
 - Visual Studio Code **
 - Sublime
 - EditPlus
 - WebStrom etc..

3. Extentions for Visual Studio Code

- a) Live Server [Ritwick Dey]
- b) Vscode-icons
- c) IntelliSense for CSS class name in HTML

Web Site & Web Application

- Website

It is a virtual directory on web server.
It will not allow any interactions.

- Web Applications

It is a virtual directory on web server.
It allows interactions.

[- Wiki](#)

Wiki [Quick]

Build a new Project

[Web Application]

- Web Application comprises of 2 paths

a) Physical Path

b) Virtual Path

- Physical path is the location where developer keeps all resources required for a project.

E:\Shopping

- Virtual path is use by client to access the resources from physical path.

<http://localhost/shopping>

<http://127.0.0.1/shopping>

Creating a new Web Application

1. Open any location on your PC

2. Create a new folder

E:\Shopping

3. Open Visual Studio Code

4. Go to File Menu => Open Folder => E:\Shopping

5. Go to Terminal Menu => New Terminal

6. Change the terminal from Power Shell to "Command Prompt"

7. Run the following command

E:\Shopping> npm init -y

8. This will generate "package.json"

Note: Package.json comprises of project meta data. Information about your project:

- Version
- Name
- Dependencies etc..

9. Add following folders into project

- a) public
- b) src

public comprises static resources like : images, text, pdf, html, audio, video etc.
src comprises of dynamic resources like : css, scss, js, ts etc..

10. Every web application must have "index.html", it is the startup file.

Note: Index.html must be at root level in project.

11. Add some content in index.html

Welcome to Shopping Site

12. Click "Go Live" button

13. Open any browser and request

<http://127.0.0.1:5500>

(or)

<http://localhost:5500>

Note: Without index.html you can access the resources but you must know the exact path.

Web Page

- Web Page is a Hyper Text Document that provides an UI from where user can interact with our application.
- Index.html is a web page.

Hyper = Beyond

- Web Pages are classified into 2 types

- a) Static Page: does not change
.html, .htm
- b) Dynamic Page: changes
.jsp, php, aspx, asp

HTML

(Hyper Text Markup Language)

- Hyper means "beyond".
- Hyper Text is the text that contains content beyond what it displays.
- The term markup is derived from "Marking Up".
- Markup means preparing for Presentation.
- Markup Language is a language used for presentation.
- HTML is used for both static and dynamic pages.

Evolution of HTML

- In early 1960's CERN labs introduced a language GML.
[Generic Markup Language]
- SGML is the superset of GML.
[Standard Generic Markup Language]
- 1990's "Tim Berners Lee" introduced "HTML".
- **IETF** developed HTML versions upto 3.1
[Internet Engineering Task Force]
- From 2004, **WHATWG** is maintaining and developing HTML.
[Web Hyper Text Application Technology Work Group]

<https://whatwg.org/>

- Latest version of HTML is "HTML 5". [2014]
- HTML 4 version from 2004 to 2014.

Browser Architecture

Markup => Bytes => Chars => Tokens => Nodes => DOM => Layout => Render => Paint

- HTML is presenting DOM [Document Object Model]
- DOM comprises of Nodes known as Elements.
- HTML is a set of elements.
- Elements are arranged in a hierarchy called DOM.
- **HTML elements are classified into 5 groups**

1. Normal Elements
2. Void Elements
3. RC Data Elements
4. Raw Text Elements
5. Foreign Elements
SVG, Canvas, MathML

Browser Parsing

Markup => Bytes => Chars => Token [Tag] => Node [Elements] => DOM => Layout
=> Render => Paint

HTML Elements

1. Normal
2. Void
3. RC Data
4. Raw Text
5. Foreign

HTML Page Structure

1. Every Static must have the extension ".html" or ".htm"
2. Every HTML page starts with "Document Declaration" from HTML 5 version

<!DOCTYPE html>

It specifies that we are using HTML 5 version.

3. Every HTML page will have document scope after declaration.
Scope defines where document start and ends.
It is defined by using `<html> </html>`

Syntax:

```
<!DOCTYPE html>  
<html>
```

```
</html>
```

- Every document scope must define the language type.
- It is defined by using "lang" attribute.

Syntax:

```
<!DOCTYPE html>  
<html lang="en-in">
```

```
</html>
```

```
lang=en-in      //valid  
lang="en-in"    //valid => Good / Recommended  
lang='en-in'    //valid
```

4. Document scope comprises of 2 sections

- a) Head `<head> </head>`
- b) Body `<body> </body>`

- Head section defines the content that is loaded into browser memory and later used by browser.
- Body section defines the content that is directly loaded into browser.

5. Head Section of HTML page comprises of following

- a) title
- b) link

- c) meta
- d) script
- e) style

- Title specifies the text to display in browser title bar.
Title is also used for bookmarking the page.

Syntax:

```
<!DOCTYPE html>
<html lang="en-in">
  <head>
    <title> Shopping | Shopping Online </title>
  </head>
</html>
```

- Link is used to link external file to web page. such as "favicons", "style sheets" etc..

- * favicons must have the extension ".ico"
- * favicon size is between 16x16 and 32x32 pixels

Ex:

1. Go to public folder
2. Add a new file "favicon.ico"
3. Open in explorer
4. Right click on file => Open With => MsPaint
5. Set Image Properties => size : 32x32 pixels
6. Draw image and save
7. Go to index.html

```
<head>
  <link rel="shortcut icon" href="public/favicon.ico">
</head>
```

Ex:

```
<!DOCTYPE html>
<html lang="en-in">
  <head>
    <title>Shopping | Online Shopping</title>
    <link href="public/favicon.ico" rel="shortcut icon">
  </head>
</html>
```

Meta

- Meta refers to meta data.
- Meta data is information about your page given to browser and SEO.
[Search Engine Optimization]
- Meta data contains information that makes the page responsive and SEO friendly.

Syntax: Responsive

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

- a) viewport : refers to device screen type and size
- b) content : your page body content
- c) initial-scale : Zoom % [1 = 100]

```
<meta http-equiv="refresh" content="5">
```

```
<meta charset="utf-8">
```

(Unicode Transformation Format - 8 Bit Size)

English Chars	8bits
Japan, Korean	16bits
Arabic, China	32bits

Syntax: SEO Friendly

```
<meta name="keywords" content="best software training, best institute  
for Java, UI Course in Hyderabad, Online and Offline Training">
```

```
<meta name="description" content="some thing about your site">
```

Body Section

```
<!DOCTYPE html>  
<html lang="en-in">  
<head>  
  <title>Shopping | Online Shopping</title>
```

```

<link href="public/favicon.ico" rel="shortcut icon">
<meta name="viewport" content="width=device-width, initial-scale=2">
<meta http-equiv="refresh" content="4">
<meta charset="utf-8">
</head>
<body>
    The Microsoft Services Agreement is an agreement between you and
    Microsoft (or one of its affiliates) that governs your use of Microsoft consumer
    online products and services.

    We understand that you may have questions about the Microsoft Services
    Agreement. We have an FAQ page (https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx) that provides more information,
    including what Microsoft products and services it covers. Whenever we make
    changes, we provide a summary of the most notable changes to the current
    Microsoft Services Agreement. We will also continue to provide a summary of the
    changes to the previous version, and a link to the previous version, of the
    Microsoft Services Agreement on the Summary of Changes (
    https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx)
    page.
</body>
</html>

```

Body Section

- It comprises of content which is display directly in browser workspace.
- HTML body section is configure using <body> element.

- Body element attributes

1. bgcolor : It sets background color for page.

HTML colors are defined by using

- a) Color Name
- b) Color Contextual Name
- c) Hexa Decimal Code

```

<body bgcolor="red">
<body bgcolor="danger">

```

Hexa Decimal Color Code

- a) 3 chars code with "#"
- b) 6 chars code with "#"

#RGB
#RRGGBB

R = Red [0,1,2,3,4,5,6,7,8,9,a,b,c,d,e,f]
G = Green [0,1,2,3,4,5,6,7,8,9,a,b,c,d,e,f]
B = Blue [0,1,2,3,4,5,6,7,8,9,a,b,c,d,e,f]

0 = Dark => => => => Bright = f

#f00 = red
#0f0 = green
#00f = blue
#000 = black
#fff = white

#00ff00 = green
#00da00

2. text : It sets text color.

```
<body bgcolor="red" text="white">  
<body bgcolor="#f00" text="#fff">
```

3. background : It sets a background image for page.

```
<body background="public/images/banner.jpg">
```

You can control background image using styles.

```
<head>  
<style>  
  body {  
    background-size: cover;  
    background-repeat: no-repeat;  
    background-position: center center;  
    background-attachment: fixed;  
  }  
</style>
```

</head>

4. align : It aligns the page content to left, center, right or justify.

<body align="justify">

5. leftmargin]

6. rightmargin] set space between content and window

7. topmargin] width max : 1200px height max : 768 px

8. bottommargin]

<body leftmargin="350" rightmargin="350">

9. alink : Sets color for active link

10. vlink : Set color for visited link

<body alink="red" vlink="gray">

** Amazon **

</body>

Body Semantic Elements

- Semantic mean related.

- HTML 5 introduced new semantic elements to make the body section more SEO friendly.

- HTML 4 depends on tables for designing a layout.

- **HTML 5 Semantic Elements are**

1. aside : comprises of content which is not related to website.

2. article : highlights of your website. latest trends on your site.

3. dialog : it is an pop-up for handling interactions.

4. figure : image with caption

5. figcaption : caption for image.

6. header : top margin of page

7. footer : bottom margin of page

8. section : content between header and footer

9. nav : navigation area

- | | |
|----------|------------------------------|
| 10. menu | : Item in navigation area |
| 11. div | : block container |
| 12. span | : inline container |
| 13. main | : entry point of application |

- The above mentioned semantic elements are related to layout design.

www.getbootstrap.com => Themes | Icons

- For Icons you have to download "Bootstrap Icons" in your project.

```
E:\Shopping> npm install bootstrap-icons --save
```

- Bootstrap Icons are kept in a "css" file, which is present in "node_modules".
- You have to link the file to your webpage.

Ex: Home.html

```
<!DOCTYPE html>
<html lang="en-in">
  <head>
    <title>Shopper | Online Shopping</title>
    <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
    <style>
      header {
        display: flex;
        justify-content: space-between;
        font-family: Arial;
        background-color: black;
        color: white;
        padding: 10px;
      }
      span {
        margin-right: 20px;
      }
      .brand-name {
        font-weight: bold;
        font-size: 20px;
      }
    </style>
  </head>
  <body>
    <div class="container">
      <div class="row">
        <div class="col-12">
          <div class="text-center">
            <div class="img-fluid">
              <img alt="Shopper logo" data-bbox="100 100 200 200"/>
            </div>
            <div class="brand-name">Shopper</div>
          </div>
          <div class="text-center">
            <div class="text">Online Shopping</div>
            <div class="text">Best Price Guarantee</div>
          </div>
        </div>
      </div>
    </div>
  </body>
</html>
```



```
    header span {
        margin-right: 20px;
    }
    .brand-name {
        font-weight: bold;
        font-size: 20px;
    }
    section {
        height: 200px;
    }
    footer {
        display: grid;
        grid-template-columns: 3fr 2.25fr 2.25fr 2.25fr 2.25fr;
        font-family: Arial;
        background-color: black;
        color: white;
        padding: 10px;
        font-size: 16px;
    }
    .social-bar {
        margin-top: 20px;
    }
    .social-bar span {
        margin-right: 10px;
    }
    .footer-title {
        font-weight: bold;
        margin-bottom: 20px;
    }
    .footer-title~div {
        margin-bottom: 10px;
    }
    .brand-footer {
        font-size: 30px;
        font-weight: bold;
    }
}
</style>
</head>
<body>
    <header>
        <div>
            <span class="brand-name">Shopper.</span>
        </div>
```

```
<div>
  <nav>
    <span>Home</span>
    <span>Catalog</span>
    <span>Shop</span>
    <span>Blog</span>
    <span>Docs</span>
  </nav>
</div>
<div>
  <span class="bi bi-search"></span>
  <span class="bi bi-person"></span>
  <span class="bi bi-heart"></span>
  <span class="bi bi-cart"></span>
</div>
</header>
<section>

</section>
<footer>
  <div>
    <span class="brand-footer">Shopper</span>
    <div class="social-bar">
      <span class="bi bi-facebook"></span>
      <span class="bi bi-twitter"></span>
      <span class="bi bi-instagram"></span>
      <span class="bi bi-youtube"></span>
    </div>
  </div>
  <div>
    <div class="footer-title">SUPPORT</div>
    <div>Contact Us</div>
    <div>FAQs</div>
    <div>Size Guide</div>
    <div>Shipping & Returns</div>
  </div>
  <div>
    <div class="footer-title"> SHOP</div>
    <div>Men's Shopping</div>
    <div>Women's Shopping</div>
    <div>Kids' Shopping</div>
    <div>Discounts</div>
  </div>
</div>
```

```
<div>
  <div class="footer-title">COMPANY</div>
  <div>Our Story</div>
  <div>Careers</div>
  <div>Terms & Conditions</div>
  <div>Privacy & Cookie policy</div>
</div>
<div>
  <div class="footer-title"> CONTACT</div>
  <div>1-202-555-0105</div>
  <div>1-202-555-0106</div>
  <div>help@shopper.com</div>
</div>
</footer>
</body>
</html>
```

Ex: Flipkart Footer

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Flipkart | Shopping</title>
  <style>
    footer {
      display: grid;
      grid-template-columns: 6fr 6fr;
      height: 250px;
      background-color: black;
      color:white;
      padding: 5px;
      font-size: 15px;
    }
    .panel-left {
      display: grid;
      grid-template-columns: 3fr 3fr 3fr 3fr;
    }
    .panel-right {
      display: grid;
      grid-template-columns: 6fr 6fr;
```

```
        border-left: 2px solid gray;
        padding-left: 20px;
    }
    .footer-title {
        color:gray;
        margin-bottom: 20px;
    }
    .back {
        background-color:gray;
        color:white;
        padding: 10px;
        font-size: 20px;
        text-align: center;
    }
</style>
</head>
<body>
    <div class="back">
        Back to Top
    </div>
    <footer>
        <div class="panel-left">
            <div>col-1</div>
            <div>col-2</div>
            <div>col-3</div>
            <div>col-4</div>
        </div>
        <div class="panel-right">
            <div>
                <div class="footer-title">Mail Us</div>
                <div>
                    Flipkart Internet Private Limited,

                    Buildings Alyssa, Begonia &

                    Clove Embassy Tech Village,

                    Outer Ring Road, Devarabeesanahalli Village,

                    Bengaluru, 560103,

                    Karnataka, India
```

```
</div>
</div>
<div>
  <div class="footer-title">Registered Office Address</div>
  <div>
    Flipkart Internet Private Limited,

    Buildings Alyssa, Begonia &

    Clove Embassy Tech Village,

    Outer Ring Road, Devarabeesanahalli Village,

    Bengaluru, 560103,

    Karnataka, India

    CIN : U51109KA2012PTC066107

    Telephone: 044-45614700
  </div>
</div>
</div>
</footer>
</body>
</html>
```

Ex: Instagram - Footer

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Instagram</title>
  <style>
    footer {
      text-align: center;
      font-family: Arial;
    }
  </style>
</head>
<body>
  <div>
    <div class="text">
      Instagram
    </div>
    <div class="text">
      © 2014 Instagram
    </div>
  </div>
</body>
</html>
```

```

        footer span {
            margin-right: 20px;
        }
        .row-1 {
            margin-bottom: 20px;
        }
    </style>
</head>
<body>
    <footer>
        <div class="row-1">
            <span>Meta</span>
            <span>About</span>
            <span>Blog</span>
            <span>Jobs</span>
            <span>Help</span>
            <span>API</span>
            <span>Privacy</span>
            <span>Terms</span>
            <span>Top Accounts</span>
            <span>Hashtags</span>
            <span>Locations</span>
            <span>Instagram Lite</span>
            <span>Contact Uploading & Non-Users</span>
        </div>
        <div class="row-2">
            <span>English</span>
            <span> &copy; 2022 Instagram from Meta</span>
        </div>
    </footer>
</body>
</html>

```

Aside

- It comprises of information that is not related to the current page.
- It also uses content that navigates to other sources of your website.

Syntax:

```
<aside>
```

```
.....
```

```
</aside>
```


Ex:

```
<!DOCTYPE html>
<html lang="en-in">
  <head>
    <title>Shopper | Online Shopping</title>
    <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
    <style>
      header {
        display: flex;
        justify-content: space-between;
        font-family: Arial;
        background-color: black;
        color: white;
        padding: 10px;
      }
      header span {
        margin-right: 20px;
      }
      .brand-name {
        font-weight: bold;
        font-size: 20px;
      }
      section {
        height: 768px;
      }
      footer {
        display: grid;
        grid-template-columns: 3fr 2.25fr 2.25fr 2.25fr 2.25fr;
        font-family: Arial;
        background-color: black;
        color: white;
        padding: 10px;
        font-size: 16px;
      }
      .social-bar {
        margin-top: 20px;
      }
      .social-bar span {
        margin-right: 10px;
      }
      .footer-title {
        font-weight: bold;
```

```
        margin-bottom: 20px;
    }
    .footer-title~div {
        margin-bottom: 10px;
    }
    .brand-footer {
        font-size: 30px;
        font-weight: bold;
    }
    .section-socialbar {
        color: black;
        font-size: 25px;
        position: fixed;
        right: 10px;
        bottom: 5px;
        background-color: white;
        padding: 3px;
    }
    .section-socialbar div {
        margin-bottom: 10px;
    }
</style>
</head>
<body>
    <header>
        <div>
            <span class="brand-name">Shopper.</span>
        </div>
        <div>
            <nav>
                <span>Home</span>
                <span>Catalog</span>
                <span>Shop</span>
                <span>Blog</span>
                <span>Docs</span>
            </nav>
        </div>
        <div>
            <span class="bi bi-search"></span>
            <span class="bi bi-person"></span>
            <span class="bi bi-heart"></span>
            <span class="bi bi-cart"></span>
        </div>
```

</header>

<section>

We understand that you may have questions about the Microsoft Services Agreement. We have an FAQ page (<https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx>) that provides more information, including what Microsoft products and services it covers. Whenever we make changes, we provide a summary of the most notable changes to the current Microsoft Services Agreement. We will also continue to provide a summary of the changes to the previous version, and a link to the previous version, of the Microsoft Services Agreement on the Summary of Changes (<https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx>) page.

We understand that you may have questions about the Microsoft Services Agreement. We have an FAQ page (<https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx>) that provides more information, including what Microsoft products and services it covers. Whenever we make changes, we provide a summary of the most notable changes to the current Microsoft Services Agreement. We will also continue to provide a summary of the changes to the previous version, and a link to the previous version, of the Microsoft Services Agreement on the Summary of Changes (<https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx>) page.

We understand that you may have questions about the Microsoft Services Agreement. We have an FAQ page (<https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx>) that provides more information, including what Microsoft products and services it covers. Whenever we make changes, we provide a summary of the most notable changes to the current Microsoft Services Agreement. We will also continue to provide a summary of the changes to the previous version, and a link to the previous version, of the Microsoft Services Agreement on the Summary of Changes (<https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx>) page.

We understand that you may have questions about the Microsoft Services Agreement. We have an FAQ page (<https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx>) that provides more information, including what Microsoft products and services it covers. Whenever we make changes, we provide a summary of the most notable changes to the current Microsoft Services Agreement. We will also continue to provide a summary of the changes to the previous version, and a link to the previous version, of the Microsoft Services Agreement on the Summary of Changes (<https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx>) page.

We understand that you may have questions about the Microsoft Services

Agreement. We have an FAQ page (<https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx>) that provides more information, including what Microsoft products and services it covers. Whenever we make changes, we provide a summary of the most notable changes to the current Microsoft Services Agreement. We will also continue to provide a summary of the changes to the previous version, and a link to the previous version, of the Microsoft Services Agreement on the Summary of Changes (<https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx>) page.

We understand that you may have questions about the Microsoft Services Agreement. We have an FAQ page (<https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx>) that provides more information, including what Microsoft products and services it covers. Whenever we make changes, we provide a summary of the most notable changes to the current Microsoft Services Agreement. We will also continue to provide a summary of the changes to the previous version, and a link to the previous version, of the Microsoft Services Agreement on the Summary of Changes (<https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx>) page.

We understand that you may have questions about the Microsoft Services Agreement. We have an FAQ page (<https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx>) that provides more information, including what Microsoft products and services it covers. Whenever we make changes, we provide a summary of the most notable changes to the current Microsoft Services Agreement. We will also continue to provide a summary of the changes to the previous version, and a link to the previous version, of the Microsoft Services Agreement on the Summary of Changes (<https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx>) page.

We understand that you may have questions about the Microsoft Services Agreement. We have an FAQ page (<https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx>) that provides more information, including what Microsoft products and services it covers. Whenever we make changes, we provide a summary of the most notable changes to the current Microsoft Services Agreement. We will also continue to provide a summary of the changes to the previous version, and a link to the previous version, of the Microsoft Services Agreement on the Summary of Changes (<https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx>) page.

We understand that you may have questions about the Microsoft Services Agreement. We have an FAQ page (<https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx>) that provides more information, including what Microsoft products and services it covers. Whenever we make

changes, we provide a summary of the most notable changes to the current Microsoft Services Agreement. We will also continue to provide a summary of the changes to the previous version, and a link to the previous version, of the Microsoft Services Agreement on the Summary of Changes (<https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx>) page.

We understand that you may have questions about the Microsoft Services Agreement. We have an FAQ page (<https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx>) that provides more information, including what Microsoft products and services it covers. Whenever we make changes, we provide a summary of the most notable changes to the current Microsoft Services Agreement. We will also continue to provide a summary of the changes to the previous version, and a link to the previous version, of the Microsoft Services Agreement on the Summary of Changes (<https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx>) page.

We understand that you may have questions about the Microsoft Services Agreement. We have an FAQ page (<https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx>) that provides more information, including what Microsoft products and services it covers. Whenever we make changes, we provide a summary of the most notable changes to the current Microsoft Services Agreement. We will also continue to provide a summary of the changes to the previous version, and a link to the previous version, of the Microsoft Services Agreement on the Summary of Changes (<https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx>) page.

```
<aside class="section-socialbar">
  <div class="bi bi-facebook"></div>
  <div class="bi bi-twitter"></div>
  <div class="bi bi-instagram"></div>
  <div class="bi bi-linkedin"></div>
  <div class="bi bi-youtube"></div>
</aside>
</section>
<footer>
  <div>
    <span class="brand-footer">Shopper</span>
    <div class="social-bar">
      <aside>
        <span class="bi bi-facebook"></span>
        <span class="bi bi-twitter"></span>
        <span class="bi bi-instagram"></span>
        <span class="bi bi-youtube"></span>
```

```

        </aside>
    </div>
</div>
<div>
    <div class="footer-title">SUPPORT</div>
    <div>Contact Us</div>
    <div>FAQs</div>
    <div>Size Guide</div>
    <div>Shipping & Returns</div>
</div>
<div>
    <div class="footer-title"> SHOP</div>
    <div>Men's Shopping</div>
    <div>Women's Shopping</div>
    <div>Kids' Shopping</div>
    <div>Discounts</div>
</div>
<div>
    <div class="footer-title">COMPANY</div>
    <div>Our Story</div>
    <div>Careers</div>
    <div>Terms & Conditions</div>
    <div>Privacy & Cookie policy</div>
</div>
<div>
    <div class="footer-title"> CONTACT</div>
    <div>1-202-555-0105</div>
    <div>1-202-555-0106</div>
    <div>help@shopper.com</div>
</div>
</footer>
</body>
</html>

```

Main

- It refers to entry point of page.
- It specifies from where user starts using your page.

Note: How to refer any resource location?

```

./      invokes the current folder
../     invokes the folder, one level back

```

How to keep content center screen?

```
{
  display : flex;
  justify-content: center;
  align-items: center;
  height: 500px;
}
```

Ex: Kids Page

```
<!DOCTYPE html>
<html>
  <head>
    <title>Kids Page</title>
    <style>
      .header-top {
        background-color:darkcyan;
        color:white;
        padding: 5px;
        font-size: 20px;
        display: flex;
        justify-content: space-around;
      }
      nav {
        display: flex;
        justify-content: space-between;
        font-size: 20px;
        font-family: Arial;
        padding: 20px;
        background-color: white;
        border-bottom: 1px solid black;
      }
      .brand-title {
        font-size: 25px;
        font-weight: bold;
      }
      .brand-nav span {
        margin-right: 20px;
      }
      .banner {
        background-image: url("images/kid.jpg");
```

```

        background-size: cover;
        height: 400px;
        display: flex;
        justify-content: center;
        align-items: center;
        color: white;
    }
    .title {
        font-size: 50px;
        font-family: Arial;
        text-shadow: 2px 2px 1px black;
        width: 600px;
        text-align: center;
        font-weight: bold;
    }
</style>
</head>
<body>
    <header>
        <div class="header-top">
            <div> some address details </div>
            <div> company@email.com </div>
            <div> +91-9992888383 </div>
        </div>
        <nav>
            <div class="brand-title">Kiddos.</div>
            <div class="brand-nav">
                <span>Home</span>
                <span>About</span>
                <span>Contact</span>
                <span>Teacher</span>
                <span>Blog</span>
            </div>
        </nav>
    </header>
    <section>
        <main class="banner">
            <div class="title">Kids Are The Best Explores In The World</div>
        </main>
    </section>
</body>
</html>

```


Ex: Netflix Page

```
<!DOCTYPE html>
<html lang="en-in">
  <head>
    <title>Netflix | India</title>
    <style>
      body {
        background-size: cover;
      }
      .bg-shade {
        background-color: rgba(0,0,0,0.5);
        height: 600px;
        margin-top: -20px;
        margin-left: -20px;
        margin-right: -20px;
      }
      section {
        display: flex;
        justify-content: center;
        align-items: center;
        color: white;
        height: 500px;
      }
      main {
        text-align: center;
        font-family: Arial;
      }
      .title {
        font-size: 50px;
        font-weight: bold;
        width: 700px;
      }
      .sub-title {
        font-size: 25px;
        margin-top: 20px;
      }
      .text {
        margin-top: 20px;
      }
    </style>
  </head>
```

```

<body background="/images/netflixback.jpg">
  <div class="bg-shade">
    <section>
      <main>
        <div class="title">Unlimited movies, TV shows and more.</div>
        <div class="sub-title">Watch anywhere. Cancel anytime.</div>
        <div class="text">Ready to watch? Enter your email to create or restart
your membership.</div>
      </main>
    </section>
  </div>
</body>
</html>

```

Summary

- aside
- nav
- header
- footer
- section
- div
- span
- main
- details
- summary

Menu

- If navigation comprises of several categories then better to use Menu.

Syntax:

```

<nav>
  <menu>
    <details>
      <summary> Category Name </summary>
      <div> Item-1 </div>
      <div> Item-2 </div>
    </details>
  </menu>
</nav>

```

- Details in HTML is used to display expandable and collapsable content.
- Details is by default collapsed, to keep it expanded you have to use the attribute "open".

```
<details open>
```

```
</details>
```

Ex:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Menu Demo</title>
    <style>
      details div {
        width: 100px;
        border:1px solid black;
        padding: 5px;
        margin-bottom: 10px;
        margin-left: 20px;
      }
    </style>
  </head>
  <body>
    <nav>
      <menu>
        <details open>
          <summary>Electronics</summary>
          <div>Televisions</div>
          <div>Mobiles</div>
          <div>Watches</div>
        </details>
        <details>
          <summary>Footwear</summary>
          <div>Casuals</div>
          <div>Sneakers</div>
          <div>Boots</div>
        </details>
      </menu>
    </nav>
  </body>
```

</html>

Dialog

- It is used as a pop-up to handle interactions with user.
- Dialog in minimized or hidden, it pops-up when user selects.

Syntax:

```
<dialog open>  
.... your content....  
</dialog>
```

Figure and FigCaption

- Figure encapsulates an image with caption.
- Caption make the figure SEO friendly.

Syntax:

```
<figure>  
... your image here...  
<figcaption> Caption </figcaption>  
</figure>
```

Ex:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Document</title>  
  <style>  
    figure {  
      height: 200px;  
      width: 200px;  
      border:1px solid black;  
      padding: 5px;  
    }  
    figure div {  
      height: 100px;  
      width: 100px;  
      border:1px dotted gray;  
    }  
  </style>  
</head>  
<body>  
  <figure>  
    <img alt="A figure with a caption." data-bbox="150 650 350 850"/>  
    <figcaption>Caption</figcaption>  
  </figure>  
</body>  
</html>
```

```

</style>
</head>
<body>
  <figure>
    <div>your image.. </div>
    <figcaption>Fig: Nike Casuals</figcaption>
  </figure>
</body>
</html>

```

Article

- It is used to display latest trends on your page. It is to display the highlights of your page.
- Articles are usually displayed by using "marquee"
- Marquee sets scrolling and sliding content on page.

Syntax:

```

<article>
  <marquee> your content... </marquee>
</article>

```

- Marquee attributes

scrollamount	: It is used to set scrolling speed from 1 to 100.
direction	: It controls scrolling direction, left, right, up, down
behavior	: It sets the scrolling behavior to alternate [slide]
	[behaviour or direction only one is allowed]
loop	: It specifies the number of times to display.
bgcolor	: It sets background color for marquee area.
width/height	: It sets width and height for marquee area.

Syntax:

```

<marquee scrollamount="20" direction="right" loop="3" bgcolor="yellow"
width="200" height="50">
  .... your content.....
</marquee>

```

```

<marquee behavior="alternate"> </marquee>

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    body {
      display: flex;
      justify-content: center;
      align-items: center;
      height: 500px;
    }
    article {
      text-align: center;
      font-size: 30px;
      border: 1px solid red;
      padding: 20px;
    }
  </style>
</head>
<body>
  <article>
    <div>
      <marquee bgcolor="yellow" width="200" scrollamount="15"
behavior="alternate">50% OFF</marquee>
    </div>
    <div>
      <marquee direction="left" scrollamount="25">Offer ends on 5-Dec-
2022</marquee>
    </div>
  </article>
</body>
</html>

```

Summary

1. aside
2. article
3. dialog
4. figure
5. figcaption

6. div
7. span
8. header
9. footer
10. section
11. main
12. nav
13. menu
-
14. details
15. summary
16. marquee

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    nav div {
      width: 100px;
      height: 30px;
      border: 1px solid red;
      padding: 4px;
      font-size: 20px;
      margin-bottom: 20px;
      background-color: red;
      color: white;
    }
    nav div:hover {
      background-color: black;
      cursor: grab;
      font-size: 25px;
    }
  </style>
</head>
<body>
  <nav>
    <div>Home</div>
    <div>About</div>
```

```
<div>Contact</div>
<div>Blog</div>
<div>Pages</div>
</nav>
</body>
</html>
```

HTML 5 Semantic Elements

1. Aside
2. Article
3. Dialog
4. Figure
5. FigCaption
6. Header
7. Footer
8. Section
9. Nav
10. Menu
11. Div
12. Span
13. Main

Details
Summary
Marquee

Basic HTML Elements and Entities

- Browser can't present your content exactly how you defined in editor.
- You have to markup with basic elements and entities.

Element / Entity

Description

 	Line Break
 	Blank Space [Non Breakable Space]
<pre>	Pre Formatted Text. It keeps the line breaks & blank spaces exactly how you defined in editor.
<var>	It is used for variables.
<code>	It is used for code snippet.
<samp>	It is used to comments as sample text.

<address>

It is used for presenting contact address.

**Note: HTML comments are defined by using "<!-- your text -->"
These comments are only for developers.**

**FAQ: What is difference between
 &
 ?**

**Ans : There is no
 in HTML.**

**
 is just developers way of writing line break to tell the editor that,
it is an self ending element.**

EX:

```
<body>
  H &nbsp;&nbsp;&nbsp;T &nbsp;&nbsp;&nbsp;M &nbsp;&nbsp;&nbsp;L <br>
  It is a markup language.
</body>
```

Ex: Pre

```
<!DOCTYPE html>
<html>
  <head>
    <title>Basic Elements</title>
    <style>
      pre {
        margin-left: -30px;
        font-family: Arial;
      }
    </style>
  </head>
  <body>
    <div>Sample C Program</div>
    <div>
      <pre>
      #include &lt;stdio.h>
      main()
      {
        printf("Welcome to C Programming");
```

```
    }  
  </pre>  
</div>  
</body>  
</html>
```

Ex:

```
<!DOCTYPE html>  
<html>  
  <head>  
    <title>Basic Elements</title>  
    <style>  
      var {  
        color:gray;  
      }  
      code {  
        color:red;  
      }  
    </style>  
  </head>  
  <body>  
    <!--  
      This page is using all new elements of HTML  
      Line 2  
    -->  
    <samp>Note: New Elements are used for coding platforms.</samp>  
    <div>  
      <code>404-Page You Requested Not Found</code>  
    </div>  
    <!--Add Horizontal line here-->  
    <div>  
      const <var>x</var> = 10; <br>  
      const <var>y</var> = 20; <br>  
      const <var>z</var> = <var>x</var> + <var>y</var>;  
    </div>  
    <div>  
      <code>  
        <pre>  
          for(var i=1; i<=10; i++) {  
            console.log(i);  
          }  
        </pre>  
      </code>
```

```
    </div>
  </body>
</html>
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Flipkart | Shopping</title>
  <style>
    footer {
      display: grid;
      grid-template-columns: 6fr 6fr;
      height: 250px;
      background-color: black;
      color:white;
      padding: 5px;
      font-size: 15px;
    }
    .panel-left {
      display: grid;
      grid-template-columns: 3fr 3fr 3fr 3fr;
    }
    .panel-right {
      display: grid;
      grid-template-columns: 6fr 6fr;
      border-left: 2px solid gray;
      padding-left: 20px;
    }
    .footer-title {
      color:gray;
      margin-bottom: 20px;
    }
    .back {
      background-color:gray;
      color:white;
      padding: 10px;
      font-size: 20px;
      text-align: center;
    }
  </style>
</head>
<body>
```

```
</style>
</head>
<body>
  <div class="back">
    Back to Top
  </div>
  <footer>
    <div class="panel-left">
      <div>col-1</div>
      <div>col-2</div>
      <div>col-3</div>
      <div>col-4</div>
    </div>
    <div class="panel-right">
      <div>
        <div class="footer-title">Mail Us</div>
        <div>
          <address>
            Flipkart Internet Private Limited,<br>

            Buildings Alyssa, Begonia &<br>

            Clove Embassy Tech Village,<br>

            Outer Ring Road,<br> Devarabeesanahalli Village,<br>

            Bengaluru, 560103,<br>

            Karnataka, India
          </address>
        </div>
      </div>
      <div>
        <div class="footer-title">Registered Office Address</div>
        <div>
          <address>
            Flipkart Internet Private Limited, <br>

            Buildings Alyssa, Begonia & <br>

            Clove Embassy Tech Village,<br>
```

Outer Ring Road, Devarabeesanahalli Village,

Bengaluru, 560103,

Karnataka, India

CIN : U51109KA2012PTC066107

Telephone: 044-45614700

</address>

</div>

</div>

</div>

</footer>

</body>

</html>

Summary

- HTML Basic Structure

<!DOCTYPE>

<html>

<head> <title> <link> <meta> <style> <script>

<body>

- Semantic Elements

<aside> <article> <dialog> etc..

- Basic Elements

<code> <pre>
 <samp> etc..

Headings

- Headings are used to configure headings on page.
- They are defined in 1 to 6 levels.

Syntax:

```
<h1> heading - 1 </h1>
```

```
.....
```

```
<h6> heading - 6 </h6>
```

FAQ: Without heading style we define headings in page? Then why to use heading element?

Ans: Heading styles are used to make the content SEO friendly.

FAQ: Can we modify the heading style?

Ans: Yes. By using styles.

FAQ: Can we remove the styles? How?

Ans: Yes. By using "unset" css attribute value.

Syntax:

```
h1 {  
    all : unset;  
}
```

FAQ: Can we remove any specific style?

Yes: By using "initial" css attribute value for style.

Syntax:

```
h1 {  
    font-weight:initial;  
    font-size:initial;  
}
```

Note:

- Don't use heading style to highlight any content in a paragraph or sentence.
- Don't use too many headings in page. SEO will SPAM your page.
- Headings can be defined with "align" attribute

Syntax:

```
<h1 align="left | center | right | justify"> </h1>
```

Ex:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Basic Elements</title>
```

```

<style>
  var {
    color:gray;
  }
  code {
    color:red;
  }
  h1 {
    font-family: Arial;
    font-size: 50px;
  }
</style>
</head>
<body>
  <h1 align="center"> Web Technologies </h1>
  <h2>Front-End</h2>
  <h3>HTML</h3>
  It is a markup language.
  <h3>JavaScript</h3>
  It is a client side scripting. <br><br>
  <font size="4"><b>CSS</b></font> <br><br>
  It configures styles for HTML.
  <h2>Back-End</h2>
  <h3>MongoDB</h3>
  It is a database.
</body>
</html>

```

Paragraphs and Blockquotes

- Paragraphs are defined by using <p> element.
- Block quote is defined by using <blockquote> element.
- "align" is attribute.

Syntax:

```

<blockquote align="justify"> </blockquote>
<p align="left"> </p>

```

Ex:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Basic Elements</title>
    <style>
      blockquote {
        margin-left: 200px;
        margin-right: 200px;
        font-style: italic;
      }
    </style>
  </head>
  <body>
    <h2>Terms of Service</h2>
    <div>
      <blockquote align="justify">
        We understand that you may have questions about the Microsoft Services
        Agreement. We have an FAQ page (https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx) that provides more information,
        including what Microsoft products and services it covers. Whenever we make
        changes, we provide a summary of the most notable changes to the current
        Microsoft Services Agreement. We will also continue to provide a summary of the
        changes to the previous version, and a link to the previous version, of the
        Microsoft Services Agreement on the Summary of Changes (
        https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx)
        page.
      </blockquote>
      <p>
        We understand that you may have questions about the Microsoft Services
        Agreement. We have an FAQ page (https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx) that provides more information,
        including what Microsoft products and services it covers. Whenever we make
        changes, we provide a summary of the most notable changes to the current
        Microsoft Services Agreement. We will also continue to provide a summary of the
        changes to the previous version, and a link to the previous version, of the
        Microsoft Services Agreement on the Summary of Changes (
        https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx)
        page.
      </p>
      <p>
        We understand that you may have questions about the Microsoft Services
        Agreement. We have an FAQ page (https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx) that provides more information,
```


including what Microsoft products and services it covers. Whenever we make changes, we provide a summary of the most notable changes to the current Microsoft Services Agreement. We will also continue to provide a summary of the changes to the previous version, and a link to the previous version, of the Microsoft Services Agreement on the Summary of Changes (<https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx>) page.

</p>

<p>

We understand that you may have questions about the Microsoft Services Agreement. We have an FAQ page (<https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx>) that provides more information, including what Microsoft products and services it covers. Whenever we make changes, we provide a summary of the most notable changes to the current Microsoft Services Agreement. We will also continue to provide a summary of the changes to the previous version, and a link to the previous version, of the Microsoft Services Agreement on the Summary of Changes (<https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx>) page.

</p>

<p>

We understand that you may have questions about the Microsoft Services Agreement. We have an FAQ page (<https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx>) that provides more information, including what Microsoft products and services it covers. Whenever we make changes, we provide a summary of the most notable changes to the current Microsoft Services Agreement. We will also continue to provide a summary of the changes to the previous version, and a link to the previous version, of the Microsoft Services Agreement on the Summary of Changes (<https://www.microsoft.com/en-us/servicesagreement/upcoming-updates.aspx>) page.

</p>

</div>

</body>

</html>

Data List

- A data list comprises of Terms and definitions.
- List is defined by using <dl>.
- Terms are defined by using <dt>.
- Definitions are defined by using <dd>.

Syntax:

```
<dl>
  <dt> Term </dt>
  <dd> definition </dd>
</dl>
```

Ex:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Basic Elements</title>
    <style>
      dt {
        background-color: gray;
        color:white;
        font-weight: bold;
        padding: 2px;
        text-align: left;
        margin-bottom: 10px;
      }
    </style>
  </head>
  <body>
    <h2>Front-End Technologies</h2>
    <dl>
      <dt>HTML</dt>
      <dd>It is a markup language.</dd>
      <dd>It is used for Presentation.</dd>
      <dd>Latest Version is HTML 5.</dd>
      <dt>CSS</dt>
      <dt>Styles</dt>
      <dd>It defines styles for HTML.</dd>
      <dt>JavaScript</dt>
      <dd> It is client side scripting.</dd>
    </dl>
  </body>
</html>
```

Ex:

```
<!DOCTYPE html>
```

```

<html>
  <head>
    <title>Basic Elements</title>
    <style>
      dt {
        background-color: black;
        color:white;
        margin-bottom: 10px;
        width: 200px;
        padding: 5px;
      }
      dl {
        display: grid;
        grid-template-columns: 2fr 10fr;

      }
      dd {
        background-color: gray;
        color:white;
        margin-bottom: 10px;
        padding: 5px;
      }
    </style>
  </head>
  <body>
    <dl>
      <dt>First Name</dt>
      <dd>some name</dd>
      <dt>Last Name</dt>
      <dd>some name</dd>
      <dt>Age</dt>
      <dd>some number</dd>
      <dt>Address</dt>
      <dd>some contact</dd>
    </dl>
  </body>
</html>

```

Ex:

```

<!DOCTYPE html>
<html>
  <head>
    <title>Basic Elements</title>

```

[illegible]

[illegible]

```

<dd>Kids Clothing</dd>
<dd>Jeans</dd>
<dd>Winter Jacket</dd>
<dd>Kids Clothing</dd>
<dd>Jeans</dd>
<dd>Winter Jacket</dd>
<dd>Kids Clothing</dd>
<dd>Jeans</dd>
<dd>Winter Jacket</dd>
<dd>Kids Clothing</dd>
<dd>Jeans</dd>
<dd>Winter Jacket</dd>
<dd>Kids Clothing</dd>
<dd>Jeans</dd>
<dd>Winter Jacket</dd>
<dd>Kids Clothing</dd>
<dd>Jeans</dd>
<dd>Winter Jacket</dd>
<dd>Kids Clothing</dd>
<dd>Jeans</dd>
<dd>Winter Jacket</dd>
<dd>Kids Clothing</dd>
<dd>Jeans</dd>
<dd>Winter Jacket</dd>
</dl>
</body>
</html>

```

Fieldset and Legend

- Fieldset is a frame.
- Legend is a caption for frame.
- It is used to display a category of options.

Syntax:

```

<fieldset>
  <legend> Title </legend>
  ..... your content.....

```

`</fieldset>`

Ex:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Basic Elements</title>
    <style>
      dt {
        font-weight: bold;
        background-color: gray;
        color: white;
        margin-bottom: 10px;
      }
      dl {
        display: grid;
        grid-template-columns: 3fr 9fr;
      }
      fieldset {
        margin-bottom: 40px;
        box-shadow: 2px 2px 2px black;
      }
      legend {
        background-color: black;
        padding: 5px;
        text-align: center;
        color: white;
        border-radius: 20px;
        box-shadow: 5px 5px 5px gray;
      }
    </style>
  </head>
  <body>
    <fieldset>
      <legend>Personal Details</legend>
      <dl>
        <dt>First Name</dt>
        <dd>some name</dd>
        <dt>Last Name</dt>
        <dd>some name</dd>
        <dt>Age</dt>
        <dd>some age</dd>
      </dl>
    </fieldset>
  </body>
</html>
```

```

</fieldset>
<fieldset>
  <legend>Education Details</legend>
  <dl>
    <dt>School</dt>
    <dd>Some name</dd>
    <dt>College</dt>
    <dd>Some name</dd>
  </dl>
</fieldset>
</body>
</html>

```

Ex:

```

<!DOCTYPE html>
<html>
  <head>
    <title>Basic Elements</title>
    <style>
      legend {
        text-align: center;
        padding-right: 10px;
        padding-left: 10px;
      }
      fieldset {
        text-align: center;
        border-left: none;
        border-right: none;
        border-bottom: none;
        width: 300px;
        border-top: 1px solid gray;
      }
      body {
        display: flex;
        justify-content: center;
      }
    </style>
  </head>
  <body>
    <fieldset>
      <legend>New to Amazon ?</legend>
      <button>Create you Amazon account</button>
    </fieldset>
  </body>
</html>

```



```
</body>
</html>
```

Ordered and Unordered List

- ordered list defines auto numbering.
- is for list.
- is for item in list.

Syntax:

```
<ol>
  <li> Item - 1 </li>
  <li> Item - 2 </li>
</ol>
```

- You can change numbering style by using "type" attribute.

```
<ol type="1,a,A,i,I">
```

- You can set start level for numbering by using "start" attribute.

```
<ol type="1" start="101">
<ol type="A" start="5">
```

- HTML 5 introduced "reversed" attribute.

```
<ol type="A" reversed>
```

Attributes:

- type
- start
- reversed

- You can create nested list.

Note: Always keep nested list inside element. Not below.

Ex:

```
<!DOCTYPE html>
<html>
  <head>
```

```

<title>Basic Elements</title>
<style>

</style>
</head>
<body>
  <ol>
    <li>Electronics
      <ol type="a">
        <li>Mobiles</li>
        <li>Televisions</li>
      </ol>
    </li>
    <li>Footwear
      <ol type="a">
        <li>Casuals
          <ol type="i">
            <li>Nike</li>
            <li>Puma</li>
          </ol>
        </li>
        <li>Sneakers</li>
      </ol>
    </li>
  </ol>
</body>
</html>

```

FAQ: How to keep list side by side?

Ans: You can set display-flex, or display-inline
 [flex is for container]
 [inline is for item]

Ex:

```

<!DOCTYPE html>
<html>
  <head>
    <title>Basic Elements</title>
    <style>
      ol {
        display: flex;
      }
      li {

```

```

        margin-right: 50px;
    }
</style>
</head>
<body>
    <ol>
        <li>Home</li>
        <li>About</li>
        <li>Contact</li>
    </ol>
</body>
</html>

```

Ex:

```

<!DOCTYPE html>
<html>
    <head>
        <title>Basic Elements</title>
        <style>
            li {
                display: inline;
                margin-left: 20px;
            }
        </style>
    </head>
    <body>
        <ol>
            <li>Home</li>
            <li>About</li>
            <li>Contact</li>
        </ol>
    </body>
</html>

```

FAQ: How to display in multiple columns?

Ans: There are 2 techniques

display-grid [individual columns]
columns [continous columns]

Ex:

```

<!DOCTYPE html>

```

```

<html>
  <head>
    <title>Basic Elements</title>
    <style>
      .choice {
        margin-top: 20px;
        columns:2;
      }
    </style>
  </head>
  <body>
    <ol type="1">
      <li>_____ elements will not required end token.
        <ol type="a" class="choice">
          <li>Normal Elements</li>
          <li>Void Elements</li>
          <li>Raw Text</li>
          <li>Foreign</li>
        </ol>
      </li>
    </ol>
  </body>
</html>

```

Syntax:

```

.choice {
  display:grid;
  grid-template-columns: 6fr 6fr
}

```

FAQ: How to create scrollable list?

Ans: By using border, height and overflow.

Ex:

```

<!DOCTYPE html>
<html>
  <head>
    <title>Basic Elements</title>
    <style>
      ol {
        border:1px solid black;
        width: 100px;
        height: 55px;
      }
    </style>
  </head>
  <body>
    <ol>
      <li>_____ elements will not required end token.
        <ol type="a" class="choice">
          <li>Normal Elements</li>
          <li>Void Elements</li>
          <li>Raw Text</li>
          <li>Foreign</li>
        </ol>
      </li>
    </ol>
  </body>
</html>

```

```

        overflow: auto;
    }
</style>
</head>
<body>
    <ol>
        <li>HTML</li>
        <li>CSS</li>
        <li>JavaScript</li>
        <li>jQuery</li>
        <li>Bootstrap</li>
        <li>Sass</li>
        <li>Less</li>
        <li>Node JS</li>
    </ol>
</body>
</html>

```

FAQ: How to remove numbering?

Ans : By using "list-style" css attribute

```

ol {
    list-style : none;
}

```

Ex: Footer

```

<!DOCTYPE html>
<html>
    <head>
        <title>Basic Elements</title>
        <style>
            footer {
                background-color: black;
                color:white;
                padding: 5px;
                font-family: Arial;
                display: grid;
                grid-template-columns: 3fr 3fr 3fr 3fr;
            }
            ol {

```

[illegible]

Unordered List

- It sets bullet symbol for list of items.
- for list
- for items

Syntax:

```
<ul>
  <li> Item-1 </li>
  <li> Item-2 </li>
</ul>
```

- Attribute "type" is used to change bullet symbol type, which can be circle, disc, square

Syntax:

```
<ul type="circle | disc | square">
```

FAQ: How to define custom bullets?

Ans : By using the CSS attribute "list-style-image"

Syntax:

```
ul {
  list-style-image: url("images/bullet.png");
}
```

Ex

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    ul {
      list-style-image: url("images/bullet.png");
      font-size: 20px;
    }
  </style>
</head>
<body>
  <ul>
```

```
        <li>HTML</li>
        <li>CSS</li>
        <li>JavaScript</li>
    </ul>
</body>
</html>
```

Ex.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <style>

        </style>
</head>
<body>
    <ol>
        <li>HTML
            <ul type="square">
                <li>Normal Elements
                    <ol type="a">
                        <li>Bold</li>
                        <li>Italics</li>
                    </ol>
                </li>
                <li>Void Elements</li>
            </ul>
        </li>
        <li>CSS</li>
        <li>JavaScript</li>
    </ol>
</body>
</html>
```

Text Formatting in HTML

1. Font
2. Font Styles
3. Font Effects

1.Font:

- It is defined by using element.
- It can change face, size, color.

face : Font Family Name [Arial, Brush Script MT,...]
size : From 1 to 7 level
color : Text color, name or hexa code

Syntax:

```
<font size="7" color="#0000ff">  
Welcome to HTML  
</font>
```

FAQ: What are web-safe fonts?

Ans : The fonts that can run and display on every browser.

- a) serif
- b) sans-serif
- c) monospace

Syntax:

```
<font size="5" color="red" face="Brush Script MT">  
Welcome to HTML  
</font>
```

Ex:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Document</title>  
  <style>  
  
    </style>  
</head>  
<body>  
  <font size="7" face="Brush Script MT" color="#0000ff">  
    Welcome to HTML  
  </font>  
  <font size="5" face="Algerian">
```

```
<p>It is a markup language.</p>
</font>
</body>
</html>
```

2. Font Style and Effects

bold	
italic	<i> Emphasized
underline	<u> <ins> Inserted
strikeout	<strike> Deleted
superscript	<sup>
subscript	<sub>

FAQ: What is difference between & ?

Ans : Bold is used in design mode.

** is used in review mode.**

Ex.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    .product {
      display: grid;
      grid-template-columns: 2fr 7fr 3fr;
      font-family: Arial;
    }
    .rating {
      background-color: green;
      color:white;
      padding: 3px;
      border-radius: 5px;
    }
    li {
```

```
        color:gray;
        margin-bottom: 8px;
        margin-left: -24px;
    }
</style>
<link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body>
    <div class="product">
        <div>

            </div>
            <div>
                <div> <span class="bi bi-heart-fill"></span> <span class="title"><font
size="5" color="blue"><b>realme C33 (Aqua Blue, 32 GB)</b></font></span>
</div>
                <br>
                <div>
                    <span class="rating">4.5 <span class="bi bi-star-fill"></span> </span>
<span>14,673 Ratings & 452 Reviews</span>
                </div>
                <div>
                    <ul>
                        <li>3 GB RAM | 32 GB ROM | Expandable Upto 1 TB</li>
                        <li>16.51 cm (6.5 inch) HD+ Display</li>
                        <li>50MP + 0.3MP | 5MP Front Camera</li>
                        <li>5000 mAh Lithium Ion Battery</li>
                        <li>Unisoc T612 Processor</li>
                        <li>1 Year Manufacturer Warranty for Phone and 6 Months Warranty for
In-Box Accessories</li>
                    </ul>
                </div>
            </div>
            <div>
                <div><font size="6"><b>₹8,999</b></font> </div>
                <br>
                <div><font size="4" color="gray"><del> ₹11,999</del></font>
<span><font color="green"> <b>25% Off</b></font></span> </div>
                <br>
                <div> Free delivery </div>
                <br>
                <div> Upto <b>₹8,400</b> Off on Exchange </div>
            </div>
        </div>
    </div>
</body>
</html>
```

```
<div><font color="green"><b>Bank Offer</b></font> </div>
</div>
</div>
<hr>

<div class="product">
<div>

</div>
<div>
<div><span class="bi bi-heart-fill"></span><span class="title"><font
size="5" color="blue"><b>realme C33 (Silver, 64 GB)</b></font></span> </div>
<br>
<div>
<span class="rating">4.5 <span class="bi bi-star-fill"></span> </span>
<span>14,673 Ratings & 452 Reviews</span>
</div>
<div>
<ul>
<li>3 GB RAM | 32 GB ROM | Expandable Upto 1 TB</li>
<li>16.51 cm (6.5 inch) HD+ Display</li>
<li>50MP + 0.3MP | 5MP Front Camera</li>
<li>5000 mAh Lithium Ion Battery</li>
<li>Unisoc T612 Processor</li>
<li>1 Year Manufacturer Warranty for Phone and 6 Months Warranty for In-Box Accessories</li>
</ul>
</div>
</div>
<div>
<div><font size="6"><b>₹8,377; 9,599</b></font> </div>
<br>
<div><font size="4" color="gray"><strike> ₹8,377; 14,999</strike></font>
<span><font color="green"> <b>25% Off</b></font></span> </div>
<br>
<div>Free delivery </div>
<br>
<div>Upto <b>₹8,377; 8,400</b> Off on Exchange </div>

<div><font color="green"><b>Bank Offer</b></font> </div>
</div>
</div>
```

</body>
</html>

Images in HTML

- Images are various types.
- Web will not support all types of images.
- Image types supported for web are:

PNG [Portable Network Graphics]	.png
APNG [Animated PNG]	.apng
JPEG [Joint Photographic Expert Group]	.jpg, .jpeg, .jfif
GIF [Graphic Interchange Format]	.gif
TIFF [Tagged Image File Format]	.tiff
SVG [Scalar Vector Graphics]	.svg
BMP [Bitmap]	.bmp
WebP [Web Picture]	.webp
ICO [Microsoft Icon]	.ico

- Image Types and their purpose

PNG, APNG	HD, More Pixels, [Animated PNG] More space in memory Providing image for download
JPG	HD, Compressed Image Less Pixels than PNG Good for On Screen Presentation
GIF	Only 256 Colors Size no-change Bullets, Icons, Logos Animated but less colors.
SVG	Not Pixel Based It is XML format Not infected by virual Not blocked by firewall Diagrams, Architecture, Maps

ICO	Icons [Favicon]
TIFF	Scanned Documents Sliced Image Format
WebP	Embedded into page.
BMP	Binary Format

- HTML can embed image into page by using token [tag]

- Attributes

- | | |
|-----------|---|
| 1. alt | : alternate text to display when image fails to load. |
| 2. src | : it specifies the image path and name |
| 3. width | : it sets width |
| 4. height | : it sets height |
| 5. title | : it sets screen tip for image |
| 6. align | : it sets alignment left or right for wrapping. |
| 7. vspace | : vertical padding space. |
| 8. hspace | : horizontal padding space. |
| 9. border | : sets border with size |

Syntax:

```

```

Image Attributes

- alt
- src
- width
- height
- align
- vspace
- hspace
- border

FAQ: What is difference between "href" & "src" ?

```
<link href="images/favicon.ico">
```

``

Ans: "href" is known as "setter", it is an accessor used to set path.
"src" is known as "getter", it is an accessor used to get path.

setter is used to set path in the URL, so that file is requested and loaded.
getter is used to get path, with out loading URL it will just access the resource.

Advanced Attributes:

- **crossorigin** : It defines how the restrictions can be defined for image which is accessed from remote server.

- a) allow-anonymous
- b) use-credentials

- **decoding** : It defines how image loads along with other content in page.

- a) sync : It blocks the other content.
- b) async: It loads along with other content
- c) auot: It changes according to situation.

- **importance** : It sets priority for loading images.

- a) low
- b) high

- **srcset** : It configure multiple images to display.

Design Cards

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cars</title>
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
```

```
<style>
  .card {
    width: 200px;
    box-shadow: 2px 2px 2px gray;
    padding: 5px;
    margin: 10px;
  }
  .card-img {
    width: 100%;
    height: 160px;
  }
  .card-header {
    text-align: center;
    height: 70px;
  }
  .card-body {
    text-align: center;
    height: 90px;
  }
  .rating {
    color: goldenrod;
    margin-bottom: 10px;
  }
  .container {
    display: flex;
    flex-wrap: wrap;
  }
</style>
</head>
<body>
  <div class="container">
    <div class="card">
      
      <div class="card-header">
        <p>Digital Marketing</p>
      </div>
      <div class="card-body">
        <div class="rating">
          <span class="bi bi-star-fill"></span>
          <span class="bi bi-star-fill"></span>
          <span class="bi bi-star-fill"></span>
          <span class="bi bi-star-fill"></span>
          <span class="bi bi-star-fill"></span>
        </div>
      </div>
    </div>
  </div>
</body>
```



```
        </div>
        <div>View Course</div>
    </div>
</div>
```

```
<div class="card">
    
    <div class="card-header">
        <p>Android Training</p>
    </div>
    <div class="card-body">
        <div class="rating">
            <span class="bi bi-star-fill"></span>
            <span class="bi bi-star-fill"></span>
            <span class="bi bi-star-fill"></span>
            <span class="bi bi-star-fill"></span>
            <span class="bi bi-star-fill"></span>
        </div>
        <div>View Course</div>
    </div>
</div>
```

```
<div class="card">
    
    <div class="card-header">
        <p>ASP Training</p>
    </div>
    <div class="card-body">
        <div class="rating">
            <span class="bi bi-star-fill"></span>
            <span class="bi bi-star-fill"></span>
            <span class="bi bi-star-fill"></span>
            <span class="bi bi-star-fill"></span>
            <span class="bi bi-star-fill"></span>
        </div>
        <div>View Course</div>
    </div>
</div>
```

```
<div class="card">
    
    <div class="card-header">
        <p>RPA Training</p>
```

```

</div>
<div class="card-body">
  <div class="rating">
    <span class="bi bi-star-fill"></span>
    <span class="bi bi-star-fill"></span>
    <span class="bi bi-star-fill"></span>
    <span class="bi bi-star-fill"></span>
    <span class="bi bi-star-fill"></span>
  </div>
  <div>View Course</div>
</div>
</div>

<div class="card">
  
  <div class="card-header">
    <p>PHP Training</p>
  </div>
  <div class="card-body">
    <div class="rating">
      <span class="bi bi-star-fill"></span>
      <span class="bi bi-star-fill"></span>
      <span class="bi bi-star-fill"></span>
      <span class="bi bi-star-fill"></span>
      <span class="bi bi-star-fill"></span>
    </div>
    <div>View Course</div>
  </div>
</div>

</div>
</body>
</html>

Ex.
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Amazon</title>
  <style>

```

```
    section {
      display: grid;
      grid-template-columns: 3fr 3fr 3fr 3fr;
      margin-top: -300px;
    }
    .card {
      height: 400px;
      background-color: white;
      box-shadow: 2px 2px 2px black;
      margin: 10px;
      padding: 5px;
    }
    .row {
      display: grid;
      grid-template-columns: 6fr 6fr;
      margin-bottom: 10px;
    }
    .row div {
      margin: 5px;
    }
    .card-header {
      font-family: Arial;
      font-size: 20px;
      font-weight: bold;
      margin-bottom: 10px;
    }
  </style>
</head>
<body>
  <header>
    
  </header>
  <section>
    <div class="card">
      <div class="card-header">
        Top picks for your home
      </div>
      <div class="card-body">
        <div class="row">
          <div>
            
            <p>Airconditions</p>
          </div>
```

```

        <div>
            
            <p>Refrigrators</p>
        </div>
    </div>
    <div class="row">
        <div>
            
            <p>Microwaves</p>
        </div>
        <div>
            
            <p>Washing</p>
        </div>
    </div>
    <div>
        more..
    </div>
</div>
<div class="card">

</div>
<div class="card">

</div>
<div class="card">

</div>
</section>
</body>
</html>

```

HyperLinks in HTML

- Hyper mean beyond.
- Hyper link is clickable text, picture or graphic that navigates the user to any specified location when clicked.

- Hyper links are classified into 2 types

- a) Intra Document : Navigates from one location to another in same page.
- b) Inter Document : Navigates from one page to another.

- Hyper links are created by using anchor element <a>.

- The navigation path is defined by using "href" attribute.

Intra Document Links:

- 1. It handles navigation from one location to another within the same page.
- 2. The target location is defined by using "id".

```
<h2 id="home"> Home </h2>
<figure id="">
<ol id="">
<img id="">
```

3. You can navigate to any specific ID by using <a> element.

```
<a href="#home"> Text | Picture | Graphic </a>
```

Note:

1. How to change link colors?

A. CSS Classes

```
a:link { }      Normal Link
a:active{ }     Active Link
a:visited{ }    Visited Link
a{ }
```

2. How to remove underline for hyperlink?

A. By using CSS attribute "text-decoration:none"

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
```

```
<style>
  section {
    display: grid;
    grid-template-columns: 3fr 9fr;
  }
  header {
    background-color: black;
    color:white;
    text-align: center;
  }
  main {
    height: 500px;
    overflow: auto;
  }
  ul {
    list-style: none;
  }
  li {
    background-color: black;
    color:white;
    font-size: 20px;
    margin-bottom: 20px;
    width: 100px;
    border-radius: 20px;
    padding: 10px;
  }
  li:hover {
    background-color: green;
    color:black;
  }
  nav a {
    color:white;
    text-decoration: none;
  }
</style>
</head>
<body>
  <header>
    <h1>Online Tutorials</h1>
  </header>
  <section>
    <nav>
      <ul>
```

```
<li><a href="#home">Home</a></li>
<li><a href="#asp">ASP.NET</a></li>
<li><a href="#android">Android</a></li>
<li><a href="#php">PHP</a></li>
</ul>
</nav>
<main>
```

```
<h2 id="home">Home</h2>
```

```
<p>Depending on <span><a href="#frameworks">Mobile
Frameworks</a></span> how you obtained the Windows software, this is a
license agreement between (i) you and the device manufacturer or software
installer that distributes the software with your device; or (ii) you and Microsoft
Corporation (or, based on where you live or, if a business, where your principal
place of business is located, one of its affiliates) if you acquired the software
from a retailer. Microsoft is the device manufacturer for devices produced by
Microsoft or one of its affiliates, and Microsoft is the retailer if you acquired the
software directly from Microsoft. Note that if you are a volume license customer,
use of this software is subject to your volume license agreement rather than this
agreement.</p>
```

```
<h2 id="asp">ASP.NET</h2>
```

```
<div> <a href="#home">Back to Top</a></div>
```

```

```

```
<p>Depending on how you obtained the Windows software, this is a
license agreement between (i) you and the device manufacturer or software
installer that distributes the software with your device; or (ii) you and Microsoft
Corporation (or, based on where you live or, if a business, where your principal
place of business is located, one of its affiliates) if you acquired the software
from a retailer. Microsoft is the device manufacturer for devices produced by
Microsoft or one of its affiliates, and Microsoft is the retailer if you acquired the
software directly from Microsoft. Note that if you are a volume license customer,
use of this software is subject to your volume license agreement rather than this
agreement.</p>
```

```
<h2 id="android">Android Training</h2>
```

```
<div><a href="#home">Back to Top</a></div>
```

```

```

```
<p>Depending on how you obtained the Windows software, this is a
license agreement between (i) you and the device manufacturer or software
installer that distributes the software with your device; or (ii) you and Microsoft
Corporation (or, based on where you live or, if a business, where your principal
place of business is located, one of its affiliates) if you acquired the software
from a retailer. Microsoft is the device manufacturer for devices produced by
Microsoft or one of its affiliates, and Microsoft is the retailer if you acquired the
software directly from Microsoft. Note that if you are a volume license customer,
```

use of this software is subject to your volume license agreement rather than this agreement.</p>

```
<dl id="frameworks">
  <dt>Mobile Frameworks</dt>
  <dd>Apache Cordova</dd>
  <dd>Ionic</dd>
  <dt>Mobile Platforms</dt>
  <dd>Android</dd>
  <dd>IOS</dd>
</dl>
```

```
<h2 id="php">PHP Tutorial</h2>
```

```
<div><a href="#home">Back to Top</a></div>
```

```

```

<p>Depending on how you obtained the Windows software, this is a license agreement between (i) you and the device manufacturer or software installer that distributes the software with your device; or (ii) you and Microsoft Corporation (or, based on where you live or, if a business, where your principal place of business is located, one of its affiliates) if you acquired the software from a retailer. Microsoft is the device manufacturer for devices produced by Microsoft or one of its affiliates, and Microsoft is the retailer if you acquired the software directly from Microsoft. Note that if you are a volume license customer, use of this software is subject to your volume license agreement rather than this agreement.</p>

```
</main>
```

```
</section>
```

```
</body>
```

```
</html>
```

Ex:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Document</title>
```

```
  <style>
```

```
    .topic {
```

```
      width: 200px;
```

```
      padding: 10px;
```

```
      box-shadow: 2px 2px 2px black;
```



```
        height: 200px;
        border: 1px solid black;
        margin: 20px;
    }
    .topic:target {
        background-color: black;
        color: white;
        font-size: 25px;
        border-radius: 30px;
    }
    .container {
        display: flex;
    }
    ul {
        list-style: none;
        display: flex;
        background-color: black;
        color: white;
        padding: 10px;
    }
    li {
        font-size: 20px;
        margin-right: 50px;
    }
    a {
        color: white;
        text-decoration: none;
    }
    li: hover {
        text-decoration: underline;
    }
</style>
</head>
<body>
    <nav>
        <ul>
            <li><a href="#html">HTML</a></li>
            <li><a href="#css">CSS</a></li>
            <li><a href="#js">JavaScript</a></li>
        </ul>
    </nav>
    <div class="container">
        <div id="html" class="topic">
```

```

    <h2>HTML</h2>
    <p>It is a markup language.</p>
</div>
<div id="css" class="topic">
    <h2>CSS</h2>
    <p>It defines styles for HTML.</p>
</div>
<div id="js" class="topic">
    <h2>JavaScript</h2>
    <p>It is a client side language.</p>
</div>
</div>
</body>
</html>

```

Hyper Links

- Intra Document
- Inter Document

Inter Document Link

- Handling navigation from one document to another.
- You can link to any
 - URL
 - File
 - Application

- Links are created by using "<a>" with "href" attribute.

Note:

- You can link any type of file from web page.
- Every file can't open and display its content in browser.
- To open and display file content in browser, it required a special software extension known as "Plugin".
- If plugin is available, then it can open the file and display its content.
- If plugin is not available then file will download.

FAQ: How to download a linked document always?

Ans : By using "download" attribute for <a> element.

Syntax:

```

<a href="images/banner.jpg" download> Text </a>
<a href="images/banner.jpg" download="fileName">

```

- You can create links that can invoke various types of apps in your device.

mailto:	Used to invoke any Email application
skype:	It invokes skype
tel:	It invokes phone dial
javascript:	It invokes any JavaScript function

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h2>Inter Document Links</h2>
  <ol>
    <li><a href="amazon-card.html">Amazon Template</a></li>
    <li><a href="http://www.amazon.in">Amazon Shopping</a></li>
    <li><a href="images/amazonBack.jpg" download="Amazon-
Banner">Amazon Banner</a></li>
    <li><a href="docs/onlineclass.pdf">Class Tutorial PDF</a></li>
    <li><a href="docs/class.docx">Class Tutorial Word Document</a></li>
    <li>Contact Us : <a href="hr@nareshit.in">hr@nareshit.in</a></li>
    <li>Skype Us : <a href="skype:nitinfo@outlook.com">NITinfo@outlook.com
</a></li>
    <li>Call Us : <a href="tel:+919876543210">+919876543210</a></li>
  </ol>
</body>
</html>
```

FAQ'S

1. How to open linked document in a new tab?

A. By using "target=_blank"

```
<a href="images/banner.jpg" target="_blank">
```

2. How to open linked document in a new window?

A. By using "javascript" window object.

Syntax:

```
<a href="javascript:window.open('images/asp.jpg','ASP','width=400
height=400')"> Text </a>
```

3. How to open linked document in the same window along with existing content?

A. By using <iframe> </iframe>

Note: <iframe> can embed any content into page along with existing content.

```
<iframe src="" name=""> </iframe>
```

<https://www.youtube.com/watch?v=HMzB3zGEuqE>

<https://www.youtube.com/embed/HMzB3zGEuqE>

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    .container {
      display: grid;
      grid-template-columns: 4fr 8fr;
    }
    iframe {
      border:none;
    }
  </style>
</head>
<body>
  <h2>Inter Document Links</h2>
  <div class="container">
```

```

<div>
<ol>
<li><a href="amazon-card.html" target="_blank">Amazon
Template</a></li>
<li><a href="http://www.amazon.in">Amazon Shopping</a></li>
<li><a href="javascript:window.open('images/asp.jpg','ASP','width=300
height=300')">  </a></li>
<li><a href="images/android.jpg" target="frameBody">Android</a></li>
<li><a href="images/amazonBack.jpg" download="Amazon-
Banner">Amazon Banner</a></li>
<li><a href="docs/onlineclass.pdf" target="frameBody">Class Tutorial
PDF</a></li>
<li><a href="docs/class.docx">Class Tutorial Word Document</a></li>
<li>Contact Us : <a href="hr@nareshit.in">hr@nareshit.in</a></li>
<li>Skype Us : <a href="skype:nitinfo@outlook.com">
NITinfo@outlook.com</a></li>
<li>Call Us : <a href="tel:+919876543210">+919876543210</a></li>
<li><a href="https://www.youtube.com/embed/HMzB3zGEuqE">
target="frameBody">Core Java - Hindi</a></li>
</ol>
</div>
<div>
<iframe width="100%" name="frameBody" height="350"></iframe>
</div>
</div>
</body>
</html>

```

Tables in HTML

- Tables are used to arrange data in rows and columns.
- Tables are used to design Grids and Dashboards.

www.telerik.com
 devExpress
 jQuery UI
 MUI

- Table Elements in HTML

<table> start and end of table

<caption>	make table SEO friendly
<thead>	Header
<tbody>	Body
<tfoot>	Footer
<tr>	Row in head, body, foot
<th>	Heading style for table cell
<td>	Ordinary table cell
<colgroup>	spans a set of columns to define style

EX:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <table border="1" width="600">
    <caption>NareshIT - Courses</caption>
    <colgroup span="3"></colgroup>
    <thead>
      <tr>
        <th>Course Id</th>
        <th>Course Name</th>
        <th>Banner</th>
        <th>Course Content</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>101</td>
        <td>ASP.NET</td>
        <td>
          
        </td>
        <td>
          <ul>
            <li>Topic-1</li>
            <li>Topic-2</li>
            <li>Topic-3</li>
          </ul>
        </td>
      </tr>
    </tbody>
  </table>

```

```

        </td>
    </tr>
    <tr>
        <td>102</td>
        <td>Android</td>
        <td>
            
        </td>
        <td>
            <ul>
                <li>Topic-1</li>
                <li>Topic-2</li>
                <li>Topic-3</li>
            </ul>
        </td>
    </tr>
</tbody>
<tfoot>
    <tr>
        <td></td>
        <td>&copy; ; copyright</td>
        <td></td>
        <td></td>
    </tr>
</tfoot>
</table>
</body>
</html>

```

Table Attributes:

1. frame, rules and border

frame : It sets a frame for table with values:
box, above, below, rhs, lhs, void

```
<table frame="box|void|rhs..">
```

rules : It sets rules for rows, columns and groups, it have values:
all, rows, cols, groups, none

```
<table frame="box" rules="all | rows | cols | groups">
```

border : It sets border for cells , it can have only 2 values

0[false] : no border

1[true] : border

Note: You can't set border if rules are defined. Rules attributes must be removed

2. cellpadding and cellspacing

cellspacing : It sets space between cells.

cellpadding : It sets space between content in cell and its border

```
<table border="1" cellspacing="10" cellpadding="5">
```

3. bgcolor and background

bgcolor : It sets background color for table, group, tr, cell.

background : It sets a background image for table, group, tr, cell.

```
<table bgcolor="">
```

```
<table background="">
```

```
<tr bgcolor="">
```

4. width and height

width : It can be defined only for table, th, td

height : It can be defined only for table, group, tr, td, th

```
<tr height="200">
```

```
<td width="100">
```

5. colspan and rowspan

colspan : It can merge all columns into single cell
rowspan : It can merge all rows into single cell

6. align and valign

align : left, center, right, justify
valign : top, center, bottom

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    td {
      background-size: cover;
    }
  </style>
</head>
<body>
  <table frame="box" cellspacing="10" cellpadding="5" border="1"
width="800">
    <caption>NareshIT - Courses</caption>
    <colgroup span="3"></colgroup>
    <thead >
      <tr height="50">
        <th>Course Id</th>
        <th>Course Name</th>
        <th>Banner</th>
        <th width="300">Course Content</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>101</td>
        <td>ASP.NET</td>
        <td background="/images/asp.jpg">
          <img alt="ASP.NET Banner" data-bbox="205 852 548 868" />
        </td>
      </tr>
    </tbody>
  </table>
```

```

        <td>
            <ul>
                <li>Topic-1</li>
                <li>Topic-2</li>
                <li>Topic-3</li>
            </ul>
        </td>
    </tr>
    <tr>
        <td width="50">102</td>
        <td>Android</td>
        <td>
            
        </td>
        <td>
            <ul>
                <li>Topic-1</li>
                <li>Topic-2</li>
                <li>Topic-3</li>
            </ul>
        </td>
    </tr>
</tbody>
<tfoot>
    <tr>
        <td></td>
        <td>&copy; copyright</td>
        <td></td>
        <td></td>
    </tr>
</tfoot>
</table>
</body>
</html>

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>

```

```

</head>
<body>
  <table width="800" border="1">
    <thead>
      <tr>
        <th colspan="2">Name</th>
        <th colspan="3">Address</th>
      </tr>
      <tr>
        <th>First Name</th>
        <th>Last Name</th>
        <th>City</th>
        <th>State</th>
        <th>Postal Code</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>&nbsp;</td>
        <td>&nbsp;</td>
        <td rowspan="5" align="center">
          D <br>
          E <br>
          L <br>
          H <br>
          I
        </td>
        <td>&nbsp;</td>
        <td>&nbsp;</td>
      </tr>
      <tr>
        <td>&nbsp;</td>
        <td>&nbsp;</td>
        <td>&nbsp;</td>
        <td>&nbsp;</td>
      </tr>
      <tr>
        <td>&nbsp;</td>
        <td>&nbsp;</td>
        <td>&nbsp;</td>
        <td>&nbsp;</td>
      </tr>
      <tr>
        <td>&nbsp;</td>
        <td>&nbsp;</td>
        <td>&nbsp;</td>
        <td>&nbsp;</td>
      </tr>
      <tr>
        <td>&nbsp;</td>
        <td>&nbsp;</td>
        <td>&nbsp;</td>
        <td>&nbsp;</td>
      </tr>
    </tbody>
  </table>

```

```
</tr>
<tr>
  <td>&nbsp;</td>
  <td>&nbsp;</td>

  <td>&nbsp;</td>
  <td>&nbsp;</td>
</tr>
<tr>
  <td>&nbsp;</td>
  <td>&nbsp;</td>

  <td>&nbsp;</td>
  <td>&nbsp;</td>
</tr>
</tbody>
<tfoot>
  <tr>
    <td colspan="5" align="center">&copy; copyright 2022</td>
  </tr>
</tfoot>
</table>
</body>
</html>
```

Table Elements

```
<table>
<caption>
<thead>
<tbody>
<tfoot>
<tr>
<th>
<td>
<colgroup>
```

Table Attributes

- frame
- rules
- border
- cellspacing
- cellpadding
- width

- height
- align
- valign
- bgcolor
- background
- colspan
- rowspan

Note: Tables lead to a situation called "Kiss-of-Death". It is not sure about where and which content can load on page.

Ex: Nested Table

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    h2 {
      font-family: Arial;
    }
  </style>
</head>
<body>
  <table width="100%" border="1" cellspacing="15" frame="void">
    <tr height="400">
      <td width="300">
        <table cellspacing="5" cellpadding="2">
          <tr>
            <td colspan="2"><h2>Top picks for your home</h2></td>
          </tr>
          <tr>
            <td width="150">
              
              <p>Airconditions</p>
            </td>
            <td width="150">
              
              <p>Refrigerators</p>
            </td>
          </tr>
          <tr>
            <td>
            </td>
          </tr>
        </table>
      </td>
    </tr>
  </table>
```

```

        
        <p>Microwaves</p>
    </td>
    <td>
        
        <p>Washing</p>
    </td>
</tr>
<tr>
    <td colspan="2">
        <a href="#">more..</a>
    </td>
</tr>
</table>
</td>
<td></td>
<td></td>
<td></td>
</tr>
<tr height="200">
    <td colspan="4"></td>
</tr>
<tr height="400">
    <td width="300">
        <table cellspacing="5" cellpadding="2">
            <tr>
                <td colspan="2"><h2>Top picks for your home</h2></td>
            </tr>
            <tr>
                <td width="150">
                    
                    <p>Airconditions</p>
                </td>
                <td width="150">
                    
                    <p>Refrigerators</p>
                </td>
            </tr>
            <tr>
                <td>
                    
                    <p>Microwaves</p>
                </td>
                <td>
                    
                    <p>Washing</p>
                </td>
            </tr>
        </table>
    </td>
    <td></td>
    <td></td>
    <td></td>
</tr>

```

```

        </tr>
        <tr>
            <td colspan="2">
                <a href="#">more..</a>
            </td>
        </tr>
    </table>
</td>
<td></td>
<td></td>
<td></td>
</tr>
</table>
</body>
</html>

```

Forms in HTML

- Form provides an UI for interacting with data.
- Form is a Container that provides a set of elements , which are used to interact with our application.
- Form provides UI for handling CRUD operations
 - C - Create
 - R - Read
 - U - Update
 - D - Delete
- Form comprises of elements like
textbox, checkbox, radio, listbox, dropdown, textarea, button etc..

The <form> Element

- It is a container with set of elements, which it can submit to server.

Syntax:

```
<form>
```

```
</form>
```

- Every page can have multiple forms.
- You can't have nested forms.

Attributes:

1. id : It is a unique reference name.
2. name : It can be common for several elements
3. class : It defines style and allows multiple classes

<form id="frmRegister" name="Register" class="form-register">

4. method : It defines the actions performed by form.
A form can perform only 2 actions
a) GET
b) POST

GET is a request method used to fetch data from server.
POST is a request method used to submit data to server.

FAQ: What is the default method for Form?

Ans : GET

FAQ: Can we submit data to server on GET request?

Ans: Yes. But not recommended.

GET Method

- It submits the data and appends into URL as query string.
- Everyone can view your data.
- It is not safe
- It is easy to hack.
- It is stored in browser logs.
- It can be bookmarked.
- You can submit only 2048 chars
- Can't submit complex data like binary
- It can cache your data.
- It saves round trips

POST Method

- It submits data as form body
- It is not visible
- Not store in browser logs
- Can't bookmarked.
- No limit
- Complex data
- It can't cache

Form Element <form>

1. id
2. name
3. class
4. method : get and post

FAQ: Where data is submitted on GET and POST requests?

Ans : GET - Query String [2048] - Cache Data - Saves Round Trip
POST - Form Body

FAQ: What is round-trip?

Ans: It is the process where client communicates with server every time on request.

5. action : It specifies where to submit the form data. Usually it will be any server side page like: jsp, php, asp, aspx etc..

Syntax:

```
<form method="post" action="page.jsp">
```

6. novalidate: It is used to by-pass the validations of HTML.

HTML 5 provides several built-in validations, which are not according to client requirements. Hence we have to by-pass.

```
<form novalidate>
```

```
</form>
```

Text Box Element

- It is an input element used in form to view, input and edit string.
- String is a literal with group chars:
 - a) alphabet
 - b) number
 - c) special character
- Textbox is defined by using <input> element with "type" as "text".

Syntax:

```
<input type="text">
```

Attributes:

1. id
2. name
3. class

Note: Form can't submit the value of any element if it is not defined with name.
"Name" attribute is mandatory for every element.

4. value : It specifies the default value to display in textbox.

```
<input type="text" name="UserName" value="John">
```

5. readonly : It will not allow to change the value in textbox but it can submit.

```
<input type="text" value="John" readonly>
```

6. disabled : It will not allow to edit and also submit.

```
<input type="text" value="John" disabled>
```

7. size : It sets width for textbox. Default size is 20.

```
<input type="text" size="4" name="OTP">
```

8. minlength : It ensures that the value you entered is according to the specified lower bound.

9. maxlength: It restricts the maximum number of characters.

```
<input type="text" minlength="4" maxlength="4" size="4" name="OTP">
```

10. placeholder: It specifies the watermark text to display in textbox.

```
<input type="text" placeholder="UserName">
```

11. autofocus : It sets focus to element automatically when page is loaded.

```
<input type="text" autofocus>
```

12. required : It will not allow to submit empty.

```
<input type="text" required>
```

13. Pattern

pattern

- It is used to verify the format of input value.
- Pattern uses a Regular Expression to verify the input value.
- Regular Expression is built by using
 - a) Meta Characters
 - b) Quantifiers

Meta Characters

Description

? Zero or one occurrence of character

Ex:
pattern="colou?r" => color, colour

*** Zero or more occurrence of characters**

Ex:
pattern="colou*r" => color, colour, colour

+ One or more occurrence of character

Ex:
pattern="colou+r" => colour, colour

. Any single character [A-Za-z0-9!@#\$%&]

Ex:
pattern=".at" => cat, bat, \$at, 1at
pattern="?.at" => cat, bat, at

\ It is known as escape sequence character

Ex:
pattern="gmail.com" => gmail.com
pattern="gmail.com" => gmail.com , gmail1com

\d Any single digit number [0-9]

Ex:
pattern="\d" => 1, 3, 5, 2

pattern="\d\d?" => 22, 4

\D Any single non-numeric character

Ex:

pattern="\D" => [a-zA-Z!@#\$\$%^&]

\w Any word character [A-Z,a-z,0-9,_]

Ex:

pattern="\w" => [A-Z,a-z,0-9,_]

\W Any non-word character, only special chars not _.

| OR

Ex:

pattern="red|green|blue"

pattern="r|g|b\d" => r or g or b1, b4

\s Single blank space

Ex:

pattern="\d\s\d" => 4 5

|i Ignore case

EX:

pattern="Red" => Red

pattern="Red|i" => Red, red, rEd, RED

[A-Z] only uppercase letter

[a-z] only lower case letter

[0-9] only number

[!@#\$\$%^&] only specified special

Pattern - Regular Expression

Meta Characters

?

+

*

|

.

\

\d

\D

\w

\W

\s

\i

[A-Z]

[a-z]

[A-Za-z] (or) [a-Z]

[0-9]

[A-Za-z0-9]

[a,d,s] Exactly the specified chars

[^a,d,s] Excluding specified chars

[a-mA-M4-9] Only chars in specified range

[^a-mA-M4-9] Excluding the chars in specified range.

[!@#\$%&] Only specified special chars

^ Starts with

\$ Ends with

() union

Quantifiers { }

{4} Exactly 4 chars allowed

{4,10} Min 4 and Max 10

{4, } Min 4 and Max any

- Write a regular expression to validate user name, which can be 4 to 15 chars alpha numeric with underscore.

```
pattern="\w{4,15}"
```

- User Name must be 4 to 15 chars uppercase letters only.

```
pattern="[A-Z]{4,15}"
```

- Write expression for validating Bank IFSC code

```
SBIN0023479  
SBIN0029475  
SBIN0065635
```

```
pattern="SBIN00\d{5}"
```

- Write expression for mobile number starting with +91 and 10 digits.

```
pattern="\+91[0-9]{10}"
```

- Write expression for UK phone number

```
pattern="\+?(44|)\([0-9]{2}\)\s\d{4}\s\d{4}"
```

- Pre-defined Expressions

```
(?=.*[A-Z])
```

 at least one upper case letter.

```
(?=.*[0-9])
```

```
(?=.*[!@#$%&])
```

- User Name 4 to 15 chars alpha numeric with atleast one uppercase letter.

```
pattern="(?=.*[A-Z])\w{4,15}"
```

- Name 4 to 15 chars with at least one number, uppercase letter and special character.

Ex:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Document</title>  
  <style>
```

```

    dt {
        font-weight: bold;
    }
    .UserName:invalid+div {
        display: block;
        color:red;
    }
    .UserName:valid+div {
        display: none;
    }
    .Mobile:invalid+div {
        display: block;
        color:red;
    }
    .Mobile:valid+div {
        display: none;
    }
</style>
</head>
<body>

    <form>
        <dl>
            <dt>User Name</dt>
            <dd>
                <input type="text" class="UserName" required>
                <div>User Name Required</div>
            </dd>
            <dt>Mobile</dt>
            <dd>
                <input type="text" pattern="+91\d{10}" class="Mobile">
                <div>Invalid Mobile : +91 and 10 digits required</div>
            </dd>
        </dl>
    </form>

</body>
</html>

```

- Styles for disabled and readonly are defined by using classes

:disabled

:read-only

```
.UserName:read-only {  
    cursor:not-allowed;  
}
```

Ex:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <meta http-equiv="X-UA-Compatible" content="IE=edge">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>Document</title>  
    <style>  
        dt {  
            font-weight: bold;  
        }  
        .UserName:invalid+div {  
            display: block;  
            color:red;  
        }  
        .UserName:valid+div {  
            display: none;  
        }  
        .Mobile:invalid+div {  
            display: block;  
            color:red;  
        }  
        .Mobile:valid+div {  
            display: none;  
        }  
        .Mobile:invalid {  
            border:2px solid red;  
            box-shadow: 2px 2px 2px red;  
        }  
        .Mobile:valid {  
            border:2px solid green;  
            box-shadow: 2px 2px 2px green;  
        }  
        .UserName:read-only {  
            cursor:not-allowed;  
        }  
    </style>
```



```

</head>
<body>

  <form>
    <dl>
      <dt>User Name</dt>
      <dd>
        <input type="text" value="John" readonly class="UserName">

      </dd>
      <dt>Mobile</dt>
      <dd>
        <input type="text" pattern="\+91\d{10}" class="Mobile">
        <div>Invalid Mobile : +91 and 10 digits required</div>
      </dd>
    </dl>
  </form>

</body>
</html>

```

Form Elements

- TextBox String

Password Input

- It is also textbox type but it can mask the chars with "*".
- Attributes are same like textbox, but will not support "list".

Syntax:

```

<input type="password" name="Password" required minlength
pattern="(?!.*[A-Z]{4,15})>

```

Number Input

- It allows only number.
- All attributes are same like textbox, but the following attributes are used for number
 - a) min
 - b) max
 - c) step

Syntax:

Age: `<input type="number" min="15" max="30" value="22" step="1">`

Rate: `<input type="number" min="10.4" max="18.5" value="10.4" step="0.1">`

Range Input

- It is similar to number but have a slider instead of numeric up & down.

Syntax:

`<input type="range" min="10000" max="40000" value="10000" step="5000" name="Price">`

Color Input

- It allows user to pick a color.
- Color value is defined by using 6 chars hexa decimal.

Syntax:

`<input type="color" name="color" value="#ff0000">`

Email Input

- It is similar to textbox but have default email validation.

Syntax:

`<input type="email">`

URL Input

- It is used to validate URL.
- URL format is

`protocol://www.domain.com`

Syntax:

`<input type="url">`

Date Input

- It is used to display a date and time picker.
- So that user can select a date or time.

- Data Type includes the following

- date
- time
- datetime-locale [local]
- month
- week
- year

- You can set restriction for date or time by using

- a) min
- b) max

- Min and Max will accept date or time in standard format

date : "YY-MM-DD"

time : "hrs:min:sec.milliSeconds"

Syntax:

```
<input type="date" min="2022-12-19" max="2022-12-25">
```

File Input

- It allows the user to select a file from local computer.
- It is not a file uploading element, it is just a selector.
- You can't restrict the size or type of file, but you can set filter, which can be changed.
- Filter is defined by using "accept".

```
<input type="file" accept=".jpg, .png">
```

- You can also design to select "multiple"

```
<input type="file" multiple>
```

Radio Button

- It allows to select any one or multiple options.
- It will not allow to uncheck.
- Radios are used with "Mutex" [Mutual Exclusion] Mechanism
- Mutex can be defined by configuring same name for all radios.
- You can make any radio selected by using "checked" attribute.

Syntax:

`<input type="radio" name="gender" checked>`

- By default radio will submit "on" as value.
- You have to configure a "value".

`<input type="radio" name="gender" value="Male"> Male`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <form action="">
    <dl>
      <dt>gender</dt>
      <dd><input type="radio" name="gender" checked id="male" value="male">male</dd>
      <dd><input type="radio" name="gender" id="female" value="female">female</dd>
    </dl>
    <button>submit</button>
  </form>
</body>
</html>
```

Label

- It is used to set captions and titles for any element.
- It can bind with any element by using "for" attribute.

Syntax:

`<div>`

`<label for="UserName"> User Name </label>`

`<input type="text" name="UserName">`

CheckBox

- It is similar to radio, but allows to check and uncheck.
- You can use radio for selecting one or multiple.
- Every checkbox submits "on"
- You have to define name and value.

- Even you defined same name it allows to select multiple.

Syntax:

```
<input type="checkbox" name="course" value="UI" checked>UI
```

FAQ: How to design a checkbox list?

Ans: By using "overflow:auto"

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<style>
```

```
font-weight: bold;
```

```
dt {
```

```
}
```

```
ul {
```

```
}
```

```
list-style: none;
```

```
border:1px solid gray;
```

```
width: 120px;
```

```
padding: 10px;
```

```
height: 50px;
```

```
overflow: auto;
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<form>
```

```
<div>
```

```
<label>Select Course</label>
```

```
<div>
```

```
<ul>
```

```
<li><input type="checkbox"> UI </li>
```

```
<li><input type="checkbox"> MongoDB </li>
```

```
<li><input type="checkbox"> Node JS </li>
```

```
<li><input type="checkbox"> Oracle </li>
```

```
<li><input type="checkbox"> MySql </li>
```

```
<li><input type="checkbox"> JSP </li>
```

```
</ul>
</div>
<button>Submit</button>
</div>
</form>
</body>
</html>
```

FAQ: How to define effects using checked property?

Ans: You can use "checked" class of css.

```
yourCheckbox:checked {
}
```

Exp

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    ul{
      list-style: none;
      border: 1px solid gray;
      height: 50px;
      overflow: auto;
      width: 150px;
      padding: 5px;
    }
    .terms{
      border: 1px solid gray;
      width: 100px;
      padding: 10px;
    }
    #notes:checked+label{
      color: green;
    }
    #notes+label{
```

```

        color: rgb(128, 0, 15);

    }
    #notes~button{
        display: none;
    }
    #notes:checked~button{
        display: block;
    }

</style>
</head>
<body>
    <form >
        <div>
            <label for="">select course</label>

            <div>
                <ul>
                    <li><input type="checkbox" name="course" id="">ui</li>
                    <li><input type="checkbox" name="course" id="">mando dv</li>
                    <li><input type="checkbox" name="course" id="">node js</li>
                    <li><input type="checkbox" name="course" id="">oracle</li>
                    <li><input type="checkbox" name="course" id="">my sql</li>
                    <li><input type="checkbox" name="course" id="">jsp</li>

                </ul>
            </div>
            </div>
            <div>
                <label for="">terms of service</label>

                <div class="terms">
                    read our terms of service <br> something....
                </div>
                <div>
                    <input type="checkbox" name="" id="notes"><label for="">i
accept</label>
                    <button>submit</button>
                </div>
            </div>

        </form>
    </body>

```

```
</html>
```

+ adjacent

~ any occurrence after element

Dropdown List

[ComboBox]

- It provides a list of options from where user can select any one option.
- List is defined by using following elements.

<select>

<option>

<optgroup>

- Every <option> contains 3 attributes

- a) value : It defines the value to submit
- b) selected : It makes the option selected
- c) disabled. : It will not allow to select and submit

- Every option is RC data type. It will not allow icons or formats for options.

Ex

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <form >
    <dl>
      <dt>select gender</dt>
      <dd>
        <select name="gender">
          <option >choose gender</option>
```



```

        <option >male</option>
        <option >female</option>
    </select>
</dd>
<dt>select catagory</dt>
<dd>
    <select name="products">
        <option >choose product</option>
        <optgroup label="electronics">
            <option >mobiles</option>
            <option >laptos</option>
            <option >tvs</option>
            <option >radios</option>

        </optgroup>
        <optgroup label="footwear">
            <option >boots</option>
            <option >shoes</option>
            <option >chapals</option>
            <option >bata</option>

        </optgroup>
    </select>
</dd>
</dl>
<button>submit</button>
</form>
</body>
</html>

```

ListBox

- It is same as dropdown but allows to select multiple.
- Dropdown is transformed into listbox by using 2 attributes

- size
- multiple

Syntax:

```

<select>
<select size="2">

```

<select size="3" multiple>

Ex

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <form >
    <dl>
      <dt>select gender</dt>
      <dd>
        <select name="gender" size="2">
          <option >choose gender</option>
          <option >male</option>
          <option >female</option>
        </select>
      </dd>
      <dt>select catagory</dt>
      <dd>
        <select name="products" size="3" multiple>
          <option >choose product</option>
          <optgroup label="electronics">
            <option >mobiles</option>
            <option >laptos</option>
            <option >tvs</option>
            <option >radios</option>
          </optgroup>
          <optgroup label="footwear">
            <option >boots</option>
            <option >shoes</option>
            <option >chapals</option>
            <option >bata</option>
          </optgroup>
        </select>
      </dd>
    </dl>
    <button>submit</button>
  </form>
```

```
</body>
</html>
```

21 dec 2022

Radio, Checkbox, Dropdown, ListBox

TextArea

- It is used for multiline text.
- You can set "rows and cols" attribute to change size.
- It allows some default to display.
- It is RC data type.

Ex

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <fieldset>
    <legend>your comments</legend>
    <textarea name="" id="" cols="30" rows="2"></textarea>
  </fieldset>
</body>
</html>
```

Progress

- It is used to show the status of any task performed in page.
- Progress is shown by using attributes.

- a)min
- b)max
- c)value

ex

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <fieldset>
    <legend>downloading</legend>
    <dl>
      <dt>prepairing for download</dt>
      <dd><progress ></progress></dd>
      <dt>30% complete</dt>
      <dd>
        <progress min="1" max="100" value="30"></progress>
      </dd>
      <dt>download complete</dt>
      <dd>
        <progress min="1" max="100" value="100"></progress>
      </dd>
    </dl>
  </fieldset>
</body>
</html>
```

Meter

- It is used to show grade meter
- You can set min,max and value.

- You can set colors by using “low and high”.
- If more gap between low and high then “yellow”
- If less gap between low and high then “red”
- If low and high set to 0 then green

Ex

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="../node_modules/bootstrap-
icons/font/bootstrap-icons.css">
  <style>
    form{
      display: grid;
      grid-template-columns: 3fr 6fr;
    }
    .rate{
      font-size: 25px;
    }
    .rate span{
      font-size: 15px;
    }
    .review{
      color: gray;
      font-size: 15px;
    }
    ul{
      list-style: none;
    }
    meter{
      width: 200px;
    }
  </style>
</head>
<body>
  <form >
```

```

        <div>
            <div class="rate">
                4.6 <span class="bi bi bi-star-fill"></span>
            </div>
            <div class="review">
                750 rating <br>
                109 reviews
            </div>
        </div>
        <div>
            <ul>
                <li>5 <span class="bi bi bi-star-fill"></span> <meter min=
"1" max="5" value="5"></meter>529
                </li>
                <li>4 <span class="bi bi bi-star-fill"></span> <meter min=
"1" max="5" value="4"></meter>529
                </li>
                <li>3 <span class="bi bi bi-star-fill"></span> <meter min=
"1" max="5" value="3"></meter>529
                </li>
                <li>2 <span class="bi bi bi-star-fill"></span> <meter min=
"1" max="5" value="2"></meter>529
                </li>
                <li>1 <span class="bi bi bi-star-fill"></span> <meter min=
"1" max="5" value="1" low="4" high="4"></meter>529
                </li>
            </ul>
        </div>

    </form>
</body>
</html>

```

Buttons

- Button are used for confirming the actions
- HTML buttons are 2 types
 - a) Generic Buttons
 - b) Non Generic Buttons

- Generic button have default functionality.

a) Submit

b) Reset

- Non Generic button are without any default functionality.

a) Button

- HTML can use buttons from 4 and 5 version

-HTML 4 Buttons are defined by using

`<input type="submit" >`

`<input type="reset">`

`<input type="button">`

You can define a value to display as text

-HTML 5 Buttons are defined by using

`<button type="submit">`

`<button type="reset">`

`<button type="button">`

Ex

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="../node_modules/bootstrap-
icons/font/bootstrap-icons.css">
</head>
<body>
  <div>
    <form >
```

```

        User Name :
        <input type="text" name="username" id="">
        <input type="button" value="print">
        <input type="reset" value="cancal">
        <input type="submit" value="register">
    </form>
</div>
<div>
    <form>
        user name :
        <input type="text" name="user name" id="">
        <button type="submit"><span class="bi bi bi-person-fill"></span>
register</button>
        <button type="reset">cancal</button>
        <button type="button">print</button>
    </form>
</div>
</body>
</html>

```

Transform Form Elements

- You can change the appearance of elements and presentation St grils bootstrap.

- To setup bootstrap you have install

>npm install bootstrap --save

- Bootstrap CSS file location

node_modules/bootstrap/dist/css/bootstrap.css

- Bootstrap.css will provides various classes for presentation

Class Name	Purpose
p	padding 1to4 [p-4]

m margin 1 to 4

t top

b bottom

s left

e right

Ex: ps-4 — ' [Padding Left]

p-4 [padding all directions]

d-flex flex

justify-content-center Horizontal center

align-items-center vertical center

Class Name Control

.form-label Label

.form-check-label Label for radio or checkbox

.form-control textbox, password, number, date, email, url

.form-range range

.form-select dropdown , listbox

.form-check-input radio and checkbox

.btn

.form-switch group of radios and check box

.btn-primary|danger|warning

.btn-outline-primary|danger...

.btn-close

-btn-link

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="./node_modules/bootstrap-icons/font/bootstrap-
icons.css">
  <style>
    body {
      height: 600px;
    }
  </style>
</head>
<body class="container-fluid d-flex justify-content-center align-items-center">
  <form class="border border-3 border-primary p-3">
    <h2> <span class="bi bi-person-fill"></span> Register User</h2>
    <div class="mb-3">
      <label for="UserName" class="form-label">User Name</label>
      <div>
        <input type="text" class="form-control" name="UserName">
      </div>
    </div>
    <div class="mb-3">
      <label class="form-label" for="Password">Password</label>
      <div>
        <input type="password" class="form-control" name="Password">
      </div>
    </div>
  </form>
</body>
```

```

</div>
<div class="mb-3">
  <label class="form-label" for="City">Your City</label>
  <div>
    <select name="City" class="form-select">
      <option>Choose City</option>
      <option>Delhi</option>
      <option>Hyd</option>
    </select>
  </div>
</div>
<div class="mb-3">
  <label>Price Range</label>

  <div>
    <input type="range" class="form-range">
  </div>
</div>
<div class="mb-3">
  <label class="form-label" for="Subscribe">Subscribe</label>
  <div class="form-switch">
    <input type="checkbox" class="form-check-input"> <label class="form-
check-label">Yes</label>
  </div>
</div>
<div>
  <button class="btn btn-primary w-100">Register</button>
</div>
</form>
</body>
</html>

```

Bootstrap Input Group

```

.input-group
.input-group-text
.input-group-lg
.input-group-sm

```

```
> npm install bootstrap --save
> npm install bootstrap-icons --save
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="./node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body class="container-fluid">
  <h2>Input Group</h2>
  <div class="input-group w-25">
    <span class="input-group-text">&#8377;</span>
    <input type="text" class="form-control">
    <span class="input-group-text">.00</span>
  </div>
  <div class="input-group">
    <input type="email" class="form-control">
    <span class="input-group-text">@</span>
    <input type="text" class="form-control">
  </div>
  <h2>Amazon Search</h2>
  <div class="input-group">
    <select class="input-group-text">
      <option>All</option>
      <option>Mobiles</option>
      <option>Fashion</option>
    </select>
    <input type="text" class="form-control">
    <button class="btn btn-warning">
      <span class="bi bi-search"></span>
    </button>
  </div>
  <h2>Netflix Register</h2>
  <div class="input-group input-group-lg bg-dark p-4">
```

```

        <input type="email" placeholder="your email" class="form-control">
        <button class="btn btn-danger">
            Get Started <span class="bi bi-chevron-right"></span>
        </button>
    </div>
    <h2>User Login</h2>
    <div class="input-group w-25">
        <span class="bi bi-person-fill input-group-text"></span>
        <input type="text" class="form-control">
        <span class="bi bi-check-square-fill input-group-text"></span>
    </div>
    <div class="input-group w-25">
        <span class="bi bi-key-fill input-group-text"></span>
        <input type="password" class="form-control">
        <span class="bi bi-check-square-fill input-group-text"></span>
    </div>
</body>
</html>

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
    <style>
        body {
            display: flex;
            justify-content: center;
            align-items: center;
            height: 600px;
        }
        form {
            width: 350px;
            border: 1px solid gray;
            padding: 20px;
        }
    </style>

```

```

a {
    text-decoration: none;
}
.left-bar, .right-bar {
    border-top: 1px solid gray;
    width: 100px;
}
.new {
    display: flex;
}
.bar-title {
    margin-top: -20px;
    padding: 10px;
    color: gray;
    font-size: 15px;
}
</style>
</head>
<body class="container-fluid">
<div>
<form class="mb-3">
<h1>Sign in</h1>
<div class="mb-2">
<label class="form-label"><b>Email or mobile phone number</b></label>
<div>
<input type="text" class="form-control">
</div>
</div>
<div class="mb-2">
<button class="btn btn-warning w-100">Continue</button>
</div>
<div class="mb-2">
<p>By continuing, you agree to Amazon's Conditions of Use and Privacy
Notice.</p>
</div>
<div class="mb-2">
<details>
<summary><a href="#">Need Help?</a></summary>
<dl class="mt-2 me-2">
<dd><a href="#">Forget Password</a></dd>
<dd><a href="#">Other Issues with Sign-in</a></dd>
</dl>
</details>

```

```

    </div>
  </form>
  <div class="mt-4">
    <div class="new">
      <div class="left-bar"></div>
      <span class="bar-title">New to Amazon ?</span>
      <div class="right-bar"></div>
    </div>
  </div>
</div>
</body>
</html>

```

Multi Media

- Multimedia refers to animation, audio and video.

- HTML provides

```

<video>
<audio>
<embed>

```

Syntax: .mp4, mkv, avi, ogg

```
<video src="" controls width height poster> </video>
```

```

<body>
  <h2>Class Video</h2>
  <video src="../public/docs/class.mkv" poster="../public/images/asp.jpg"
width="400" height="400" controls></video>
</body>

```

CSS

Styles in HTML

- Styles are attributes defined for HTML elements to make the presentation more interactive and responsive.

- Styles are defined in 3 ways

1. Inline Styles
2. Embedded Styles
3. External Style Sheet [CSS]

1. Inline Styles

- Styles are defined for every element individually by using "style" attribute.

Syntax:

```
<h1 style="color:red"> Welcome </h1>
```

- You can't reuse.

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h1 style="background-color: red; color:white; text-align: center;">HTML</h1>
  <h1>CSS</h1>
  <h1>Bootstrap</h1>
</body>
</html>
```

2. Embedded Style

- Styles are defined for elements in a separate <style> block.
- You can reuse the styles.
- It is slower than inline.

Syntax:

```
<style>  
.....  
</style>
```

FAQ: Where to embed styles? Head or Body?

Ans: You can embed in both head and body.

FAQ: What is difference between styles in head and in body?

Ans: Styles in head are intended to load into browser memory and then applied to elements.

Styles in body are intended to load into browser directly.

FAQ: What is the MIME type of styles?

Ans:

- MIME is "Multipurpose Internet Mail Extensions"
- It defines the type of code, so that browser can understand.
- CSS MIME is "text/css"

Syntax:

```
<style type="text/css">  
  
</style>
```

FAQ: What is the media type of styles?

Ans: Media type defines style for various devices.

The media types are

- a) all
- b) screen
- c) print
- d) speech

Syntax:

```
<style type="text/css" media="print">  
  
</style>
```

Ex:

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style type="text/css" media="screen">
    .qr {
      display: none;
    }
    .details {
      display: grid;
      grid-template-columns: 6fr 6fr;
    }
    dt {
      background-color: blueviolet;
      color:white;
      margin-bottom: 20px;
    }
    dd {
      background-color: black;
      color:white;
      margin-bottom: 20px;
    }
  </style>
  <style type="text/css" media="print">
    .poster, button {
      display: none;
    }
    .qr {
      display: block;
    }
    dt,dd {
      background-color: none;
      color:black;
    }
  </style>
</head>
<body>
  <dl style="width: 300px;">
    <dt>Booking Summary</dt>
    
    
```

```

<dl class="details">
  <dt>Movie Name</dt>
  <dd>Avatar</dd>
  <dt>Cinema</dt>
  <dd>PVR Hyd</dd>
  <dt>Show Time</dt>
  <dd>10:30AM</dd>
</dl>
</dl>
<button onclick="window.print()">Print Ticket</button>
</body>
</html>

```

1. Inline Styles
2. Embedded Styles

3. External Style Sheet

Cascading Style Sheet

- Styles are defined in separate style sheet that have extension ".css"
- You can link to any page and access the styles.
- You can reuse styles across pages.

1. Create a new folder by name "styles" and add into "src"

src\styles

2. Add a new style sheet

```

headings.css
h1 {
background-color: red;
color:white;
text-align: center;
}

```

3. Link style sheet to any page

```

<link rel="stylesheet" href="../src/styles/headings.css">

```

Note: Using external style sheet will increase the number of requests for page and also page load time.

Minification

- It is a coding technique followed by developers to compress the code and reduce the file size.

Ex:

1. Copy your actual CSS code

```
h1 {  
  background-color: red;  
  color: white;  
  text-align: center;  
}
```

2. Visit the website

<https://www.toptal.com/developers/cssminifier>

3. Paste the actual CSS code and click "Minify" button

4. This will generate Minified code

```
h1{background-color:red;color:#fff;text-align:center}
```

5. Create a new file "headings.min.css" and copy minified code into it.

6. Link the minified file to page.

```
<link rel="stylesheet" href="../src/headings.min.css">
```

Note: Always use the un-compressed file in "Development" and compressed file [minified] in "Production".

CDN [Content Delivery Network]

- It is the process of maintaining repository in a "sandbox" server and access from any project.
- You are accessing file directly from server instead of downloading into your project.

www.getbootstrap.com

EX:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>

  <link rel="stylesheet" href="../src/styles/headings.min.css">

  <link href="
https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/bootstrap.min.css
  &quot;ot; rel="stylesheet" integrity="sha384-
  rbsA2VBKQhggwzxH7pPCaAqO46MgnOM80zW1RWuH61DGLwZJEdK2Kad
  q2F9CUG65" crossorigin="anonymous">

  <link rel="stylesheet" href="
https://cdn.jsdelivr.net/npm/bootstrap-
  icons@1.10.2/font/bootstrap-icons.css
  &quot;&gt;

</head>
<body class="container">
  <h1 class="bg-dark"> <span class="bi bi-globe"></span> HTML</h1>
  <h1>CSS</h1>
  <h1>Bootstrap</h1>
</body>
</html>
```

Rules

FAQ: If styles are defined for any element in page by using all 3 techniques, then which one will apply?

Ans:

- a) If same attribute are defined then styles are applied according to

priority

1st	Inline
2nd	Embedded
3rd	External

b) If different attributes are defined then all will apply.

FAQ: How CSS selects HTML elements?

Ans : If you define inline then no need of selecting.

```
<h1 style="color:red">
```

If you define embedded or external file then you need a selector.

```
<style>
  selector
  {
    color:red;
  }
</style>
```

CSS Selectors are classified into various categories

1. Primary Selectors
2. Rational Selectors
3. Pseudo Classes
4. Attribute Selectors
5. Structural Selectors etc...

1. Primary Selectors

- a) Type Selector
- b) ID Selector
- c) Class Selector

Type Selector

- It defines directly the element name.
h2, h1, table, p, dd, li
- It will apply effects to every occurrence of element.

- You can't ignore for any specific occurrence.

Syntax:

```
h2      p      table
{        {        {
}        }        }
```

ID Selector

- It is defined by using "#" reference name.
- It can be accessed by using "id" attribute.
- You can select and apply effects to specific element.

Syntax:

```
#head-style
{
  color:red;
}
```

```
<h1 id="head-style">
<p id="head-style">
<div id="head-style">
```

- Every element can refer to only one ID.
- You can apply only one set of effects.

Class Selector

- A class selector is defined by using "." and reference name.
- It is accessed by using "class" attribute.
- You implement multiple classes for one element.

Syntax:

```
.className
{

}

<h1 class="className"> </h1>
<div class="className"> </div>
```

Ex:

```
<!DOCTYPE html>
```

```

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    .text-style {
      color:red;
      text-align: center;
      font-family: Arial;
    }
    .border-style {
      border:2px solid red;
    }
    .bg-style {
      background-color: yellow;
    }
  </style>
</head>
<body>
  <h1 class="border-style bg-style text-style">Web Technologies</h1>
  <h1 class="text-style">HTML</h1>
  <h1 class="border-style text-style">CSS</h1>
  <h1 class="bg-style">JavaScript</h1>
  <p>It is a language.</p>
</body>
</html>

```

Rules FAQ:

Q. If styles are defined by using all 3 selectors to any element, which set of effects will apply?

A.

If same attributes are defined then it will apply according to priority.

- 1st Id
- 2nd class
- 3rd Type

If different attributes are defined then all will apply.

Ex:


```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    h1 {
      color:red;
      text-align: center;
    }
    #head-style {
      color:green;
      background-color: yellow;
    }
    .heading {
      color:blue;
      border:2px solid red;
    }
  </style>
</head>
<body>
  <h1 class="heading" id="head-style">HTML</h1>
</body>
</html>

```

2. Dynamic Pseudo Classes

- Dynamic allows to change according state and situation.
- It is a false representation.
- Class is a program template with set of values and functions, which you can implement and customize according to requirement.
- A class is implemented by any element by using inheritance operator " : "
- Inheritance is used to re-uses and extend the logic.
- CSS dynamic pseudo classes are

:hover : It defines effects when mouse is over the element.

:active : Effects when you hold down mouse button on element

:link : It defines effects for hyperlink in normal state

:visited : It defines effects for hyperlink in visited state

Ex:

```

<!DOCTYPE html>

```

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    h1:hover {
      color:red;
    }
    img {
      width: 100px;
      height: 100px;
      transition: 5s;
    }
    img:active {
      width: 300px;
      height: 300px;
      transition: 3s;
    }
    a:link {
      text-decoration: none;
    }
    a:hover {
      text-decoration: underline;
      font-size: 30px;
      transition:2s;
    }
    a:active {
      color:red;
    }
    a:visited {
      color:gray;
    }
  </style>
</head>
<body>
  <a href="home.html">Home</a> |
  <a href="http://www.ebay.in">Shop</a>&gt;
  <h1>Welcome to HTML</h1>
  
</body>
</html>
```

CSS Selectors

1. Primary Selectors

type
id
class

2. Dynamic Pseudo Classes

:hover
:active
:link
:visited

3. Element State Pseudo Classes

:read-only	: It will not allow to change but can submit
:disabled	: It will not allow to change and submit
:checked	: Selected [Checkbox, Radio]
:focus	: Element gets focus

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    #UserName+div {
      display: none;
    }
    #UserName:focus+div {
      display: block;
    }
    #UserName:focus {
      background-color: yellow;
    }
    #UserId:read-only {
      cursor: not-allowed;
      color: gray;
    }
  </style>
</head>
<body>
  <div id="UserName">
    <input type="text" value="User Name" />
    <div id="UserName+div">
      <div id="UserName+div">
        <input type="text" value="User Name" />
      </div>
    </div>
  </div>
  <div id="UserId">
    <input type="text" value="User ID" />
  </div>
</body>
</html>
```

```

    }
    button:disabled {
        cursor: not-allowed;
    }
    #terms+span {
        color:red;
    }
    #terms:checked+span {
        color:green;
    }
}
</style>
</head>
<body>
<dl>
    <dt>User Id</dt>
    <dd>
        <input type="text" id="UserId" readonly value="John_nit">
    </dd>
    <dt>User Name</dt>
    <dd>
        <input type="text" id="UserName" placeholder="Block Letters">
        <div>Name in Block Letters</div>
    </dd>
    <dt>Terms of Service</dt>
    <dd>
        <input type="checkbox" id="terms"> <span>I Accept</span>
    </dd>
</dl>
<button disabled>Submit</button>
</body>
</html>

```

4. Element Validation State Pseudo Classes

:required	: If input is defined with required
:optional	: If input is not defined with required
:valid	: All validation attributes evaluate true
:invalid	: Any one validation evaluate to false
:in-range	: All values are within specified range
:out-of-range	: If value is out of range

Ex:

```
<!DOCTYPE html>
```

```

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    input:optional+span {
      display: none;
    }
    input:required+span {
      display: inline;
      color:red;
    }
    input:optional {
      background-color: yellow;
    }
    input:required {
      background-color: red;
    }
  </style>
</head>
<body>
  <dl>
    <dt>User Name</dt>
    <dd><input type="text" required><span>*</span></dd>
    <dt>Mobile</dt>
    <dd><input type="text"><span>*</span></dd>
    <dt>Age</dt>
    <dd><input type="number"><span>*</span></dd>
  </dl>
  <button>Submit</button>
</body>
</html>

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>

```

```
<style>
  input:optional+span {
    display: none;
  }
  input:required+span {
    display: inline;
    color:red;
  }
  input:optional {
    background-color: yellow;
  }
  input:required {
    background-color: red;
  }
  #UserName:valid~div {
    display: none;
  }
  #UserName:valid {
    background-color: lightgreen;
  }
  #UserName:valid+span {
    display: none;
  }
  #UserName:invalid~div {
    color:red;
    display: block;
  }
  #Mobile:invalid~div {
    color:red;
    display: block;
  }
  #Mobile:valid~div {
    display: none;
  }
  #Age:in-range~div {
    display: none;
  }
  #Age:out-of-range~div {
    display: block;
    color:red;
  }
</style>
</head>
```

```

<body>
  <dl>
    <dt>User Name</dt>
    <dd>
      <input type="text" id="UserName" required><span>*</span>
      <div>User Name Required</div>
    </dd>
    <dt>Mobile</dt>
    <dd>
      <input type="text" id="Mobile"
pattern="\+91\d{10}"><span>*</span>
      <div>Invalid Mobile</div>
    </dd>
    <dt>Age</dt>
    <dd><input type="number" id="Age" min="15"
max="30"><span>*</span>
      <div>Age 15 to 30 Only</div>
    </dd>
  </dl>
  <button>Submit</button>
</body>
</html>

```

5. Rational Selectors

parent child	all child elements
parent > child	only direct child elements
element+element	adjacent sibling
element~element	general sibling

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    .container > p {
      color:red;
    }
  </style>

```

```

</head>
<body>
  <div class="container">
    <p>Para-1</p>
    <p>Para-2</p>
    <p>Para-3</p>
    <div>
      <p>Para-4</p>
    </div>
    <p>Para-5</p>
  </div>
</body>
</html>

```

6. Structural Pseudo Classes

:first-child	: first occurrence
:last-child	: last occurrence
:nth-child()	: specific occurrence
:nth-of-type()	: every occurrence after given value [from top]
:nth-last-of-type()	: every occurrence from bottom

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    p:nth-last-of-type(3n+1){
      color:red;
    }
    thead tr {
      background-color: darkblue;
      color:white;
    }
    tbody tr:nth-child(even) {
      background-color: lightblue;
    }
    tbody tr:nth-child(odd) {

```



```
        background-color: lemonchiffon;
    }
</style>
</head>
<body>
    <table width="500" border="1">
        <thead>
            <tr>
                <th>Name</th>
                <th>Price</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td>&nbsp;</td>
                <td>&nbsp;</td>
            </tr>
            <tr>
                <td>&nbsp;</td>
                <td>&nbsp;</td>
            </tr>
            <tr>
                <td>&nbsp;</td>
                <td>&nbsp;</td>
            </tr>
            <tr>
                <td>&nbsp;</td>
                <td>&nbsp;</td>
            </tr>
            <tr>
                <td>&nbsp;</td>
                <td>&nbsp;</td>
            </tr>
            <tr>
                <td>&nbsp;</td>
                <td>&nbsp;</td>
            </tr>
            <tr>
                <td>&nbsp;</td>
                <td>&nbsp;</td>
            </tr>
            <tr>
                <td>&nbsp;</td>
                <td>&nbsp;</td>
            </tr>
            <tr>
                <td>&nbsp;</td>
                <td>&nbsp;</td>
            </tr>
        </tbody>
    </table>
</body>
</html>
```

```

        <td>&nbsp;</td>
    </tr>
</tbody>
</table>
<div class="container">
    <p>Para-1</p>
    <p>Para-2</p>
    <p>Para-3</p>
    <p>Para-4</p>
    <p>Para-5</p>
    <p>Para-6</p>
    <p>Para-7</p>
    <p>Para-8</p>
</div>
</body>
</html>

```

7. Attribute Selectors

element [attribute]
 element [attribute=value]

Conditions

element [attribute=value]	Equal
element [attribute^=value]	Starts with
element [attribute\$=value]	Ends with
element [attribute =value]	Starts and separated with "-"
element [attribute~=value]	Contain and separated with space

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    p[class="Effect"] {
      color:red;
    }
  </style>

```

```

</head>
<body>
  <p class="Effect">Para-1</p>
  <p class="ParaEffect">Para-2</p>
  <p class="EffectPara">Para-3</p>
  <p class="Para-Effect">Para-4</p>
  <p class="Para Effect">Para-5</p>
  <p class="Effect-Para">Para-6</p>

</body>
</html>

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    p[id] {
      color:red;
    }
  </style>
</head>
<body>
  <p>Para-1</p>
  <p id="p2">Para-2</p>
  <p id="p3">Para-3</p>
  <p>Para-4</p>
</body>
</html>

```

Behavioural and Occurance Selectors

: inheritance operator

:: inheritance and implement

::first-letter	It returns the first letter of any sentence or para
::first-line	It returns the first line
::before	It defines effects for occurance before element
::after	It defines effects for occurance after element
::placeholder	It defines effects for place holder
::selection	It sets effects for selection.

Ex :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    p::first-letter {
      font-size: 40px;
      font-family: Arial;
      color:red;
      float:left;
    }
    p::first-line {
      color:red;
    }
    p::selection {
      background-color: yellow;
    }
    ul {
      display: flex;
      list-style: none;
    }
    li::after {
      content: " >> ";
    }
    li:last-child::after {
      content: "";
    }
    li:last-child {
      color:gray;
    }
  </style>
</head>
<body>
```

```

        input::placeholder {
            color:red;
        }
    </style>
</head>
<body>
    <input type="text" placeholder="User Name Block Letters">
    <ul>
        <li>Home</li>
        <li>Electronics</li>
        <li>Mobiles</li>
        <li>Smasung</li>
    </ul>
    <p align="justify">We understand that you may have questions about the
Microsoft Services Agreement. We have an FAQ page (
https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx)
that provides more information, including what Microsoft products and
services it covers. Whenever we make changes, we provide a summary of
the most notable changes to the current Microsoft Services Agreement. We
will also continue to provide a summary of the changes to the previous
version, and a link to the previous version, of the Microsoft Services
Agreement on the Summary of Changes (https://www.microsoft.com/en-
us/servicesagreement/upcoming-updates.aspx) page.</p>
</body>
</html>

```

Universal and Root Selector

- It selects all element is page from body.

```

*{
}

```

- Root selector will select all elements from "html" root.

```

:root {

}

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">

```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<style>
  p::first-letter {
    font-size: 40px;
    font-family: Arial;
    color:red;
    float:left;
  }
  p::first-line {
    color:red;
  }
  p::selection {
    background-color: yellow;
  }
  ul {
    display: flex;
    list-style: none;
  }
  li::after {
    content: " >> ";
  }
  li:last-child::after {
    content: "";
  }
  li:last-child {
    color:gray;
  }
  input::placeholder {
    color:red;
  }
  *{
    font-family: Arial;
  }
  :root {
    font-family: Arial;
  }
</style>
</head>
<body>
  <input type="text" placeholder="User Name Block Letters">
  <ul>
    <li>Home</li>
```

```

        <li>Electronics</li>
        <li>Mobiles</li>
        <li>Smasung</li>
    </ul>
    <p align="justify">We understand that you may have questions about the
    Microsoft Services Agreement. We have an FAQ page (
    https://www.microsoft.com/en-us/servicesagreement/upcoming-faq.aspx)
    that provides more information, including what Microsoft products and
    services it covers. Whenever we make changes, we provide a summary of
    the most notable changes to the current Microsoft Services Agreement. We
    will also continue to provide a summary of the changes to the previous
    version, and a link to the previous version, of the Microsoft Services
    Agreement on the Summary of Changes (https://www.microsoft.com/en-
    us/servicesagreement/upcoming-updates.aspx) page.</p>
</body>
</html>

```

Summary:

- Primary Selectors
- Rational Selectors
- Attribute Selectors
- Structural Pseudo Classes
- Dynamic Pseudo Classes
- Element State Pseudo Classes
- Validation State Pseudo Classes
- Behavioural and Occurance
- Universal and Root

CSS Colors

- Color Name
- Shade Name
- Hexa Decimal
- RGB
- RGBA
- HSL
- HSLA
- Gradient

Color Name:

body {

```
background-color: red;
}
```

Shade Name:

```
body {
background-color: lightgreen;
}
```

Hexa Decimal [# 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f]

```
body {
background-color: #f00;
color:#ff00ff;
}
```

```
#rgb
#rrggbb
```

RGB

- rgb(red, green, blue)
- color value will start with 0 to 255.

Syntax:

```
body {
background-color: rgb(255,0,0);
}
```

RGBA

- rgba(red, green, blue, alpha)
- alpha is opacity 0 to 1.

Syntax:

```
body {
background-color: rgba(0,0,0, 0.4);
}
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```



```

<title>Document</title>
<style>
  body {
    background-size: cover;
  }
  .shade {
    height: 600px;
    width: 105%;
    background-color: rgba(0,0,0,0.2);
    margin-left: -20px;
    margin-top: -20px;
  }
</style>
</head>
<body background="../images/netflixback.jpg">
  <div class="shade">

</div>
</body>
</html>

```

HSL

Hue angle 0 to 360 deg [0=red, 180=green, 360=blue]
Saturation - Gery 0 to 100%
Light - Brightness 0 to 100%

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    body {
      height: 600px;
      background-color: hsl(250, 15%, 50%);
    }

  </style>
</head>
<body>

```

```
</body>
</html>
```

HSLA

- hue 0 to 360
- sat 0 to 100%
- light 0 to 100%
- alpha 0 to 1

Gradient

- Linear-gradient()
- Radial-gradient()
- You can define only by using background-image

Syntax:

```
body {
    height: 600px;
    background-image: linear-gradient(to top right, red, yellow, green) ;
}
```

```
body {
    height: 600px;
    background-image: radial-gradient(rgb(239, 142, 142), yellow,
rgb(173, 228, 173), blue) ;
}
```

CSS Rules

- Priority for inline, embedded and external file.
- Priority for id, class and type selectors.

CSS Units

- Absolute Units

cm
in
px
%
pt
pi

- Relative Units

em
rem

CSS Selectors

CSS Colors

name, shade, hexa, rgb, rgba, hsl, hsla, gradient

CSS Units

1. Absolute Units

in
cm
mm
pc
pt
px

2. Relative Units

em	parent element
rem	root element

Relative units change the dimensions according to their parent.

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    .container {
      font-size: 40px;
    }
    p {
      font-size: 0.5em;
    }
    .title {
      font-size: 1rem;
    }
  </style>
</head>
<body>
  <div class="container">
    <p>Hello World</p>
    <h1 class="title">Hello World</h1>
  </div>
</body>
</html>
```

```

    }
  </style>
</head>
<body>
  Normal Body Text
  <div class="container">
    HTML
    <p>Markup language</p>
    <div class="title">It is used for presentation.</div>
  </div>
</body>
</html>

```

CSS Inheritance

1. **inherit** : Inherits the parent effects
2. **initial** : Resets the effects to initial state. [set to default]
3. **unset** : Remove the pre-defined effects.

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    .container {
      border:2px double red;
      background-color: yellow;
      color:red;
      padding: 20px;
    }
    p {
      border:inherit;
      padding: inherit;
      color:initial;
    }
    h1 {
      all: unset;
    }
  </style>

```

```

</head>
<body>
  <h1>CSS Inheritance</h1>
  <div class="container">
    Container Text
    <p>Markup language</p>
  </div>
</body>
</html>

```

CSS Rules

- If styles are defined inline, embedded and external
 - 1st inline
 - 2nd embedded
 - 3rd external
- If styles are defined by using id, class and type
 - 1st id
 - 2nd class
 - 3rd type

CSS Box Model

1. Margins
2. Padding
3. Border
4. Border Image
5. Border Radius
6. Width
7. Height
8. Box Shadow

CSS Margins

margin : short hand, set margin all directions.

```

{
  margin: 20px;    // all directions 20 px
}

```

```

{

```

```
margin: 0px 0px 0px 0px;    [top, right, bottom, left]
}
```

margin-left
margin-right
margin-top
margin-bottom

CSS Padding

padding : It is short hand for all directions.
padding-left
padding-right
padding-top
padding-bottom

CSS Border

border : short hand for border to set width, color and style.
border-style : dotted, dashed, solid, groove etc..
border-width : size
border-color : color
border-left : left short hand
border-left-style
border-left-color
border-left-width
.....
all directions

CSS Border Radius

border-radius : short hand to set radius all directions.
border-top-left-radius
border-top-right-radius
border-bottom-left-radius
border-bottom-right-radius

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<style>
    .container {
        padding: 20px;
        border-left: 10px double red;
        border-top: 5px double blue;
        border-right: 10px solid red;
        border-bottom: 5px solid blue;
        border-radius: 30px;
    }

</style>
</head>
<body>
    <div class="container">
        Container Text
        <p>Markup language</p>
    </div>
</body>
</html>

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <style>
        img {
            border-top-right-radius: 150px;
            border-bottom-left-radius: 150px;
        }
    </style>
</head>
<body>
    
</body>
</html>

```

CSS Border Image

border-image:url() style 0% [offset]

Note: You have to set border and image.

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    div {
      text-align: justify;
      border: 20px solid transparent;
      border-image: url("../images/border.gif") stretch 70;
      padding: 20px;
    }
  </style>
</head>
<body>
  <div>
    Depending on how you obtained the Windows software, this is a license
    agreement between (i) you and the device manufacturer or software
    installer that distributes the software with your device; or (ii) you and
    Microsoft Corporation (or, based on where you live or, if a business, where
    your principal place of business is located, one of its affiliates) if you
    acquired the software from a retailer. Microsoft is the device manufacturer
    for devices produced by Microsoft or one of its affiliates, and Microsoft is
    the retailer if you acquired the software directly from Microsoft. Note that if
    you are a volume license customer, use of this software is subject to your
    volume license agreement rather than this agreement.
  </div>
</body>
</html>
```

CSS Box Shadow

box-shadow : horizontal, vertical blur color;

`box-shadow: 20px 10px 13px gray;`

CSS Units

CSS Colors

CSS Inheritance

CSS Box Model

CSS Positions

1. Static
2. Absolute
3. Fixed
4. Sticky
5. Relative

Static Position

- It keeps the element according to normal flow of document.
- It will not allow to change position of element.
- It is not effected by top, left, right and bottom positions.
- It is default position.

Absolute Position

- It removes the element from normal flow of document.
- It can change the position using left, right, top or bottom.
- It keeps the element attached to content in page.
- It is not attached to page.

Fixed Position

- It removes the element from normal flow of document.
- It can change the position left, right, top or bottom.
- It is attached to page and not to content.
- It locks scrolling.

Sticky Position

- It keeps the element according to normal flow.
- It can stick the element after reaching specified point.
- So that element will lock scrolling after reaching the specified.

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet"
href="../../node_modules/bootstrap/dist/css/bootstrap.css">
  <style>
    header {
      margin-bottom: 50px;
    }
    nav {
      display: flex;
      justify-content: space-between;
      background-color: black;
      padding: 5px;
      position: sticky;
      top: 0px;
    }
  </style>
</head>
<body class="container-fluid">
  <header>
    
  </header>
  <nav>
    <div>
      <select class="form-select">
        <option>Select Movie</option>
      </select>
    </div>
    <div>
      <select class="form-select">
        <option>Select Cinema</option>
      </select>
    </div>
    <div>
      <select class="form-select">
        <option>Select Date</option>
      </select>
    </div>
  </nav>

```

```

<div>
  <select class="form-select">
    <option>Show Time</option>
  </select>
</div>
<div>
  <select class="form-select">
    <option>Select Seats</option>
  </select>
</div>
<div>
  <button class="btn btn-danger">Book</button>
</div>
</nav>
<section>

```

Depending on how you obtained the Windows software, this is a license agreement between (i) you and the device manufacturer or software installer that distributes the software with your device; or (ii) you and Microsoft Corporation (or, based on where you live or, if a business, where your principal place of business is located, one of its affiliates) if you acquired the software from a retailer. Microsoft is the device manufacturer for devices produced by Microsoft or one of its affiliates, and Microsoft is the retailer if you acquired the software directly from Microsoft. Note that if you are a volume license customer, use of this software is subject to your volume license agreement rather than this agreement.

```

</section>
</body>
</html>

```

Relative Position

- It removes the child element from normal flow and keeps with relative to the parent.
- It is the position defined for parent element.

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">

```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<link rel="stylesheet" href="../../node_modules/bootstrap-
icons/font/bootstrap-icons.css">
<link rel="stylesheet" href="../../node_modules
/bootstrap/dist/css/bootstrap.css">
<style>
    .count {
        background-color: black;
        color:white;
        padding: 3px;
        border-radius: 30px;
        font-weight: bold;
        position: absolute;
        right: -10px;
        top: -10px;
        width: 30px;
        height: 30px;
    }
    button {
        position: relative;
    }
</style>
</head>
<body class="container-fluid">
    <h2>Relative</h2>
    <button class="btn btn-primary btn-lg">
        <span class="bi bi-bell"></span>
        Notifications
        <span class="count">4</span>
    </button>
    <h2>Cart</h2>
    <div>
        <span class="bi bi-cart"></span>
        <span class="cart-count">2</span>
    </div>
</body>
</html>

```

CSS Z-Index

- It changes the position of element back, backward, front or forward.
- Z Index sets value 0 for back and front depends on the stack of elements.

Syntax:

```
.div-1 {
  z-index:0;
}
.div-2 {
  z-index:1;
}
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    div {
      width: 200px;
      height: 250px;
      box-shadow: 2px 3px 3px gray;
      padding: 10px;
      position: absolute;
    }
    .card-1 {
      background-color: red;
      color:white;
      z-index: 2;
    }
    .card-2 {
      background-color: green;
      color:white;
      margin-left: 50px;
      margin-top: 50px;
      z-index: 1;
    }
    .card-3 {
      background-color: yellow;
      color:red;
      margin-left: 15px;
      margin-top: 100px;
      z-index: 0;
    }
  </style>

```

```
</style>
</head>
<body>
  <div class="card-1">
    <h2>Card-1</h2>
  </div>
  <div class="card-2">
    <h2>Card-2</h2>
  </div>
  <div class="card-3">
    <h2>Card-3</h2>
  </div>
</body>
</html>
```

CSS - 3

CSS Selectors

CSS Colors

CSS Units

CSS Inheritance

CSS Box Model

CSS Positions

CSS Z-Index

CSS Display

display

- inline

- block

- none

- flex

- grid

display:none

- It is used to hide element and wrap the relative content.

FAQ: What is difference between display:none and visibility:hidden?

Ans: display:none will hide the element and wraps the relative.

visibility:hidden will hide but will not wrap the relative content.

Syntax:

```
img {
```

```
display:none;
}
img {
  visibility:hidden;
}
```

display:inline

- It is used to display elements side by side.

FAQ: What is difference between flex and inline?

Ans: inline is used for elements.

flex is used for container that have elements.

Syntax:

```
li {
  display:inline;
}
ul {
  display:flex;
}
```

display:block

- It arranges elements in a column.
- Inline arranges in a row.
- It is used for elements individually not for container.

Syntax:

```
nav span {
  display:block;
}
```

display:grid

- Grid is used for creating responsive layout.
- It can arrange content into rows and columns.
- It can change the position of row and column.
- Responsive layout will change according to device and its orientation.

Attributes:

display : grid
grid-row
grid-column
grid-gap

grid-rule

FAQ: What are the basic options required to create responsive layout?

Ans :

1. Meta Viewport
2. Fluid Images [img size in %]
3. Fluid Containers [display:grid]
4. Media Queries [@media]

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    header, nav, main, aside, footer {
      border:1px solid black;
      margin: 5px;
      padding: 10px;
      box-shadow: 2px 2px 2px gray;
    }
    main {
      height: 350px;
    }
    @media screen and (orientation:landscape)
    {
      body {
        display: grid;
        grid-template-columns: 2fr 8fr 2fr;
      }
      header {
        grid-row: 1;
        grid-column: 1/4;
      }
      nav {
        grid-row: 2;
        grid-column: 3;
      }
      main {
        grid-row: 2;
```



```

        grid-column: 2;
    }
    aside {
        grid-row: 2;
        grid-column: 1;
    }
    footer {
        grid-row: 3;
        grid-column: 1/4;
    }
}
@media screen and (orientation:portrait) {
    body {
        display: grid;
        grid-template-columns: 12fr;
    }
    header {
        grid-row: 1;
    }
    nav {
        grid-row: 2;
    }
    main {
        grid-row: 3;
    }
    aside {
        grid-row: 4;
    }
    footer {
        grid-row: 5;
    }
}
</style>
</head>
<body>
    <header>
        Header
    </header>
    <nav>
        Navbar
    </nav>
    <main>
        Main

```

```

</main>
<aside>
  Ads..
</aside>
<footer>
  Footer
</footer>
</body>
</html>

```

FAQ: What is difference between display:grid and columns in CSS?

Ans : Columns refer to contineous columns.

Grid refers to individual columns.

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    .options {
      columns:2;
    }
  </style>
</head>
<body>
  <ol>
    <li>HTML Elements
      <ol type="a" class="options">
        <li>Normal</li>
        <li>Void</li>
        <li>RC Data</li>
        <li>Raw Text</li>
      </ol>
    </li>
  </ol>
</body>
</html>

```

Column Attributes

- columns
- column-gap
- column-rule
- column-width
- column-span

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    header {
      font-size: 50px;
      text-align: center;
      background-color: black;
      color:white;
      text-shadow: 2px 2px 2px gray;
      padding: 5px;
    }
    section {
      columns:5;
      column-rule: 2px dotted black;
      column-gap: 50px;
      text-align: justify;
      margin-top: 20px;
    }
    h2 {
      column-span: all;
      text-align: center;
      background-color: black;
      color:white;
    }
    .banner {
      column-span: all;
    }
  </style>
</head>
<body>
  <header>
    THE HINDU
```

```
</header>
<section>
  <p>Depending on how you obtained the Windows software, this is a
  license agreement between (i) you and the device manufacturer or software
  installer that distributes the software with your device; or (ii) you and
  Microsoft Corporation (or, based on where you live or, if a business, where
  your principal place of business is located, one of its affiliates) if you
  acquired the software from a retailer. Microsoft is the device manufacturer
  for devices produced by Microsoft or one of its affiliates, and Microsoft is
  the retailer if you acquired the software directly from Microsoft. Note that if
  you are a volume license customer, use of this software is subject to your
  volume license agreement rather than this agreement.</p>
  <p>This agreement describes your rights and the conditions upon
  which you may use the Windows software. You should review the entire
  agreement, including any supplemental license terms that accompany the
  software and any linked terms, because all of the terms are important and
  together create this agreement that applies to you. You can review linked
  terms by pasting the (aka.ms/) link into a browser window.</p>
  <p>Depending on how you obtained the Windows software, this is a
  license agreement between (i) you and the device manufacturer or software
  installer that distributes the software with your device; or (ii) you and
  Microsoft Corporation (or, based on where you live or, if a business, where
  your principal place of business is located, one of its affiliates) if you
  acquired the software from a retailer. Microsoft is the device manufacturer
  for devices produced by Microsoft or one of its affiliates, and Microsoft is
  the retailer if you acquired the software directly from Microsoft. Note that if
  you are a volume license customer, use of this software is subject to your
  volume license agreement rather than this agreement.</p>
  <h2>Sports Update</h2>
  <div class="banner">
    
  </div>

</section>
</body>
</html>
```

CSS Display

display: Flex

- It is used for container.
- It can keep the element flexible in row or column.

- "display: flex" is used with attributes

flex-wrap	: wrap, no-wrap
flex-row	: horizontal orientation
flex-row-reverse	: latest first
flex-column	: vertical orientation
flex-column-reverse	: latest top
flex-direction	: row or column
justify-content	: space between, around, evenly
align-items	: baseline, top, center

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    .box {
      width: 100px;
      height: 100px;
      padding: 5px;
      margin: 10px;
      border: 1px solid black;
    }
    body {
      display: flex;
      flex-wrap: wrap;
      flex-direction: column-reverse;
    }
  </style>
</head>
<body>
  <div class="box">
    Box-1
  </div>
  <div class="box">
    Box-2
  </div>
  <div class="box">
    Box-3
```

```
</div>
<div class="box">
  Box-4
</div>
<div class="box">
  Box-5
</div>
<div class="box">
  Box-6
</div>
<div class="box">
  Box-7
</div>
<div class="box">
  Box-8
</div>
<div class="box">
  Box-9
</div>
<div class="box">
  Box-10
</div>
</body>
</html>
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    ul {
      list-style: none;
      height: 400px;
      border: 2px solid gray;
      width: 150px;
      display: flex;
      flex-direction: column;
      justify-content: space-around;
      align-items: stretch;
    }
  </style>
```

```

    }
    li {
      width: 100px;
      border: 1px solid black;
      border-radius: 10px;
      background-color: red;
      color: white;
      padding: 5px;
    }
  </style>
</head>
<body>
  <ul>
    <li>Home</li>
    <li>Electronics</li>
    <li>Footwear</li>
    <li>Fashion</li>
  </ul>
</body>
</html>

```

CSS Float

- It makes the element float left or right.
- It can wrap the content around floating element.

Syntax:

```

{
  float: left | right;
}

```

- If you want to clear floating then have to use "clear" attribute.

Syntax:

```

{
  clear: left, right, both
}

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>

```

```

<meta charset="UTF-8">
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<style>
  body {
    background-color: black;
  }
  .banner-1 {
    border-top: 15px solid rgb(65, 65, 65);
    border-bottom: 15px solid rgb(84, 84, 84);
    height: 300px;
    color: white;
    font-family: Arial;
  }
  .banner1-img {
    float: right;
  }
  .banner1-title {
    float: left;
    margin-left: 100px;
    width: 500px;
  }
  .banner1-title h1 {
    font-size: 50px;
  }
  .banner1-title p {
    font-size: 25px;
  }
</style>
</head>
<body>
  <div class="banner-1">
    <div class="banner1-title">
      <h1>Enjoy on your TV.</h1>
      <p>Watch on smart TVs, PlayStation, Xbox, Chromecast, Apple TV,
Blu-ray players and more.</p>
    </div>
    <div class="banner1-img">
      
    </div>
  </div>
</body>

```


</html>

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    .h1 {
      float: left;
    }
    .h2 {
      float: left;
    }
    .pic {
      clear:left;
    }
  </style>
</head>
<body>
  <div class="h1">
    <h1>Heading-1</h1>
  </div>
  <div class="h2">
    <h1>Heading-2</h1>
  </div>
  <div class="pic">
    
  </div>

</body>
</html>
```

CSS Display

CSS Positions

CSS Float

CSS Clear

CSS Backgrounds

background-color

background-image

background-repeat : no-repeat, repeat-x, repeat-y, repeat

background-position : left, center, right, top, center, bottom

background-size : pixels, cover, contain

background-attachment : scroll, fixed

FAQ: Can we set multiple background images?

Ans: Yes.

Ex:

```
<style>
  body {
    background-image: url("../images/asp.jpg"),
    url("../images/android.jpg");
    background-repeat: no-repeat, repeat;
    background-size: 200px 200px, cover;
    background-position: center center, center center;
    background-attachment: fixed, fixed;
  }
</style>
```

FAQ: Can we change background-image dynamically?

Ans : Yes, By using media query

Ex:

```
<style>
  @media screen and (orientation:landscape) {
    body {
      background-image: url("../images/netflixback.jpg");
      background-size: cover;
    }
  }
  @media screen and (orientation:portrait) {
    body {
      background-image: url("../images/netflixlogo.jpg");
      background-size: 100% 700px;
    }
  }
</style>
```

```
        background-repeat: no-repeat;
    }
}
</style>
```

Summary

- CSS Selectors
- CSS Units
- CSS Colors
- CSS Inheritance
- CSS Box Model
- CSS Positions
- CSS Display
- CSS Float
- CSS Backgrounds

Bootstrap Theme

https://themes.getbootstrap.com/preview/?theme_id=37702

Template Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Shopper Template</title>
  <link rel="stylesheet" href="../node_modules/bootstrap-
icons/font/bootstrap-icons.css">
  <link rel="stylesheet"
href="../node_modules/bootstrap/dist/css/bootstrap.css">
  <style>
    header {
      display: flex;
      justify-content: space-between;
    }
    .brand-title {
      font-size: 25px;
```

```
        font-weight: bold;
    }
    .brand-menu {
        font-size: 20px;
    }
    .brand-menu span {
        margin-right: 20px;
    }
    .brand-icons {
        font-size: 20px;
    }
    .brand-icons span {
        margin-right: 20px;
    }
    .brand-offer {
        background-color: black;
        color: white;
        padding: 5px;
        text-align: center;
    }
    .brand-nav {
        margin-top: 10px;
        height: 450px;
        display: grid;
        grid-template-columns: 4fr 4fr 4fr;
    }
    .women {
        background-image: url("../images/fashion1.jpg");
        background-size: cover;
        text-align: center;
        display: flex;
        flex-direction: column;
        justify-content: space-evenly;
        opacity: 0.6;
    }
    .women:hover {
        opacity: 1;
    }
    .men {
        background-image: url("../images/fashion2.jpg");
        background-size: cover;
        text-align: center;
        display: flex;
```

```
        flex-direction: column;
        justify-content: space-evenly;
        opacity: 0.6;
    }
    .men:hover {
        opacity: 1;
    }
    .kids {
        background-image: url("../images/fashion3.png");
        background-size: cover;
        text-align: center;
        display: flex;
        flex-direction: column;
        justify-content: space-evenly;
        opacity: 0.6;
    }
    .kids:hover {
        opacity: 1;
    }
    .banner-title {
        font-size: 40px;
        font-weight: bold;
        color: white;
        text-shadow: 2px 2px 2px black;
    }
    .men button {
        width: 200px;
        height: 45px;
        background-color: white;
    }
    .women button {
        width: 200px;
        height: 45px;
        background-color: white;
    }
    .kids button {
        width: 200px;
        height: 45px;
        background-color: white;
    }
}
</style>
</head>
<body class="container-fluid">
```

```

<header>
  <div class="brand-title">
    Shopper.
  </div>
  <div class="brand-menu">
    <span>Home</span>
    <span>Catalog</span>
    <span>Shop</span>
    <span>Blog</span>
    <span>Pages</span>
    <span>Docs</span>
  </div>
  <div class="brand-icons">
    <span class="bi bi-search"></span>
    <span class="bi bi-person"></span>
    <span class="bi bi-heart"></span>
    <span class="bi bi-cart"></span>
  </div>
</header>
<div class="brand-offer">
  <span class="bi bi-lightning-fill text-warning"></span> HAPPY
HOLIDAY DEALS ON EVERYTHING <span class="bi bi-lightning-fill text-
warning"></span>
</div>
<div class="brand-nav">
  <div class="women">

    <div class="banner-title">
      Women
    </div>
    <div>
      <button>Shop Women <span class="bi bi-arrow-right"></span>
</button>
    </div>

  </div>
  <div class="men">
    <div class="banner-title">
      Men
    </div>
    <div>
      <button>Shop Men <span class="bi bi-arrow-right"></span>
</button>

```

```

        </div>
    </div>
    <div class="kids">
        <div class="banner-title">
            Kids
        </div>
        <div>
            <button>Shop Kids <span class="bi bi-arrow-right"></span>
        </button>
        </div>
    </div>
</div>
</body>
</html>

```

CSS Transforms

- Transform changes the element position, size and orientation.
- Transforms are 2 types
 - a) 2D Transforms
 - b) 3D Transforms

2D Transform

- It transforms any element along "X & Y" axis.
- It is 2 Dimensional [X & Y]
- Transform is defined by using "transform" attribute
- Transform comprises of various methods

scale()

translate()

rotate()

skew()

matrix()

Syntax:

```

selector
{
    transform : scale() | translate() | rotate();
}

```

- Transform is not compatible with several browsers.
- You have to apply transform using browser plugin's.

-webkit	safari, edge, chrome
-moz	firefox
-o	opera
-ms	internet explorer

Syntax:

```
selector
{
  transform: scale();
  -webkit-transform:scale();
  -moz-transform:scale();
  -o-transform:scale();
  -ms-transform:scale();
}
```

translate()

- It is used to change the position of any element along X and Y axis.

translate()	Both directions
translateX()	Horizontal
translateY()	Vertical

Syntax:

```
transform:translate(200px, 100px);    [horizontal, vertical]
transform:translateX(200px);
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    img {
      transform: translate(0px, 0px);
      transition:2s;
    }
    img:hover {
      transform: translate(300px, 50px);
      transition: 3s;
```



```

    }
  </style>
</head>
<body>
  
</body>
</html>

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    .men-banner {
      transform: translateX(0px);
    }
    .men-banner:hover {
      transform: translateX(-10px);
    }
    .men-banner:hover img {
      width: 310px;
    }
    .men {
      border: 1px solid gray;
      width: 300px;
      overflow: hidden;
    }
  </style>
</head>
<body>
  <div class="men">
    <div class="men-banner">
      
    </div>
  </div>
</body>
</html>

```

Note: Transforms are with regard to the page not to the specific element.

Element related transform is configured using margins, padding, width and height.

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="../../node_modules/bootstrap-
icons/font/bootstrap-icons.css">
  <style>
    button {
      transform: translateX(0px);
      transition: 2s;
      width: 100px;
      position: relative;
    }
    button:hover span {
      margin-left: 20px;
      transition: 2s;
    }
  </style>
</head>
<body>
  <button>
    <span class="bi bi-arrow-right"></span>
  </button>
</body>
</html>
```

Scale()

- It is used to transform the size of element.

scale()	: width and height
scaleX()	: width
scaleY()	: height

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="../../node_modules/bootstrap-
icons/font/bootstrap-icons.css">
  <style>
    body {
      display: flex;
      justify-content: center;
      align-items: center;
      height: 500px;
    }
    img {
      transform: scale(3,3);
      transition: 3s;
    }
    img:hover {
      transform: scale(6,6);
      transition: 2s;
    }
    .box {
      width: 200px;
      height: 200px;
      border: 1px solid gray;
      overflow: hidden;
    }
  </style>
</head>
<body>
  <div class="box">
    
  </div>
</body>
</html>

```

CSS 2D Transforms

translate()

scale()

rotate()

- It can rotate any element by specified deg or turns.
- It uses an angle for rotating along x and y axis

rotate() short hand
 rotateX()
 rotateY()

- "deg" can be from 0 to 360.
- "turns" can be from 0.0 to 0.99

Syntax:

```
img {
  transform: rotate(90deg);
}
img {
  transform: rotate(0.25turn);
}
```

skew()

- It is used to tilt any element by specified angle.

skew() short hand
 skewX()
 skewY()

Syntax:

```
img {
  transform: skew(20deg);
}
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="../node_modules/bootstrap-
icons/font/bootstrap-icons.css">
  <style>
    body {
      display: flex;
```

```

        justify-content: center;
        align-items: center;
        height: 500px;
    }

    .box {
        width: 200px;
        height: 200px;
        border: 1px solid gray;
        overflow: hidden;
    }
    img {
        transform: skew(0deg);
        transition: 2s;
    }
    img:hover {
        transform: skewY(-20deg);

        transition: 3s;
    }
</style>
</head>
<body>
    <div class="box">
        
    </div>
</body>
</html>

```

Note: You can't apply multiple transform to any element. You can use **matrix()**.

matrix()

- It can apply multiple transforms which includes translate, scale and skew

Syntax:

matrix(scaleX(),skewX(),skewY(),scaleY(),translateX(),translateY());

Ex:

<!DOCTYPE html>

```

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="../../node_modules/bootstrap-
icons/font/bootstrap-icons.css">
  <style>
    img {
      transform: matrix(1,0,0,1,0,0);
      transition: 2s;
    }
    img:hover {
      transform: matrix(2,3,2,2,500,100);
      transition: 3s;
    }
  </style>
</head>
<body>
  
</body>
</html>

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="../../node_modules/bootstrap-
icons/font/bootstrap-icons.css">
  <style>
    .box-1 {
      background-color: yellow;
      border: 2px solid black;
      height: 200px;
      width: 50px;
      position: absolute;

```

```

    }
    .box-2 {
        background-color: greenyellow;
        border: 2px solid black;
        height: 200px;
        width: 50px;
        position: absolute;
        margin-left: 50px;
        transform: skewX(20deg);
        box-shadow: 10px 10px 3px black;
    }
    .box-3 {
        background-color: lightpink;
        border: 2px solid black;
        height: 50px;
        width: 150px;
        position: absolute;
        margin-top: 150px;
        transform:skewX(60deg);
    }
</style>
</head>
<body>
    <div class="box-1">

    </div>
    <div class="box-2">

    </div>
    <div class="box-3">

    </div>
    <div class="box-4">

    </div>
</body>
</html>

```

3D Transform

- It transforms in 3 Dimensions X, Y and Z axis.

translate3d()

scale3d()
rotate3d()
skew3d()

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="../node_modules/bootstrap-
icons/font/bootstrap-icons.css">
  <style>
    body {
      display: flex;
      justify-content: center;
      align-items: center;
      height: 500px;
    }

    img {
      transform: scale3d(1,1,1);
      transition: 2s;
      margin: 30px;
    }
    img:hover {
      transform: scale3d(1.5,1.5,1);
      transition: 3s;
      box-shadow: 10px 10px 2px gray;
    }
    marquee {
      padding: 50px;
    }
  </style>
</head>
<body>
  <marquee height="350" scrollamount="10" onmouseover="this.stop()"
onmouseout="this.start()">
    
    
    
```



```

        
    </marquee>
</body>
</html>

```

CSS Transition

transition-duration	: Time taken to complete the transform
transition-delay	: Time taken to start transform
transition-property	: Specified the property name that have to
	use the transition timing
transition-animation-function	: predefined animation function
transition	: short hand

Syntax:

```

{
  transition: 2s 4s width ease-in;
}

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="../node_modules/bootstrap-
icons/font/bootstrap-icons.css">
  <style>
    .box {
      width: 100px;
      background-color: yellow;
      height: 100px;
      transition-duration: 2s;
      transition-timing-function: steps(3);
    }
    .box:hover {
      width: 1000px;
      background-color: red;
      transition-duration: 4s;
      transition-delay: 2s;
    }
  </style>

```

```

        transition-property: width;
        transition-timing-function: steps(10);
    }
</style>
</head>
<body>
    <div class="box">

        </div>
</body>
</html>

```

@Keyframes

- Frames are used in animation
- Video have frames 30fps - 4K 60fps
- Keyframe is a frame with object that can change position, shape or orientation.

Syntax:

```

@Keyframe Name
{
    from { Initial State }
    to { Final State }
}

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link rel="stylesheet" href="../node_modules/bootstrap-
icons/font/bootstrap-icons.css">
    <style>
        @keyframes Zoom
        {
            from {

```

```

        margin-left: 110%;
        margin-top: 20%;
        font-size: 20px;
    }
    50% {
        font-size: 120px;
    }
    to {
        margin-left: 0%;
        margin-top: 0%;
        font-size: 40px;
    }
}
@keyframes ZoomImage
{
    from {
        margin-left: 110%;
        margin-top: 20%;
        width: 50px;
        height: 50px;
    }
    10%{
        width: 200px;
        height: 200px;
        transform: skew(50deg);
    }
    50% {
        width: 400px;
        height: 400px;
        transform: rotate(180deg);
    }
    80% {
        width:200px;
        height: 200px;
        transform: skewY(60deg);
    }
    to {
        margin-left: 0%;
        margin-top: 0%;
    }
}
body {
    overflow: hidden;

```

```

    }
    h1 {
      animation-name: Zoom;
      animation-duration: 5s;
      animation-iteration-count: infinite;
      animation-direction: alternate;
    }
    img {
      animation-name: ZoomImage;
      animation-duration: 5s;
      animation-iteration-count: infinite;
      animation-direction: alternate;
    }
  </style>
</head>
<body>
  <h1>Welcome</h1>
  
</body>
</html>

```

Animation Properties

animation-name : It refers to keyframe name.
 animation-duration
 animation-delay
 animation-timing-function
 animation-iteration-count
 animation-direction

CSS Text Effects
 CSS Media Query

Responsive Web Design

- Responsive makes the page adjust its content according device screen.

1. Meta

```

<meta name="viewport" content="width=device-width, initial-
scale=1">

```

2. Width and Height

% and em

3. Display - Grid, Flex

4. Media Queries

Media Queries

```
@media type query  
{  
  
}
```

type : screen, print, speech

query : orientation [landscape, portrait], min-width, max-width

FAQ: What is difference between width and min-width or max-width?

Ans:

width : It defines exact pixels. [400px = exactly 400px]

min-width : It defines the width from specified pixels. [400px = from 400px]

max-width : It defines the width upto specified pixels [400px = upto 400px]

Ex:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Document</title>
```

```
  <style>
```

```
    ul {
```

```
      list-style: none;
```

```
    }
```

```
    li {
```

```
      border: 1px solid black;
```

```
      padding: 5px;
```

```
      width: 100px;
```

```

    }
    @media screen and (min-width:500px) {
        ul {
            display: flex;
        }
        li {
            margin-right: 20px;
        }
    }
    @media screen and (max-width:499px) {
        ul {
            display: flex;
            flex-direction: column;
        }
        li {
            margin-top: 20px;
        }
    }
</style>
</head>
<body>
    <nav>
        <ul>
            <li>Home</li>
            <li>About</li>
            <li>Contact</li>
            <li>Fashion</li>
            <li>Footwear</li>
        </ul>
    </nav>
</body>
</html>

```

CSS Text Effects

font-family	: Sets font family [Arial..]
font-size	: Sets font size [absolute, relative]
font-variant	: Changes Capitalization [small-caps, lowercase]
font-weight	: Sets bold
font-style	: Sets Italics
text-align	: left, center, right, justify

text-decoration : short hand for text decoration
text-decoration-style : dotted, wavy, dashed, solid
text-decoration-line : underline, overline, line through, none
text-decoration-color : sets color for line
text-decoration-thickness: sets size for line

Syntax:

text-decoration : 3px underline wavy red;

text-transform : uppercase, lowercase, capitalize
text-shadow : sets shadow for text
letter-spacing : sets space between letters
word-spacing : sets space between words
line-height : sets space between line
text-indent : sets first line indentation
word-break : It controls the lengthy words at right margin.
It breaks the word and wraps to next line.

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    p {
      letter-spacing: 5px;
      word-spacing: 10px;
      line-height: 40px;
      text-indent: 100px;
    }
  </style>
</head>
<body>
  <p>Depending on how you obtained the Windows software, this is a
license agreement between (i) you and the device manufacturer or software
installer that distributes the software with your device; or (ii) you and
Microsoft Corporation (or, based on where you live or, if a business, where
```

your principal place of business is located, one of its affiliates) if you acquired the software from a retailer. Microsoft is the device manufacturer for devices produced by Microsoft or one of its affiliates, and Microsoft is the retailer if you acquired the software directly from Microsoft. Note that if you are a volume license customer, use of this software is subject to your volume license agreement rather than this agreement.</p>

</body>

</html>

CSS List Styles

list-style-image : sets image a custom bullet

list-style-type : changes the bullet type or numbering type

list-style-position : inside, outside

Ex:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Document</title>
```

```
  <style>
```

```
    ol {
```

```
      list-style-type: lower-alpha;
```

```
      list-style-position: inside;
```

```
    }
```

```
    li {
```

```
      border: 1px solid red;
```

```
      width: 100px;
```

```
    }
```

```
  </style>
```

```
</head>
```

```
<body>
```

```
  <ol>
```

```
    <li>HTML</li>
```

```
    <li>CSS</li>
```

```
    <li>Bootstrap</li>
```

```
  </ol>
```

```
</body>
```


</html>

Online Portfolio

1. Create a new account on GIT Hub

<https://github.com/>

2. Download and Install Github Desktop tool

<https://desktop.github.com/>

Publishing on GIT Hub Pages Portfolio

1. Create a new account on GIT.

2. Login into GIT

3. Click "New" Repository button

Owner	:	your_name	
Repository	:	portfolio	[ui-portfolio]
Select	:	public	

4. Click on "Create Repository"

<https://github.com/fullstack-shopping/ui-portfolio.git>

5. After creating repository go to "Settings"

6. Select "Pages" tab [GIT Hub Pages]

7. Open GIT Hub Desktop

8. Go to "Current Repository"

9. Click "Add" button

10. Select "Clone Repository"

**11. Choose your Repository name
"ui-portfolio"**

12. Copy the "Local Path"

C:\Users\.....username....\Documents\GitHub\ui-portfolio

13. Cut all your project files from "E:\portfolio" and paste into local path.

14. All files are cloned into GIT Desktop

15. Go to GIT Desktop

Summary : Ver-1

16. Click "Commit"

17. Click "Publish Branch"

18. Goto "GIT Hub Website => Settings => Pages => Branch Name "Main"

19. Click Save

20. Your Domain is created

<https://yourname.github.io/ui-portfolio/>

BOOT STRAP (VERSION 5)

- It is component library.
- A component comprises of
 - a) Presentation
 - b) Styles
 - c) Logic
- Library comprises of pre-defined functions and values, you can implement and customize according to the requirements.
- The other 3rd parties like bootstrap
 - twitter bootstrap
 - material ui
 - angular material
 - telerik
 - dev express etc..
- It is open source.
- Open source allows to customize according requirements.

Setup Bootstrap in Project:

1. Install the following libraries

```
> npm install bootstrap --save
> npm install bootstrap-icons --save
> npm install jquery --save
```

2. Link the following files into web page

bootstrap.css	<link>
bootstrap-icons.css	<link>
jquery.js	<script>
bootstrap.bundle.js	<script>

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet"
href="../node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
```

```

icons.css">
</head>
<body>

    <script src="../node_modules/jquery/dist/jquery.js"></script>
    <script src="../node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Bootstrap Layout Classes

- Bootstrap provides pre-defined classes for all types of layout designs.
- You can apply the classes and customize if required.

1. Container Classes: They are used to target your content according to the device width. They use media queries implicitly.

.container	Small Screens like Tabs
.container-sm	Mobile
.container-lg	Desktop
.container-xl	Desktop wide 1400px
.container-xxl	Laptop 1600px
.container-fluid	Responsive

*Note: Bootstrap uses "Roboto" font for elements.
Bootstrap uses lot of type selectors with pre-defined effects.*

2. Background Color and Text Color

.bg-{contextual} primary, secondary, danger, warning, info, success, dark..
.text-{contextual} primary,...

3. Box Model Class

a) Margin and Padding

.m	margin all directions
.p	padding all directions

{s}	left [start]
-----	--------------

{e} right [end]
{t} top
{b} bottom

.mt margin-top
.ms margin-left
.me margin-right

Size of margin and padding can be from 0 to 4.

.m-4
.mt-4
.p-4
.pt-3

b) Border

.border
.border-{0 to 3}
.border-{contextual}

c) Border Radius

.rounded
.rounded-{0 to 3}
.rounded-pill
.rounded-circle

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet"
href="../node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">

</head>
<body class="container-fluid">
```

```


<h1>Welcome</h1>
<p class="ps-4 pt-4 pb-2 border border-3 border-danger">Bootstrap is a
component library.</p>
<dl>
  <dt>HTML</dt>
  <dd class="mt-2 ms-4">It is a markup language.</dd>
</dl>
<script src="../node_modules/jquery/dist/jquery.js"></script>
<script src="../node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Classes

- Container Classes
- Box Model
 - margin, padding, border, rounded, w
- Bg Color
- Text Color

- Width

```

.w-{0,25,50,75,100}      Width
.h-{0,50,100}

```

Bootstrap Positions

```

.position-static
.position-absolute
.position-relative
.position-sticky
.position-fixed
.top
.start
.end
.bottom

```

Ex:

```

<!DOCTYPE html>

```

```

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="./node_modules/bootstrap-icons/font/bootstrap-
icons.css">

</head>
<body class="container-fluid">
  <h2>Positions</h2>
  <button class="btn btn-primary position-relative">
    <span class="bi bi-cart4"></span> Your Cart <span class="badge bg-dark
position-absolute rounded rounded-circle">3</span>
  </button>
  <div class="position-fixed top-50 end-0">
    Offer 50%
  </div>
  <script src="./node_modules/jquery/dist/jquery.js"></script>
  <script src="./node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Ex:

```
<h2 class="bg-dark text-white p-2 position-sticky top-0">Sports News</h2>
```

Display Class

```

.d-none
.d-inline
.d-flex
.d-block
.flex-wrap
.flex-row
.flex-column
.flex-row-reverse
.flex-column-reverse
.justify-content-between

```

.justify-space-between
.justify-space-around
.justify-content-center
.align-items-center
.align-items-baseline

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="./node_modules/bootstrap-icons/font/bootstrap-
icons.css">

</head>
<body class="container-fluid">
  <div class="d-flex justify-content-center align-items-center"
style="height:500px">
    <h1>Bootstrap</h1>
  </div>
  <script src="./node_modules/jquery/dist/jquery.js"></script>
  <script src="./node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>
```

Gird Classes

.row
.col
.col-{1 to 12}

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```



```

<title>Document</title>
<link rel="stylesheet"
href="../node_modules/bootstrap/dist/css/bootstrap.css">
<link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">

</head>
<body class="container-fluid">
  <h2>Product Details</h2>
  <dl class="row">
    <dt class="col-4">Name</dt>
    <dd class="col-8">Samsung TV</dd>
    <dt class="col-4">Price</dt>
    <dd class="col-8">56000.44</dd>
    <dt class="col-4">Stock</dt>
    <dd class="col-8">Available</dd>
  </dl>
  <script src="../node_modules/jquery/dist/jquery.js"></script>
  <script src="../node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet"
href="../node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">

</head>
<body class="container-fluid">
  <h2>Grid</h2>
  <div class="row m-2">
    <div class="col-2 border border-1 border-primary">Col-1</div>
    <div class="col-8 border border-1 border-primary">Col-2</div>
  </div>

```

```

    <div class="col border border-1 border-primary">Col-3</div>
  </div>
  <div class="row m-2">
    <div class="col border border-1 border-primary">
      Navigation
    </div>
    <div class="col border border-1 border-primary">
      Main
    </div>
  </div>
  <script src="./node_modules/jquery/dist/jquery.js"></script>
  <script src="./node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Components

1. alerts : Alerts are used to display embedded messages in page.
You can control the display of messages.

Classes:

- .alert
- .alert-{contextual}
- .alert-title
- .alert-subtitle
- .alert-link
- .alert-dismissible

jquery attribute: for functionality

data-bs-dismiss : It can close any component.

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

```

```

<title>Document</title>
<link rel="stylesheet"
href="../node_modules/bootstrap/dist/css/bootstrap.css">
<link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">

</head>
<body class="container-fluid">
  <h2>Alerts</h2>
  <div class="alert alert-danger alert-dismissible">
    <h2 class="alert-title">Delete Record</h2>
    <button class="btn-close" data-bs-dismiss="alert"></button>
    <p class="alert-text">Record will be deleted.</p>
    <a href="#" class="alert-link">Help?</a>
  </div>
  <div class="alert alert-success alert-dismissible">
    <h2 class="alert-title">Registered Successfully..</h2>
    <button class="btn-close" data-bs-dismiss="alert"></button>
    <button data-bs-dismiss="alert" class="btn btn-success">OK</button>
  </div>
  <script src="../node_modules/jquery/dist/jquery.js"></script>
  <script src="../node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

2. Badge : It is used to highlight any specific content with background and border. It uses an icon style for display content.

```

class
.badge
.bg-{contextual}
.text-{contextual}

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet"

```

```

href="../node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">

</head>
<body class="container-fluid">
  <h2 class="text-center">Table of Contents</h2>
  <ul class="list-unstyled">
    <li class="bg-dark p-2 text-white mb-2 d-flex justify-content-between">
<span>HTML</span> <span class="badge bg-light text-dark">2</span> </li>
    <li class="bg-dark p-2 text-white mb-2 d-flex justify-content-between">
<span>CSS</span> <span class="badge bg-light text-dark">6</span> </li>
    <li class="bg-dark p-2 text-white mb-2 d-flex justify-content-between">
<span>Bootstrap</span> <span class="badge bg-light text-dark">8</span> </li>
  </ul>
  <script src="../node_modules/jquery/dist/jquery.js"></script>
  <script src="../node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

3. Cards : Card defines content presented in 3 section, head, body, footer.

Classes

```

.card
.card-header
.card-body
.card-footer
.card-title
.card-subtitle
.card-text
.card-img-top
.card-img-overlay
.card-img-bottom

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>

```

```

<link rel="stylesheet"
href="../node_modules/bootstrap/dist/css/bootstrap.css">
<link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
<style>
    .card-header {
        height: 150px;
    }
</style>
</head>
<body class="container-fluid">
<h2>Cards</h2>
<div class="d-flex flex-wrap">
    <div class="card m-2 p-2 bg-primary text-white" style="width:250px">
        
        <div class="card-header">
            <h3 class="card-title">ASP.NET</h3>
            <p class="card-subtitle">Server Side</p>
        </div>
        <div class="card-body">
            <dl>
                <dt>Front End</dt>
                <dd>HTML, CSS</dd>
                <dt>Back End</dt>
                <dd>MySql, ASP</dd>
            </dl>
        </div>
        <div class="card-footer">
            <button class="btn btn-dark w-100">Join Course</button>
        </div>
    </div>

    <div class="card m-2 p-2" style="width:250px">

        <div class="card-header">
            <h3 class="card-title">Digital Marketing</h3>
            <p class="card-subtitle">Server Side</p>
        </div>
        <div class="card-body">
            
            <dl>
                <dt>Front End</dt>

```

```

        <dd>HTML, CSS</dd>
        <dt>Back End</dt>
        <dd>MySql, ASP</dd>
    </dl>
</div>
<div class="card-footer">
    <button class="btn btn-primary w-100">Join Course</button>
</div>
</div>
</div>
<script src="./node_modules/jquery/dist/jquery.js"></script>
<script src="./node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

4. Collapse : It can show or hide any content dynamically

Classes

```

.collapse
.show
.fade

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
    <link rel="stylesheet" href="./node_modules/bootstrap-icons/font/bootstrap-
icons.css">
    <style>
        .card-header {
            height: 150px;
        }
    </style>
</head>
<body class="container-fluid">
    <h1 class="text-center">Netflix Frequently Asked Questions</h1>
    <div>

```

```

<button data-bs-toggle="collapse" data-bs-target="#q1" class="btn btn-dark
w-100">What is Netflix?</button>
<div class="mt-1 bg-dark text-white p-2 collapse show fade" id="q1">
  <p>Netflix is a streaming service that offers a wide variety of award-
winning TV shows, movies, anime, documentaries and more – on thousands of
internet-connected devices.</p>
  <p>You can watch as much as you want, whenever you want, without a
single ad – all for one low monthly price. There's always something new to
discover, and new TV shows and movies are added every week!</p>
</div>
</div>
<div class="mt-2">
  <button data-bs-toggle="collapse" data-bs-target="#q2" class="btn btn-dark
w-100">How much does Netflix cost?</button>
  <div class="mt-1 bg-dark text-white p-2 collapse fade" id="q2">
    <p>Netflix is a streaming service that offers a wide variety of award-
winning TV shows, movies, anime, documentaries and more – on thousands of
internet-connected devices.</p>
    <p>You can watch as much as you want, whenever you want, without a
single ad – all for one low monthly price. There's always something new to
discover, and new TV shows and movies are added every week!</p>
  </div>
</div>
<script src="./node_modules/jquery/dist/jquery.js"></script>
<script src="./node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

alerts
 badge
 cards
 collapse

Accordion

- It exhibits mutex [Mutual Exclusion] mechanism.
- Only one category of options can expand and others will collapse.

Classes:

.accordion
 .accordion-header
 .accordion-body
 .accordion-footer
 .accordion-collapse

.accordion-button
.accordion-item
.show [display]
.collapse [hidden]

Attribute:

data-bs-target : refers to collapse id
data-bs-toggle : refers to collapse
data-bs-parent : refers to accordion id, it is defined to accordion-collapse

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="../node_modules/bootstrap/dist/css/bootstrap.css"
">
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body class="container-fluid">
  <h1 class="text-center">Netflix Frequently Asked Questions </h1>
  <div class="accordion" id="faqs">
    <div class="accordion-item">
      <div class="accordion-header">
        <button data-bs-toggle="collapse" data-bs-target="#q1"
class="accordion-button">What is Netflix?</button>
      </div>
      <div class="accordion-collapse collapse show" id="q1" data-bs-
parent="#faqs">
        <div class="accordion-body">
          <p>Netflix is a streaming service that offers a wide variety of award-
winning TV shows, movies, anime, documentaries and more – on thousands of
internet-connected devices.</p>
          <p>You can watch as much as you want, whenever you want, without
a single ad – all for one low monthly price. There's always something new to
discover, and new TV shows and movies are added every week!</p>
        </div>
      </div>
    </div>
  </div>
```



```

<div class="accordion-item">
  <div class="accordion-header">
    <button data-bs-target="#q2" data-bs-toggle="collapse"
class="accordion-button">How much does Netflix cost?</button>
  </div>
  <div class="accordion-collapse collapse" id="q2" data-bs-parent="#faqs">
    <div class="accordion-body">
      <p>Netflix is a streaming service that offers a wide variety of award-
winning TV shows, movies, anime, documentaries and more – on thousands of
internet-connected devices.</p>
      <p>You can watch as much as you want, whenever you want, without
a single ad – all for one low monthly price. There's always something new to
discover, and new TV shows and movies are added every week!</p>
    </div>
  </div>
</div>
</div>
</div>
<script src="./node_modules/jquery/dist/jquery.js"></script>
<script src="./node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Carousel

- Creating sliding and fading banners.

Basic Carousel Classes:

```

.carousel
.carousel-inner
.carousel-item
.active      : It is defined for carousel item. Any one item must be active.
.slide      : It is defined for carousel to set sliding effects
.fade       : It is defined for carousel to set fading effects

```

Basic Carousel Attributes:

```

.data-bs-ride : It starts animation for carousel

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">

```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Carousel</title>
<link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
<link rel="stylesheet" href="./node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body class="container-fluid">
  <div class="carousel slide" id="banner" data-bs-ride="carousel">
    <div class="carousel-inner">
      <div class="carousel-item active">
        
      </div>
      <div class="carousel-item">
        
      </div>
      <div class="carousel-item">
        
      </div>
    </div>
  </div>
  <script src="./node_modules/jquery/dist/jquery.js"></script>
  <script src="./node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Carousel Buttons [Controls]

- They are used to navigate between slides in carousel.
- They can add previous and next functions.

Classes:

```

.carousel-control-prev
.carousel-control-prev-icon
.carousel-control-next
.carousel-control-next-icon
.carousel-dark

```

Attributes:

```

data-bs-slide="prev"
data-bs-slide="next"
data-bs-target="#Carouse Id"

```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Carousel</title>
  <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="./node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body class="container-fluid">
  <div class="carousel slide carousel-dark" id="banner" data-bs-
ride="carousel">
    <div class="carousel-inner">
      <div class="carousel-item active">
        
      </div>
      <div class="carousel-item">
        
      </div>
      <div class="carousel-item">
        
      </div>
    </div>
    <button data-bs-slide="prev" data-bs-target="#banner" class="carousel-
control-prev">
      <span class="carousel-control-prev-icon"></span>
    </button>
    <button data-bs-slide="next" data-bs-target="#banner" class="carousel-
control-next">
      <span class="carousel-control-next-icon"></span>
    </button>
  </div>
  <script src="./node_modules/jquery/dist/jquery.js"></script>
  <script src="./node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>
```

Carousel Indicators

- They allows to navigate to any specific slide directly.

Classes:

.carousel-indicators

Attribute:

data-bs-slide-to="indexNumber"

data-bs-target="carousel_id"

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Carousel</title>
  <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="./node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body class="container-fluid">
  <div class="carousel slide carousel-dark" id="banner" data-bs-
ride="carousel">
    <div class="carousel-inner">
      <div class="carousel-item active">
        
      </div>
      <div class="carousel-item">
        
      </div>
      <div class="carousel-item">
        
      </div>
    </div>
    <button data-bs-slide="prev" data-bs-target="#banner" class="carousel-
control-prev">
      <span class="carousel-control-prev-icon"></span>
    </button>
    <button data-bs-slide="next" data-bs-target="#banner" class="carousel-
control-next">
      <span class="carousel-control-next-icon"></span>
    </button>
```

```

    <div class="carousel-indicators">
      <button class="active" data-bs-slide-to="0" data-bs-
target="#banner"></button>
      <button data-bs-slide-to="1" data-bs-target="#banner"></button>
      <button data-bs-slide-to="2" data-bs-target="#banner"></button>
    </div>
  </div>
  <script src="../node_modules/jquery/dist/jquery.js"></script>
  <script src="../node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Note: You can control timing for every slide by using "data-bs-interval=3000".

1000 = 1sec

Timing is defined for carousel item.

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Carousel</title>
  <link rel="stylesheet"
href="../node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body class="container-fluid">
  <div class="carousel slide carousel-dark" id="banner" data-bs-
ride="carousel">
    <div class="carousel-inner">
      <div class="carousel-item active" data-bs-interval="3000">
        
      </div>
      <div class="carousel-item" data-bs-interval="1000">
        
      </div>
      <div class="carousel-item" data-bs-interval="4000">

```

```

        
    </div>
</div>
<button data-bs-slide="prev" data-bs-target="#banner" class="carousel-
control-prev">
    <span class="carousel-control-prev-icon"></span>
</button>
<button data-bs-slide="next" data-bs-target="#banner" class="carousel-
control-next">
    <span class="carousel-control-next-icon"></span>
</button>
<div class="carousel-indicators">
    <button class="active" data-bs-slide-to="0" data-bs-
target="#banner"></button>
    <button data-bs-slide-to="1" data-bs-target="#banner"></button>
    <button data-bs-slide-to="2" data-bs-target="#banner"></button>
</div>
</div>
<script src="../node_modules/jquery/dist/jquery.js"></script>
<script src="../node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Buttons

```

.btn
.btn-{contextual}
.btn-outline-{contextual}
.btn-sm | lg
.btn-close
.btn-link
.btn-group-vertical
.btn-toolbar
.btn-group

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Carousel</title>

```

```

<link rel="stylesheet"
href="../node_modules/bootstrap/dist/css/bootstrap.css">
<link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body class="container-fluid">
  <h2>Special</h2>
  <button class="btn-close"></button>
  <button class="btn btn-link">Signout</button>
  <h2>Toolbar</h2>
  <div class="btn-toolbar bg-danger justify-content-between">
    <div class="btn-group">
      <button class="btn btn-danger">Home</button>
      <button class="btn btn-danger">About</button>
      <button class="btn btn-danger">Contact</button>
    </div>
    <div class="btn-group">
      <button class="btn btn-danger">
        <span class="bi bi-person-fill"></span>
      </button>
      <button class="btn btn-danger">
        <span class="bi bi-heart-fill"></span>
      </button>
      <button class="btn btn-danger">
        <span class="bi bi-cart4"></span>
      </button>
    </div>
  </div>
  <h2>Button Contextual</h2>
  <button class="btn btn-primary">Insert</button>
  <button class="btn btn-info">Update</button>
  <button class="btn btn-danger">Delete</button>
  <h2>Button Outline Contextual</h2>
  <button class="btn btn-outline-primary">Insert</button>
  <button class="btn btn-outline-info">Update</button>
  <button class="btn btn-outline-danger">Delete</button>
  <h2>Button Group</h2>
  <div class="btn-group-vertical">
    <button class="btn btn-primary">Insert</button>
    <button class="btn btn-info">Update</button>
    <button class="btn btn-danger">Delete</button>
  </div>
  <h2>Button Size</h2>

```

```
<button class="btn btn-primary">Insert</button>
<button class="btn btn-info btn-sm">Update</button>
<button class="btn btn-danger btn-lg">Delete</button>
<script src="./node_modules/jquery/dist/jquery.js"></script>
<script src="./node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>
```

Forms

.form-control text, number, password, color, date...

.form-select dropdown

.form-check-input radio, checkbox

.form-label label

.form-check-label labels for radio and checkbox

.form-switch checkbox and radio with switch type

.form-range input type range

.form-input-file input type file

Input Group

.input-group

.input-group-sm | lg

.input-group-text

Modals

- It is used to design dialog box.
- A dialog box can popup in page.

Classes

.modal

.modal-dialog

.modal-content

.modal-header

.modal-body

.modal-footer

.modal-dialog-centered

.modal-fullscreen

.modal-dialog-scrollable

Attributes

data-bs-target
data-bs-toggle

Note: If you are using a hyperlink <a> element to open any modal then data-bs-target is not required, as it is defined by using "href".

```
<a data-bs-toggle="modal" href="#details"> Details </a>
```

```
<button data-bs-toggle="modal" data-bs-target="#details"> Details </button>
```

Nested Modals are disabled from bootstrap 5 version.

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Modals</title>
  <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="./node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body class="container-fluid">
  <h2>Modals</h2>
  <button class="btn btn-primary" data-bs-toggle="modal" data-bs-
target="#login">Login</button>
  <button class="btn btn-success" data-bs-toggle="modal" data-bs-
target="#details">Course Details</button>
  <a data-bs-toggle="modal" href="#details">Course Details</a>
  <div class="modal fade" id="login">
    <div class="modal-dialog modal-fullscreen modal-dialog-centered modal-
dialog-scrollable">
      <div class="modal-content">
        <div class="modal-header">
          <h3> <span class="bi bi-person-fill"></span> User Login</h3>
          <button class="btn-close" data-bs-dismiss="modal"></button>
        </div>
        <div class="modal-body">
```

```

<dl>
  <dt>User Name</dt>
  <dd><input type="text" class="form-control"></dd>
  <dt>Password</dt>
  <dd><input type="password" class="form-control"></dd>
  <dt>User Name</dt>
  <dd><input type="text" class="form-control"></dd>
  <dt>Password</dt>
  <dd><input type="password" class="form-control"></dd>
  <dt>User Name</dt>
  <dd><input type="text" class="form-control"></dd>
  <dt>Password</dt>
  <dd><input type="password" class="form-control"></dd>
  <dt>User Name</dt>
  <dd><input type="text" class="form-control"></dd>
  <dt>Password</dt>
  <dd><input type="password" class="form-control"></dd>
  <dt>User Name</dt>
  <dd><input type="text" class="form-control"></dd>
  <dt>Password</dt>
  <dd><input type="password" class="form-control"></dd>
  <dt>User Name</dt>
  <dd><input type="text" class="form-control"></dd>
  <dt>Password</dt>
  <dd><input type="password" class="form-control"></dd>
  <dt>User Name</dt>
  <dd><input type="text" class="form-control"></dd>
  <dt>Password</dt>
  <dd><input type="password" class="form-control"></dd>
</dl>
</div>
<div class="modal-footer">
  <button class="btn btn-primary">Login</button>
  <button class="btn btn-danger" data-bs-
dismiss="modal">Cancel</button>
</div>
</div>
</div>
<div class="modal fade" id="details">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">

```

```

        <h2>Course Details</h2>
        <button class="btn-close" data-bs-dismiss="modal"></button>
    </div>
    <div class="modal-body">
        <div class="row">
            <div class="col">
                
            </div>
            <div class="col">
                <h2>ASP.NET</h2>
                <ul>
                    <li>Web Development</li>
                    <li>Web API</li>
                </ul>
            </div>
        </div>
    </div>
</div>
</div>
<script src="../node_modules/jquery/dist/jquery.js"></script>
<script src="../node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Dropdown

- Bootstrap dropdown allows icons and symbols for list and items.
- It is good for designing static list in web page.
- It is good for designing navigation in page.
- It is not good for dynamic list. [List that have to change according to situation]

Classes

.dropdown
 .dropup
 .dropend
 .dropstart
 .dropdown-toggle
 .dropdown-menu
 .dropdown-item
 .dropdown-text
 .dropdown-link
 .dropdown-divider

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Carousel</title>
  <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="./node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body class="container-fluid">
  <h2>Netflix</h2>
  <div class="dropdown">
    <button data-bs-toggle="dropdown" class="btn btn-dark dropdown-toggle">
      <span class="bi bi-globe"></span>
      Language
    </button>
    <ul class="dropdown-menu">
      <li class="dropdown-item"><span class="dropdown-item-
text">English</span></li>
      <li class="dropdown-item"><span class="dropdown-item-text">हिंदी
</span></li>
      <li class="dropdown-divider"></li>
      <li class="dropdown-item"><span class="dropdown-item-
text">Help?</span></li>
    </ul>
  </div>
  <script src="./node_modules/jquery/dist/jquery.js"></script>
  <script src="./node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>
```

Navbar

- It is used to create a responsive navigation bar.

Classes:

.navbar

.nav-bar-expand-lg | sm
 .nav-bar-brand
 .nav-bar-toggler
 .nav-bar-toggler-icon
 .nav-bar-collapse
 .nav-bar-nav
 .nav-menu
 .nav-item
 .nav-link
 .bg-{contextual}(dark/light/primary)
 .nav-bar-dark
 .nav-bar-light

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Carousel</title>
  <link rel="stylesheet"
href="../node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body class="container-fluid">
  <div class="nav-bar nav-bar-expand-lg nav-bar-dark bg-dark">
    <div class="container-fluid">
      <button class="nav-bar-toggler" data-bs-toggle="collapse" data-bs-
target="#NavItems">
        <span class="nav-bar-toggler-icon"></span>
      </button>
      <a href="#" class="nav-bar-brand">Amazon</a>
      <div class="nav-bar-collapse collapse" id="NavItems">
        <ul class="nav-bar-nav">
          <li class="nav-item">
            <a href="#" class="nav-link">Home</a>
          </li>
          <li class="nav-item">
            <a href="#" class="nav-link">Kid's</a>
          </li>
          <li class="nav-item">

```

```

        <a href="#" class="nav-link">Men's</a>
    </li>
    <li class="nav-item">
        <a href="#" class="nav-link">Women's</a>
    </li>
    <li class="nav-item">
        <div class="input-group">
            <input type="text" class="form-control">
            <button class="btn btn-warning">
                <span class="bi bi-search"></span>
            </button>
        </div>
    </li>
    <li class="nav-item">
        <a href="#" class="nav-link">
            <span class="bi bi-person-fill"></span>
        </a>
    </li>
    <li class="nav-item">
        <a href="#" class="nav-link">
            <span class="bi bi-cart4"></span>
        </a>
    </li>
</ul>
</div>
</div>
</div>
<script src="./node_modules/jquery/dist/jquery.js"></script>
<script src="./node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Nav with Tabs

- It is used to create tab based navigation to display various categories of content on screen.

```

.nav
.nav-tabs
.nav-pills
.nav-menu
.nav-item
.nav-link
.nav-text

```

- To design tab pages

.tab-content

.tab-pane

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Carousel</title>
  <link rel="stylesheet"
href=" ../node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href=" ../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body class="container-fluid">
  <h2>Product Details</h2>

  <ul class="nav nav-tabs">
    <li class="nav-item">
      <a href="#basic" data-bs-toggle="tab" class="nav-link"> <span
class="nav-text">Basic Details</span> </a>
    </li>
    <li class="nav-item">
      <a href="#preview" data-bs-toggle="tab" class="nav-link"> <span
class="nav-text"></span> </a>
    </li>
    <li class="nav-item">
      <a href="#description" data-bs-toggle="tab" class="nav-link"> <span
class="nav-text">Description</span> </a>
    </li>
  </ul>
  <div class="tab-content">
    <div class="tab-pane active" id="basic">
      <dl>
        <dt>Name</dt>
        <dd>RealMe Mobile</dd>
```

```

        <dt>Price</dt>
        <dd>23000.44</dd>
    </dl>
</div>
<div class="tab-pane" id="preview">
    
</div>
<div class="tab-pane" id="description">
    <p>Something about realme mobile..</p>
</div>
</div>

<script src="../node_modules/jquery/dist/jquery.js"></script>
<script src="../node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Progress Bar

Classes:

```

.progress
.progress-bar
.progress-bar-striped
.progress-bar-animated
.bg-{contextual}

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Carousel</title>
    <link rel="stylesheet"
href="../node_modules/bootstrap/dist/css/bootstrap.css">

```



```

    <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body class="container-fluid">
    <h2>HTML Progress</h2>
    <progress></progress>
    <h2>Bootstrap Progress</h2>
    <div class="progress">
        <div class="progress-bar progress-bar-animated progress-bar-striped"
style="width: 40%;">
            40% Completed
        </div>
        <div class="progress-bar progress-bar-animated progress-bar-striped bg-
warning" style="width: 60%;">
            60% Remaining
        </div>
    </div>
    <script src="../node_modules/jquery/dist/jquery.js"></script>
    <script src="../node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Spinners

Classes:

```

.spinner-border
.spinner-grow
.spinner-border-sm|lg
.spinner-grow-sm|lg

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Carousel</title>
    <link rel="stylesheet"

```

```

href="../node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body class="container-fluid">
  <h2>Spinners</h2>
  <button class="btn btn-warning">
    <span class="spinner-border spinner-border-sm text-danger"></span>
Loading
  </button>
  <button class="btn btn-warning">
    <span class="spinner-grow spinner-grow-sm text-danger"></span> Loading
  </button>
  <script src="../node_modules/jquery/dist/jquery.js"></script>
  <script src="../node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Bread Crumb

Class:

.breadcrumb
.breadcrumb-item

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Carousel</title>
  <link rel="stylesheet"
href="../node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>

```

```

<body class="container-fluid">
  <ul class="breadcrumb">
    <li class="breadcrumb-item">Amazon</li>
    <li class="breadcrumb-item">Home</li>
    <li class="breadcrumb-item">Electronics</li>
    <li class="breadcrumb-item">Mobiles</li>
    <li class="breadcrumb-item active">Samsung</li>
  </ul>
  <script src="./node_modules/jquery/dist/jquery.js"></script>
  <script src="./node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Pagination

Classes

```

.pagination
.page-item
.page-link
.page-text

```

Tables

Classes

```

.table
.table-hover
.table-bordered
.table-striped
.table-border-less
.table-contextual
.table-responsive
.caption-top
.caption-bottom

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

```

[illegible]

```

</thead>
<tbody>
  <tr>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
  </tr>
  <tr>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
  </tr>
  <tr>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
  </tr>
  <tr>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
  </tr>
  <tr>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
  </tr>
  <tr>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
  </tr>
  <tr>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
  </tr>
  <tr>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
  </tr>
  <tr>
    <td colspan="3">
      <ul class="pagination">
        <li class="page-item"><a class="page-link"
href="#">&laquo;</a></li>
        <li class="page-item"><a href="#" class="page-link">1</a></li>
        <li class="page-item active"><a href="#" class="page-
link">2</a></li>
        <li class="page-item"><a href="#" class="page-link">3</a></li>

```

```

        <li class="page-item"><a href="#" class="page-link">4</a></li>
        <li class="page-item"><a href="#" class="page-
link">&raquo;</a></li>
    </ul>
</td>
</tr>
</tfoot>
</table>
</div>
<script src="../node_modules/jquery/dist/jquery.js"></script>
<script src="../node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Summary - Bootstrap Components

- alerts
- badge
- card
- carousel
- dropdown
- breadcrumb
- accordion
- collapse
- navbar
- nav and tabs
- modal
- form
- button
- button group
- tables
- pagination
- progress
- spinners

Sass

[Syntactically Awesome Style Sheet]

- Sass is a CSS pre-processor.

- Sass provides several programming features that makes styles more easier for developers.
- Browser can't understand Sass, you have to trans compile sass into css.
- In real-world progress common CSS pre-processors are

a) Sass

b) Less

- Download and Install Sass on your PC

```
C:\>npm install -g sass
```

- Add a new file into project
effects.scss

```
.container {
  border:1px solid red;
  box-shadow: 2px 2px 2px gray;
  background-color: yellow;
  padding: 10px;
  h2 {
    color:red;
  }
  p {
    color:blue;
  }
}
```

- Open File location in Terminal

```
> sass effects.scss effects.css
```

- Link CSS file to your index.html

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="effects.css">
</head>
<body>
```

```
<div class="container">
  <h2>Sass</h2>
  <p>Syntactically Awesome Style Sheet</p>
</div>

</body>
</html>
```

- Sass files will have extension ".scss"

```
effects.scss
.container {
  .....
  h2 {
  }
}
```

- Trans Compile scss into css

```
>sass effects.scss effects.css
```

Sass Variables

- Variables are storage locations in memory, where you can store a value and use it as a part of any expression.
- Variable allows to re-use values.
- Sass variables are declared by using "\$"

Syntax:

```
$variableName : value;
```

- You have to access and use the value with reference of variable name.

```
box-shadow : $variableName;
```

- Variable Name Rules

- * Name must start with an alphabet or _ .
- * Name can be alpha numeric, but can't start with number.
- * Name can't have special chars or spaces, only underscore allowed.
- * Name can be max 255 chars long.
- * Name must be unique.

Syntax:

```
$shadow: 2px 2px 3px black;
```

```
.container {  
  box-shadow : $shadow;  
}  
h2 {  
  text-shadow: $shadow;  
}
```

Ex:

effects.scss

```
$shadow : 2px 3px 3px yellow;  
$border : 2px solid red;  
$padding: 20px;
```

```
.container {  
  border: $border;  
  box-shadow: $shadow;  
  padding: $padding;  
  h2 {  
    color:red;  
    padding: $padding;  
    text-shadow: $shadow;  
    border:$border;  
  }  
}
```

```
> sass effects.scss effects.css
```

index.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Document</title>  
  <link rel="stylesheet" href="effects.css">  
</head>
```

```

<body>
  <div class="container">
    <h2>Sass</h2>
    <p>Syntactically Awesome Style Sheet</p>
  </div>
</body>
</html>

```

- Variables are block scoped. You can access only in the block where it is declared.

- If you want global scope, then declare variables outside block.

```

.container
{
  $varName : values;      // block scope
  border : $varName;
}
h2
{
  border: $varName;      // invalid
}

```

```

$varName : values;      // Global Scope
.container
{
  border : $varName;
}
h2
{
  border: $varName;      // valid
}

```

- Variable declaration must be before using the variable.

```

.container {
  color: $color;          // invalid - you can't use before declaring.
}
$color: red;

```

Sass Mixins and Functions

- Mixin is used for refactoring the code.
- Refactoring is the process of encapsulating a set of line and extracting into a method.
- You can create a method that contains set of attributes, which you can reuse for any element.

Syntax:

```
@mixin MethodName
{
    // attributes..
}

.container {
    @include MethodName;
}
h2 {
    @include MethodName;
}
```

Ex:

effects.scss

```
$shadow: 2px 2px 2px black;
$border: 2px solid red;
$padding : 20px;
```

```
@mixin LayoutEffects
{
    border: $border;
    box-shadow: $shadow;
    padding: $padding;
}
```

```
.container {
    @include LayoutEffects;
}
h2 {
    @include LayoutEffects;
    color:red;
}
```

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="effects.css">
</head>
<body>
  <div class="container">
    <h2>Sass</h2>
    <p>Syntactically Awesome Style Sheet</p>
  </div>
</body>
</html>
```

- Every mixin can be

- a) Parameterized
- b) Parameter less

- Parameterized mixin allows to modify the functionality.

Syntax:

```
@mixin MethodName(param1, param2,...)
{
  attribute: param1;
}
```

```
@include MethodName(val1, val2);
```

Ex:

effects.scss

```
@mixin LayoutEffects($shadowColor, $borderSize , $borderColor, $padding)
{
  border: $borderSize solid $borderColor;
  box-shadow: 2px 2px 2px $shadowColor;
  padding: $padding;
```

```
}
```

```
.container {  
  @include LayoutEffects(black, 2px , black, 20px);  
  h2 {  
    @include LayoutEffects(red, 1px , blue , 10px);  
  }  
}
```

Sass Variables

Sass Mixins

Sass Function

- Function is similar to mixin but it have to return a value.
- Function is used to build an expression.
- Expression returns a value.

FAQ: What is difference between function and mixin?

Ans : Mixin will not return any value, it is void type.
Function will return value.

- Function is defined by using "@function"

Syntax:

```
@function Name  
{  
}  
}
```

- Function can be parameterized or parameter less

Syntax:

```
@function Name(params)  
{  
}  
}
```

- Functions are accessed directly by reference name

border : functionName(value);

Ex:

```
$borderSize : 2px;
```

```
@function CalculateBorder($size){  
  @return $borderSize*$size;  
}
```

```
.container {  
  border: CalculateBorder(2) solid black;  
  h2 {  
    border: CalculateBorder(0.5) dotted red;  
  }  
}
```

Built-in Functions

@if

@else

@for

@if function

- It is used in decision making.
- It is used for conditional rendering.
- It can take decision and render one set of attributes when condition evaluates to true and another set of attribute when it is false.

Syntax:

```
@if condition1  
{  
  attributes to apply when condition1 is true;  
}  
@else if condition2  
{  
  attributes to apply when condition2 is true;  
}  
@else  
{  
  attributes to apply when app conditions evaluate to false;  
}
```

Ex:

effects.scss

```
$theme : yellow;
```

```
@mixin ApplyTheme($borderColor, $bgColor, $hColor, $pColor)
```

```
{  
  .container {  
    border: 2px solid $borderColor;  
    background-color: $bgColor;  
    padding: 20px;  
    h2 {  
      color: $hColor;  
    }  
    p {  
      color: $pColor;  
    }  
  }  
}
```

```
@if ($theme==red)
```

```
{  
  @include ApplyTheme(darkred, rgb(254, 185, 185) , rgb(216, 4, 4) , red );  
}  
@else if($theme==green) {  
  @include ApplyTheme(darkgreen, rgb(190, 255, 190) , rgb(3, 179, 3) , rgb(48,  
156, 48) );  
}  
@else {  
  @include ApplyTheme(black, lightgray , rgb(99, 98, 98) , black);  
}
```

@for function

- It is an iterator.
- Iterator executes a set of statement repeatedly until the condition is matching.

Syntax:

```
@for ref from initialValue through finalValue  
{  
}  
}
```

- You have to embed any dynamic value with string by using "#{ }"

```
$i=1;  
.box-#{ $i }      .box-1
```

Ex:

effects.scss

```
div {
  width: 100px;
  height: 100px;
  border: 2px solid black;
}
img {
  width: 100px;
  height: 100px;
}
@for $i from 1 through 4
{
  .box-#{ $i } {
    margin-left: $i*100px;
  }
}
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="effects.css">
</head>
<body>
  <div class="box-1">
    
  </div>
  <div class="box-2">
    
  </div>
  <div class="box-3">
    
  </div>
  <div class="box-4">
    
  </div>
```



```
</body>
</html>
```

Sass Modules

- Module is a set of variables, functions and mixins.
- Module is used to build a library.
- Library is used in computer programming to reuse the values or functions or mixins accross pages.
- It allows to bundle the files [bundling in CSS]
- Every module file must start with "_"

```
"_alert.scss"
```

Ex:

```
_variables.scss
```

```
$shadow: 2px 2px 2px black;
$border: 2px solid black;
$padding: 10px;
```

```
_mixins.scss
```

```
@import "variables.scss";
```

```
@mixin ApplyEffects()
{
    .container {
        box-shadow: $shadow;
        border:$border;
        padding: $padding;
    }
}
```

```
effects.scss
```

```
@import "mixins.scss";
@include ApplyEffects();
```

JAVA SCRIPT

JavaScript

- JavaScript is light weight, Just-in-Time compiled and interpreted programming language.
- Interpreted language translates line-by-line.
- Compiler translates all lines simultaneously at the same time.

- There are 2 compiling techniques

a) JIT

b) AOT

- JIT is Just-in-Time, It compiles the program in browser.
- AOT is Ahead-of-Time It compiles the program at application level.
- JIT and AOT is handled by using various compilers of JavaScript like

a) V8

b) Babel

- JavaScript is a language, which can be used

Client Side	HTML
Server Side	NodeJS
Database	MongoDB
Animations	3DSMax, CAD Systems etc..

- JavaScript is a language that supports various programming approaches

- 1]. Functional Programming
- 2]. Structural Programming
- 3]. Imperative Programming
- 4]. Object Oriented Programming etc..

- JavaScript is not OOP language. It supports only few features of OOP.

History of JavaScript

- Internet started with a language for scripting and a language for markup

GML, SGML	Markup
ECMA Script	Scripting
Mosaic	Browser

- HTML also started using ECMA Script in browser.

- Netscape communications introduced a browser called Netscape Communicator.

HTML - ECMAScript

- Netscape appointed "Brendan Eich" to develop a script for browser. [1994]
- He developed a script and named it as "Mocha".
- After few months it was renamed to "Live Script".
- Netscape given the Live Script maintenance to a company called "Sun Microsystems".
- Sun Micro Systems company changed the name of Live Script to JavaScript.
- 2000 Netscape stopped its services.
- Netscape give the JavaScript to ECMA.
- JavaScript version is ECMA version.
 - ECMAScript 2014, 2015, 2016, 2017, 2018, ... 2022
 - ES4, ES5, ES6, ES7
- MDN is official for JavaScript.

JavaScript is used with HTML Client Side

- HTML presents DOM.
- HTML DOM is static.
- JavaScript converts the static DOM into dynamic.
- JavaScript is used with HTML for

a) DOM Manipulations

- Adding Elements
- Removing Elements
- Updating data into Elements

b) Client Side Interactions

c) Client Side Validations

Features and Drawbacks of JS

JavaScript is used as client side script with HTML to reduce the burden on server.

- It can reduce burden on server by handling all interactions client side.

DrawBacks

- JavaScript is not strongly typed language.
 - var x = 10;
 - x = "A";
- JavaScript is not implicitly strictly typed.

- JavaScript is not an OOP language, It supports only few features of OOP.
- If its not completely OOP
 - a) Extensibility Issues
 - b) Code Security Issues
 - c) No Dynamic Polymorphism

Solution:

- TypeScript => JavaScript
- TypeScript belongs to Microsoft

FAQ: What is a Script?

Ans: A script can bound to any technology and define a functionality, or It can individually build an application.

ECMA Script 2017

[ES7]

1. How JavaScript is integrated into HTML page?

A. JavaScript can be

- a) Inline
- b) Embedded
- c) External File

Inline

- JavaScript functions are defined for every element individually.
- It is fast but can't re-use.

Syntax:

`<button onclick="window.print()"> Print </button>`

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Inline</title>
</head>
```

```
<body>
  <h2>Click Print Button to Print Page</h2>
  <button onclick="window.print()">Print</button>
</body>
</html>
```

Embedded

- You can embed script in page by using <script> element
- You can embed in <head> or <body> section
- JavaScript MIME type is "text/javascript" (or) language="javascript"

Syntax:

```
<head>
  <script type="text/javascript" or language="javascript">

  </script>
</head>
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Inline</title>
  <script type="text/javascript">
    function PrintPage(){
      window.print();
    }
  </script>
</head>
<body>
  <h2>Click Print Button to Print Page</h2>
  <button onclick="PrintPage()">Print</button>
</body>
</html>
```

External File

- JavaScript function can be in an external file with extension ".js"
- JavaScript file are linked to a page by using <script> element.

Syntax:

```
<script type="text/javascript" src="printing.js"> </script>
```

Ex:

1. Add a new file "printing.js"

```
function PrintPage()
{
    window.print();
}
```

2. Link to your HTML page

```
<head>
    <script type="text/javascript src="printing.js"> </script>
</head>
<body>
    <button onclick="PrintPage()"> Print </button>
</body>
```

FAQ: How to check the status of JavaScript in browser?

Ans : By using HTML element `<noscript>`.

```
<body>
    <noscript> Please enable JavaScript in your browser </noscript>
</body>
```

Strict Mode for JavaScript

- JavaScript is not implicitly strictly typed.
- You have to **turn ON** strict mode by using `"use strict";`

Example:

```
<script>
    x = 10;                // Valid in JavaScript
    document.write("X=" + x);
</script>
```

```
<script>
    "use strict";
```

```
x = 10; // Invalid - x is not defined
document.write("X=" + x);
</script>
```

How JavaScript refers HTML elements?

1. JavaScript can refer HTML elements by using DOM hierarchy.

```
window.document.images[]
window.document.forms[].elements[]
```

- Index issues, if you change the position of any element then you have to update the index number in logic every time.
- Hierarchy issues, you have to refer all the parent and child elements in a sequential order.

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script type="text/javascript">
    function bodyload(){
      window.document.images[0].src = "../public/images/asp.jpg";
      window.document.forms[0].elements[0].value = "Register";
      window.document.forms[1].elements[1].value = "Login";
    }
  </script>
</head>
<body onload="bodyload()">
  <div>
    <img width="100" height="100" border="1">
  </div>
  <div>
    <form>
      <h2>Register</h2>
```

```

        Your Name : <input type="button"> <input type="text">
    </form>
</div>
<div>
    <form>
        <h2>Login</h2>
        Your Email : <input type="email"> <input type="button">
    </form>
</div>
</body>
</html>

```

2. JavaScript can refer elements by using name.

```
<img name="pic">
```

```
pic.src="";
```

- You can't refer any child element directly.
- You have to refer the child element through the parent.

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script type="text/javascript">
        function bodyload(){
            pic.src= "../public/images/dm.jpg";
            frmRegister.btnRegister.value = "Register";
            frmLogin.btnLogin.value = "Login";
        }
    </script>
</head>
<body onload="bodyload()">
    <div>
        <img width="100" height="100" border="1" name="pic">
    </div>
</div>

```



```

    <form name="frmRegister">
      <h2>Register</h2>
      Your Name : <input type="text"> <input type="button"
name="btnRegister">
    </form>
  </div>
  <div>
    <form name="frmLogin">
      <h2>Login</h2>
      Your Email : <input type="email"> <input name="btnLogin"
type="button">
    </form>
  </div>
</body>
</html>

```

3. JavaScript can refer elements by using their ID.

document.getElementById()

- You can refer any child element directly.

```
<img id="pic">
```

```
document.getElementById("pic").src = " ";
```

- ID is also used by CSS.
- CSS ID can be common for many elements.
- JavaScript ID can't be common.

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script type="text/javascript">
    function bodyload(){
      document.getElementById("pic").src = "../public/images/asp.jpg";
      document.getElementById("btnRegister").value = "Register";
    }
  </script>

```

```

        document.getElementById("btnLogin").value = "Login";
    }
</script>
</head>
<body onload="bodyload()">
    <div>
        <img width="100" id="pic" height="100" border="1" name="pic">
    </div>
    <div>
        <form name="frmRegister">
            <h2>Register</h2>
            Your Name : <input type="text"> <input id="btnRegister" type="button"
name="btnRegister">
        </form>
    </div>
    <div>
        <form name="frmLogin">
            <h2>Login</h2>
            Your Email : <input type="email"> <input id="btnLogin"
name="btnLogin" type="button">
        </form>
    </div>
</body>
</html>

```

4. JavaScript can refer HTML elements by using CSS selectors

- JavaScript can use the method "document.querySelector()" to refer any css selector.

```

<img>
<p id="p1">
<input type="button" class="btn-primary">

document.querySelector("img")    // type selector
document.querySelector("#p1")
document.querySelector(".btn-primary")
document.querySelector("nav+div");

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>

```

```

<meta charset="UTF-8">
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<script type="text/javascript">
    function bodyload(){
        document.querySelector("img").src= "../public/images/dm.jpg";
        document.querySelector("#btnRegister").value = "Register";
        document.querySelector(".btn-primary").value = "Login";
    }
</script>
</head>
<body onload="bodyload()">
    <div>
        <img width="100" id="pic" height="100" border="1" name="pic">
    </div>
    <div>
        <form name="frmRegister">
            <h2>Register</h2>
            Your Name : <input type="text"> <input id="btnRegister" type="button"
name="btnRegister">
        </form>
    </div>
    <div>
        <form name="frmLogin">
            <h2>Login</h2>
            Your Email : <input type="email"> <input id="btnLogin" class="btn-
primary" name="btnLogin" type="button">
        </form>
    </div>
</body>
</html>

```

Note: JavaScript provides several methods to access multiple elements

```

collection = document.getElementsByTagName("p", "div", "form",...);
collection.length = 2

```

- `getElementsByTagName()`
- `getElementsByClassName()`
- `getElementsByName()`

- JavaScript Input and Output functions

JavaScript Output Techniques

1. alert()
2. confirm()
3. document.write()
4. console methods
 log(), warn(), error(), debug(), info()
5. innerHTML
6. outerHTML
7. innerText

alert

- It pops up a message box and can display message.
- It will not have any option for cancel.
- You have to confirm only with OK.

Syntax:

```
alert("your message");    // single line
alert("Line-1\nLine2");    // multi line
```

\n line break

- alert is RC data type.

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<script type="text/javascript">
    function DeleteClick(){
        alert("Delete Record\nRecord will be deleted\nAre you sure?");
    }
</script>
</head>
<body>
    <button onclick="DeleteClick()">Delete</button>
</body>
</html>

```

Confirm

- It is same like alert but allows to cancel.
- Confirm is a boolean type, it returns true on OK and false on Cancel.

Syntax:

```
flag = confirm("your message");
```

flag = true => on OK

flag = false => on Cancel

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script type="text/javascript">
        function DeleteClick(){
            flag = confirm("Delete Record\nRecord will be deleted\nAre you sure?");
            if(flag==true)
            {
                alert("Record Deleted Successfully..");
            } else {
                alert("You canceled..");
            }
        }
    </script>

```

```
</head>
<body>
  <button onclick="DeleteClick()">Delete</button>
</body>
</html>
```

document.write()

- It can display output on a new screen.
- It allows to present a markup or plain string.

syntax:

```
document.write("your message");
document.write(<markup> </markup>);
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script type="text/javascript">
    function DeleteClick(){
      flag = confirm("Delete Record\nRecord will be deleted\nAre you sure?");
      if(flag==true)
      {
        document.write("<font color='red'><b><i>Record Deleted
Successfully..</b></i></font><br><a href='output.html'>back</a>");
      } else {
        alert("You canceled..");
      }
    }
  </script>
</head>
<body>
  <button onclick="DeleteClick()">Delete</button>
</body>
</html>
```

innerText | innerHTML | outerHTML

- innerText : display output in any container element that can show string.
It can't used formats.
It is RC data type.

- innerHTML : It is same as innerText but supports all formats with markup or methods.

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script type="text/javascript">
    function DeleteClick(){
      flag = confirm("Delete Record\nRecord will be deleted\nAre you sure?");
      if(flag==true)
      {
        document.querySelector("p").innerHTML = "<font size='5'
color='red'>Record Deleted Successfully..</font>";
      } else {
        alert("You canceled..");
      }
    }
  </script>
</head>
<body>
  <button onclick="DeleteClick()">Delete</button>
  <div style="display: flex; justify-content: center; align-items: center; height:
400px;">
    <p></p>
  </div>
</body>
</html>
```

- outerHTML : It can replace the existing element with new element.

Syntax:

```
document.querySelector("h1").outerHTML = "<p> </p>";
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script type="text/javascript">
    function DeleteClick(){
      flag = confirm("Delete Record\nRecord will be deleted\nAre you sure?");
      if(flag==true)
      {
        document.querySelector("h1").outerHTML = "<p><font size='5'
color='red'>Record Deleted Successfully..</font></p>";
      } else {
        alert("You canceled..");
      }
    }
  </script>
</head>
<body>
  <button onclick="DeleteClick()">Delete</button>
  <div style="display: flex; justify-content: center; align-items: center; height:
400px;">
    <h1></h1>
  </div>
</body>
</html>
```

Console Methods

- There are the output methods used by developers to display output in browser console.
- They are used from tracking and logging errors, warnings, debug messages etc..

Syntax:

```
console.warn("message");
console.error("message");
console.log("message");
```


Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script type="text/javascript">
    function DeleteClick(){
      console.warn("Delete Button Clicked");
      flag = confirm("Delete Record\nRecord will be deleted\nAre you sure?");
      if(flag==true)
      {
        console.error("OK Clicked..");
        document.querySelector("h1").outerHTML = "<p><font size='5'
color='red'>Record Deleted Successfully..</font></p>";
      } else {
        alert("You canceled..");
        console.log("Cancel Clicked..");
      }
    }
  </script>
</head>
<body>
  <button onclick="DeleteClick()">Delete</button>
  <div style="display: flex; justify-content: center; align-items: center; height:
400px;">
    <h1></h1>
  </div>
</body>
</html>
```

JavaScript Input Technique

1. prompt()

2. form input elements

3. query string

1 .prompt()

- It is similar to a message box [alert , confirm].
- It is technically an input box that allows plain string input.

Syntax:

```
prompt("Your Message", "Default_Value");
```

- Prompt returns

" "	If no value provided and clicked OK
null	If it is canceled
string	If value provided and OK

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
  <link rel="stylesheet"
href="../node_modules/bootstrap/dist/css/bootstrap.css">
  <script type="text/javascript">
    function NewClick(){
      folder = prompt("Enter Folder Name","New_Folder_Name");
      msg = document.getElementById("msg");
      if(folder==null) {
        alert("You canceled..");
      }else if(folder=="") {
        alert("Please Enter a Name");
      } else {
        msg.innerHTML += "Folder Created : " + folder + "<br>";
      }
    }
  </script>
</head>
<body class="container-fluid">
  <button class="btn" onclick="NewClick()"> <span class="bi bi-file-earmark-
plus"></span> New Folder </button>
  <p id="msg"></p>
</body>
```

</html>

2.JavaScript Input using Form Elements

- HTML <form> provides various input elements like textbox, checkbox, radio, dropdown, listbox etc..
- You can design a form with input elements and accept input from Form.

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form Input</title>
  <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="./node_modules/bootstrap-icons/font/bootstrap-
icons.css">
  <script type="text/javascript">
    function RegisterClick(){
      document.getElementById("registerContainer").style.display = "none";
      document.getElementById("detailsContainer").style.display = "block";

      document.getElementById("lblName").innerHTML =
document.getElementById("Name").value;
      document.getElementById("lblPrice").innerHTML =
document.getElementById("Price").value;
      document.getElementById("lblCity").innerHTML =
document.getElementById("City").value;

      stockStatus = "";
      stockCheckBox = document.getElementById("Stock");
      if(stockCheckBox.checked)
      {
        stockStatus = "Available";
      } else {
```

```

        stockStatus = "Out of Stock";
    }

    document.getElementById("lblStock").innerHTML = stockStatus;
}
function EditClick(){
    document.getElementById("btnRegister").innerHTML = "Save";
    document.getElementById("btnRegister").className = "btn btn-
success";
}
</script>
</head>
<body class="container-fluid">
    <div id="registerContainer" class="mt-3">
        <button data-bs-target="#register" data-bs-toggle="modal" class="btn btn-
primary">Register Product</button>
    </div>
    <div id="detailsContainer" style="display: none;">
        <h3>Product Details</h3>
        <dl>
            <dt>Name</dt>
            <dd id="lblName"></dd>
            <dt>Price</dt>
            <dd id="lblPrice"></dd>
            <dt>City</dt>
            <dd id="lblCity"></dd>
            <dt>Stock</dt>
            <dd id="lblStock"></dd>
        </dl>
        <div>
            <button onclick="EditClick()" data-bs-target="#register" data-bs-
toggle="modal" class="btn btn-warning"> <span class="bi bi-pencil-
square"></span> Edit</button>
            <a class="ms-4 btn btn-info" href="form-input.html">New Product</a>
        </div>
    </div>
    <div class="modal fade" id="register">
        <div class="modal-dialog">
            <div class="modal-content">
                <div class="modal-header">
                    <h3>Register Product</h3>
                    <button class="btn-close" data-bs-dismiss="modal"></button>
                </div>

```

```

<div class="modal-body">
  <dl>
    <dt>Product Name</dt>
    <dd><input type="text" id="Name" class="form-control"></dd>
    <dt>Price</dt>
    <dd><input type="text" id="Price" class="form-control"></dd>
    <dt>Shipped To</dt>
    <dd>
      <select id="City" class="form-select">
        <option>Delhi</option>
        <option>Hyd</option>
      </select>
    </dd>
    <dt>Stock</dt>
    <dd class="form-switch">
      <input class="form-check-input" type="checkbox" id="Stock">
Available
    </dd>
  </dl>
</div>
<div class="modal-footer">
  <button id="btnRegister" onclick="RegisterClick()" class="btn btn-
primary" data-bs-dismiss="modal">Register</button>
  <button class="btn btn-danger" data-bs-
dismiss="modal">Cancel</button>
</div>
</div>
</div>
<script src="../node_modules/jquery/dist/jquery.js"></script>
<script src="../node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Inox</title>
  <link rel="stylesheet"

```

```

href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="./node_modules/bootstrap-icons/font/bootstrap-
icons.css">
  <script type="text/javascript">
    function BookClick(){
      document.getElementById("toolBar").style.display="none";
      document.getElementById("summary").style.display="block";

      document.getElementById("lblMovie").innerHTML =
document.getElementById("Movie").value;
      document.getElementById("lblCinema").innerHTML =
document.getElementById("Cinema").value;
      document.getElementById("lblDate").innerHTML =
document.getElementById("Date").value;
      document.getElementById("lblTiming").innerHTML =
document.getElementById("Timing").value;
      document.getElementById("lblSeats").innerHTML =
document.getElementById("Seats").value;

      movieName = document.getElementById("Movie").value;
      if(movieName=="Avatar : Way of Water"){

document.getElementById("poster").src="./public/images/avatar1.jpg";
      }
      if(movieName=="Pathaan") {

document.getElementById("poster").src="./public/images/pathaan.jpg";
      }

    }
  </script>
</head>
<body class="container-fluid">
  <div id="toolBar" class="mt-3 d-flex justify-content-between bg-dark p-2">
    <div>
      <select class="form-select" id="Movie">
        <option>Select Movie</option>
        <option>Avatar : Way of Water</option>
        <option>Pathaan</option>
      </select>
    </div>

```

```
<div>
  <select class="form-select" id="Cinema">
    <option>Select Cinema</option>
    <option>GVK B'Hills</option>
    <option>GVK Ameerpet</option>
  </select>
</div>
<div>
  <select class="form-select" id="Date">
    <option>Select Date</option>
    <option>Today 1-Feb</option>
    <option>Tomorrow 2-Feb</option>
  </select>
</div>
<div>
  <select class="form-select" id="Timing">
    <option>Select Timing</option>
    <option>10:30 AM</option>
    <option>07:30 PM</option>
  </select>
</div>
<div>
  <select class="form-select" id="Seats">
    <option>Select Seats</option>
    <option>1</option>
    <option>2</option>
    <option>3</option>
    <option>4</option>
  </select>
</div>
<div>
  <button onclick="BookClick()" class="btn w-100 btn-
danger">Book</button>
</div>
</div>
<div id="summary" class="mt-3" style="display: none;">
  <dl style="width:250px">
    <h3 class="bg-dark text-white p-1">Booking Summary</h3>
    <dt>
      <img id="poster" width="100%" height="200">
    </dt>
    <dt>Movie Name</dt>
    <dd id="lblMovie"></dd>
```

```

        <dt>Cinema</dt>
        <dd id="lblCinema"></dd>
        <dt>Date</dt>
        <dd id="lblDate"></dd>
        <dt>Timing</dt>
        <dd id="lblTiming"></dd>
        <dt>Seats</dt>
        <dd id="lblSeats"></dd>
    </dl>
</div>
</body>
</html>

```

JavaScript Language Basics

1. Variables

2. Data Types

3. Operators

4. Statements

5. Functions

Variables

- Variables are storage locations in memory where you can store a value and use it as a part of any expression.

- Variable configuration comprises of 3 phases

a) Declaring

b) Assigning

c) Initialization

```

var username;           // declaring
username = "John";      // assigning

```

```

var age = 22;           // Initialization

```

- JavaScript variable declaration is mandatory only when it is "Strict Mode".

Syntax:

```

<script>
    x = 10;                // valid
    document.write("x=" + x);

```



```
</script>
```

```
<script>
  "use strict";
  x = 10;           // invalid - x is not defined
  document.write("x=" + x);
</script>
```

- JavaScript variables are declared by using 3 keywords

- a) var
- b) let
- c) const

Syntax:

```
var x = 10;
let x = 10;
const x = 10;
```

FAQ: What is difference between var, let and const ?

var

- It defines function scope variable.
- You can declare in any block of a function and you can access from any another block inside function.
- It allows declaring, assigning and initialization.

Ex:

```
<script>
  function f1()
  {
    var x;      // declaring
    x = 10;     // assigning
    if(x==10)
    {
      var y = 20; // initialization
    }
    document.write("x=" + x + "<br>" + "y=" + y);
  }
  f1();
</script>
```

- var allows shadowing.
- shadowing is the process of re-declaring or re-initializing a variable within the scope. [same name variable]

Ex:

```
<script>
  function f1()
  {
    var x;      // declaring
    x = 10;     // assigning
    var x;      // shadowing
    x = 20;
    if(x==20)
    {
      var y = 20; // initialization
      var y = 50; // shadowing
    }
    document.write("x=" + x + "<br>" + "y=" + y);
  }
  f1();
</script>
```

- var supports hoisting.
- Hoisting allows to use a variable before declaring.
- You can use and declare later. The declaration of variable can be after assignment.

Ex:

```
<script>
  "use strict";
  function f1()
  {
    x = 10;
    document.write("x=" + x);
    var x;
  }
  f1();
</script>
```

Let

- It defines a block scope variable.
- A block scope variable is accessible only within the specified block and its

inner blocks.

- It support declaring, initialization and assignment.

Ex:

```
<script>
  "use strict";
  function f1()
  {
    let x;      // declaring
    x = 10;     // assigning
    if(x==10)
    {
      let y = 20; // initialization
      document.write("x=" + x + "<br>" + "y=" + y);
    }
  }
  f1();
</script>
```

- It will not allow shadowing and hoisting.

const

- It is also block scope.

- It allows only initialization, no declaring, no assigning.

- No shadowing and hoisting.

Ex:

```
<script>
  "use strict";
  function f1()
  {
    const x = 5;
    document.write("x=" + x);
  }
  f1();
</script>
```

JavaScript Variables

- Declaration

- Assignment
- Initialization
- var
- let
- const

Global Scope for Variable

- If you want a variable to access from any function, then you have to define as Global.
- Declaring or initializing variable outside function and within script or module is Global.

<pre> <script> var x; // global function f1() { access x } function f2() { access x } </script> </pre>	demo.js	<pre> module var x; function f1() { } </pre>
---	----------------	--

- Global variables can be defined with var, let and const.

Ex:

```

<script>
  var x = 10;
  let y = 20;
  const z = 30;
  function f1(){
    document.write("function 1 x=" + x + "<br>" + "y=" + y + "<br>" + "z=" + z +
    "<br>");
  }
  function f2(){
    document.write("function 2 x=" + x + "<br>" + "y=" + y + "<br>" + "z=" + z +
    "<br>");
  }
  f1();
  f2();
</script>

```

FAQ: Can we declare a global variable in side function?

Ans : Yes if you are using JavaScript with HTML DOM

You can define global variable inside function using "window" object.
[window is a DOM object].

Syntax:

window.variableName = value;

Ex:

```
<script>
  "use strict";
  var x = 10;
  let y = 20;
  const z = 30;

  function f1(){
    window.a = "John";
    document.write("function 1 <br> x=" + x + "<br>" + "y=" + y + "<br>" + "z="
+ z + "<br>Name=" + a + "<br>");
  }
  function f2(){
    document.write("function 2 <br> x=" + x + "<br>" + "y=" + y + "<br>" + "z="
+ z + "<br>Name=" + a + "<br>");
  }
  f1();
  f2();
</script>
```

Variable Naming Rules

- Variable name must start with an alphabet or "_" only.
- Variable name can't start with number or any other special char.

```
var _user;    // valid
var  user;    // valid
var 1user;    // invalid
```

- Variable name can be alpha numeric but not starting with number.

```
var year2023;    // valid
var 2023Year;    // invalid
```

- Variable name can't have blank space or periods "." and not recommended to use special chars except "_"

```
var user name;    // invalid
var user.name;    // invalid
var user_name;    // valid
```

- Variable name can be max 255 chars long.
- Variable name must speak what it is.

```
var userName;
var userAge;
```

JavaScript Data Types

- Data type defines the data structure.
- Data structure specifies the type and range of data that you can store in a variable.
- JavaScript is not strongly typed, you can't restrict the type.
- JavaScript is implicitly typed, the type of variable is determined according to value assigned or initialized.

```
var x = 10;    x is number
x = "John";    x is string
```

- JavaScript data type are classified into 2 groups

a) Primitive Types

b) Non Primitive Types

Primitive Types

- They are immutable types.
- They have a fixed range for value.
- Structure can't change.
- They are stored in memory stack.
- Stack use "LIFO" [Last-in-First-out]

- JavaScript primitive types are

- a) number
- b) string
- c) boolean
- d) null
- e) undefined
- f) symbol [ES6]

Number Type

- JavaScript number type can be
Signed Integer -5

Unsigned Integer	5
Floating Point	45.32
Double	346.34
Decimal	4569.55
Hexa	0,f
Octa	735o
Binary	0b1010
Exponent	2e3 [2x10(3)] = 2000

- Every value entered from HTML forms is a string type.
- If you want a numeric value to handle any mathematical operation then you have to convert string type into number.
- **JavaScript provides parsing techniques [methods]**
 - a) **parseInt()**
 - b) **parseFloat()**

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function SubmitClick(){
      var age = parseInt(document.getElementById("Age").value);
      document.write("You will be " + (age+1) + " next year.");
    }
  </script>
</head>
<body>
  Your age : <input type="number" min="15" max="30" value="15" id="Age">
  <button onclick="SubmitClick()">Submit</button>
</body>
</html>
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<script>
    function SubmitClick(){
        var rate = parseFloat(document.getElementById("Rate").value);
        document.write("You interest rate is increase by 1 and it is :" + (rate+1));
    }
</script>
</head>
<body>
    Interest Rate : <input type="number" min="10.24" max="18.34" step="0.1"
value="10.24" id="Rate"> <button onclick="SubmitClick()">Submit</button>
</body>
</html>

```

- You can validate number type by using the method "isNaN()".

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script>
        function SubmitClick(){
            var age = document.getElementById("Age").value;
            if(isNaN(age)){
                alert("Age must be a number");
            } else {
                document.write("Your Age : " + age);
            }
        }
    </script>
</head>
<body>
    Age : <input type="text" id="Age"> <button
onclick="SubmitClick()">Submit</button>
</body>
</html>

```


FAQ: How Math Operations are handled over number?

Ans: By using general arithmetic operators and Math object.

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus Division
**	Power [2**3] = 8

Math object in JavaScript provides various math functions

Math.PI
Math.sqrt()
Math.pow()
Math.random()
Math.round()
Math.sin()
Math.cos()
Math.avg() etc..

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>EMI Calculator</title>
  <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <script type="text/javascript">
    function AmountChange(){
      document.getElementById("txtAmount").value =
document.getElementById("rangeAmount").value;
    }
    function YearChange(){
      document.getElementById("txtYears").value =
document.getElementById("rangeYears").value;
    }
    function RateChange(){
      document.getElementById("txtRate").value =
document.getElementById("rangeRate").value;
```

```

    }
    function CalculateClick(){
        var P = parseInt(document.getElementById("txtAmount").value);
        var r = parseFloat(document.getElementById("txtRate").value)/12/100;
        var n = parseInt(document.getElementById("txtYears").value) * 12;

        var EMI = P * r * Math.pow(1 + r,n)/Math.pow(1 + r, n) - 1;

        document.getElementById("result").innerHTML = "Monthly Installment
for your amount " + P + " for " + n + "months is " + "<b> ₹";
        Math.round(EMI) + "</b>";

    }
</script>
</head>
<body class="container-fluid">
    <h2>EMI Calculator</h2>
    <div class="p-3 border border-2 border-dark">
        <div class="row">
            <div class="col">
                Amount you need ₹; <input type="text" id="txtAmount"
size="15">
            </div>
            <div class="col">
                for <input type="text" id="txtYears" size="4"> years
            </div>
            <div class="col">
                Interest rate <input type="text" id="txtRate" size="4"> %
            </div>
        </div>
        <div class="row mt-4">
            <div class="col">
                ₹; 10,000 <input type="range" onchange="AmountChange()"
id="rangeAmount" value="10000" min="10000" max="1000000"> ₹;
10,00,000
            </div>
            <div class="col">
                1 <input type="range" id="rangeYears" onchange="YearChange()"
value="1" min="1" max="5"> 5
            </div>
            <div class="col">
                10.45% <input type="range" id="rangeRate"
onchange="RateChange()" value="10.45" step="0.1" min="10.45" max="18.45">

```

```

18.45%
    </div>
  </div>
  <div class="row mt-3">
    <div class="col text-center">
      <button class="btn btn-primary" onclick="
CalculateClick()">Calculate</button>
    </div>
  </div>
</div>
<div class="mt-4 text-center">
  <p id="result"></p>
</div>
</body>
</html>

```

Task : BMI Calculator

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="./node_modules/bootstrap-icons/font/bootstrap-
icons.css">
  <style>
    #pointer {
      margin-left: 400px;
    }
  </style>
  <script>
    document.getElementById("pointer").style.marginLeft = "500px";
  </script>
</head>
<body class="container-fluid">
  <div class="mt-4">
    <div class="progress">
      <div class="progress-bar bg-dark text-white" style="width:400px">
        < 52 Under Weight

```

```

    </div>
    <div class="progress-bar bg-info text-white" style="width:400px">
      53 to 72 Normal Weight
    </div>
    <div class="progress-bar bg-dark text-white" style="width:400px">
      > 72 Over Weight
    </div>
  </div>
</div>
<div>
  <div class="bi bi-triangle-fill" id="pointer">

    </div>
  </div>
</div>
</body>
</html>

```

String Type

- String is a literal with group chars enclosed in

- a) Double Quotes " "
- b) Single Quotes ' '
- c) Back Tick ` `

- Literal is combination of alphabet, number and special chars.
- Single and double quotes are used for outer and inner strings.

Syntax:

```

var link = "<a href='home.html'> Home </a>";
var link = '<a href="home.html"> Home </a>';

```

- Single and double quote strings require lot of "+" operator to concat any dynamic value or expression.

Ex:

```

<script>
  var username = "John";
  var age = 22;
  var msg1 = "Hello !" + " " + username + " " + "you will be" + " " + (age+1) + " "
+ "next year.";
  document.write(msg1);
</script>

```

- JavaScript ES5 introduced "backtick" for string which allows embedded expression. You can embed any dynamic value into string by using "\${ }".

\${ } => Binding Expression => only with backtick

`your string \${ expression | dynamic } your string`;

Ex:

```
<script>
  var username = "John";
  var age = 22;
  var msg1 = "Hello !" + " " + username + " " + "you will be" + " " + (age+1) + " "
+ "next year.";
  document.write(msg1 + "<br>");
  var msg2 = `Hello ! ${username} you will be ${age+1} next year.`;
  document.write(msg2);
</script>
```

- Several chars in a string are not printable. They escape printing.

- You have to print the non-printable chars by using "\".

Ex:

```
<script>
  var path = "\"D:\\Images\\pic.jpg\"";
  document.write(path);
</script>
```

- JavaScript provides several methods to format and manipulate string.
- String formatting allows to change the appearance of string dynamically.

bold()
italics()
sup()
sub()
fontcolor()
fontsize()
toUpperCase()
toLowerCase() etc...

Ex:

```
<script>
    var msg = "Welcome to JavaScript";
    document.write(msg.fontSize(7).italics().bold().fontcolor('red'));
</script>
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script>
        function ChangeCase(){
            var username = document.getElementById("UserName").value;
            document.getElementById("UserName").value =
username.toUpperCase();
            if(username==""){
                document.getElementById("userError").innerHTML = "User Name
Required".fontcolor('red').italics();
            } else {
                document.getElementById("userError").innerHTML = "";
            }
        }
    </script>
</head>
<body>
    <dl>
        <dt>User Name</dt>
        <dd><input type="text" onblur="ChangeCase()" id="UserName"
placeholder="Block Letter Only"></dd>
        <dd id="userError"></dd>
    </dl>
</body>
</html>
```

Note: All string formatting methods are RC data type. You can't used on any RC data type presentation. They convert the format into Markup.

- JavaScript provides several members for manipulating string.

length : It returns the total count of chars in a string.

charAt()

charCodeAt()

indexOf()

lastIndexOf()

substr()

substring()

slice()

trim()

split()

match() etc..

Ex: Length

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function VerifyName(){
      var username = document.getElementById("UserName").value;
      var userError = document.getElementById("userError");
      if(username==""){
        userError.innerHTML = "User Name Required".fontcolor('red');
      } else {
        if(username.length<4) {
          userError.innerHTML = "Name too short min 4
chars".fontcolor('red');
        } else if(username.length>10) {
          userError.innerHTML = "Name too long max 10
chars.".fontcolor('red')
        } else {
          userError.innerHTML = "";
        }
      }
    }
  </script>
</head>
<body>
  <dl>
```

```

        <dt>User Name</dt>
        <dd><input type="text" onkeyup="VerifyName()" id="UserName" ></dd>
        <dd id="userError"></dd>
    </dl>
</body>
</html>

```

Method	Description
--------	-------------

charAt()	It returns the character at specified index.
-----------------	---

Syntax:
string.charAt(0)

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script>
        function VerifyCode(){
            var code = document.getElementById("code").value;
            var error = document.getElementById("error");
            if(code.charAt(0)=="S") {
                error.innerHTML = "";
            } else {
                error.innerHTML = "Invalid Code".fontcolor('red');
            }
        }
    </script>
</head>
<body>
    SBI IFSC Code : <input onkeyup="VerifyCode()" type="text" id="code">
    <span id="error"></span>
</body>
</html>

```

charCodeAt()	It retruns the chars code (ASCII) at specified index.
---------------------	--

A = 65, Z = 90

Syntax:

```
var str = "Welcome";  
str.charCodeAt(0);    86
```

Ex:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Document</title>  
  <script>  
    function VerifyUser(){  
      var username = document.getElementById("UserName").value;  
      var error = document.getElementById("error");  
      if(username.charCodeAt(0)>=65 && username.charCodeAt(0)<=90){  
        error.innerHTML = "";  
      } else {  
        error.innerHTML = "Name must start with Uppercase  
letter".fontcolor('red');  
      }  
    }  
  </script>  
</head>  
<body>  
  Your Name : <input type="text" onkeyup="VerifyUser()" id="UserName">  
<span id="error"></span>  
</body>  
</html>
```

indexOf() It returns the index number of given character
if the char not found then it returns -1.

Syntax:

```
var str = "Welcome";  
str.indexOf("W");    0  
str.indexOf("a");    -1  
str.indexOf("e");    1
```

lastIndexOf() It returns the last occurrence index number

Syntax:

str.lastIndexOf("e"); 6

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function VerifyEmail(){
      var email = document.getElementById("Email").value;
      var error = document.getElementById("error");
      if(email.indexOf("@")==-1){
        error.innerHTML = "Invalid Email : Please Include @".fontcolor('red');
      } else {
        error.innerHTML = "";
      }
    }
  </script>
</head>
<body>
  Your Email : <input type="text" onblur="VerifyEmail()" id="Email"> <span
id="error"></span>
</body>
</html>
```

startsWith() It returns "true" if string starts with specified chars.

endsWith() It returns "true" if string ends with specified chars.

EX:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```

<title>Document</title>
<link rel="stylesheet"
href="../node_modules/bootstrap/dist/css/bootstrap.css">
<script>
    function VerifyCard(){
        var card = document.getElementById("Card").value;
        var cardimg = document.getElementById("cardImg");
        if(card.startsWith("4550")) {
            cardimg.src = "../public/images/visa.png";
        } else if(card.startsWith("5540")) {
            cardimg.src = "../public/images/master.png";
        }
    }
    function VerifySkype(){
        var skype = document.getElementById("skype").value;
        var error = document.getElementById("error");
        if(skype.endsWith("outlook.com")) {
            error.innerHTML = "";
        } else {
            error.innerHTML = "Invalid Skype ID".fontcolor('red');
        }
    }
</script>
</head>
<body class="container-fluid">
    <dl>
        <dt>Your Card Number</dt>
        <dd class="input-group">
            <input type="text" onblur="VerifyCard()" id="Card"> <img class="input-
group-text" width="60" id="cardImg">
        </dd>
        <dt>Your Skype ID</dt>
        <dd>
            <input type="text" onblur="VerifySkype()" id="skype">
        </dd>
        <dd id="error"></dd>
    </dl>
</body>
</html>

```

slice() : It extracts the chars between specified index

The end index must be greater than start index.

Syntax:

```
str.slice(startIndex, endIndex)
str.slice(startIndex); // read upto end
```

substr() : It reads specified number of chars from given index.

Syntax:

```
str.substr(7, 3);    from 7 it reads 3 chars
str.substr(startIndex, howMany)
```

substring() : It is bi-directional in reading string from given index

Syntax:

```
str.substring(startIndex, endNumber);
endNumber can be left or right index
```

Ex:

```
<script>
  var email = prompt("Enter Your Email");
  var id = email.substring(email.indexOf("@"),0);
  var domain = email.slice(email.indexOf("@")+1)
  document.write(`Your ID : ${id}<br>Your Domain : ${domain}`);
</script>
```

String Methods / Properties

length
charAt()
charCodeAt()
indexOf()
lastIndexOf()
slice()
substr()
substring()
startsWith()
endsWith()

split() It splits the string at specified character and returns an array of string. [array is a collection of string values]

Ex:

```
<script>
```

```

var email = prompt("Enter Your Email");
var [id, domain] = email.split('@');
document.write(`Your ID : ${id} <br> Domain : ${domain}`);
</script>

```

Ex:

```

<script>
var contacts = "john-9992838482, david-8822284853, sam-9938223483";
var [john, david, sam, a] = contacts.split(',');
document.write(david + "<br>" + sam);
document.write(a); undefined
</script>

```

Note: [a, b, c] = collection => destructuring - ES6

var result = string.split(',');

result[0]

result[1]

trim() It is used to remove the leading spaces in a string.
 leading space is the blank space before and after string.

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function VerifyUserId(){
      var userid = document.getElementById("UserId").value;
      if(userid.trim()=="john_nit") {
        document.getElementById("error").innerHTML = "";
        document.getElementById("UserId").value = userid.trim();
      } else {
        document.getElementById("error").innerHTML = "Invalid User
Id".fontcolor('red');
      }
    }
  </script>

```

```

</head>
<body>
  User Id : <input type="text" onblur="VerifyUserId()" id="UserId"> <span
id="error"></span>
</body>
</html>

```

match() It is used to verify the string by matching it against a regular expression.

Syntax:
string.match(/regExp/)

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet"
href=" ../node_modules/bootstrap/dist/css/bootstrap.css">
  <script>
    var regExp = / /;
    function CountryChanged(){
      var country = document.getElementById("Country").value;
      var flag = document.getElementById("flag");
      var mobile = document.getElementById("Mobile");
      if(country=="India") {
        flag.src = "../public/images/india.jpg";
        mobile.placeholder = "+91 and 10 digits";
        regExp = /\+91\d{10}/;

      }else if(country=="US") {
        flag.src = "../public/images/us.jpg";
        mobile.placeholder = "+(1)(425) 555-0100";
        regExp = /\+?(1)\([0-9]{3}\)\s[0-9]{3}-[0-9]{4}/;

      } else if(country=="UK") {
        flag.src = "../public/images/uk.jpg";
        mobile.placeholder = "+(44)(20) 1234 5678";
      }
    }
  </script>

```

```

    regExp = /^[+](44)\|([0-9]{2})\s[0-9]{4}\s[0-9]{4}/;

    } else {
        mobile.placeholder = "Please Select Country";
        flag.src = "";
        flag.alt = "N/A";
    }
}
function VerifyClick(){
    var mobile = document.getElementById("Mobile").value;
    if(mobile.match(regExp)){
        document.write("Your mobile verified successfully...");
    } else {
        document.getElementById("error").innerHTML = "Invalid Mobile - " +
document.getElementById("Country").value + "-" +
document.getElementById("Mobile").placeholder;
    }
}
</script>
</head>
<body class="container-fluid">
    <h2>Verify Your Mobile</h2>
    <dl class="w-25">
        <dt>Select Your Country</dt>
        <dd>
            <select id="Country" onchange="CountryChanged()" class="form-select">
                <option>Choose Country</option>
                <option>India</option>
                <option>US</option>
                <option>UK</option>
            </select>
        </dd>
        <dt>Your Mobile Number</dt>
        <dd class="input-group">
            <input type="text" id="Mobile" class="form-control">
            <span class="input-group-text"><img id="flag" width="50"></span>
        </dd>
        <button onclick="VerifyClick()" class="btn btn-primary w-
100">Verify</button>
    </dl>
    <h2 class="text-center text-danger" id="error"></h2>
</body>
</html>

```

- JavaScript can format string by using styles and classes.

```
element.style.attributeName = "value";
```

CSS	=> attribute name	"font-size"	"background-color"
JS	=> style attribute name	"fontSize"	"backgroundColor"

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function bodyload(){
      var heading = document.querySelector("h1");
      heading.innerHTML = "Welcome to JavaScript";
      heading.style.backgroundColor = "yellow";
      heading.style.padding = "10px";
      heading.style.color = "red";
      heading.style.textAlign = "center";
    }
  </script>
</head>
<body onload="bodyload()">
  <h1></h1>
</body>
</html>
```

- You can use class attribute, which is defined as className.

```
element.className = "css class1 class2";
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```



```

<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<style>
    .effects {
        background-color: yellow;
        color:red;
        text-align: center;
        padding: 5px;
        border:2px solid red;
    }
</style>
<script>
    function bodyload(){
        var heading = document.querySelector("h1");
        heading.innerHTML = "Welcome to JavaScript";
        heading.className = "effects";
    }
</script>
</head>
<body onload="bodyload()">
    <h1></h1>
</body>
</html>

```

Boolean Type

- Boolean type can handle 2 values
 - a) true
 - b) false
- Boolean type are used in decision making.
- JavaScript boolean true is handled by using 1 and false by using 0.

```

true = 1
false = 0

```

Syntax:

```
var x = true;
```

```

if(x==1) { }    //valid
if(x==true) { } // valid [ recommended]

```

FAQ: true + true = ? 2

```
var x = 1;
```

```
if(x==true) { }    // invalid
```

- HTML element attributes that use boolean are

```
readonly = true / false  
disabled = true / false  
checked = true / false  
required = true / false  
autofocus = true / false  
selected = true / false
```

Ex:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Document</title>  
  <script>  
    function AllChange(){  
      var all = document.getElementById("all");  
      if(all.checked==true) {  
        document.getElementById("stock").checked = true;  
        document.getElementById("visible").checked = true;  
      } else {  
        document.getElementById("stock").checked = false;  
        document.getElementById("visible").checked = false;  
      }  
    }  
  </script>  
</head>  
<body>  
  <input type="checkbox" onchange="AllChange()" id="all"> Select All <br>  
  <input type="checkbox" id="stock"> In Stock  
  <br>  
  <input type="checkbox" id="visible"> Is Visible
```

```
</body>
</html>
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function VerifyName(){
      var username = document.getElementById("UserName").value;
      var button = document.querySelector("button");
      if(username=="") {
        button.disabled = true;
      } else {
        button.disabled = false;
      }
    }
  </script>
</head>
<body>
  <dl>
    <dt>User Name</dt>
    <dd><input type="text" onkeyup="VerifyName()" placeholder="User Name
Required" id="UserName"></dd>
  </dl>
  <button disabled>Register</button>
</body>
</html>
```

Ex: Toggle with boolean

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
```

```

function StockChange(){
    var stock = document.getElementById("Stock");
    var lblStock = document.getElementById("lblStock");
    if(stock.checked==1){
        lblStock.innerHTML = "Available";
    } else {
        lblStock.innerHTML = "Out of Stock";
    }
}
</script>
</head>
<body>
    <input type="checkbox" onchange="StockChange()" id="Stock"> <span
id="lblStock">Out of Stock</span>
</body>
</html>

```

Ex: Button Toggle

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script>
        function ToggleClick(){
            var button = document.querySelector("button");
            var img = document.querySelector("img");
            img.style.display = (img.style.display=="none")?"block":"none";
            button.innerHTML = (button.innerHTML=="Show")?"Hide":"Show";
        }
    </script>
</head>
<body>
    <button onclick="ToggleClick()">Show</button>
    <br><br>
    
</body>
</html>

```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    button {
      display: flex;
      justify-content: space-between;
      width: 100%;
    }
  </style>
  <script>
    function ToggleClick(){
      var icon = document.getElementById("icon");
      var details = document.getElementById("details");
      icon.innerHTML = (icon.innerHTML=="+"?"X":"+");
      details.style.display = (details.style.display=="none")?"block":"none";
    }
  </script>
</head>
<body>
  <button onclick="ToggleClick()">
    <span>What is Netflix ?</span>
    <span id="icon">+</span>
  </button>
  <div id="details" style="display:none">
    <p>    Netflix is a streaming service that offers a wide variety of award-
winning TV shows, movies, anime, documentaries and more – on thousands of
internet-connected devices.
  </p>
  <p>
    You can watch as much as you want, whenever you want, without a single
ad – all for one low monthly price. There's always something new to discover, and
new TV shows and movies are added every week!
  </p>
  </div>
</body>
</html>
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    button {
      display: flex;
      justify-content: space-between;
      width: 100%;
    }
  </style>
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
  <script>
    function ToggleClick(){
      var icon = document.getElementById("icon");
      var details = document.getElementById("details");
      icon.className = (icon.className=="bi bi-chevron-up")?"bi bi-chevron-
down":"bi bi-chevron-up";
      details.style.display = (details.style.display=="none")?"block":"none";
    }
  </script>
</head>
<body>
  <button onclick="ToggleClick()">
    <span>What is Netflix ?</span>
    <span id="icon" class="bi bi-chevron-up"></span>
  </button>
  <div id="details" style="display:none">
    <p>    Netflix is a streaming service that offers a wide variety of award-
winning TV shows, movies, anime, documentaries and more – on thousands of
internet-connected devices.
  </p>
  <p>
    You can watch as much as you want, whenever you want, without a single
ad – all for one low monthly price. There's always something new to discover, and
new TV shows and movies are added every week!
  </p>
```

```
</div>
</body>
</html>
```

Primitive Types

1. number
2. string
3. boolean
4. undefined
5. null

undefined

- It specifies that reference is defined but there is no value in the reference.

Syntax:

```
var x ;
document.write("x=" + x);    x=undefined

if(x==undefined)
{
}
```

Ex:

```
<script>
  var name = "John";
  var age;
  if(age==undefined)
  {
    document.write(`Name=${name}`);
  } else {
    document.write(`Name=${name}<br>Age=${age}`);
  }
</script>
```

- You can also verify as defined.

if(age) => If age is defined

Ex:

```
<script>
  var name = "John";
  var age = 33;
  if(age)
  {
    document.write(`Name=${name}<br>Age=${age}`);
  } else {
    document.write(`Name=${name}`);
  }
</script>
```

FAQ: What is difference between undefined and not-defined?

Ans : undefined states that value is not defined.

no-defined states that reference is not defined.

```
var x;
document.write(`x=${x}<br>y=${y}`)
```

x = undefined // Error y is not defined

null

- Null specifies that there is no value provided into reference during runtime.

Syntax:

```
var x;                                      // undefined
var y = prompt("Enter value");
```

- The null types are verified by using "null" keyword.

Ex:

```
<script>
  var x = prompt("Enter Number");
  if(x==null){
    alert("You canceled");
  } else if(x=="") {
```



```
    alert("Please provide some value - It can't be empty");
  } else {
    document.write("x=" + x);
  }
</script>
```

Symbol

- It is a new data type introduces into JavaScript from ES6 version.
- It is used to define a unique key reference.
- It is not identified in iterations. It is hidden.
- It is "Primitive Type".

Syntax:

```
var UserId = Symbol();
```

Ex: Form Hidden Type

```
<form>
  <dl>
    <dd><input type="hidden" name="UserId" value="john_nit"></dd>
    <dt>User Name</dt>
    <dd><input type="text" name="UserName" ></dd>
    <dt>Password</dt>
    <dd><input type="password" name="Password"></dd>
    <dt>Age</dt>
    <dd><input type="number" name="Age"></dd>
  </dl>
  <button>Submit</button>
</form>
```

Summary:

- number
- string
- boolean
- null
- undefined
- symbol

Non Primitive Types

- The non-primitive types are mutable types.
- Their structure can change according to state and situation.
- There is no fixed range for value.
- Value range can change
- They are stored in memory heap.
- JavaScript Non-Primitive types are

a) Array

b) Object

c) Map

Array

- Arrays are used in computer programming to reduce overhead and complexity.
- Arrays can reduce overhead by storing values in sequential order.
- Arrays can reduce complexity by storing multiple values under one name.
- Array can store any type of value in sequential order.
- Array size can change dynamically.
- Array is a name given for a type of formation.
- It stores in sequential order but can access in random order.

- Reducing Overhead
- Reducing Complexity
- Array can handle various types of values.
- Array size can change dynamically.

Declaring Array:

- Array declaration is same like other variables.
- You can use var , let or const.
- Usually array name is defined with a plural term.

```
var categories;  
let products;  
const sales;
```

Initialize or Assign memory:

- Memory for array is initialized or assigned by using

a) meta character `[]`

b) constructor `Array()`

Syntax:

```
let categories = [];  
let categories = new Array();
```

(or)

```
let categories;  
categories = [];
```

```
let categories;  
categories = new Array();
```

FAQ: What is difference between `[]` and `Array()`?

Ans : "new" is dynamic memory allocation operator.

It allocates memory for array everytime when requested.

It is known as disconnected memory.

`[]` uses continuous memory.

Memory once allocated for array will be available across multiple requests.

Storing Data in Array

- Data is stored in array with reference of its properties.
- Property is string type that maps to index in memory.
- Property reference in array starts with "0".

Syntax:

```
var values = [];  
values[0] = 10;  
values["1"] = "A";
```

```
document.write(values["0"] + "<br>" + values[1]);
```

Ex:

```
<script>  
var values = [];
```

```

values[0] = 10;
values["1"] = true;
values["2"] = "john";
for(var property in values)
{
    document.write(`${property}[${typeof property}] - ${values[property]}
[${typeof values[property]]}<br>`);
}
</script>

```

Type of Data to Store:

- Array allows to store any type of data

a) Primitive

b) Non Primitive

c) Function

Syntax:

```

var values = [ 10, "TV", true, ["Delhi", "Hyd"], function(){ document.write("Hello");
} ]

```

```

document.write(values[3][1]);
values[4]();

```

Ex:

```

<script>
    var values = [10, "TV", true, ["Delhi", "Hyd"], function(){document.write("Hello
from Array")});
    document.write(` City = ${values[3][1]}<br>`);
    values[4]();

</script>

```

Array Manipulations:

- JavaScript "Array" object comprises various methods and properties that are used to manipulate array.

1. Methods for reading data from Array

toString()	It can read and return all values from array separated with ","
------------	---

join() It can read and return all values using a custom delimiter [separator].

slice() It can read and return values between specified index.

map() It returns all values from array and presents using specified markup. It is an iterator.

find() It returns the first occurrence value that exactly matches with the condition.

filter() It is similar to find but returns all values that match.

Ex:

```
<script>
  var sales = [45000, 56000, 57000, 53000, 23000, 46000];
  var menuitems = ["Home", "Electronics", "Mobiles", "Realme"];
  document.write(menuitems.toString() + "<br>");
  document.write(menuitems.join(" / ") + "<br>");
  document.write(menuitems.slice(1,3) + "<br>");
  menuitems.map(function(item){
    document.write("<a href='#">${item}</a><br>");
  });
  var result = sales.filter(function(value){
    return value >= 50000;
  });
  document.write(result);
</script>
```

for..in It reads and returns all properties.

for..of It reads and returns all values.

Ex:

```
<script>

  var menuitems = ["Home", "Electronics", "Mobiles", "Realme"];
  for(var property of menuitems)
  {
    document.write(property + "<br>");
  }
```

```
}  
</script>
```

Ex:

```
<script>  
  
    var menuitems = ["Home", "Electronics", "Mobiles", "Realme"];  
    for(var property in menuitems)  
    {  
        document.write(`${property} - ${menuitems[property]}<br>`);  
    }  
</script>
```

Reading Values from Array

- toString()
- join()
- slice()
- find()
- filter()
- map()
- for..in
- for..of

FAQ: How to present the collection in UI?

Ans : - Use any iterator for reading all values from collection

- Create a new element for every item in collection.
- Append element into its parent.

FAQ: How to create a new element of HTML using JavaScript?

Ans : document.createElement()
document.createElement("button")
document.createElement("img")

FAQ: How to add element into page?

Ans : appendChild()

Syntax:

```
var btn = document.createElement("button");
```

```
btn.innerHTML = "Submit";  
document.querySelector("div").appendChild(btn);
```

Ex:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Document</title>  
  <style>  
    div {  
      width: 200px;  
      height: 300px;  
      border:1px solid black;  
      padding: 30px;  
    }  
  </style>  
  <script>  
    function bodyload(){  
      var btn = document.createElement("button");  
      btn.innerHTML = "Submit";  
      btn.style.marginTop = "50px";  
      btn.style.display = "block";  
      document.getElementById("container").appendChild(btn);  
  
      var pic = document.createElement("img");  
      pic.width = "100";  
      pic.height = "100";  
      pic.src = "../public/images/asp.jpg";  
      pic.style.marginTop = "100px";  
      document.getElementById("container").appendChild(pic);  
    }  
  </script>  
</head>  
<body onload="bodyload()">  
  <div id="container"></div>  
</body>  
</html>
```

Ex: Presenting Array Elements

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    var categories = ["All", "Electronics", "Fashion", "Footwear"];
    function bodyload(){
      for(var item of categories)
      {
        var li = document.createElement("li");
        li.innerHTML = item;
        document.querySelector("ol").appendChild(li);

        var opt = document.createElement("option");
        opt.text = item;
        opt.value = item;
        document.querySelector("select").appendChild(opt);

        var ul_li = document.createElement("li");
        var input = document.createElement("input");
        input.type = "checkbox";
        var span = document.createElement("span");
        span.innerHTML = item;
        ul_li.appendChild(input);
        ul_li.appendChild(span);
        document.querySelector("ul").appendChild(ul_li);
      }
    }
  </script>
</head>
<body onload="bodyload()">
  <ol></ol>
  <select></select>
  <ul style="list-style:none"></ul>
</body>
</html>
```


Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    var categories = ["All", "Electronics", "Fashion", "Footwear"];
    function bodyload(){
      for(var item of categories)
      {
        var li = document.createElement("li");
        li.innerHTML = item;
        document.querySelector("ol").appendChild(li);

        var opt = document.createElement("option");
        opt.text = item;
        opt.value = item;
        document.querySelector("select").appendChild(opt);

        var ul_li = document.createElement("li");
        ul_li.innerHTML = `<input type="checkbox"> ${item}`;
        document.querySelector("ul").appendChild(ul_li);

        var tr = document.createElement("tr");
        var td = document.createElement("td");
        td.innerHTML = item;
        tr.appendChild(td);
        document.querySelector("tbody").appendChild(tr);

      }
    }
  </script>
</head>
<body onload="bodyload()">
  <ol></ol>
  <select></select>
  <ul style="list-style:none"></ul>
  <table border="1" width="200">
```

```

<thead>
  <tr>
    <th>Categories</th>
  </tr>
</thead>
<tbody>

  </tbody>
</table>
</body>
</html>

```

Adding elements into Array:

push()	It adds new element(s) into array as last item.
unshift()	It adds new element(s) into array as first item.
splice()	It adds new element(s) into array at specific position.

Syntax:

```

arrayName.push("item1", "item2"... )
arrayName.unshift("item1", "item2"..)
arrayName.splice(indexNumber, deleteCount, "items...")

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    var categories = ["Electronics", "Fashion", "Footwear"];
    function bodyload(){
      categories.unshift("All","Kids Fashion");
      categories.splice(3,0,"Women's Fashion");
      for(var item of categories)
      {
        var li = document.createElement("li");
        li.innerHTML = item;
        document.querySelector("ol").appendChild(li);
      }
    }
  </script>

```

```

        var opt = document.createElement("option");
        opt.text = item;
        opt.value = item;
        document.querySelector("select").appendChild(opt);

        var ul_li = document.createElement("li");
        ul_li.innerHTML = `<input type="checkbox"> ${item}`;
        document.querySelector("ul").appendChild(ul_li);

        var tr = document.createElement("tr");
        var td = document.createElement("td");
        td.innerHTML = item;
        tr.appendChild(td);
        document.querySelector("tbody").appendChild(tr);
    }
}
</script>
</head>
<body onload="bodyload()">
    <ol></ol>
    <select></select>
    <ul style="list-style:none"></ul>
    <table border="1" width="200">
        <thead>
            <tr>
                <th>Categories</th>
            </tr>
        </thead>
        <tbody>

        </tbody>
    </table>
</body>
</html>

```

Removing Elements from Array:

pop()	It removes and returns the last item.
shift()	It removes and returns the first item.
splice()	It removes and returns specific item.

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    var categories = ["All", "Electronics", "Fashion", "Footwear"];
    function bodyload(){
      alert(`${categories.splice(1,1)} Removed`);
      for(var item of categories)
      {
        var li = document.createElement("li");
        li.innerHTML = item;
        document.querySelector("ol").appendChild(li);

        var opt = document.createElement("option");
        opt.text = item;
        opt.value = item;
        document.querySelector("select").appendChild(opt);

        var ul_li = document.createElement("li");
        ul_li.innerHTML = `☐
```

```
<thead>
  <tr>
    <th>Categories</th>
  </tr>
</thead>
<tbody>

  </tbody>
</table>
</body>
</html>
```

Sorting Array Elements:

`sort()`
`reverse()`

Syntax:

```
categories.sort()
categories.reverse()
```

Array Methods

- Read
- Insert
- Delete
- Search
- Sort
- length

Create Element
Append Element

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Array</title>
  <link rel="stylesheet"
```

```

href="../node_modules/bootstrap/dist/css/bootstrap.css">
<link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
<script>
var cities = ["Delhi", "Hyd", "Chennai"];
function LoadCities(){
    document.getElementById("lstCities").innerHTML="";
    for(var city of cities)
    {
        var option = document.createElement("option");
        option.text = city;
        option.value = city;
        document.getElementById("lstCities").appendChild(option);
    }
    document.getElementById("lblCount").innerHTML = `Total Number of
Cities : ${cities.length}`;
}

function bodyload(){
    LoadCities();
}
function AddClick(){
    var cityname = document.getElementById("txtCity").value;
    if(cities.indexOf(cityname)==-1){
        cities.push(cityname);
        alert(`${cityname} Added to list`);
        LoadCities();
        document.getElementById("txtCity").value = "";
    } else {
        alert(`${cityname} Exists`);
    }
}

function RemoveClick(){
    var selectedCity = document.getElementById("lstCities").value;
    var selectedIndex = cities.indexOf(selectedCity);
    var flag = confirm(`Delete City : ${selectedCity}\nAre you sure want to
Delete?`);
    if(flag==true) {
        cities.splice(selectedIndex,1);
        LoadCities();
    }
}

```

```

    }
    function SortAsc(){
        cities.sort();
        LoadCities();
    }
    function SortDsc(){
        cities.sort();
        cities.reverse();
        LoadCities();
    }
</script>
</head>
<body class="container-fluid" onload="bodyload()">
    <h2>Array Manipulations</h2>
    <div class="w-25">
        <div>
            <label class="form-label">Add New City</label>
            <div class="input-group">
                <input type="text" id="txtCity" class="form-control">
                <button class="btn btn-primary" onclick="AddClick()">Add</button>
            </div>
        </div>
        <div class="mt-3">
            <label class="form-label">Select City</label>
            <div>
                <select id="lstCities" size="3" class="form-select">

                    </select>
            </div>
            <div class="mt-2">
                <span id="lblCount"></span>
            </div>
        </div>
        <div class="mt-3">
            <button class="btn btn-danger" onclick="RemoveClick()"> <span
class="bi bi-trash"></span> Remove</button>
            <button class="btn btn-success" onclick="SortAsc()">
                <span class="bi bi-sort-alpha-down"></span>
            </button>
            <button class="btn btn-dark" onclick="SortDsc()">
                <span class="bi bi-sort-alpha-up"></span>
            </button>
        </div>
    </div>

```

```
</div>
</body>
</html>
```

Ex: Cascading Dropdown

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    var categories = ["Select Category", "Electronics", "Footwear"];
    var electronics = ["Select Electronics", "Televisions", "Mobiles"];
    var footwear = ["Select Footwear", "Casuals", "Sneakers", "Boots"];
    var products = [];
    function LoadCategories(){
      for(var category of categories) {
        var option = document.createElement("option");
        option.text = category;
        option.value = category;
        document.getElementById("lstCategories").appendChild(option);
      }
    }
    function LoadProducts(){
      document.getElementById("lstProducts").innerHTML = "";
      for(var product of products) {
        var option = document.createElement("option");
        option.text = product;
        option.value = product;
        document.getElementById("lstProducts").appendChild(option);
      }
    }
    function bodyload(){
      LoadCategories();
    }
    function CategoryChanged(){
      var categoryName = document.getElementById("lstCategories").value;
      if(categoryName=="Electronics") {
        products = electronics;
      }
    }
  </script>

```



```

        LoadProducts();
    } else if(categoryName=="Footwear") {
        products = footwear;
        LoadProducts();
    } else {
        products = ["Please Select Category"];
        LoadProducts();
    }
}
</script>
</head>
<body onload="bodyload()">
<dl>
    <dt>Select Category</dt>
    <dd>
        <select id="lstCategories" onchange="CategoryChanged()"></select>
    </dd>
    <dt>Select Product</dt>
    <dd>
        <select id="lstProducts"></select>
    </dd>
</dl>
</body>
</html>

```

Object

- Object keeps all related data and logic together.
- Object encapsulates the data and logic.
- "Alan Kay" introduced the concept of object into programming in early 1960's.
- It is a key and value collection.

```
let obj = { key:value, key:function(){} }
```

- Every key is "string" type and value can be any type [primitive, non-primitive].
- If object comprises of only data then it is referred as "JSON".
[JavaScript Object Notation]
- JSON is a format used for presenting data.
- Another popular format for data is XML.
- Object enables clean separation and reusability.

Syntax:

```
var product = {
```

```
"Name": "TV",  
"Price": 45000.33  
}
```

- You can access the data of object using the object name and its key reference.

```
document.write(product.Price);  
document.write(product[key]);
```

Ex:

```
<script>  
  var product = {  
    "Name" : "Samsung TV",  
    "Price": 44500.33,  
    "Stock": true,  
    "Cities": ["Delhi", "Hyd"],  
    "Rating": {"Rate": 4.3, "Count": 3500}  
  };  
  document.write(  
    Name : ${product.Name} <br>  
    Price: ${product.Price} <br>  
    Stock: ${product.Stock} <br>  
    Cities: ${product.Cities.toString()} <br>  
    Rating: ${product.Rating.Rate} [ ${product.Rating.Count} ]  
  );  
</script>
```

- You can use object as a prototype

Ex:

```
<script>  
  var product = {  
    "Name" : "",  
    "Price": 0,  
    "Stock": false,  
    "Cities": [],  
    "Rating": {"Rate": 0, "Count": 0}  
  };  
  
  product.Name = prompt("Enter Name");  
  product.Price = prompt("Enter Price");
```

```

product.Stock = prompt("Enter Stock", "true|false");
product.Cities[0] = prompt("Enter City 1");
product.Cities[1] = prompt("Enter City 2");
product.Rating.Rate = prompt("Enter User Rating");
product.Rating.Count = prompt("Enter Users Count");

document.write(
    Name : ${product.Name} <br>
    Price: ${product.Price} <br>
    Stock: ${product.Stock} <br>
    Cities: ${product.Cities.toString()} <br>
    Rating: ${product.Rating.Rate} [${product.Rating.Count}]
);
</script>

```

Ex:

```

<script>
    var product = {
        "Name": "Samsung TV",
        "Price" : 34000.33
    }
    for(var property in product)
    {
        document.write(`${property} : ${product[property]}<br>`);
    }
</script>

```

- A JavaScript object can contain both data and logic, which is defined in functions.
- The keys are accessed outside object by using object name.
- The keys are accessed inside object by using "this" keyword.

Ex:

```

<script>
    var product = {
        "Name": "Samsung TV",
        "Price" : 34000.33,
        "Qty": 2,
        "Total": function(){
            return this.Qty * this.Price
        },
        "Print": function(){

```

```

document.write(`Name=${this.Name}<br>Price=${this.Price}<br>Qty=${this.Qty}<br>Total=${this.Total()}`);
    }
}
product.Print();
document.write("<hr>");
product.Name = "Nike Casuals";
product.Price = 3500.44;
product.Qty = 3;
product.Print();
document.write("<hr>");
product.Name = "Watch";
product.Price= 2000.33;
product.Qty = 2;
product.Print();
</script>

```

Ex::

```

<script>
    var products = [
        {Name: "TV", Price: 45000.44, Stock:true},
        {Name: "Mobile", Price:23000.33, Stock:false}
    ];
    document.write("Mobile Price : " + products[1].Price);
    document.write("<hr>");
    for(var item of products)
    {
        document.write(`${item.Name} - ${item.Price} - ${item.Stock} <br>`);
    }
</script>

```

Database [Oracle | MongoDB | MySQL] => Server Side App [JSP | PHP | ASP] => API => XML | JSON => Present HTML, CSS, JavaScript, jQuery, Angular, React

```

[
{
}
]

```

JavaScript Fetch Promise

- Promise is a function that returns result or any exception if failed.
- Fetch is a promise in JavaScript, which can interact with any API data and return the data on success or return exception details on failed.
- API is Application Programming Interface, which provides data understandable to every platform and device.

Syntax:

```
fetch("url").then(function(){data binary}).then(function(){data JSON}).catch(function(){catch the exception if promise failed})
```

Ex:

1. Add a new folder into project by name "data"

2. Add a new file into data folder by name

```

    "products.json"
[
  {
    "Name": "Samsung TV",
    "Price": 35000.55,
    "Stock": true,
    "Rating": {"Rate":4.5, "Count":6000},
    "Photo": "../public/images/tv.jpg"
  },
  {
    "Name": "Nike Casuals",
    "Price": 5000.35,
    "Stock": true,
    "Rating":{"Rate":3.5, "Count":300},
    "Photo": "../public/images/shoe.jpg"
  },
  {
    "Name": "Speaker",
    "Price": 4000.22,
    "Stock": false,
    "Rating":{"Rate": 4.2, "Count": 5000},
    "Photo": "../public/images/speaker.jpg"
  }
]

```

3. Add a new HTML page

demo.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <script type="text/javascript">
    function LoadClick(){
      fetch("./data/products.json")
      .then(function(response){
        return response.json();
      })
      .then(function(products){
        for(var product of products)
        {
          var tr = document.createElement("tr");
          var tdName = document.createElement("td");
          var tdPrice = document.createElement("td");
          var tdStock = document.createElement("td");
          var tdRating = document.createElement("td");
          var tdPhoto = document.createElement("td");

          tdName.innerHTML = product.Name;
          tdPrice.innerHTML = product.Price;
          tdStock.innerHTML = product.Stock;
          tdRating.innerHTML = `${product.Rating.Rate}
[${product.Rating.Count}]`;
          tdPhoto.innerHTML = `<img src=${product.Photo} width="50"
height="50">`;

          tr.appendChild(tdName);
          tr.appendChild(tdPrice);
          tr.appendChild(tdStock);
          tr.appendChild(tdRating);
          tr.appendChild(tdPhoto);

          document.querySelector("tbody").appendChild(tr);
        }
      })
    }
  </script>
</head>
<body>
  <table>
    <tr>
      <th>Name</th>
      <th>Price</th>
      <th>Stock</th>
      <th>Rating</th>
      <th>Photo</th>
    </tr>
    <tr>
      <td>Product 1</td>
      <td>100</td>
      <td>10</td>
      <td>4.5</td>
      <td></td>
    </tr>
    <tr>
      <td>Product 2</td>
      <td>200</td>
      <td>20</td>
      <td>4.0</td>
      <td></td>
    </tr>
    <tr>
      <td>Product 3</td>
      <td>300</td>
      <td>30</td>
      <td>4.8</td>
      <td></td>
    </tr>
  </table>
</body>
</html>
```

```

    }
  })
  .catch(function(error){
    alert(error);
  })
}
</script>
</head>
<body class="container-fluid">
  <h2>Products Table <button onclick="LoadClick()" class="btn btn-
primary">Load Data</button> </h2>
  <table class="table table-hover">
    <thead>
      <tr>
        <th>Name</th>
        <th>Price</th>
        <th>Stock</th>
        <th>Rating</th>
        <th>Preview</th>
      </tr>
    </thead>
    <tbody>

    </tbody>
  </table>
</body>
</html>

```

Ex: Dynamic Cards

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet"
href="../node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
  <script type="text/javascript">

```

```

function bodyload(){
  fetch("../data/products.json")
  .then(function(res){
    return res.json();
  })
  .then(function(products){
    for(var product of products)
    {
      var div = document.createElement("div");
      div.className = "card p-2 m-2";
      div.style.width = "300px";
      div.innerHTML = `
        <img src=${product.Photo} class="card-img-top" height="160">
        <div class="card-header">
          <h3>${product.Name}</h3>
        </div>
        <div class="card-body">
          <dl>
            <dt>Price</dt>
            <dd>${product.Price}</dd>
            <dt>Stock</dt>
            <dd>${product.Stock}</dd>
            <dt>Rating</dt>
            <dd>
              <span class="bi bi-star-fill text-success"> </span> ${
product.Rating.Rate} [${product.Rating.Count}]
            </dd>
          </dl>
        </div>
        <div class="card-footer">
          <button class="btn btn-danger w-100"> <span class="bi bi-
cart4"></span> Add to Cart</button>
        </div>
      `;
      document.getElementById("catalog").appendChild(div);
    }
  })
}
</script>
</head>
<body class="container-fluid" onload="bodyload()">
  <div id="catalog" class="d-flex flex-wrap">

```



```
</div>
</body>
</html>
```

api.nasa.gov

URL :

https://api.nasa.gov/mars-photos/api/v1/rovers/curiosity/photos?sol=1000&api_key=DEMO_KEY

Result:

```
{ }      Object
{ "photos": [] }    photos array
photos: [ { } ]     id, sol, camera{ }, img_src, earth date, rover { }
```

Ex: Nasa Mars Rover Photos - Table Format

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Nasa</title>
  <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <script>
    function bodyload(){
      fetch("https://api.nasa.gov/mars-photos/api/v1/rovers/curiosity/photos?sol=1000&api_key=DEMO_KEY")
      .then(function(response){
        return response.json();
      })
      .then(function(data){
        for(var photo of data.photos)
        {
          var tr = document.createElement("tr");
          var tdId = document.createElement("td");
```

```

        var tdCameraName = document.createElement("td");
        var tdPreview = document.createElement("td");
        var tdRoverName = document.createElement("td");

        tdId.innerHTML = photo.id;
        tdCameraName.innerHTML = photo.camera.full_name;
        tdPreview.innerHTML = `

```

Ex: Card Style

```

<!DOCTYPE html>
<html lang="en">

```

```

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Nasa</title>
  <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <script>
    function bodyload(){
      fetch("https://api.nasa.gov/mars-
photos/api/v1/rovers/curiosity/photos?sol=1000&api_key=DEMO_KEY")
      .then(function(response){
        return response.json();
      })
      .then(function(data){
        for(var photo of data.photos)
        {
          var div = document.createElement("div");
          div.className= "card m-2 p-2";
          div.style.width= "200px";
          div.innerHTML = `
            <img src=${photo.img_src} class="card-img-top" height="150">
            <div class="card-header"> ${photo.id} </div>
            <div class="card-body">
              <dl>
                <dt>Camera Name</dt>
                <dd>${photo.camera.full_name} </dd>
                <dt>Rover Name</dt>
                <dd>${photo.rover.name}</dd>
              </dl>
            </div>
          `;
          document.querySelector("body").appendChild(div);
        }
      })
    }
  </script>
</head>
<body class="container-fluid d-flex flex-wrap" onload="bodyload()">

</body>
</html>

```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Nasa</title>
  <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <script>
    function bodyload(){
      fetch("https://api.nasa.gov/mars-
photos/api/v1/rovers/curiosity/photos?sol=1000&api_key=DEMO_KEY&quot;")
      .then(function(response){
        return response.json();
      })
      .then(function(data){
        var lstCameras = document.getElementById("lstCameras");
        for(var photo of data.photos)
        {
          var div = document.createElement("div");
          div.className= "card m-2 p-2";
          div.style.width= "200px";
          div.innerHTML = `
            <img src=${photo.img_src} class="card-img-top" height="150">
            <div class="card-header"> ${photo.id} </div>
            <div class="card-body">
              <dl>
                <dt>Camera Name</dt>
                <dd>${photo.camera.full_name} </dd>
                <dt>Rover Name</dt>
                <dd>${photo.rover.name}</dd>
              </dl>
            </div>
          `;
          document.querySelector("main").appendChild(div);

          var option = document.createElement("option");
          option.text = photo.camera.full_name;
          option.value = photo.camera.name;
          lstCameras.appendChild(option);
        }
      })
    }
    bodyload();
  </script>
</head>
<body>
  <div class="container">
    <div class="row">
      <div class="col">
        <div class="card">
          <div class="card-body">
            <div class="card-text">
              <div class="list-group">
                <div class="list-group-item">
                  <div class="input-group">
                    <input type="text" value="Search for camera name" />
                    <button type="button" value="Search" />
                  </div>
                </div>
                <div class="list-group-item">
                  <div class="input-group">
                    <input type="text" value="Search for rover name" />
                    <button type="button" value="Search" />
                  </div>
                </div>
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</body>
</html>
```

```

    }
  })
}
function CameraChange(){
  document.querySelector("main").innerHTML = "";
  var cameraName = document.getElementById("lstCameras").value;
  fetch("https://api.nasa.gov/mars-
photos/api/v1/rovers/curiosity/photos?sol=1000&api_key=DEMO_KEY")
  .then(function(response){
    return response.json();
  })
  .then(function(data){
    var result = data.photos.filter(function(photo){
      return photo.camera.name==cameraName;
    });
    for(var photo of result)
    {
      var div = document.createElement("div");
      div.className= "card m-2 p-2";
      div.style.width= "200px";
      div.innerHTML = `
        <img src=${photo.img_src} class="card-img-top" height="150">
        <div class="card-header"> ${photo.id} </div>
        <div class="card-body">
          <dl>
            <dt>Camera Name</dt>
            <dd>${photo.camera.full_name} </dd>
            <dt>Rover Name</dt>
            <dd>${photo.rover.name}</dd>
          </dl>
        </div>
      `;
      document.querySelector("main").appendChild(div);
    }
  })
}
</script>
</head>
<body class="container-fluid" onload="bodyload()">
  <section class="row">
    <nav class="col-3">
      <h3>Filter Photos</h3>
      <select id="lstCameras" onchange="CameraChange()" class="form-

```

```

select"></select>
  </nav>
  <main class="col-9 d-flex flex-wrap">

    </main>
  </section>
</body>
</html>

```

fakestoreapi.com

- Provides data for ERP - Shopping Online
- API Requests

<http://fakestoreapi.com/products> [] of products "object" - 20
<http://fakestoreapi.com/products/1> { } specific product
<http://fakestoreapi.com/products/categories> [] of categories "string"
<http://fakestoreapi.com/products/category/jewelery> [] of specific category

```

products[ ]
  products.filter(function(item){
    return item.category=="jewelery"
  })

  products.find(function(item){
    return item.id==2;
  })

```

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Shopper</title>
  <link rel="stylesheet"

```

```

href="../node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
  <style>
    a{
      color:white;
      text-decoration: none;
    }
    a:hover {
      color:yellow;
    }
  </style>
  <script type="text/javascript">
    function LoadCategories(){
      fetch("http://fakestoreapi.com/products/categories");
      .then(function(response){
        return response.json();
      })
      .then(function(categories){
        categories.unshift("all");
        for(var category of categories){
          var option = document.createElement("option");
          option.text = category.toUpperCase();
          option.value = category;
          document.getElementById("lstCategories").appendChild(option);
        }
      })
    }

    function LoadProducts(url){
      fetch(url)
      .then(function(response){
        return response.json();
      })
      .then(function(products){
        document.querySelector("main").innerHTML="";
        for(var product of products){
          var div = document.createElement("div");
          div.className = "card m-2 p-2";
          div.style.width = "200px";
          div.innerHTML = `
            <img src=${product.image} class="card-img-top" height="160">
            <div class="card-header"

```

```

style="height:130px"><p>${product.title}</p></div>
    <div class="card-body">
        <dl>
            <dt>Price</dt>
            <dd>${product.price}</dd>
            <dt>Rating</dt>
            <dd><span class="bi bi-star-fill text-success"></span>
                ${product.rating.rate} [ ${product.rating.count} ]
            </dd>
        </dl>
    </div>
    <div class="card-footer">
        <button onclick="AddClick('${product.id}')" class="btn btn-danger
w-100">
            <span class="bi bi-cart4"></span>
            Add to Cart
        </button>
    </div>
    `;
    document.querySelector("main").appendChild(div);
}
})
}

function bodyload(){
    LoadCategories();
    LoadProducts("http://fakestoreapi.com/products");
    GetCartItemsCount();
}
function CategoryChanged(){
    var categoryName = document.getElementById("lstCategories").value;
    if(categoryName=="all") {
        LoadProducts("http://fakestoreapi.com/products");
    } else {
        LoadProducts(`http://fakestoreapi.com/products/category/${categoryName}`);
    }
}
function NavClicked(categoryName){
    if(categoryName=="all"){
        LoadProducts("http://fakestoreapi.com/products");
    } else {
        LoadProducts(`http://fakestoreapi.com/products/category/${categoryName}`);
    }
}

```



```

    }
  }
  var cartItems = [];
  function GetCartItemsCount(){
    document.getElementById("lblCount").innerHTML = cartItems.length;
  }
  function AddClick(id) {
    fetch(`http://fakestoreapi.com/products/${id}`)
    .then(function(response){
      return response.json();
    })
    .then(function(product){
      cartItems.push(product);
      alert(`${product.title}\nAdded to Cart`);
      GetCartItemsCount();
    })
  }
}
</script>
</head>
<body class="container-fluid" onload="bodyload()">
  <header class="p-2 mt-2 d-flex justify-content-between bg-primary text-white">
    <div>
      <h3>Shopper.</h3>
    </div>
    <div>
      <span class="me-2"><a
href="javascript:NavClicked('all')">Home</a></span>
      <span class="me-2"><a
href="javascript:NavClicked('electronics')">Electronics</a></span>
      <span class="me-2"><a
href="javascript:NavClicked('jewelery')">Jewelery</a></span>
      <span class="me-2"><a>Men's Fashion</a></span>
      <span class="me-2"><a>Women's Fashion</a></span>
    </div>
    <div>
      <span class="me-2"><a><span class="bi bi-person"></span></a></span>
      <span class="me-2"><a><span class="bi bi-heart"></span></a></span>
      <span class="me-2 position-relative"><a href="#cart" data-bs-
toggle="modal"><span class="bi bi-cart3"></span> <span class="badge bg-
danger rounded rounded-circle position-absolute" id="lblCount"></span>
</a></span>
      <div class="modal fade" id="cart">
        <div class="modal-dialog">

```

```
<div class="modal-content">
  <div class="modal-header">
    <h2 class="text-dark">Your Cart Items</h2>
    <button class="btn-close" data-bs-dismiss="modal"></button>
  </div>
  <div class="modal-body">
    <table class="table table-hover">
      <thead>
        <tr>
          <th>Name</th>
          <th>Price</th>
          <th>Preview</th>
        </tr>
      </thead>
      <tbody>

      </tbody>
    </table>
  </div>
</div>
</div>
</div>
</div>
</div>
</header>
<section class="mt-2 row">
  <nav class="col-2">
    <div>
      <label class="form-label">Select Category</label>
      <div>
        <select id="lstCategories" onchange="CategoryChanged()"
class="form-select">

        </select>
      </div>
    </div>
  </nav>
  <main class="col-10 d-flex flex-wrap overflow-auto" style="height:500px">

  </main>
</section>
<script src="../node_modules/jquery/dist/jquery.js"></script>
<script src="../node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
```

</html>

<!DOCTYPE html>

<html lang="en">

<head>

 <meta charset="UTF-8">

 <meta http-equiv="X-UA-Compatible" content="IE=edge">

 <meta name="viewport" content="width=device-width, initial-scale=1.0">

 <title>Shopper</title>

 <link rel="stylesheet"

href="../node_modules/bootstrap/dist/css/bootstrap.css">

 <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-icons.css">

 <style>

 a{
 color:white;
 text-decoration: none;

 }

 a:hover {
 color:yellow;

 }

 </style>

 <script type="text/javascript">

 function LoadCategories(){

 fetch("http://fakestoreapi.com/products/categories")

 .then(function(response){

 return response.json();

 })

 .then(function(categories){

 categories.unshift("all");

 for(var category of categories){

 var option = document.createElement("option");

 option.text = category.toUpperCase();

 option.value = category;

```

        document.getElementById("lstCategories").appendChild(option);
    }
}

function LoadProducts(url){
    fetch(url)
    .then(function(response){
        return response.json();
    })
    .then(function(products){
        document.querySelector("main").innerHTML="";
        for(var product of products){
            var div = document.createElement("div");
            div.className = "card m-2 p-2";
            div.style.width = "200px";
            div.innerHTML = `
                <img src=${product.image} class="card-img-top" height="160">
                <div class="card-header"
style="height:130px"><p>${product.title}</p></div>
                <div class="card-body">
                    <dl>
                        <dt>Price</dt>
                        <dd>${product.price}</dd>
                        <dt>Rating</dt>
                        <dd><span class="bi bi-star-fill text-success"></span>
${product.rating.rate} [${product.rating.count}]
                    </dl>
                </div>
                <div class="card-footer">
                    <button onclick="AddClick(${product.id})" class="btn btn-danger
w-100">
                        <span class="bi bi-cart4"></span>
                        Add to Cart
                    </button>
                </div>
            `;
            document.querySelector("main").appendChild(div);
        }
    })
}

function bodyload(){

```

```

    LoadCategories();
    LoadProducts("http://fakestoreapi.com/products");
    GetCartItemsCount();
}
function CategoryChanged(){
    var categoryName = document.getElementById("lstCategories").value;
    if(categoryName=="all") {
        LoadProducts("http://fakestoreapi.com/products");
    } else {
        LoadProducts(`http://fakestoreapi.com/products/category/${categoryName}`);
    }
}
function NavClicked(categoryName){
    if(categoryName=="all"){
        LoadProducts("http://fakestoreapi.com/products");
    } else {
        LoadProducts(`http://fakestoreapi.com/products/category/${categoryName}`);
    }
}
var cartItems = [];
function GetCartItemsCount(){
    document.getElementById("lblCount").innerHTML = cartItems.length;
}
function AddClick(id) {
    fetch(`http://fakestoreapi.com/products/${id}`)
    .then(function(response){
        return response.json();
    })
    .then(function(product){
        cartItems.push(product);
        alert(`${product.title}\nAdded to Cart`);
        GetCartItemsCount();
    })
}
function ShowCartClick(){
    document.querySelector("tbody").innerHTML = "";
    for(var item of cartItems)
    {
        var tr = document.createElement("tr");
        var tdName = document.createElement("td");
        var tdPrice = document.createElement("td");
    }
}

```

```

    var tdPreview = document.createElement("td");
    var tdAction = document.createElement("td");

    tdName.innerHTML = item.title;
    tdPrice.innerHTML = item.price;
    tdPreview.innerHTML = `

```

```
<div class="modal-dialog">
  <div class="modal-content">
    <div class="modal-header">
      <h2 class="text-dark">Your Cart Items</h2>
      <button class="btn-close" data-bs-dismiss="modal"></button>
    </div>
    <div class="modal-body">
      <table class="table table-hover">
        <thead>
          <tr>
            <th>Name</th>
            <th>Price</th>
            <th>Preview</th>
          </tr>
        </thead>
        <tbody>

          </tbody>
      </table>
    </div>
  </div>
</div>
</div>
</div>
</div>
</div>
</header>
<section class="mt-2 row">
  <nav class="col-2">
    <div>
      <label class="form-label">Select Category</label>
      <div>
        <select id="lstCategories" onchange="CategoryChanged()"
class="form-select">

          </select>
        </div>
      </div>
    </nav>
    <main class="col-10 d-flex flex-wrap overflow-auto" style="height:500px">

    </main>
  </section>
<script src="../node_modules/jquery/dist/jquery.js"></script>
<script src="../node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
```

```
</body>
</html>
```

Non-Primitive Types

1. Array

2. Object

Symbol for Object

- Symbol is a primitive type introduced from ES6.
- Symbol configures a hidden field, which sends data but not discovered in iterations.

Syntax:

```
var UserId = Symbol();

var user = {
  [UserId] : 1001,
  UserName: "John",
  Age: 22
}

document.write(user[UserId]);
```

Ex:

```
<script>
var UserId = Symbol();
var User = {
  [UserId]: 1001,
  UserName: "John",
  Age: 22
}
for(var property in User)
{
  document.write(`${property} - ${User[property]}<br>`);
}
```



```
document.write("User Id : " + User[UserId]);  
</script>
```

FAQ's:

1. How to remove any property from object?

A. By using JavaScript "delete" operator.

Syntax:

```
delete object.Property|Key
```

Ex:

```
<script>  
var product = {  
    Name : "TV",  
    Price: 45000.33,  
    Rating: 4.3  
};  
delete product.Price;  
for(var property in product)  
{  
    document.write(`${property} : ${product[property]}<br>`);  
}  
</script>
```

2. How to verify the availability of any property in object?

A. By using JavaScript "in" operator

Syntax:

```
"PropertyName" in Object;    // true - false
```

Ex:

```
<script>  
var product = {  
    Name : "TV",  
    Price: 45000.33,  
    Rating: 4.3,  
};
```

```
document.write(`Is Qty Available : ${"Qty" in product}`);  
</script>
```

3. How to know the data type of every property in object?

A. By using JavaScript "typeof" operator.

Syntax:

```
typeof product.Price // number  
typeof product.Name // string
```

Issues with Object

- Keys are always string type.
- You can't have size for keys

Ex:

```
<script>  
var product = {  
  Name : "TV",  
  Price: 45000.33,  
  Rating: {Rate:4.3, Count:2340},  
  Stock: true,  
};  
var keys = [];  
for(var property in product)  
{  
  keys.push(property);  
}  
document.write("Total Number of Keys : " + keys.length);  
</script>
```

- You need explicit(external) iterators to read all keys and values
[for..in]
- You need explicit operators to manipulate.
[delete, in]
- It is slow in access and in manipulations.

What is Good ?

It can have structured data. [JSON]

JavaScript Map Type

- It is a new type available from ES5.
- It is a key and value collection same like object.
- Key can be any type

```
{  
  "Name": "TV",  
  1 : "value",  
  true: "value"  
}
```

- It provides implicitly methods for querying and manipulations

map.keys()	-It returns all keys
map.values()	-it returns all Values
map.entries()	-it returns both keys and values
map.has()	-it is used to verify the availability of keys .it is

a boolean method that returns true if key is

available or false if keys not available

map.delete()	-it is built in method which can delete any keys
map.clear()	-it is used to delete all entities
map.set()	-it is used to set a new key and value
map.get()	-it is used to get a value with reference of key

- It is faster than object.
- It is schema less(structure less means one key is string another key is number...etc).

Ex:

```
<script>  
  var data = new Map();  
  data.set(1, "HTML is a markup language");  
  data.set("CSS", "CSS defines styles for HTML");  
  data.delete(1);  
  document.write("Total Keys :" + data.size + "<br>");  
  document.write(data.get("CSS"));  
</script>
```

Ex: (delete any key) and (access any keys)

```
var data = new Map();  
  data.set(1, "HTML is a markup language");  
  data.set("CSS", "CSS defines styles for HTML");  
  data.delete(1);  
  document.write("Total Keys :" + data.size + "<br>");  
  document.write(data.get("CSS"));
```

Ex:

```
<script>
  var data = new Map();
  data.set("html", "HTML is a markup language");
  data.set("CSS", "CSS defines styles for HTML");
  for(var item of data.entries()){
    document.write(item + "<br>");
  }
  if(data.has("html")){
    document.write(data.get("html"));
  } else {
    document.write("HTML Not Found");
  }
</script>
```

Object Map

Keys are string	Keys can be any type
Structured	Schema less
Need explicit operator	Have implicit methods
Need explicit iterators	Have implicit iterators
No Size	Can define size
Slow	Faster

Date Type

- JavaScript date and time values are handled by "Date" constructor.

Syntax:

var mfd = new Date(); loads current date and time.

var mfd = new Date("yy-MM-dd hrs:min:sec.milliSec"); loads mentioned date and time

{new—dynamic memory allocating operator}

- JavaScript provides various methods for presenting date and time

getHours() –it returns the hour number in 24 hours (Hour Number 0 to 23)

getMinutes() –it returns the minutes number 0-59(Minutes 0 to 59)

getSeconds() —it returns the Seconds number 0 to 59

getMilliseconds()—it returns the Milli Seconds number 0 to 99

1000-1 sec(CPU Clock)

100-1 sec (System Clock)

getDay()—it returns the Weekday number (0 = Sunday ,1-Monday)

getDate()—it returns the Date number(ex- today date= "24")

getMonth()—it returns the Month number 0 = January

getFullYear()—it returns the Year Number in 4 digit (2023)

toLocaleDateString()—it returns only date(ex-1/20/2023)

toDatestring()—it returns only date(ex--Sat Feb 25 2023)

toLocaleTimeString()—it returns only time in 12hr format (ex--8:30:45 AM)

toTimeString()—it returns only time in 24hr format (ex-16:43:30 GMT+0900
(Korean Standard Time)

toString()—it returns both date and time in a string format

Ex:

```
<script>
  var Mfd = new Date("2023-02-18 08:30:45.88");
  var weekdays = ["Sunday", "Monday", "Tue", "Wed", "Thu", "Friday",
"Staturday"];
  var months = ["January", "February", "March", "Apr", "May"];
  document.write(
    WeekDay : ${weekdays[Mfd.getDay()]} <br>
```

```

        Date    : ${Mfd.getDate()} <br>
        Month   : ${months[Mfd.getMonth()]} <br>
        Year    : ${Mfd.getFullYear()} <br>
        Time    : ${Mfd.toLocaleTimeString()}
    );
</script>

```

Ex:

```

<script>
    var Mfd = new Date("2023-02-18 08:30:45.88");
    var weekdays = ["Sunday", "Monday", "Tue", "Wed", "Thu", "Friday",
"Saturday"];
    var months = ["January", "February", "March", "Apr", "May"];
    document.write(
        ${Mfd.getDate()} ${weekdays[Mfd.getDay()]}, ${months[Mfd.getMonth()]}
        ${Mfd.getFullYear()}
    );
</script>

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
    <link rel="stylesheet" href="./node_modules/bootstrap-icons/font/bootstrap-
icons.css">
    <script>
        function LoadTime(){
            var now = new Date();
            document.getElementById("lblTime").innerHTML =
now.toLocaleTimeString();
        }
        function LoadDate(){
            var now = new Date();
            document.getElementById("lblDate").innerHTML = now.toDateString();
            var hrs = now.getHours();
            var img = document.querySelector("img");

```

```

var p = document.querySelector("p");
if(hrs>=0 && hrs<=12) {
    img.src = "../public/images/morning.gif";
    p.innerHTML = "Hello ! Good Morning";
} else if (hrs>=13 && hrs<=17) {
    img.src= "../public/images/afternoon.jpg";
    p.innerHTML = "Hello ! Good Afternoon";
} else if(hrs>17 && hrs<=23) {
    img.src = "../public/images/evening.gif";
    p.innerHTML = "Hello ! Good Evening";
}
}

function bodyload(){
    setInterval(LoadTime, 1000);
    LoadDate();
}
</script>
<style>
img {
    width: 200px;
    height: 200px;
    border:1px solid red;
    border-radius: 100px;
}
</style>
</head>
<body class="container-fluid" onload="bodyload()">
    <div class="btn-toolbar bg-danger d-flex mt-2 justify-content-between">
        <div>
            <button class="btn btn-danger">Home</button>
            <button class="btn btn-danger">Fashion</button>
            <button class="btn btn-danger">Electronics</button>
        </div>
        <div>
            <button class="btn btn-danger"><span id="lblDate"></span></button>
        </div>
        <div>
            <button class="btn btn-danger"> <span class="bi bi-clock"></span> <span
id="lblTime"></span> </button>
        </div>
    </div>
    <main style="height: 400px;" class="d-flex justify-content-center align-items-

```

```
center">
  <div class="text-center">
    <img>
    <p></p>
  </div>
</main>
</body>
</html>
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function SubmitClick(){
      var date = new Date(document.getElementById("date").value);
      var weekdays = ["Sunday", "Monday", "Tuesday",
"Wednesday", "Thu", "Friday", "Saturday"];
      document.write(
        You have to leave on ${weekdays[date.getDay()]}.
      );

    }
  </script>
</head>
<body>
  Departure : <input type="date" id="date"><button
onclick="SubmitClick()">Submit</button>
</body>
</html>
```

Summary

- Primitive Types

number, string, boolean, null, undefined, symbol

- Non Primitive

array, object, map

- Date

JavaScript Operators

- Operator is an object that evaluates a value.
- Operator stores data in operands

Left operand entity Right operand

X + Y

- Based on how many operands an operator can handle, they are classified into following groups

1. Unary Operator

2. Binary Operator

3. Ternary Operator

- Unary Operator uses only one "operand".

x++, x--, --x, ++x

x = 0;

x++; x = x + 1

x = 1

x++; x = x + 1

 x = 1 + 1

x = 2

- Increment ++ and Decrement --

a) Post Increment and Post Decrement

```
x = 10;  
y = x++;      ? y=10, x=11  
y = x-- ;     ? y=10, x=9
```

b) Pre Increment and Pre Decrement [Compiler uses RHS assigned to LHS]

```
x = 10;  
y = ++x;      ? x=11, y=11  
y = --x;      ? x=9, y=9
```

Ex:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Document</title>  
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-  
icons.css">  
  <link rel="stylesheet"  
href="../node_modules/bootstrap/dist/css/bootstrap.css">  
  <script>  
    function LoadProduct(id) {  
      fetch( http://fakestoreapi.com/products/${id} )  
      .then(function(response){  
        return response.json();  
      })  
      .then(function(product){  
        document.getElementById("title").innerHTML = product.title;  
        document.getElementById("pic").src= product.image;  
        document.getElementById("details").innerHTML = `&#8377;  
${product.price}<br><span class="bi bi-star-fill"></span> ${product.rating.rate}`;  
      })  
    }  
    function bodyload(){  
      LoadProduct(1);
```

```

    }
    var count = 1;
    function NextClick(){
        count++;
        LoadProduct(count);
    }
    function PreviousClick(){
        count--;
        LoadProduct(count);
    }
</script>
<style>
    #details {
        position: fixed;
        top: 100px;
        right: 200px;
        border:2px solid green;
        width: 100px;
        height: 60px;
        font-size: 20px;
    }
</style>
</head>
<body onload="bodyload()" class="container-fluid d-flex justify-content-center align-items-center">
    <div class="card p-2 m-2" style="width:500px">
        <div class="card-header text-center">
            <p id="title"></p>
        </div>
        <div class="card-body text-center">
            <img height="300" width="100%" id="pic">
        </div>
        <div class="card-footer text-center">
            <button class="btn btn-danger" onclick="PreviousClick()">
                <span class="bi bi-chevron-left"></span>
            </button>
            <button class="btn btn-danger" onclick="NextClick()">
                <span class="bi bi-chevron-right"></span>
            </button>
        </div>
    </div>

    <div id="details" class="badge bg-success text-white"></div>

```

```
</body>
</html>
```

- Binary operators handle 2 operands

$a + b$, a / b , $a * b$
 $a + b * c$

- Ternary Operators handle 3 operands

?:

first ? second : third
(condition)?true:false

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function ToggleClick(){
      var button = document.getElementById("toggle");
      button.innerHTML = (button.innerHTML=="Hide")?"Show":"Hide";
      var poster = document.getElementById("poster");
      poster.style.display = (poster.style.display=="block")?"none":"block";
    }
  </script>
</head>
<body>
  <button id="toggle" onclick="ToggleClick()">Hide</button>
  <br><br>
  
</body>
</html>
```

Ternary

(condition) ? value_on_true : value_on_false => Minified

```
if (condition)
{
    value_on_true;
}
else
{
    value_on_false;
}
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script>
        function StockChanged(){
            var stock = document.getElementById("Stock");
            var lblStock = document.getElementById("lblStock");
            lblStock.innerHTML = (stock.checked)?"Available":"Out Of Stock";
        }
    </script>
</head>
<body>
    <input type="checkbox" onchange="StockChanged()" id="Stock"> <label
id="lblStock">Out of Stock</label>
</body>
</html>
```

Task:

1. Create a new Array

```
var cities = [ "A", "D", "B", "C" ]
```

2. Display Cities list in

3. Keep a Toggle button for sorting [once click - ascending, again click -

descending]

Ans→

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="../node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-icons.css">
  <script>
    var alphas=["A","C","D","F","B","E"];
    function loadAlpha(){

      document.getElementById("txt").innerHTML=""
      for(item of alphas){
        var li=document.createElement("li");
        li.innerHTML=item;
        document.querySelector("ol").appendChild(li)
      }

    }
    function bodyload(){
      loadAlpha()
    }
    function sortdownClick(){

      alphas.sort()
      loadAlpha()
    }
    function sortupClick(){

      alphas.sort()
      alphas.reverse()
      loadAlpha()
    }

  </script>
</head>
<body onload="bodyload()" class="container-fluid">
  <ol id="txt">

  </ol>
```

```

<button class="btn btn-primary" onclick="sortdownClick()"><span class="bi bi-sort-alpha-down"></span></button>
<button class="btn btn-danger" onclick="sortupClick()"><span class="bi bi-sort-alpha-up"></span></button>

</body>
</html>

```

- Operators are again classified into various groups based on the type of value they return.

1. Arithmetic
2. Assignment
3. Comparison
4. Logical
5. Bitwise
6. Special

1 Arithmetic

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus Division
**	Exponent => Math.pow() => 2**3 = 8 [Math.pow(2,3)]
++	Increment
--	Decrement

Syntax:

number + number	= number
number + string	= string 1 + "A" = 1A
number + boolean	= number 1 + true = 2
string + string	= string "A" + "B" = AB
boolean+string	= string true + "A" = trueA
boolean+boolean	= number true + true = 2

number - string	= NaN
string - string	= NaN

<u>string - boolean</u>	<u>= NaN</u>
<u>number-number</u>	<u>= number</u>
<u>number-boolean</u>	<u>= number</u>

2. Assignment Operators

<u>+=</u>	<u>Add and Assign</u>
<u>-=</u>	<u>Subtract and Assign</u>
<u>/=</u>	<u>Divide and Assign</u>
<u>%=</u>	<u>Modulus and Assign</u>
<u>*</u>	<u>Multiply and Assign</u>

Syntax:

```
x = 10;
x += 20;    x = x + 20
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function AddClick(){
      var Name = document.getElementById("Name").value;
      document.getElementById("result").innerHTML += Name + "<br>";
      document.getElementById("Name").value = "";
    }
  </script>
  <link rel="stylesheet" href="/node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body>
  Product Name :
  <input type="text" id="Name"> <button onclick="AddClick()">Add</button>
  <br><br>
  <p id="result"></p>
</body>
</html>
```


Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    var margin = 1;
    function MoveClick(){

      margin *= 5;
      document.querySelector("img").style.marginLeft = margin + "px";
    }
  </script>
  <link rel="stylesheet" href="/node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body>
  <button onclick="MoveClick()">Move 20px</button>
  <br><br>
  
</body>
</html>
```

Arithmetic Operators

+
-
*
/
%
**
++
--

Assignment Operators

+=
-=

```
*=  
/=   
% =
```

Ex:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Document</title>  
  <script>  
    var margin = 0;  
    function MoveRightClick(){  
  
      margin += 20;  
      document.querySelector("img").style.marginLeft = margin + "px";  
    }  
    function MoveLeftClick(){  
  
      margin -= 20;  
      document.querySelector("img").style.marginLeft = margin + "px";  
    }  
  </script>  
  <link rel="stylesheet" href="/node_modules/bootstrap-icons/font/bootstrap-  
icons.css">  
</head>  
<body>  
  <button onclick="MoveRightClick()">Move Right</button>  
  <button onclick="MoveLeftClick()">Move Left </button>  
  <br><br>  
    
</body>  
</html>
```

Ex: KFC Form

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>KFC-Online Order</title>
<link rel="stylesheet"
href="../node_modules/bootstrap/dist/css/bootstrap.css">
<link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
<script type="text/javascript">
    function OrderClick(){
        document.getElementById("lblName").innerHTML =
document.getElementById("txtName").value;
        document.getElementById("lblMobile").innerHTML=
document.getElementById("txtMobile").value;

        var mealName = "";
        var mealCost = 0;

        var adOnName = "";
        var adOnCost = 0;

        var total = 0;

        var optBurger = document.getElementById("optBurger");
        var optRoller = document.getElementById("optRoller");
        var mealPic = document.getElementById("mealPic");

        if(optBurger.checked) {
            mealPic.src= "../public/images/OMG1.png";
            mealName = optBurger.value;
            mealCost = 120;
        }
        if(optRoller.checked) {
            mealPic.src= "../public/images/OMG2.png";
            mealName = optRoller.value;
            mealCost = 100;
        }

        var optKrusher = document.getElementById("optKrusher");
        var optWings = document.getElementById("optWings");

        if(optKrusher.checked) {
            adOnName += optKrusher.value + "<br>";
            adOnCost = 40;

```

```

        mealCost+=adOnCost;
    }

    if(optWings.checked) {
        adOnName += optWings.value + "<br>";
        adOnCost = 70;
        mealCost += adOnCost;
    }
    total = mealCost;

    document.getElementById("lblMeal").innerHTML = mealName;
    document.getElementById("lblAdOn").innerHTML = adOnName;
    document.getElementById("lblTotal").innerHTML = `<b>₹
    ${total}</b>`

    }
</script>
</head>
<body class="container-fluid">
    <header>
        
    </header>
    <section>
        <div class="accordion" id="kfcForm">
            <div class="accordion-item">
                <div class="accordion-header">
                    <button class="btn btn-danger w-100" data-bs-target="#customer"
data-bs-toggle="collapse">
                        Customer Info
                    </button>
                </div>
                <div class="accordion-collapse collapse show" id="customer" data-bs-
parent="#kfcForm">
                    <div class="accordion-body">
                        <dl>
                            <dt>Customer Name</dt>
                            <dd><input type="text" id="txtName" class="form-control"></dd>
                            <dt>Mobile Number</dt>
                            <dd><input type="text" id="txtMobile" class="form-control"></dd>
                        </dl>
                    </div>
                </div>
            </div>
        </div>
    </div>

```

```

<div class="accordion-item">
  <div class="accordion-header">
    <button class="btn btn-danger w-100" data-bs-target="#meal" data-
bs-toggle="collapse">
      Select a Meal
    </button>
  </div>
  <div class="accordion-collapse collapse" id="meal" data-bs-
parent="#kfcForm">
    <div class="accordion-body">
      <div class="row">
        <div class="col">
          <div class="card">
            

            <div class="card-header">
              <h2>OMG Burger</h2>
            </div>
            <div class="card-body">
              <h3> <input type="radio" class="form-check-input"
name="meal" value="OMG Burger" id="optBurger"> &#8377; 120/-</h3>
            </div>
          </div>
        </div>
        <div class="col">
          <div class="card">
            

            <div class="card-header">
              <h2>OMG Roller</h2>
            </div>
            <div class="card-body">
              <h3> <input type="radio" class="form-check-input"
name="meal" value="OMG Roller" id="optRoller"> &#8377; 100/-</h3>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
  <div class="accordion-item">
    <div class="accordion-header">

```


Comparison Operators

>	Greater than
>=	Greater than or equal
<	Less than
<=	Less than or equal
==	Equal
===	Identical equal
!=	Not equal
!==	Not Identical

- Comparison operators are used in decision making.
- All comparison operators return boolean true or false.

FAQ: What is difference between "==" and "===" ?

Ans: "==" can compare value of various types.

10=="10" = true

"===" can compare only values of similar type.

10=== "10" = false

Logical Operators

&&	AND
	OR
!	NOT

- Logical operators are used to build a statement using multiple expressions.
- AND will return true when all expressions evaluate to "true".
- OR will return true when any one expression evaluate to true.

```
var x = 23;
```

```
x > 20 && x < 30    = true
```

```
x > 20 && x == 0    = false
```

```
x > 20 || x==0      = true
```

- NOT [!] will return the negation of an expression


```
var sort = true;
if(sort==false) => not recommended
if(!sort)
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Logical</title>
  <script>
    function LoginClick(){
      var username = document.getElementById("UserName").value;
      var password = document.getElementById("Password").value;
      fetch("../data/users.json")
      .then(function(response){
        return response.json();
      })
      .then(function(users){
        for(var user of users)
        {
          if((user.UserName===username||user.Email===username) &&
user.Password===password)
          {
            document.write("Login Success..");
          } else {
            document.getElementById("msg").innerHTML = "Invalid User Name
/ Password".fontcolor('red');
          }
        }
      })
    }
  </script>
</head>
<body>
  <h2>User Login</h2>
  <dl>
    <dt>User Name or Email</dt>
    <dd><input type="text" id="UserName"></dd>
    <dt>Password</dt>
```

```
<dd><input type="password" id="Password"></dd>
</dl>
<button onclick="LoginClick()">Login</button>
<p align="center" id="msg"></p>
</body>
</html>
```

Special Operators

1. Ternary Operator

?:

2. delete : It deletes any property from an object

3. typeof : It returns the data type of value stored in a reference.

4. in : It is used to verify the existence of a property in object.

```
"Price" in Product => true / false
typeof Price      => number
delete Product.Price => removes price from product
delete Math.PI     => Invalid [Built-in objects read-only]
delete Products.length => Invalid
```

5. new : It is dynamic memory allocation operator.

```
new Array()
new Map()
new Date()
```

6. void : It discards the return of any element or object.

```
<a href="javascript:void()">Home</a>
```

JavaScript Statements

JavaScript Statements

- Statements are used to control the execution flow of a program.
- Statements are classified into various types

1. Selection Statements

if, else, switch, case, default

2. Iteration and Lopping Control Statements

for, while, do while, for..in, for..of

3. Jump Statements

break, continue, return

4. Exception Handling Statements

try, catch, throw, finally

1. Selection Statements

- They are used in decision making
- The keywords are
if, else, switch, case, default

The IF Statement

- It is decision making statement that executes a set of statements when condition evaluates to true and another set of statement when it is false.

- The IF statement have various forms

1. Forward Jump

2. Simple Decision

3. Multiple Choice

4. Multi Level Decisions

Forward Jump:

- It is a decision making without any alternative process.
- The statements will execute only when condition is true.

Syntax:

```
if (condition )
{
    statements on true;
}
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script>
        var userDetails = {
            "Card": "4444555577776890",
            "Cvv": "3456",
            "Expiry": "2024"
        }
        function VerifyCard(){
            var card = document.getElementById("Card").value;
            var cvv = document.getElementById("Cvv");
            if(userDetails.Card===card){
                cvv.disabled = false;
                document.getElementById("tick").className = "bi bi-check-circle-fill";
                document.getElementById("tick").style.color = "green";
                cvv.focus();
            }
        }
        function VerifyCvv(){
            var cvv = document.getElementById("Cvv").value;
            if(userDetails.Cvv===cvv) {
                document.getElementById("Expiry").disabled = false;
            }
        }
        function VerifyExpiry(){
            var expriy = document.getElementById("Expiry").value;
            if(userDetails.Expiry===expriy) {
                document.getElementById("btnPay").disabled = false;
            }
        }
    </script>
```

```

    <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body>
  <fieldset>
    <legend>Payment Details</legend>
    <dl>
      <dt>Card Number</dt>
      <dd><input type="text" id="Card" onkeyup="VerifyCard()"><span
style="color:red" class="bi bi-x-circle-fill" id="tick"></span></dd>
      <dt>CVV</dt>
      <dd><input type="text" id="Cvv" onkeyup="VerifyCvv()" size="4"
disabled></dd>
      <dt>Expiry</dt>
      <dd>
        <select id="Expiry" disabled onchange="VerifyExpiry()">
          <option>2023</option>
          <option>2024</option>
          <option>2025</option>
        </select>
      </dd>
    </dl>
    <button id="btnPay" disabled>Pay</button>
  </fieldset>
</body>
</html>

```

Ex: Show and Hide Elements

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    var userDetails = {
      "Card": "4444555577776890",
      "Cvv": "3456",
      "Expiry": "2024"
    }
    function VerifyCard(){

```

```

var card = document.getElementById("Card").value;

if(userDetails.Card===card){
    document.getElementById("tick").className = "bi bi-check-circle-fill";
    document.getElementById("tick").style.color = "green";
    document.getElementById("cvvContainer").style.display = "block";
}
}
function VerifyCvv(){
    var cvv = document.getElementById("Cvv").value;
    if(userDetails.Cvv===cvv) {
        document.getElementById("expiryContainer").style.display = "block";
    }
}
function VerifyExpiry(){
    var expriy = document.getElementById("Expiry").value;
    if(userDetails.Expiry===expriy) {
        document.getElementById("btnPay").style.display = "block";
    }
}
</script>
<link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body>
<fieldset>
    <legend>Payment Details</legend>
    <dl>
        <dt>Card Number</dt>
        <dd><input type="text" id="Card" onkeyup="VerifyCard()"><span
style="color:red" class="bi bi-x-circle-fill" id="tick"></span></dd>
        <div id="cvvContainer" style="display:none">
            <dt>CVV</dt>
            <dd><input type="text" id="Cvv" onkeyup="VerifyCvv()" size="4"></dd>
        </div>
        <div id="expiryContainer" style="display:none">
            <dt>Expiry</dt>
            <dd>
                <select id="Expiry" onchange="VerifyExpiry()">
                    <option>2023</option>
                    <option>2024</option>
                    <option>2025</option>
                </select>
            </dd>
        </div>
    </dl>
</fieldset>

```

```

        </dd>
    </div>
</dl>
    <button id="btnPay" style="display:none" >Pay</button>
</fieldset>
</body>
</html>

```

2. Simple Decision

- It is a decision making statement that provides one alternative to execute when condition is false.
- Alternative is configured by using "else" clause.

"IF ... Else"

Syntax:

```

if (condition )
{
    statement on true;
}
else
{
    statement on false;
}

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script>
        var userDetails = {
            "Card": "4444555577776890",
            "Cvv": "3456",
            "Expiry": "2024"
        }
        function VerifyCard(){
            var card = document.getElementById("Card").value;

```

```

    if(userDetails.Card===card){
        document.getElementById("tick").className = "bi bi-check-circle-fill";
        document.getElementById("tick").style.color = "green";
        document.getElementById("cvvContainer").style.display = "block";
        document.getElementById("btnPay").disabled = false;
    } else {
        document.getElementById("btnPay").disabled = true;
        document.getElementById("tick").className = "bi bi-x-circle-fill";
        document.getElementById("tick").style.color = "red";
    }
}
function VerifyCvv(){
    var cvv = document.getElementById("Cvv").value;
    if(userDetails.Cvv===cvv) {
        document.getElementById("expiryContainer").style.display = "block";
    }
}
function VerifyExpiry(){
    var expiry = document.getElementById("Expiry").value;
    if(userDetails.Expiry===expiry) {
        document.getElementById("btnPay").style.display = "block";
    }
}
</script>
<link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body>
<fieldset>
<legend>Payment Details</legend>
<dl>
<dt>Card Number</dt>
<dd><input type="text" id="Card" onkeyup="VerifyCard()"><span
style="color:red" class="bi bi-x-circle-fill" id="tick"></span></dd>
<div id="cvvContainer" style="display:none">
<dt>CVV</dt>
<dd><input type="text" id="Cvv" onkeyup="VerifyCvv()" size="4"></dd>
</div>
<div id="expiryContainer" style="display:none">
<dt>Expiry</dt>
<dd>
<select id="Expiry" onchange="VerifyExpiry()">
<option>2023</option>

```



```

        <option>2024</option>
        <option>2025</option>
    </select>
</dd>
</div>
</dl>
    <button id="btnPay" style="display:none" >Pay</button>
</fieldset>
</body>
</html>

```

Selection Statements

- IF selection

- a) Forward Jump if { }
- b) Simple Decision if { }...else { }
- c) Multi Level
- d) Multiple Choice

Multi Level

- It comprises of a condition within the context of another condition.

Syntax:

```

if (condition-1)
{
    if(condition-2) {
        statements when all conditions evaluate to true;
    } else {
        statements when condition 2 is false;
    }
} else {
    statements when condition 1 is false;
}

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>

```

```

<script>
    var userDetails = {
        "UserName": "john123",
        "Password": "john@11"
    }
    function LoginClick(){
        var username = document.getElementById("UserName").value;
        var password = document.getElementById("Password").value;
        var error = document.getElementById("Error");
        if(username== "") {
            error.innerHTML= "User Name Required".fontcolor('red');
        } else {
            if(userDetails.UserName===username){
                if(userDetails.Password===password){
                    document.write("Login Success..");
                } else {
                    error.innerHTML = "Invalid Password".fontcolor('red');
                }
            } else {
                error.innerHTML = "Invalid User Name".fontcolor('red');
            }
        }
    }
}
</script>
</head>
<body>
    <dl>
        <h3>User Login</h3>
        <dt>User Name</dt>
        <dd><input type="text" id="UserName"></dd>
        <dt>Password</dt>
        <dd><input type="password" id="Password"></dd>
    </dl>
    <button onclick="LoginClick()">Login</button>
    <p align="center" id="Error"></p>
</body>
</html>

```

Multiple Choice

- It is a decision making process where you can have multiple conditions for one set of task to perform.

- Every task can contain multiple choices , which are defined as multiple conditions using "else if".

Syntax:

```
if(condition-1)
{
    statement on condition 1 true;
}
else if(condition-2)
{
    statement on condition 2 true;
}
else
{
    statement when both conditions evaluate to false;
}
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Amazon Login</title>
    <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
    <script>
        var userDetails = {
            "Email": "david@outlook.com",
            "Mobile": "+919876543211",
            "Password": "david@11"
        }

        var flag = "";

        function ContinueClick(){
            var userid = document.getElementById("UserId").value;
            var PasswordContainer =
document.getElementById("PasswordContainer");
            var UserContainer = document.getElementById("UserContainer");
```

```

        if(userid===userDetails.Email) {
            flag = `Your Login Success - OTP sent to your Email ${userid}`;
            PasswordContainer.style.display = "block";
            UserContainer.style.display = "none";

        } else if(userid===userDetails.Mobile) {
            flag = `Your Login Success - OTP sent to your Mobile ${userid}`;
            PasswordContainer.style.display = "block";
            UserContainer.style.display = "none";
        } else {
            document.getElementById("UserError").innerHTML = `${userid} is not
registered`;
        }
    }
}
function LoginClick(){
    var password = document.getElementById("Password").value;
    var PwdError = document.getElementById("PwdError");
    if(password===userDetails.Password) {
        document.write(flag);
    } else {
        PwdError.innerHTML = "Invalid Password";
    }
}
</script>
</head>
<body class="container-fluid">
    <div class="d-flex justify-content-center align-items-center"
style="height:500px">
        <div class="w-25">
            <h2>Sign In</h2>
            <div id="UserContainer">
                <label class="form-label"><b>Email or mobile phone
number</b></label>
                <div>
                    <input type="text" id="UserId" class="form-control">
                    <div id="UserError" class="text-danger"></div>
                </div>
                <div class="mt-3">
                    <button onclick="ContinueClick()" class="btn btn-warning w-
100">Continue</button>
                </div>
            </div>
        </div>
    </div>

```

```

<div id="PasswordContainer" style="display:none">
  <label class="form-label"><b>Password</b></label>
  <div>
    <input type="password" id="Password" class="form-control">
    <div id="PwdError" class="text-danger"></div>
  </div>
  <div class="mt-3">
    <button onclick="LoginClick()" class="btn btn-warning w-
100">Login</button>
  </div>
</div>
</div>
</div>
</body>
</html>

```

Note: If there are multiple choices to handle, then better to use a "switch".

Decision Making Statements

IF Select

- Forward Jump
- Simple Decision
- Multiple
- Multi Level

Comparision Operators

==

!=

===

!==

>

>=

<

<=

Programs:

1. Write a program to find if the given number is with in the specified range?

```
function FindRange(number, min, max) {
```

```
}
```

```
FindRange(3, 1, 10)    => In range  
FindRange(12, 20,30)   => Out of range
```

Ex:

```
<script>  
  function FindRange(n, min, max) {  
    if(n>=min && n<=max) {  
      document.write(`Your number ${n} is in range of ${min} - ${max}`);  
    } else {  
      document.write(`Your number ${n} is out of range ${min} - ${max}`);  
    }  
  }  
  FindRange(14, 10, 20);  
</script>
```

2. Write a program to find the largest among 3 numbers.

```
function FindLargest(a, b, c) {  
  
  }
```

Ex:

```
<script>  
  function FindLargest(a,b,c){  
    if(a>b && a>c) {  
      document.write("A is Large");  
    } else if (b>c) {  
      document.write("B is Greater");  
    } else {  
      document.write("C is Greater");  
    }  
  }  
  FindLargest(10,34, 64);  
</script>
```

3. Write a program to find if the given values are for a triangle or square?

```
function FindShape(values)  
{  
}
```

```
FindShape([100, 200, 50]);
```

Ex:

```
<script>
function FindShape(values){
    if(values.length==3) {
        document.write("Values are for Triangle");
    } else if (values.length==4) {
        document.write("values are for Square");
    } else {
        document.write("Please provide values only for square or triangle");
    }
}
FindShape([10,40,20,20, 20]);
</script>
```

4. Write a program to find if the values are for a square or rectangle?

```
function FindSquare(side1, side2, side3, side4)
{
}
```

Ex:

```
<script>
function FindSquare(side1, side2, side3, side4) {
    if(side1==side2 && side2==side3 && side3==side4) {
        document.write("Values are for Square");
    } else {
        document.write("Values are for Rectangle");
    }
}
FindSquare(10,20,20,10);
</script>
```

5. Write to program to Check if a triangle is equilateral, scalene, or isosceles

```
function findTrangleType(side1, side2, side3) {

}
```

```
side1==side2 and side1==side3          = Equilateral
side1==side2 or side2==side3 or side1==side3      = Isosceles
```

```
findTriangleType(12,12,12) //"Equilateral triangle."  
findTriangleType(20,20,31) //"Isosceles triangle."  
findTriangleType(5,4,3) //"Scalene triangle."
```

6. Write a program to convert number into words?

453 = Four Hundred Fifty Three

Switch Statement

- Switch is used to interrupt the flow of electrons.
- There are various types of switches
 - a) Toggle Button
 - b) Push Button
 - c) Selector
 - d) JoyStick etc..
- A selector switch is used in programming to select exactly the block that matches given condition.

Syntax:

```
switch (value | expression)  
{  
  case value|expression:  
    statements_on_match;  
    jump_statement;           [break, retrun]  
  default:  
    statements_on_mismatch;  
    jump_statement;  
}
```

- You can define multiple cases.
- If many cases are matching with the value or expression, then the first matching case will execute.

Ex:

```
<script>  
  var x = parseInt(prompt("Enter Number"));
```



```
switch(x)
{
  case 1:
    document.write("One");
    break;
  case 2:
    document.write("Two");
    break;
  case 3:
    document.write("Three");
    break;
  default:
    document.write("Please Enter values from 1 to 3 only");
    break;
}
</script>
```

FAQ's:

1. Can we define switch without "default"?

A. Yes.

2. Can we define "default" before or between cases?

A. Yes.

3. Can we define case or default without break?

A. Yes. Switch will stop at next break occurrence, If there is no break in switch then it will stop at the end of switch. It executes all blocks from the matching case.

4. Can we define "return" as jump statement for case?

A. Yes.

5. What is difference between break and return?

A. Break will exit the block but still stays in function.

Return will exit the block and stops the compiling, it exits the script.

Ex:

```
<script>
function f1(){
  var x = parseInt(prompt("Enter Number"));
  switch(x)
  {
```

```

    case 1:
    document.write("One");
    break;
    case 2:
    document.write("Two");
    return;
    case 3:
    document.write("Three");
    break;
    default:
    document.write("Please Enter values from 1 to 3 only");
    break;
}
}
f1();
</script>

```

6. How to handle same block for multiple cases?

A. Directly by defining multiple cases one below another for same block.

Synta:

```

    case 5:
    case 8:
        statements;
        break;

```

Ex:

```

<script>
    var product = prompt("Enter Product Name");
    switch(product)
    {
        case "watch":
        case "tv":
        case "mobile":
            document.write(`Your product ${product} belongs to Electronics`);
            break;
        case "casuals":
        case "boots":
        case "shirt":
            document.write(`Your product ${product} belongs to Fashion`);
            break;
        default:

```

```
        document.write("Sorry - We can find only Electronics and Fashion  
Products");  
        break;  
    }  
</script>
```

Ex:

```
<script>  
    var choice = prompt("Enter choice", "y / n");  
    switch(choice){  
        case "y":  
        case "Y":  
            document.write("You selected yes to continue");  
            break;  
        case "n":  
        case "N":  
            document.write("You selected No - It quits");  
            break;  
        default:  
            document.write("You choice must be Y or N");  
            break;  
    }  
</script>
```

7. What can be the value or expression type for case?

A. number, boolean, string.

8. How to define case for any capitalization?

A. By using change case methods.

toUpperCase()

toLowerCase()

Ex:

```
<script>  
    var choice = prompt("Enter choice", "yes / no");  
    switch(choice.toLowerCase()){  
        case "yes":  
            document.write("You selected yes to continue");  
            break;  
        case "no":
```

```

        document.write("You selected No - It quits");
        break;
    default:
        document.write("You choice must be Y or N");
        break;
    }
</script>

```

9. How to define a block of statements for a range of values?

A. By using boolean "true" in switch and boolean conditions in case.

Ex:

```

<script>
    var n = parseInt(prompt("Enter Number"));
    switch(true)
    {
        case (n>=1 && n<=10):
            document.write(`Your Number ${n} is between 1 to 10`);
            break;
        case (n>10 && n<=20):
            document.write(`Your Number ${n} is between 11 to 20`);
            break;
    }
</script>

```

Ex: Cascading Dropdown

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <style>
        select {
            width: 150px;
        }
    </style>
</script>

```

```

var categories = ["Select a Category", "Electronics", "Footwear"];
var electronics = ["Select Electronics", "Televisions", "Mobiles"];
var footwear = ["Select Footwear", "Casuals", "Boots"];
var products = [];
function LoadCategories(){
    for(var category of categories) {
        var option = document.createElement("option");
        option.text = category;
        option.value = category;
        document.getElementById("lstCategories").appendChild(option);
    }
}
function LoadProducts(){
    document.getElementById("lstProducts").innerHTML = "";
    for(var product of products) {
        var option = document.createElement("option");
        option.text = product;
        option.value = product;
        document.getElementById("lstProducts").appendChild(option);
    }
}
function bodyload(){
    LoadCategories();
}
function CategoryChanged(){
    var categoryName = document.getElementById("lstCategories").value;
    switch(categoryName)
    {
        case "Electronics":
            products = electronics;
            LoadProducts();
            break;
        case "Footwear":
            products = footwear;
            LoadProducts();
            break;
        default:
            products = ["Please Select a Category"];
            LoadProducts();
            break;
    }
}
</script>

```

```

</head>
<body onload="bodyload()">
  <div style="display: flex;">
    <div>
      <select id="lstCategories" onchange="CategoryChanged()"></select>
    </div>
    <div>
      <select id="lstProducts"></select>
    </div>
  </div>

</body>
</html>

```

Summary - Selection Statements

1. If..else if..else
2. switch.. case.. default

Looping Control Statements

- Looping is the process of executing a set of statements repeatedly based on given condition
- Loop comprises of initialization, condition and counter.

- Loop is created by using

- a) for
- b) while
- c) do while

The For() loop:

- For loop is used when the exact number of iterations are known to developer and iteration count will not change dynamically.

Syntax:

```

for(initialization ; condition; counter)
{
}

```

Ex:

```

<script>
  for(var i=1; i<=6; i++){
    document.write(`<h${i}>Welcome</h${i}>`);
  }
</script>

```

- Iteration is design pattern, which is used to read elements from a collection in sequential order.
- Iterator will not have initialization, condition and counter.

```
for(var item of collection)
{
}
```

EX: Loop

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    var categories = ["Electronics", "Footwear", "Fashion"];
    function bodyload(){
      for(var i = 0; i<categories.length; i++) {
        var li = document.createElement("li");
        li.innerHTML = categories[i];
        document.querySelector("ol").appendChild(li);
      }
    }
  </script>
</head>
<body onload="bodyload()">
  <ol>

  </ol>
</body>
</html>
```

Ex: Loops

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```

<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<script>
    var menuList = [
        {Category: "Electronics", Products: ["TV", "Mobile"]},
        {Category: "Footwear", Products: ["Casuals", "Sneakers"]}
    ];
    function bodyload(){
        for(var i=0; i<menuList.length; i++)
        {
            var olLi = document.createElement("li");
            olLi.innerHTML = menuList[i].Category;
            for(var j=0; j<menuList[i].Products.length; j++) {
                var ul = document.createElement("ul");
                var ulLi = document.createElement("li");
                ulLi.innerHTML = menuList[i].Products[j];
                ul.appendChild(ulLi);
                olLi.appendChild(ul);
                document.querySelector("ol").appendChild(olLi);
            }
        }
    }
</script>
</head>
<body onload="bodyload()">
    <ol>

    </ol>
</body>
</html>

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>

```



```

<script>
  var menuList = [
    {Category: "Electronics", Products: ["TV", "Mobile", "Watch"]},
    {Category: "Footwear", Products: ["Casuals", "Sneakers", "Boots"]}
  ];
  function bodyload(){
    for(var item of menuList){
      var olLi = document.createElement("li");
      olLi.innerHTML = item.Category;
      for(var product of item.Products)
      {
        var ul = document.createElement("ul");
        var ulLi = document.createElement("li");
        ulLi.innerHTML = product;
        ul.appendChild(ulLi);
        olLi.appendChild(ul);
        document.querySelector("ol").appendChild(olLi);
      }
    }
  }
</script>
</head>
<body onload="bodyload()">
  <ol>

  </ol>
</body>
</html>

```

Ex: For with decrement and step value

```

<script>
  for(var i=10; i>=1; i=i-2){
    document.write(i + "<br>");
  }
</script>

```

Ex: You can initialize, check condition or increment outside the for() method signature.

for() => signature
{ } => definition

```
<script>
  var i=20;
  for( ; ; i=i-2){
    if(i>=1) {
      document.write(i + "<br>");
    }
  }
</script>
```

Ex: Write a program to print the given pattern

```
*
* *
* * *
* * * *
```

```
<script type="text/javascript">
  var howMany = parseInt(prompt("Enter Number"));
  let string = " ";
  for (let i = 1; i <= howMany; i++) {
    for (let j = 1; j <= i; j++) {
      string += "*" + "&nbsp; &nbsp;";
    }
    string +="<br>";
  }
  document.write(string);
</script>
```

Ex: Write a program to print factorial of given number.

```
<script type="text/javascript">
function factorialcalc()
{
  var number = parseInt(prompt("Enter Number"));
  factorial = 1
  for (i=1; i <= number; i++)
  {
```

```
factorial = factorial * i
}
document.write("the factorial of " + number + " is " + factorial)
}
factorialcalc();

</script>
```

Ex: Write a program to print multiplication table for any number.

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 13
....
5 x 10 = 50
```

Ex

```
<script type="text/javascript">
    var yourNumber = parseInt(prompt("Enter Number"));
    var table = "";
    var n = 1;
    for(var i=1; i<=10; i++) {
        n = yourNumber * i;
        table += `${yourNumber} x ${i} = ${n} <br>`;
    }
    document.write(table);
</script>
```

Task: Write a program to verify pin max 3 times can be wrong, 4 th time it will block.

While and Do While

- If developer is not sure about the number of iterations and the iteration counter may change dynamically then better to use "while and do while".
- While will execute statements only when the given condition evaluates to true.

- do while will execute statements at least once even when the condition is false.

Syntax:

<pre>while(condition) { counter; statements; }</pre>	<pre>do { counter; statements; } while (condition)</pre>
--	--

Ex:

```
<script type="text/javascript">
  var x = 1;
  while(x<=10){
    document.write(x + "<br>");
    x++;
  }
</script>
```

Ex:

```
<script type="text/javascript">
  var x = 11;
  do{
    document.write(x + "<br>");
    x++;
  }while(x<=10);
</script>
```

Iterators

- Iterator is a pattern which is used to access elements from a collection in sequential order.
- It doesn't require initialization, condition and counter.
- JavaScript iteration statements are

for...in	It can read all properties from collection
for..of	It can read all values from collection

Syntax:

```
for(var property in collection)
{
}
```

```
for(var value of collection)
{
}
```

}

Jump Statements

- The jump statements are used to control the code block, they allow to skip and terminate the block.

break : It stop the current block but will stay in the function.

return : It terminates the block and stops the compilation.

continue : It is used to to skip the counter.

Ex:

```
<script type="text/javascript">
  for(var i=1; i<=20; i++){
    if(i==5 || i==8 || i==14) {
      continue;
    }
    document.write(i + "<br>");
  }
</script>
```

Ex:

```
<script type="text/javascript">
  function LoadProducts(){
    fetch("http://fakestoreapi.com/products")
    .then(function(response){
      return response.json();
    })
    .then(function(products){
      for(var product of products){
        if(product.category=="electronics") {
          continue;
        }
        document.write(`${product.title} - <b>${product.category}</b> <br>`);
      }
    })
  }
  LoadProducts();
</script>
```

Ex:

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Shopper</title>
  <link rel="stylesheet"
href="./node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="./node_modules/bootstrap-icons/font/bootstrap-
icons.css">
  <style>
    a{
      color:white;
      text-decoration: none;
    }
    a:hover {
      color:yellow;
    }
  </style>
  <script type="text/javascript">
    function LoadCategories(){
      fetch("http://fakestoreapi.com/products/categories")
      .then(function(response){
        return response.json();
      })
      .then(function(categories){
        categories.unshift("all");
        for(var category of categories){
          var option = document.createElement("option");
          option.text = category.toUpperCase();
          option.value = category;
          document.getElementById("lstCategories").appendChild(option);
        }
      })
    }

    function LoadProducts(url){
      fetch(url)
      .then(function(response){
        return response.json();
      })
      .then(function(products){
        document.querySelector("main").innerHTML="";
```

```

    for(var product of products){
        var div = document.createElement("div");
        div.className = "card m-2 p-2";
        div.style.width = "200px";
        div.innerHTML = `
            <img src=${product.image} class="card-img-top" height="160">
            <div class="card-header"
style="height:130px"><p>${product.title}</p></div>
            <div class="card-body">
                <dl>
                    <dt>Price</dt>
                    <dd>${product.price}</dd>
                    <dt>Rating</dt>
                    <dd><span class="bi bi-star-fill text-success"></span>
${product.rating.rate} [${product.rating.count}]
                </dl>
            </div>
            <div class="card-footer">
                <button onclick="AddClick(${product.id})" class="btn btn-danger
w-100">
                    <span class="bi bi-cart4"></span>
                    Add to Cart
                </button>
            </div>
        `;
        document.querySelector("main").appendChild(div);
    }
})
}

```

```

function bodyload(){
    LoadCategories();
    LoadProducts("http://fakestoreapi.com/products");
    GetCartItemsCount();
}
function CategoryChanged(){
    var categoryName = document.getElementById("lstCategories").value;
    if(categoryName=="all") {
        LoadProducts("http://fakestoreapi.com/products");
    } else {
        LoadProducts(`http://fakestoreapi.com/products/category/${categoryName}`);
    }
}

```

```

}
function NavClicked(categoryName){
    if(categoryName=="all"){
        LoadProducts("http://fakestoreapi.com/products");
    } else {
        LoadProducts("http://fakestoreapi.com/products/category/${categoryName}");
    }
}
var cartItems = [];
function GetCartItemsCount(){
    document.getElementById("lblCount").innerHTML = cartItems.length;
}
function AddClick(id) {
    fetch(`http://fakestoreapi.com/products/${id}`)
    .then(function(response){
        return response.json();
    })
    .then(function(product){
        cartItems.push(product);
        alert(`${product.title}\nAdded to Cart`);
        GetCartItemsCount();
    })
}
function ShowCartClick(){
    document.querySelector("tbody").innerHTML = "";
    for(var item of cartItems)
    {
        var tr = document.createElement("tr");
        var tdName = document.createElement("td");
        var tdPrice = document.createElement("td");
        var tdPreview = document.createElement("td");
        var tdAction = document.createElement("td");

        tdName.innerHTML = item.title;
        tdPrice.innerHTML = item.price;
        tdPreview.innerHTML = `<span class="bi
bi-trash"></span></button>`;

        tr.appendChild(tdName);
        tr.appendChild(tdPrice);

```



```
<h2 class="text-dark">Your Cart Items</h2>  
<button class="btn-close" data-bs-dismiss="modal"></button>  
</div>  
<div class="modal-body">  
  <table class="table table-hover">  
    <thead>  
      <tr>  
        <th>Name</th>  
        <th>Price</th>  
        <th>Preview</th>  
      </tr>  
    </thead>  
    <tbody>  
  
    </tbody>  
  </table>  
</div>  
</div>  
</div>  
</div>  
</div>  
</header>  
<section class="mt-2 row">  
  <nav class="col-2">  
    <div class="mb-3">  
      <label class="form-label">Select Category</label>  
      <div>  
        <select id="lstCategories" onchange="CategoryChanged()"  
class="form-select">  
  
        </select>  
      </div>  
    </div>  
    <div>  
      <label>Filter Products</label>  
      <ul class="list-unstyled">  
        <li><input type="checkbox" name="category" class="form-check-input" value="electronics"> Electronics</li>  
        <li><input type="checkbox" name="category" class="form-check-input" value="jewelery"> Jewelery</li>  
        <li><input type="checkbox" name="category" class="form-check-input" value="men's clothing"> Men's Clothing</li>  
        <li><input type="checkbox" name="category" class="form-check-
```

```

input" value="women's clothing"> Women's Clothing</li>
    <button onclick="FilterClick()" class="btn btn-primary w-100 mt-
2">Apply Filter</button>

    </ul>
</div>
</nav>
<main class="col-10 d-flex flex-wrap overflow-auto" style="height:500px">

</main>
</section>
<script src="./node_modules/jquery/dist/jquery.js"></script>
<script src="./node_modules/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>

```

Exception Handling Statements

- Computer programming returns 2 types of errors

a) Compile Time Errors

b) Run Time Errors

- Compile Time errors are syntax errors, due to this program will not execute.
- Run Time errors occur at the time of using application. Program compiles and runs successfully but it stops dynamically during runtime.
- Application terminates due run time errors.
- Exception handling is used in computer programming to avoid abnormal termination.
- Exceptions are handled by using

- a) try
- b) catch
- c) throw
- d) finally

- try is monitoring block, it contains statements to execute.

- catch is handler block, it catches the problem if any statement in try block can't execute.

- throw is used to throw the exception manually.

- finally contains statements to execute always.

Ex:

```
<script>
  try {
    var a = parseInt(prompt("Enter Number-1"));
    var b = parseInt(prompt("Enter Number-2"));
    if(b==0) {
      throw "Can't Divide By Zero";
    }
    var c = a / b;
    document.write("Division=" + c);
  }
  catch(exception) {
    document.write(exception);
  }
  finally {
    document.write("<br>Program End");
  }
</script>
```

Ex:

```
<script>
  try {
    var a = parseInt(prompt("Enter Number-1"));
    var b = parseInt(prompt("Enter Number-2"));
    if(b==0) {
      throw "Can't Divide By Zero";
    }
    if(b>a) {
      throw "Number is too large..";
    }
    var c = a / b;
    document.write("Division=" + c);
  }
  catch(exception) {
    document.write(exception);
  }
  finally {
```

```
        document.write("<br>Program End");
    }
</script>
```

Functions

JavaScript Functions

- Functions are used to "Refactor".
- Refactoring is the process of extracting a set of logic and storing as a single unit under reference of a function or file.
- Refactoring is required to reuse the code.
- Function configuration comprises 3 phases
 - a) Declaration
 - b) Signature
 - c) Definition

```
function Name(params)
{

}
```

```
function Name(params)    => Declaration
Name(params)             => Signature
{ }                      => Definition
```

- Function can be parameter less

Ex:

```
function PrintNumbers() {
    for (var i = 1; i <= 10; i++) {
        document.write(i + "<br>");
    }
}
```

- Function can be parameterized
- Parameters in a function are used to modify the function.

Syntax:

```
function PrintNumbers(howMany) => Formal Parameter
{
```

```
}
```

`PrintNumbers(10);` => Actual Parameter

`PrintNumbers()` is a function that takes one "Argument".

Ex:

```
<script>
  function PrintNumbers(howMany){
    for(var i=1; i<=howMany; i++) {
      document.write(i + "<br>");
    }
  }
  PrintNumbers(16);
</script>
```

- Every function parameter can accept any type of value.

- a) Primitive
- b) Non Primitive
- c) Function

- Formal parameters are just reference name for memory, where you can store any value.

Ex:

```
<script>
  function PrintList(collection){
    for(var item of collection){
      document.write(item + "<br>");
    }
  }
  PrintList(["Delhi","Hyd","Mumbai"]);
  PrintList([10, 450, 520,]);
  PrintList(new Array("TV", "Mobile", "Watch"));
</script>
```

- A function can have multiple parameters.
- Every parameter is mandatory and have order dependency.

Ex:

```
<script>
  function Details(id, name, price, stock, cities, rating){
```

```

    if(price===undefined) {
        document.write(
            Id : ${id} <br>
            Name: ${name} <br>
            Stock: ${stock} <br>
            Cities: ${cities.toString()} <br>
            Rating: ${rating.rate} [${rating.count}]
        );
    } else {
        document.write(
            Id : ${id} <br>
            Name: ${name} <br>
            Price: ${price} <br>
            Stock: ${stock} <br>
            Cities: ${cities.toString()} <br>
            Rating: ${rating.rate} [${rating.count}]
        );
    }
}
Details(1, "TV",undefined, true, ["Delhi", "Hyd"], {rate:4.3, count:4000});
</script>

```

- ECMA standards a function can handle max 1024 params.
- ES6 introduced "Rest" parameters.
- One Rest parameter can take multiple arguments.
- Rest parameter is array type.
- Rest parameter is defined by using "...paramName".
- Every function can have only one "Rest" parameter.
- You can define other parameters with rest parameter.
- Rest parameter must be the last in formal list.

Ex:

```

<script>
function Details(title, ...product){
    var [id, name,price, stock, cities, rating] = product;
    document.write(
        <h2>${title}</h2>
        Id : ${id} <br>
        Name: ${name} <br>
        Price: ${price} <br>
        Stock: ${stock} <br>
        Cities: ${cities.toString()} <br>
        Rating: ${rating.rate} [${rating.count}]
    );
}

```

```

    );

    }
    Details( "Product Details" ,1, "TV",45000.33, true, ["Delhi", "Hyd"], {rate:4.3,
count:4000});
</script>

```

- You can define function as parameter.
- Function is used a parameter to handle callback.
- Callback is a technique where a set of functions are defined and any specific function is selected to execute according to state and situation.

Ex:

```

<script>
    function VerifyPassword(password, success, failure) {
        if(password=="admin") {
            success();
        } else {
            failure();
        }
    }
    VerifyPassword(prompt("Enter password"),function(){
        document.write("Login Success");
    },function(){
        document.write("Invalid Password");
    })
</script>

```

- Function without name are known a "Anonymous" functions.

- Function
- Function Declaration
- Function Signature
- Function Definition
- Function Parameter Less
- Function Parameterized
- Function Rest parameter
- Function as parameter

Anonymous Function

- A function without name.
- Anonymous functions are used in callbacks.
- If there is any anonymous function and you want to access the function, then you have to follow IIFE pattern. [Immediately Invoked Function Expression]

Syntax:

```
(function(){  
  
    })0;
```

Ex:

```
<script>  
    ( function(){  
        document.write("Anonymous Function");  
    })0;  
</script>
```

- Anonymous functions are used as expressions.

Syntax:

```
const f1 = function() { }
```

- IIFE pattern allows to configure a function that executes only once, it is referred as "forward only and one-time render".

Ex:

```
<script>  
    var x = prompt("Enter Number");  
  
    if(isNaN(x)) {  
        (function(){  
            document.write("Hello ! " + x);  
        })0;  
    } else {  
        ( function(){  
            x++;  
            document.write(`X is increased by 1 = ${x}`);  
        })0;  
    }  
}
```

</script>

Ex:

```
<script>
  var x = prompt("Enter Number");
  var f1;
  if(isNaN(x)) {
    f1= function(){
      document.write("Hello ! " + x + "<br>");
    };
  } else {
    f1 = function(){
      x++;
      document.write(`X is increased by 1 = ${x} <br>`);
    };
  }
  f1();
  f1();
  f1();
</script>
```

FAQ: What is a function expression?

Ans : It configures a reference name for function in memory and can manage various functionalities.

Syntax:

```
const name = function() { }
```

Function with Return

```
<script>
  function ConfirmAction(flag){
    if(flag=="OK") {
      return true;
    } else {
      return false;
    }
  }
  var choice = ConfirmAction(prompt("Enter Choice","OK|Cancel"));
```

```

    if(choice==true) {
        document.write("You Selected OK");
    } else {
        document.write("You Select Cancel");
    }
</script>

```

Anonymous Functions
 Function Expression IIFE
 Function with return

Function Closure

- It allows to configure a function with in the context of another function.
- Closure is a technique of accessing the members of inner function in outer function.

Syntax:

```

function Outer()
{
    function Inner(){
        return value;
    }
    return Inner();
}

```

Ex:

```

<script>
    function Outer(){
        let x = 10;
        function Inner(){
            var y = ++x;
            return `X=${x} <br> Y=${y}`;
        }
        return Inner();
    }
    document.write(Outer());
</script>

```

Function Recursions

- It is the process of calling a function with in the context of same function.

Syntax:

```
function f1()
{
  f1();
}
```

- It is used for designing batch operations.

Ex:

```
<script>
function Fact(n) {
  if(n==0) {
    return 1;
  }
  return n * Fact(n-1);
}
document.write(`Factorial of 6 is ${Fact(6)}`);
</script>
```

FAQ: What is difference between Rest and Spread?

Ans:

Rest is a parameter that allows multiple arguments.
It is array type.
Function can have only one Rest parameter.
It must be the last parameter in formal list.

Spread is a syntax for actual values not formal
It is array type
You can define multiple spread values.
There is no order dependency for spread syntax. [It must be in same sequence]

Syntax:

```
function Name(...rest) {
}
Name(1, "TV", true);           => rest technique
```

```
function Name(id, name, stock)
{
}
Name(...[1, "TV", true]);      => spread technique
```

Ex:

```
<script>
  function Details(id, name, price, stock, rating, count)
  {

document.write(`Id=${id}<br>Name=${name}<br>Price=${price}<br>Stock=${stoc
k}<br>Rating=${rating}<br>Count=${count}`);
  }

  Details(...[1, "TV", 34000.33], true, ...[4.3, 3000]);
</script>
```

Summary - JavaScript Language Basics for Developer [Basic Level & Advanced Level]

1. Variables
2. Data Types
3. Operators
4. Statements
5. Functions

JavaScript OOP

[Object Oriented Programming Features]

- In real world project development we use 3 types of programming systems

1. POPS
2. OBPS
3. OOPS

POPS

- Process Oriented Programming System
- It supports low level features.
- It can directly interact with hardware services.
- It is faster.
- It uses less memory.

Ex: C, Pascal

- Reusability issues
- Extensibility issues
- Code Level Security issues
- No dynamic memory allocations

OBPS
OOPS

1. POPS [Process Oriented Programming System]

- Low level
- Hardware
- Fast
- Less memory

Ex: C, Pascal

2. OBPS [Object Based Programming System]

- It supports reusability and extensibility
- It supports code separation
- It have dynamic memory

Issues

- No dynamic polymorphism
- No code level security
- No Contracts

Ex: Visual Basic, JavaScript

3. OOPS [Object Oriented Programming System]

- Code reusability
- Code Separation
- Extensibility
- Dynamic Polymorphism
- Code Level Security

Ex: C++, Java, C#

- Slow
- Use more memory
- Heavy
- Can't directly interact with hardware

Note: JavaScript is not an OOP language, but it supports few features of OOP

Evolution of OOP

- 1960's Alan Kay introduced the concept of "Object" into computer programming.
- 1967 Johan Olay, Kristian Nygaard developed OOP language called "SIMULA 67"
- SIMULA [Simulation Systems] it enables code reusability.
- 1970's Trygve introduced code separation with a framework known as "MVC" [Model View Controller]
- Small Talk formulated the concept of Code reusability and separation.
- 1975 C++
- 1990 Java
- 2000 C#, TypeScript - Microsoft - "Anders Hejlsberg"

JavaScript OOP Features

1. Modules

2. Classes

3. OOP Characteristics

Inheritance, Polymorphism..

JavaScript Modules

- A module is a collection of variables, functions and classes.
- You can build a library by using module system.
- Library enables code reusability and code separation.
- To use modules of JavaScript you need a module system installed.
- There are various module systems in JavaScript
 - a) Common JS
 - b) UMD [Universal Module Distribution]
 - c) AMD [Asynchronous Module Distribution]
 - d) Require JS
 - e) ES Module
- Every JavaScript file is considered as a module.
- File Name is used as Module Name.
- Every member in a module is private in access. You can't access outside module.
- If you want any function accessible outside module then you have to mark it as "export"

Syntax:

```
export function Name() { }  
export const name = function() { }
```

- You can import the members and access in any <script>.

```
import { functionName } from ModuleName;
```

Ex:

1. Add a new folder "library"

2. Add following files
home.module.js

```
var username = "John";  
export function Hello(){  
  return `Hello !${username}`;  
}  
math.module.js
```

```
export function Addition(a, b){  
  return a + b;  
}  
export function Multiplication(a, b){  
  return a * b;  
}
```

3. Add a new HTML page
index.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Document</title>  
  <script type="module">  
    import { Hello } from "../library/home.module.js";
```



```

import { Addition, Multiplication } from "../library/math.module.js";
document.querySelector("p").innerHTML = Hello() + "<br>" + "Addition=" +
Addition(10, 20) + "<br>" + "Multiplication=" + Multiplication(4,2);
</script>
</head>
<body>
  <p></p>
</body>
</html>

```

OOP - Modules

- Variables
- Functions
- export
- import

- default export

Syntax:

```

export function Name() {}      => Lazy loading, loads only when it is in use.
export default function Name() => Eager loading, loads even when it not in use.

```

```

import { Name } from Module;    // normal exported function
import Name from Module;        // default exported function

```

- Every module can have only one "default" export.
- The default import must be first import.

Ex:

math.module.js

```

export function Addition(a, b){
  return a + b;
}
export default function Multiplication(a, b){
  return a * b;
}

```

home.module.js

```
var username = "John";
export function Hello(){
  return `Hello ! ${username}`;
}
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script type="module">
    import { Hello } from "../library/home.module.js";
    import Multiplication, {Addition} from "../library/math.module.js";
    document.querySelector("p").innerHTML = Hello() + "<br>" + "Addition=" +
Addition(10, 20) + "<br>" + "Multiplication=" + Multiplication(4,2);
  </script>
</head>
<body>
  <p></p>
</body>
</html>
```

Classes in JavaScript

- Class is program template.
- A template comprises of sample data and logic which you can implement and customize according to requirement.
- Class is known as logical entity or model or blue print.
- If a class is mapping to business requirements then it is referred as Entity.
- If a class is mapping to data requirements then it is referred as Model.
- JavaScript object type is referred as a "Pseudo Class"(Class like behaviour). It

comprises of set of properties and functions.

Configuring class:

1. Class Declaration

```
class Product
{
    // members;
}
```

2. Class Expression

```
var Product = class {

}
```

- Declaration configures a memory for class, where only specific class members are allowed.
- Expression configures a memory where various classes can function.

Class Members

- Every class can have only following members

1. Property

2. Accessor

3. Method

4. Constructor

FAQ: Can we define a variable as class member?

Ans : No.

FAQ: Why variables are not allowed as class members?

Ans: Variables are Immutable(can not change its structure). Class is a template and it can have only Mutable members as class members.

Note: You can define a variable in class as a member of method.

```
class Product
{
    var x = 10;    // invalid
    Print() {
        var y = 20; // valid
    }
}
```

FAQ: Can we define a function as class member?

Ans: No. Functions are Immutable

- In a class data is stored in Property.
- In a class functionality is defined with Method.

```
var Price = 45000.44;
```

```
class Product
{
    Price = 5000.44;
}
```

Class Members

1. Property
2. Accessor
3. Method
4. Constructor

Property

- Property is used to store data.
- Class properties are defined with in class without any keywords.
- You can initialize values into property or you can assign value into property by using an object for class.

Syntax:

```
class Product
{
    Name;           // Declared
    Price = 45000.33; // Initialized
}
```

- To access members of class , you have to create an instance for class [object].

```
let tv = new Product();    => new is dynamic memory allocation operator.
tv.Name = "Samsung TV";    => assigning value into property
```

```
document.write("Name=" + tv.Name); => print the value
```

- You can store any type of data in property

a) Primitive Types

b) Non Primitive Types

Syntax:

```
class Product
{
    ProductId = 1;=>number
    Name = "TV";=>string
    Stock = true;=>boolean
    Rating = { Rate:3.5, Count:5600 }; =>object
    Cities = [ "Delhi", "Hyd" ]; =>array
}
```

In object every property is separate with comma(,) but in class it is separated with semicolon(;

Ex:

<script>

```
class Product
{
    Name = "Samsung TV";
    Stock = true;
    Price = 45000.44;
```

```

    Rating = {Rate: 3.5, Count:6000};
    Cities = ["Delhi", "Hyd"];
}
let tv = new Product();
tv.Price = 54000.33;
document.write(
    Name = ${tv.Name} <br>
    Price = ${tv.Price} <br>
    Stock = ${tv.Stock} <br>
    Rating = ${tv.Rating.Rate} [${tv.Rating.Count}] <br>
    Cities = ${tv.Cities.toString()}
);
</script>

```

Accessors

- Accessor is like a function which controls read and write operation of property.
- Accessors are 2 types

| | | |
|-----------|-------|--|
| a) Getter | get() | It is used to read value from a property |
| b) Setter | set() | It is used to write value into a property(storing a value) |

```

class Product
{
    _Price = 45000.23; //(underscore "_" defined by developer is ,it is not final.it can be
modified or implimated)

    get Price() {                //(here price is alias name,ie we can acess by
this name)
        return this._Price;
    }

    set Price(newPrice) {
        this._Price = newPrice;
    }

}

```

Ex

```
<script>
    var username = prompt("Enter User Name");
    var password = prompt("Enter Password");
    var role = prompt("Enter Your Role", "Admin | Customer");
    var productPrice = prompt("Enter Product Price");

    class Product
    {
        Name = "Samsung TV";
        Stock = true;

        _Price;
        get Price(){
            return this._Price;
        }
        set Price(newName) {
            if(role=="Admin") {
                this._Price = newName;
            } else {
                document.write(`Hello ! ${username} You are having ${role} role and not
authorized to set price`);
            }
        }
    }

    Rating = {
        CustomerRating: {Rate: 3.6, Count:6700},
        VendorRating : {Rate: 4.5, Count:3}
    };
    get VendorRating(){
        return this.Rating.VendorRating.Rate;
    }
    get VendorCount(){
        return this.Rating.VendorRating.Count;
    }
    Cities = ["Delhi", "Hyd"];
```

```

    }

    let tv = new Product();
    tv.Price = productPrice;
    if(tv.Price)
    {
        document.write(
            Name = ${tv.Name} <br>
            Price = ${tv.Price} <br>
            Stock = ${tv.Stock} <br>
            Vendor Rating = ${tv.VendorRating} [${tv.VendorCount}] <br>
            Cities = ${tv.Cities.toString()}
        `);
    }
</script>

```

Methods

- Methods are similar to function.
- Functions are intended(function intension is) to return a value.
- Functions are Immutable.
- Methods are void type, they will not return a value.
- JavaScript methods can be defined with return value.

Syntax:

```

class Product
{
    MethodName() {

    }
}

```

- All rules for methods are similar to function.
 - parameters,
 - rest parameters
 - spread
 - arrow..

Ex:

```
<script>
  class Product
  {
    Name;
    Price;
    Qty;
    Total(){
      return this.Qty * this.Price;
    }
    Print(){

document.write(` Name=${this.Name}<br>Price=${this.Price}<br>Qty=${this.Qty}<br>Total=${this.Total()}`);
    }
  }
  let tv = new Product();
  tv.Name = prompt("Enter TV Name");
  tv.Price = parseFloat(prompt("Enter TV Price"));
  tv.Qty = parseInt(prompt("Enter TV Quantity"));
  tv.Print();
</script>
```

Constructor

- A constructor in class is responsible for creating an object for class.
- A constructor is used for instantiation.
- Every class have a default constructor.
- It is a special type of sub-routine that executes automatically for every object that you create.

Class Members

- Property
- Accessor
- Method
- Constructor

Constructor

- It is a special type of sub-routine which is used for instantiation.
- Constructor is responsible for creating an object for class.
- It is special type of method that executes automatically for every object.
- Every class have a default constructor.
- If any action you have to perform at the time of creating object, then you can define using explicit constructor.
- In JavaScript constructor is anonymous type.

Syntax:

```
class Product
{
  constructor(params) {
  }
}
let tv = new Product;
let tv = new Product(params); IIFE
```

- Every JavaScript class can have only one constructor, they can't overload.
[It is possible in general OOP languages]
- JavaScript constructor can't have any access modifier, which is possible in other OOP language.
- Constructor parameters are same as method params.

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    class Database
```

```

{
    constructor(dbName) {
        alert(`Connection with ${dbName} is success\nYou can carryout operations`);
    }
    Insert(){
        document.write("Record Inserted");
    }
    Delete(){
        document.write("Record Deleted");
    }
}

function InsertClick(){
    let db = new Database(document.getElementById("DbName").value);
    db.Insert();
}

function DeleteClick(){
    let db = new Database(document.getElementById("DbName").value);
    db.Delete();
}
</script>
</head>
<body>
    <div>
        <select id="DbName">
            <option>Select Database</option>
            <option>Oracle</option>
            <option>MySql</option>
        </select>
    </div>
    <br>
    <button onclick="InsertClick()">Insert</button>
    <button onclick="DeleteClick()">Delete</button>
</body>
</html>

```

Code Reusability and Code Extensibility

- Code reusability and extensibility can be handled using 2 techniques

a) Aggregation

b) Inheritance

A) Aggregation

- Aggregation is a technique where you can access the members of one class in another class by creating an instance of class. [object].
- It is referred as "Object-to-Object" communication.
- It is known as "Has-A-Relation".
- Classes are not in relation with each other, but they can access the members of one in another.

**Note: Extensibility is required to achieve "Backward Compatibility"(New version).
Developing Multi Versioning Environment.**

> npm install bootstrap (this command will download the latest version)
> npm install bootstrap@4.6 (this command will download the mentioned version [4.6])

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    dd {
      margin-top: 20px;
    }
  </style>
  <script>
    class HDFC_Version1
    {
      Personal = "Personal Banking Services <br>";
```

```

        NRI = "NRI Banking Services <br>";
        Print(){
            document.write(`${this.Personal}<br>${this.NRI}<br>`);
        }
    }
    class HDFC_Version2
    {
        Loans = "Personal and Housing Loans <br>";
        Print(){
            let obj = new HDFC_Version1();
            obj.Print();
            document.write(`${this.Loans}`);
        }
    }
    function InstallClick(){
        var version = document.getElementById("hdfc").value;
        switch(version)
        {
            case "ver1":
                document.write(`<h2>HDFC Version-1</h2>`);
                let obj1 = new HDFC_Version1();
                obj1.Print();
                break;
            case "ver2":
                document.write(`<h2>HDFC Version-2</h2>`);
                let obj2 = new HDFC_Version2();
                obj2.Print();
                break;
            default:
                alert("Please Select a Version");
                break;
        }
    }
</script>
</head>
<body>
    <h2>Install HDFC Bank App</h2>
    <dl>

```

```

<dt>Select Version</dt>
<dd>
  <select id="hdfc">
    <option>Choose Version</option>
    <option value="ver1">Version-1</option>
    <option value="ver2">Version-2</option>
  </select>
</dd>
<dd>
  <button onclick="InstallClick()">Install</button>
</dd>
</dl>
</body>
</html>

```

b) Inheritance

- Inheritance allows extensibility by using the keyword "extends"
- It is referred as "Is-A-Relation".
- Existing class known as Super class.
- New class in known as Derived class.
- Super class members are accessed by using "super" keyword.

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    dd {
      margin-top: 20px;
    }
  </style>
  <script>

```

```
class HDFC_Version1
{
    Personal = "Personal Banking Services <br>";
    NRI = "NRI Banking Services <br>";
    Print(){
        document.write(`${this.Personal}<br>${this.NRI}<br>`);
    }
}
class HDFC_Version2 extends HDFC_Version1
{
    Loans = "Personal and Housing Loans <br>";
    Print(){
        super.Print();
        document.write(`${this.Loans}`);
    }
}
function InstallClick(){
    var version = document.getElementById("hdfc").value;
    switch(version)
    {
        case "ver1":
            document.write(`<h2>HDFC Version-1</h2>`);
            let obj1 = new HDFC_Version1();
            obj1.Print();
            break;
        case "ver2":
            document.write(`<h2>HDFC Version-2</h2>`);
            let obj2 = new HDFC_Version2();
            obj2.Print();
            break;
        default:
            alert("Please Select a Version");
            break;
    }
}
</script>
</head>
<body>
```

```
<h2>Install HDFC Bank App</h2>
<dl>
  <dt>Select Version</dt>
  <dd>
    <select id="hdfc">
      <option>Choose Version</option>
      <option value="ver1">Version-1</option>
      <option value="ver2">Version-2</option>
    </select>
  </dd>
  <dd>
    <button onclick="InstallClick()">Install</button>
  </dd>
</dl>
</body>
</html>
```

Code Resusability and Extensibility

- Aggregation => Object to Object => Has-A-Relation
- Inheritance => Is-A-Relation

- Inheritance for class can be defined in 2 ways

- a) Single Inheritance
- b) Multi Level Inheritance

- Single Inheritance comprises of one super class and one derived class.

Syntax:

```
class Super1
{
}
class Derived1 extends Super1
{
}
```


- Multilevel Inheritance allows to extend a derived class.

Syntax:

```
class Super1
{
}
class Derived1 extends Super1
{
}
class Derived2 extends Derived1
{
}
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    dd {
      margin-top: 20px;
    }
  </style>
  <script>
    class HDFC_Version1
    {
      Personal = "Personal Banking Services <br>";
      NRI = "NRI Banking Services <br>";
      Print(){
        document.write(`${this.Personal}<br>${this.NRI}<br>`);
      }
    }
    class HDFC_Version2 extends HDFC_Version1
    {
```

```

    Loans = "Personal and Housing Loans <br>";
    Print(){
        super.Print();
        document.write(`${this.Loans}<br>`);
    }
}

class HDFC_Version3 extends HDFC_Version2 {
    Stock = "Stock Investment";
    Print(){
        super.Print();
        document.write(`${this.Stock}`);
    }
}

function InstallClick(){
    var version = document.getElementById("hdfc").value;
    switch(version)
    {
        case "ver1":
            document.write(`<h2>HDFC Version-1</h2>`);
            let obj1 = new HDFC_Version1();
            obj1.Print();
            break;
        case "ver2":
            document.write(`<h2>HDFC Version-2</h2>`);
            let obj2 = new HDFC_Version2();
            obj2.Print();
            break;
        case "ver3":
            document.write(`<h2>HDFC Version-3</h2>`);
            let obj3 = new HDFC_Version3();
            obj3.Print();
            break;
        default:
            alert("Please Select a Version");
            break;
    }
}
</script>

```

```
</head>
<body>
  <h2>Install HDFC Bank App</h2>
  <dl>
    <dt>Select Version</dt>
    <dd>
      <select id="hdfc">
        <option>Choose Version</option>
        <option value="ver1">Version-1</option>
        <option value="ver2">Version-2</option>
        <option value="ver3">Version-3</option>
      </select>
    </dd>
    <dd>
      <button onclick="InstallClick()">Install</button>
    </dd>
  </dl>
</body>
</html>
```

Note: If super is having a constructor defined explicitly then derived class constructor must call super class constructor.

Syntax:

```
class Super
{
  constructor(){
  }
}
class Derived extends Super
{
  constructor() {
    super();
  }
}
let obj = new Derived();
```

Ex:

```
<script>
  class Super
  {
    constructor(msg){
      document.write(' Super Class Constructor ${msg} <br>');
    }
  }
  class Derived extends Super
  {
    constructor(){
      super("Hello ! from Derived");
      document.write("Derived Class Constructor");
    }
  }
  let obj = new Derived();
</script>
```

Polymorphism

- Poly means "Many"
- Morphos means "Forms"
- The ability of any component to handle various situations is referred as "Polymorphism".
- In OOP languages a component can work for various situations by implementing the technique of "Polymorphism".

Definition: A single base [super] class object can use the memory of multiple derived classes.

An object can overload, function can overload and various other members can overload.

[JavaScript will not support function overloading]

```
<script>
  class Employee
  {
    FirstName;
    LastName;
```

```
        Designation;
        Print(){
            document.write(`${this.FirstName} ${this.LastName} - ${this.Designation}<br>`);
        }
    }
    class Developer extends Employee
    {
        FirstName = "Raj";
        LastName = "Kumar";
        Designation = "Developer";
        Role = "Developer Role : Build, Debug, Test, Deploy";
        Print() {
            super.Print();
            document.write(this.Role);
        }
    }
    class Admin extends Employee
    {
        FirstName = "Kiran";
        LastName = "Kumar";
        Designation = "Admin";
        Role = "Admin Role : Authorization, Authentication";
        Print() {
            super.Print();
            document.write(this.Role);
        }
    }
    class Manager extends Employee
    {
        FirstName = "Tom";
        LastName = "Hanks";
        Designation = "Manager";
        Role = "Manager Role : Approvals";
        Print() {
            super.Print();
            document.write(this.Role);
        }
    }
}
```

```
let employees = [new Developer(), new Admin(), new Manager()];
let designation = prompt("Enter Designation");
for(var employee of employees)
{
    if(employee.Designation==designation) {
        employee.Print();
    }
}
</script>
```

Arrow, Async

JavaScript OOPS

- Modules
- Classes & Object
- Class Members
 - a) Property
 - b) Accessor
 - c) Method
 - d) Constructor
- Code Reusability & Extensibility
 - a) Aggregation
 - b) Inheritance
- Polymorphsim

JavaScript Browser Objects

- DOM [Document Object Model]
- DOM is a part of BOM [Browser Object Model]
- Browser objects are used to control the browser
- BOM comprises of following objects

1. window

2. location

3. history

4. navigator

5. document

window

- It provides a set of properties and methods that are used to control the browser window.

alert()
prompt()
open()
close()
print() etc..

Ex.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h2>Window Object</h2>
  <button onclick="window.print()">Print</button>
  <button onclick="window.open('../public/images/asp.jpg','ASP','width=400
height=300')">Open</button>
  <button onclick="window.close()">Close</button>
</body>
</html>
```

location

- It provides properties and methods that are used to get and set location details.

host	: It returns IP address or Server Name
port	: It returns the port number
href	: It returns and sets URL
pathname	: It returns the page path.
hash	: It returns the hash location.
search	: It returns query string
reload()	: It reloads the page

protocol : It returns protocol "http | https | ftp | file ..."

Ex:

```
<script>
    document.write(
        Your Server Name | IP Address : ${location.host} <br>
        You are using Port Number : ${location.port} <br>
        You are accessing page : ${location.pathname} <br>
        ${((location.protocol=="http:")?"You are using Web Server":"You are using File
System - Please use WebServer")} <br>
        Your URL is : ${location.href}
    );
</script>
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script>
        function LoginClick(){
            var username = document.getElementById("UserName").value;
            var password = document.getElementById("Password").value;

            if(username=="john_nit" && password=="john@123"){
                location.href = "../public/css-examples/shopper-template.html";
            } else {
                location.href = "error.html";
            }
        }
    </script>
</head>
<body>
    <dl>
```



```

    <dt>User Name</dt>
    <dd><input type="text" id="UserName"></dd>
    <dt>Password</dt>
    <dd><input type="password" id="Password"> </dd>
</dl>
<button onclick="LoginClick()">Login</button>
</body>
</html>

```

Ex: location.search => to get the query string

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Results</title>
  <script>
    function bodyload(){
      var data = [
        {Topic : "JavaScript", Description: "JavaScript is a programming language."},
        {Topic : "HTML", Description: "HTML is a markup language."}
      ];
      var term = location.search.substring(location.search.indexOf("=")+1);
      for(var item of data){
        if(item.Topic==term) {
          document.querySelector("p").innerHTML = item.Description;
          break;
        } else {
          document.querySelector("p").innerHTML = `${term} - Not Found`;
        }
      }
    }
  </script>
</head>
<body onload="bodyload()">

```

```
<h2>Results</h2>
<p></p>
</body>
</html>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>

</head>
<body >
  <form method="get" action="results.html">
    <div align="center">

      <h2>Google</h2>
      <input type="text" size="40" name="search"><br><br>
      <button>search</button>

    </div>
  </form>

</body>
</html>
```

JavaScript Browser Objects

- window
- location
 - host
 - protocol
 - href
 - port
 - search
 - hash

Ex:

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    section {
      display: grid;
      grid-template-columns: 3fr 9fr;
    }
    header {
      background-color: black;
      color:white;
      text-align: center;
    }
    main {
      height: 500px;
      overflow: auto;
    }
    ul {
      list-style: none;
    }
    li {
      background-color: black;
      color:white;
      font-size: 20px;
      margin-bottom: 20px;
      width: 100px;
      border-radius: 20px;
      padding: 10px;
    }
    li:hover {
      background-color: green;
      color:black;
    }
    nav a {
      color:white;
```

```

        text-decoration: none;
    }
</style>
<script>
    function bodyload(){
        var recent = document.getElementById("recent");
        switch(location.hash){
            case "#asp":
                recent.innerHTML = "ASP.NET"
                break;
            case "#android":
                recent.innerHTML = "Android";
                break;
        }
    }
</script>
</head>
<body onload="bodyload()">
    <header>
        <h1>Online Tutorials</h1>
    </header>
    <section>
        <nav>
            <ul>
                <li><a href="#home">Home</a></li>
                <li><a href="#asp">ASP.NET</a></li>
                <li><a href="#android">Android</a></li>
                <li><a href="#php">PHP</a></li>
            </ul>
            <h3>Recently Viewed</h3>
            <p id="recent"></p>
        </nav>
        <main>
            <h2 id="home">Home</h2>
            <p>Depending on <span><a href="#frameworks">Mobile
Frameworks</a></span> how you obtained the Windows software, this is a license
agreement between (i) you and the device manufacturer or software installer that
distributes the software with your device; or (ii) you and Microsoft Corporation (or,

```

based on where you live or, if a business, where your principal place of business is located, one of its affiliates) if you acquired the software from a retailer. Microsoft is the device manufacturer for devices produced by Microsoft or one of its affiliates, and Microsoft is the retailer if you acquired the software directly from Microsoft. Note that if you are a volume license customer, use of this software is subject to your volume license agreement rather than this agreement.</p>

<h2 id="asp">ASP.NET</h2>

<div>Back to Top</div>

<p>Depending on how you obtained the Windows software, this is a license agreement between (i) you and the device manufacturer or software installer that distributes the software with your device; or (ii) you and Microsoft Corporation (or, based on where you live or, if a business, where your principal place of business is located, one of its affiliates) if you acquired the software from a retailer. Microsoft is the device manufacturer for devices produced by Microsoft or one of its affiliates, and Microsoft is the retailer if you acquired the software directly from Microsoft. Note that if you are a volume license customer, use of this software is subject to your volume license agreement rather than this agreement.</p>

<h2 id="android">Android Training</h2>

<div>Back to Top</div>

<p>Depending on how you obtained the Windows software, this is a license agreement between (i) you and the device manufacturer or software installer that distributes the software with your device; or (ii) you and Microsoft Corporation (or, based on where you live or, if a business, where your principal place of business is located, one of its affiliates) if you acquired the software from a retailer. Microsoft is the device manufacturer for devices produced by Microsoft or one of its affiliates, and Microsoft is the retailer if you acquired the software directly from Microsoft. Note that if you are a volume license customer, use of this software is subject to your volume license agreement rather than this agreement.</p>

<dl id="frameworks">

<dt>Mobile Frameworks</dt>

<dd>Apache Cordova</dd>

<dd>Ionic</dd>

<dt>Mobile Platforms</dt>

<dd>Android</dd>

<dd>iOS</dd>

</dl>

```

<h2 id="php">PHP Tutorial</h2>
<div><a href="#home">Back to Top</a></div>

<p>Depending on how you obtained the Windows software, this is a license
agreement between (i) you and the device manufacturer or software installer that
distributes the software with your device; or (ii) you and Microsoft Corporation (or,
based on where you live or, if a business, where your principal place of business is
located, one of its affiliates) if you acquired the software from a retailer. Microsoft is the
device manufacturer for devices produced by Microsoft or one of its affiliates, and
Microsoft is the retailer if you acquired the software directly from Microsoft. Note that if
you are a volume license customer, use of this software is subject to your volume
license agreement rather than this agreement.</p>

</main>
</section>
</body>
</html>

```

navigator

- It provides properties and methods that are used to access client browser details.

appName	Browser family name
appVersion	Browser version
platform	Client OS platform
cookieEnabled	Cookie status
plugins[]	The extensions supported
mimeTypes[]	The file types supported
geoLocation	Get the client location details [latitude, longitude]

Ex:

```

<script>
    document.write(
        Browser Version : ${navigator.appVersion} <br>
        Browser Family : ${navigator.appName} <br>
        Platform      : ${navigator.platform} <br>
    )

```

Cookie Status : \${((navigator.cookieEnabled)?"Cookies Enabled":"Cookies Disabled - Please enable cookies on your browser")}


```
`);  
</script>
```

Ex:

```
<script>  
    navigator.geolocation.getCurrentPosition(function(position){  
        document.write(  
            Latitude : ${position.coords.latitude} <br>  
            Longitude : ${position.coords.longitude}  
        `);  
    })  
</script>
```

FAQ: How to know the plugins availability in browser?

Ans: `navigator.plugins[]`

Ex:

```
<script>  
    for(var item of navigator.plugins){  
        document.write(item.name + "<br>");  
    }  
</script>
```

Ex:

```
<script>  
    if(navigator.plugins["PDF Viewer"]==undefined){  
        document.write("Please Install PDF Viewer");  
    } else {  
        document.write("You can view PDF documents");  
    }  
</script>
```

FAQ: What is MIME ?

Ans :

Multipurpose Internet Mail Extension

A MIME type configures multiple extensions.

.jpg, .jpeg, .jif => image/jpeg(mime type)

Ex:

```
<script>
  for(var item of navigator.mimeTypes) {
    document.write(item.type + "<br>");
  }
</script>
```

history

- It provides a set of properties and methods that can access the current browsing history.

length
back()
forward()
go() => go('index.html') , go(2) go(-2)

Ex:

```
<script>
  if(history.length>2) {
    alert("Please Register for more..");
    location.href = "../public/register.html";
  } else {
    document.write("You can watch Max 2 pages for free");
  }
</script>
```

document [DOM]

write()
getElementById()
querySelector()
getElementsByTagName()
appendChild() etc...

Browser Events

- Event is a message sent by sender to its subscriber in order to notify the change.
- Event uses a software design pattern called "Observer".
- It is a communication pattern.
- Event uses "delegate" mechanism [function pointer].

Syntax:

```
function InsertClick(){ }      => subscriber : actions to perform
onclick="InsertClick()"      => sender : notify the change to subscriber
```

JavaScript Events

- Event is a message sent by sender to its subscriber in order to notify change.
- Event follows a software design pattern called "Observer".
- Event uses delegate mechanism. [Function Pointer]

a) Sender <button onclick="InsertClick()">

b) Subscriber function InsertClick() { }

c) Observer It is a mapping technique used to locate the function. (it will verify whether the button is clicked or not)

(Delegate means function pointer)

- Event comprises of EventHandler.

```
onclick="InsertClick()"
```

```
onclick                    => Event
```

```
onclick="InsertClick()"    => Event Handler
```

- Every Event Handler have 2 default arguments

a) this

b) event

```
<button onclick="InsertClick(this, event)">
```

this : It comprises of object related properties, like
id, name, className, width, height, src...

event : It comprises of even related properted, like
clientX, clientY, keyCode, charCode, **shiftKey, ctrlKey, etc..**

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function InsertClick(obj, e){
      document.write(
        Button Id : ${obj.id} <br>
        Button Name : ${obj.name} <br>
        Button Class: ${obj.className} <br>
        Clicked At X : ${e.clientX} <br>
        Ctrl Key : ${e.ctrlKey}
      `);
    }
  </script>
</head>
<body>
  <button id="btnInsert" name="Insert" class="btn btn-primary" onclick="InsertClick(this,
event)">Insert</button>
</body>
</html>
```

Ex:

```
<!DOCTYPE html>
```

```

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function ImageClick(e){
      if(e.ctrlKey) {
        window.open("../public/images/asp.jpg","ASP","width=450 height=300");
      } else {
        alert("You can use ctrl + click to view large.");
      }
    }
  </script>
</head>
<body>
  
  <p>Ctrl + Click to open</p>
</body>
</html>

```

- JavaScript allows custom event arguments.

- Argument can be any type

a) Primitive Type

b) Non Primitive Type

Syntax:

```

<button onclick=InsertClick( {}, [], ' ', true, 0 )>

```

```

function InsertClick(obj, array, string, bool, num) {
}

```

Ex:

```

<!DOCTYPE html>

```

```

<html lang="en">

```

```

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function ConnectClick(db, dbs){
      document.write(
        Server Name : ${db.server} <br>
        Database   : ${db.databaseName} <br>
        Database List: ${dbs.join('-->')}
      `);
    }
  </script>
</head>
<body>
  <button onclick="ConnectClick({databaseName:'Oracle', server:'127.0.0.1'}, ['Oracle',
'MySql', 'MongoDB'])">Connect</button>
</body>
</html>

```

- Event argument allows to send any specific property to subscriber.

Syntax:

```

<button name="Insert" id="btnInsert" onclick="InsertClick(this.id)">

```

```

function InsertClick(id)
{
}

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">

```

```

<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<script>
    function NameChange(obj){
        document.querySelector("p").innerHTML = obj.value;
    }
    function CityChange(city) {
        document.querySelector("p").innerHTML = city;
    }
</script>
</head>
<body>
    Your Name : <input type="text" onchange="NameChange(this)">
    <br>
    Your City :
        <select onchange="CityChange(this.value)">
            <option>Delhi</option>
            <option>Hyd</option>
            <option>Chennai</option>
        </select>
    <p></p>
</body>
</html>

```

- JavaScript allows developer to add Event Listeners.
- It enables clean separation of logic and design.
- It is easy to extend the logic and modify the design.

- To add events in JavaScript developer must know the various types of events in JavaScript

1. Mouse Events

- onmouseover
- onmouseout
- onmousedown
- onmousemove
- onmouseup

2. Keyboard Events

- onkeyup
- onkeydown
- onkeypress

3. Button Events

- onclick
- oncontextmenu
- ondblclick
- onselectionstart

4. Clipboard Events

- oncut
- oncopy
- onpaste

5. Element State Events

- onchange
- onfocus
- onblur

6. Form Events

- onsubmit
- onreset

7. Timer Events

- setInterval
- clearInterval
- setTimeout
- clearTimeout

8. Touch Events

- ontouchstart
- ontouchend
- ontouchmove

Events

- Event
- Event Handler
- Event Args
 - a) Default Args [this, event]
 - b) Custom Args

Mouse Events

- onmouseover
- onmousedown
- onmouseup
- onmouseout
- onmousemove

Ex: onmouseover() and onmouseout()

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function Ad2(){
      document.getElementById("ad").src="../public/images/a2.jpg";
      document.querySelector("p").innerHTML = "Special Offers on Refrigerators";
    }
    function Ad1(){
      document.getElementById("ad").src="../public/images/a1.jpg";
      document.querySelector("p").innerHTML = "20% OFF on Air Conditioners";
    }
  </script>
</head>
<body>
  
```

```

<p>mouse over - image change</p>
<div>
  <marquee scrollamount="10" onmouseover="this.stop()"
onmouseout="this.start()">
    
    
    
    
  </marquee>
</div>
</body>
</html>

```

Ex: onmousedown and onmouseup

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    .zoom-in {
      width: 300px;
      height: 300px;
    }
    .zoom-out {
      width: 200px;
      height: 200px;
    }
  </style>
  <script>
    function ZoomImage(){
      document.getElementById("pic").className = "zoom-in";
    }
    function ZoomOut(){

```



```

        document.getElementById("pic").className = "zoom-out";
    }
</script>

</head>
<body>
    
</body>
</html>

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <style>
        .color-container {
            display: flex;
            margin: 20px;
        }
        .color-container div {
            width: 200px;
            padding: 30px;
            color: white;
            font-size: 20px;
        }
        .color-container div:hover {
            cursor: grab;
        }
    </style>
    <script>
        function GetColor(colorName){
            var pic = document.querySelector("img");

```

```
switch(colorName)
{
    case "red":
        pic.src = "../public/images/red-shirt.png";
        break;
    case "green":
        pic.src= "../public/images/green-shirt.png";
        break;
    case "blue":
        pic.src="../public/images/blue-shirt.png";
        break;
}
}
</script>
```

```
</head>
<body>
    <div class="color-container">
        <div onmouseover="GetColor('red')" style="background-color: red;">
            red
        </div>
        <div onmouseover="GetColor('green')" style="background-color: green;">
            green
        </div>
        <div onmouseover="GetColor('blue')" style="background-color: blue;">
            blue
        </div>
    </div>
    <div>
        
    </div>
</body>
</html>
```

Ex: Mouse Move

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function Animate(e) {
      var flag = document.getElementById("flag");
      flag.style.position = "fixed";
      flag.style.top = e.clientY + "px";
      flag.style.left = e.clientX + "px";
    }
  </script>
</head>
<body onmousemove="Animate(event)">
  <div style="height:1000px">
    <p>Move the mouse pointer to test.</p>
  </div>
  
</body>
</html>

```

Touch Events

- ontouchstart
- ontouchend
- ontouchmove

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

```

```
<title>Document</title>
<script>
  function MoveImage(e){
    var pic = document.querySelector("img");
    pic.style.position = "fixed";
    pic.style.left = e.touches[0].clientX + "px";
    pic.style.top = e.touches[0].clientY + "px";
  }
</script>
</head>
<body>
  
</body>
</html>
```

Mouse Events

- onmouseover
- onmouseout
- onmousedown
- onmouseup
- onmousemove

Touch Events

- ontouchstart
- ontouchend
- ontouchmove

Keyboard Events

- onkeyup
- onkeydown
- onkeypress

Note: Keyup and Keydown are good to use when any action is related to key [chars].
They can't access keyCode or charCode.

KeyPress is good to use when action is related to character code.

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function VerifyUserId(){
      var userid = document.getElementById("UserId").value;
      var error = document.getElementById("error");

      fetch("../data/users.json")
      .then(function(response){
        return response.json();
      })
      .then(function(users){
        for(var user of users) {
          if(user.UserName==userid){
            error.innerHTML = "User Id Taken - Try Another".fontcolor('red');
            break;
          } else {
            error.innerHTML= "User Id Available".fontcolor('green');
          }
        }
      })
    }

    function VerifyCaps(e){
      var pwdError = document.getElementById("pwdError");
      if(e.keyCode>=65 && e.keyCode<=90) {
        pwdError.style.display = "block";
      } else {
        pwdError.style.display = "none";
      }
    }
  </script>
</head>
</html>
```

```

    }
</script>
<link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body>
  <dl>
    <h2>Register User</h2>
    <dt>User Id</dt>
    <dd><input type="text" onkeyup="VerifyUserId()" id="UserId"></dd>
    <dd id="error"></dd>
    <dt>Password</dt>
    <dd><input type="password" onkeypress="VerifyCaps(event)" id="Password"></dd>
    <dd id="pwdError" style="color:goldenrod; display: none;">
      <span class="bi bi-exclamation-triangle"></span> <span>Warning Caps
ON</span>
    </dd>
  </dl>
</body>
</html>

```

Event Attributes

keyCode] returns the code of character

charCode]

which] returns the code of character [all types of keyboards]

shiftKey] return true when used

ctrlKey]

altKey]

Button Events

- onclick single click of mouse left button

- ondblclick double click of mouse left button

- oncontextmenu single click of mouse right button

- onselectstart mouse leftbutton drag

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function OpenImage(){
      window.open("../public/images/asp.jpg","ASP","width=400 height=400");
    }
    document.onselectstart = function(){
      return false;
    }
    document.oncontextmenu = function(){
      alert("Right Click Disabled");
      return false;
    }
  </script>
</head>
<body>
  
  <p>Double click to view large.</p>
  <h1>This page have secured data content. you can't copy</h1>
</body>
</html>
```

Element State Events

- onchange
- onblur
- onfocus

Ex:

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function VerifyUserId(){
      var userid = document.getElementById("UserId").value;
      var error = document.getElementById("error");

      fetch("../data/users.json")
      .then(function(response){
        return response.json();
      })
      .then(function(users){
        for(var user of users) {
          if(user.UserName==userid){
            error.innerHTML = "User Id Taken - Try Another".fontcolor('red');
            break;
          } else {
            error.innerHTML= "User Id Available".fontcolor('green');
          }
        }
      })
    }
    function HideMessage(){
      document.getElementById("error").innerHTML = "";
    }
    function VerifyCaps(e){
      var pwdError = document.getElementById("pwdError");
      if(e.keyCode>=65 && e.keyCode<=90) {
        pwdError.style.display = "block";
      } else {
        pwdError.style.display = "none";
      }
    }
  }
</script>
</html>
```



```

function ShowTip(){
    document.getElementById("tip").innerHTML = "Password 4 to 15 chars alpha
numeric only".fontcolor('gold').fontSize(2);
}
function HideTip(){
    document.getElementById("tip").innerHTML = "";
}
</script>
<link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
</head>
<body>
<dl>
<h2>Register User</h2>
<dt>User Id</dt>
<dd><input type="text" onkeyup="VerifyUserId()" onblur="HideMessage()"
id="UserId"></dd>
<dd id="error"></dd>
<dt>Password</dt>
<dd><input type="password" onfocus="ShowTip()" onblur="HideTip()"
onkeypress="VerifyCaps(event)" id="Password"></dd>
<dd id="pwdError" style="color:goldenrod; display: none;">
    <span class="bi bi-exclamation-triangle"></span> <span>Warning Caps
ON</span>
</dd>
<dd id="tip"></dd>
</dl>
</body>
</html>

```

Clipboard Events

- oncut
- oncopy
- onpaste

Ex.

```
<!DOCTYPE html>
```

```

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function Cut(){
      document.querySelector("p").innerHTML = "Removed - Placed on Clipboard";
    }
    function Copy(){
      document.querySelector("p").innerHTML = "Copied - Placed on Clipboard";
    }
    function Paste(){
      document.querySelector("p").innerHTML = "Inserted - from Clipboard";
    }

  </script>
</head>
<body>
  <textarea oncut="Cut()" oncopy="Copy()" onpaste="Paste()" rows="4"
cols="40"></textarea>
  <p></p>
</body>
</html>

```

Mouse Events

Keyboard Events

Element State Events

Button Events

Clipboard Events

Touch Events

Form Events

- onsubmit It defines actions to perform when form is submitted.

- onreset It defines actions when form resets.

Note: Form events can fire up only using Generic buttons.

<input type="submit">	HTML 4
<input type="reset">	HTML 4
<button type="submit">	HTML 5
<button type="reset">	HTML 5

Syntax:

```
<form onsubmit="" onreset="">

    <button type="submit"> Submit </button>
    <button type="reset"> Reset </button>

</form>
```

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <form onreset="alert('Form will Reset')" onsubmit="alert('Form will Submit')">
        User Name : <input type="text" name="UserName">
        <button type="submit">Submit</button>
        <button type="reset">Cancel</button>
    </form>
</body>
</html>
```

Timer Events

setTimeout()

clearTimeout()

setInterval()

clearInterval()

setTimeout()

- It is used to defined time interval for any task so that it is delayed in execution.
- Task is loaded into memory but released into process after specific time interval.

Syntax:

setTimeout(function(){}, timeInterval); => 1000ms = 1sec

- You can remove the task from memory using clearTimeout()

Syntax:

clearTimeout(referenceName)

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script>
    function Msg1(){
      document.querySelector("h1").innerHTML = "Hello !";
    }
    function Msg2(){
      document.querySelector("h1").innerHTML = "How are you ?";
    }
    function Msg3(){
      document.querySelector("h1").innerHTML = "Welcome to JavaScript";
    }
    var m1, m2, m3;
    function bodyload(){
      m1 = setTimeout(Msg1, 3000);
```

```

        m2 = setTimeout(Msg2, 6000);
        m3 = setTimeout(Msg3, 10000);
    }
    function CancelClick(){
        clearTimeout(m2);
    }
</script>
</head>
<body onload="bodyload()">
    <button onclick="CancelClick()">Cancel Msg2</button>
    <h1 align="center"></h1>
</body>
</html>

```

setInterval()

- It loads the task into memory and executed at regular time intervals.

Syntax:

```
setInterval(function(){}, timeInterval)
```

- You can cancel from memory using "clearInterval()"

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script>
        function ShowTime(){
            var now = new Date();
            document.querySelector("h1").innerHTML = now.toLocaleTimeString();
        }
        var time;
        function bodyload(){

```

```

        time = setInterval(ShowTime, 1000);
    }
    function StopClick(){
        clearInterval(time);
    }
    function StartClick(){
        bodyload();
    }
</script>
</head>
<body onload="bodyload()">
    <button onclick="StopClick()">Stop</button>
    <button onclick="StartClick()">Start</button>
    <h1 align="center"></h1>
</body>
</html>

```

setTimeout
 clearTimeout
 setInterval
 clearInterval

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-
icons.css">
    <link rel="stylesheet" href="../node_modules/bootstrap/dist/css/bootstrap.css">
    <script>
        var count = 0;
        function LoadProduct(){
            count++;

```

```

        fetch(`http://fakestoreapi.com/products/${count}`)
        .then(function(response){
            return response.json();
        })
        .then(function(product){
            document.getElementById("title").innerHTML = product.title;
            document.getElementById("pic").src= product.image;
        })
    }
    function bodyload(){
        LoadProduct();
    }
    var slideShow;
    function PlayClick(){
        slideShow = setInterval(LoadProduct, 5000);
        document.getElementById("status").innerHTML = "<b>Slide Show - Started</b>";
    }
    function PauseClick(){
        clearInterval(slideShow);
        document.getElementById("status").innerHTML = "<b>Slide Show -
Paused</b>";
    }
    function NextClick(){
        LoadProduct();
    }
</script>
</head>
<body onload="bodyload()" class="container-fluid d-flex justify-content-center align-
items-center" style="height:500px">
    <div class="card p-2 w-50" >
        <div class="row">
            <div class="col-2">
                <button class="btn btn-danger" onclick="PreviousClick()">
                    <span class="bi bi-chevron-left"></span>
                </button>
            </div>
            <div class="col-8">
                <div class="card-header text-center">

```

```

        <p id="title"></p>
        <p id="status"></p>
    </div>
    <div class="card-body">
        <img id="pic" width="100%" height="300">
    </div>
    <div class="card-footer text-center">
        <button class="btn btn-primary" onclick="PlayClick()">
            <span class="bi bi-play"></span>
        </button>
        <button class="btn btn-primary" onclick="PauseClick()">
            <span class="bi bi-pause"></span>
        </button>
    </div>
</div>
<div class="col-2">
    <button class="btn btn-danger" onclick="NextClick()">
        <span class="bi bi-chevron-right"></span>
    </button>
</div>
</div>
</div>
</body>
</html>

```

Ex: Status with Spinner

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link rel="stylesheet" href="../node_modules/bootstrap/dist/css/bootstrap.css">
    <script>
        var count = 0;
    </script>

```



```

function LoadingStatus(){
    count++;
    if(count<=100){
        document.getElementById("status").innerHTML = `${count} %`;
    } else {
        location.href = "../javascript-examples/shopper-template.html";
    }
}
function StartClick(){
    document.getElementById("statusContainer").style.display="block";
    document.querySelector("button").style.display = "none";
    setInterval(LoadingStatus,100);
}
</script>
</head>
<body class="container-fluid d-flex justify-content-center align-items-center"
style="height:500px">
    <div>
        <button onclick="StartClick()" class="btn btn-primary">Load Template</button>
        <div class="text-center mt-4" id="statusContainer" style="display:none">
            <p class="spinner-border text-success"></p>
            <p id="status"></p>
            <p>Loading Template</p>
        </div>
    </div>
</body>
</html>

```

Ex: Status with Progress

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>

```

```

<link rel="stylesheet" href="../node_modules/bootstrap/dist/css/bootstrap.css">
<script>
  var count = 0;
  function LoadingStatus(){
    count++;
    if(count<=100){
      document.getElementById("progress").value=count;
      document.getElementById("status").innerHTML = `${count} %`;
    } else {
      location.href = "../javascript-examples/shopper-template.html";
    }
  }
  function StartClick(){
    document.getElementById("statusContainer").style.display="block";
    document.querySelector("button").style.display = "none";
    setInterval(LoadingStatus,200);
  }
</script>
</head>
<body class="container-fluid d-flex justify-content-center align-items-center"
style="height:500px">
  <div>
    <button onclick="StartClick()" class="btn btn-primary">Load Template</button>
    <div class="text-center mt-4" id="statusContainer" style="display:none">
      <p>
        <progress id="progress" min="1" max="100" value="1"></progress>
      </p>
      <p id="status"></p>
      <p>Loading Template</p>
    </div>
  </div>
</body>
</html>

```

addEventListener : It is a function that allows to add browser events to any specific element

Syntax:

```
button.addEventListener("eventName", function(){  
  
    })
```

Ex:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <meta http-equiv="X-UA-Compatible" content="IE=edge">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>Document</title>  
    <script>  
        function bodyload(){  
            var button = document.createElement("button");  
            button.innerHTML = "Click Me";  
            button.addEventListener("click", function(){  
                alert('Button Clicked');  
            });  
            button.addEventListener("mouseover", function(){  
                var p = document.createElement("p");  
                p.innerHTML = "Mouse is Over Button";  
                document.getElementById("container").appendChild(p);  
            });  
            document.getElementById("container").appendChild(button);  
        }  
    </script>  
</head>  
<body onload="bodyload()">  
    <div id="container">  
  
        </div>  
</body>  
</html>
```

Arrow Functions

- It is a short hand technique of writing a function.
- It is good for function expressions.

()	function
=>	return
{ }	block

```
<script>
  /*
  function Hello(name){
    return `Hello ! name`;
  }
  */
  var hello = (name)=> `Hello ! ${name} <br>`;
  var welcome = ()=> document.write("Welcome to JS");
  document.write(hello("john"));
  welcome();
</script>
```

Ex:

```
<script>
  fetch("http://fakestoreapi.com/products/1")
  .then(response=>response.json())
  .then(product=>document.write(`${product.title}`));

</script>
```

Ex:

```
<script>
  var addition = (a,b) => a + b;
  document.write("Addition=" + addition(10,30));
</script>
```

jQuery

- jQuery is a JavaScript library for building effective UI.
- John Resig introduced jQuery library in early 2006.
- "Write less - Do More".
- jQuery provides a set of methods that handle
 - a) DOM manipulations
 - b) BOM interactions
 - c) API Connections etc..
- jQuery reduces the browser compatibility issues.

```
> npm install jquery --save
```

- Enable jQuery in a page

```
<script src="./node_modules/jquery/dist/jquery.js"> </script>
```

- Load jQuery library

```
<script>
  $(function(){

  })
</script>
```

```
(or)
<script>
  $(document).ready(function(){

  })
</script>
```

jQuery DOM references

- jQuery can refer any DOM element just by using all CSS selectors.

```
$("type")
$("#id")
$(".class")
$("parent child")
$("level1+level2")
```

- jQuery DOM methods

text()	innerText
html()	innerHTML
append()	add as suffix
prepend()	add as prefix
before()	add above
after()	add below
css()	defined style attributes
val()	get and set value
attr()	get and set attribute
prop()	get and set property etc..

appendTo()	
prependTo()	
before()	above
after()	etc...

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script src="../node_modules/jquery/dist/jquery.js"></script>
  <script>
    $(function(){
      $("h1").text("jQuery");
      $("p").text("Write Less to More..");
      $(".title").html("Welcome to <b>JavaScript</b> library.");
    })
  </script>
</head>
<body>
  <h1></h1>
  <p></p>
  <div class="title"></div>
</body>
</html>

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script src="../node_modules/jquery/dist/jquery.js"></script>
  <script>
    $(function(){
      var categories = ["Electronics", "Footwear", "Fashion"];
      for(var category of categories) {
        $('<li>${category}</li>').appendTo("ol");
        $('<option>${category}</option>').appendTo("select");
      }
    })
  </script>
</head>
<body>
  <ol>

  </ol>
  <select>

  </select>
</body>
</html>

```

JavaScript Test

1. What is difference between var, let and const?
2. What is difference between Rest parameter and Spread operator in JavaScript? Explain with example.
3. What is array destructuring? Explain with example
4. How to convert a string "true" into boolean true?
5. What is difference between slice(), substr() and substring() methods?

6. What is function closure? explain with example.
7. How to access the client geolocation? Give an example.
8. What is anonymous function? Give example
9. Give an example of any JavaScript promise.
10. How to remove any specific property from object?

JQuery DOM methods and Events

Sudhakar Sharma

8:41AM

Enable jQuery

jQuery Reference techniques

jQuery DOM Methods

html()	innerHTML
text()	innerText
attr()	get attribute
val()	get value
prop()	get property
css()	set css styles
append()	adds as suffix
prepend()	adds as prefix

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script src="../node_modules/jquery/dist/jquery.js"></script>
  <script>
    $(function(){
      $("p").html("<b>Welcome to jquery</b>");
      $("p").append("Hello !");
    });
  </script>
</head>
</html>
```



```

        $("p").prepend("Hi!");
        $("p").before("jQuery");
        $("p").after("Write less do more");
    })
</script>
</head>
<body>
    <p></p>
</body>
</html>

```

FAQ: What is difference between "append" and "appendTo"?

Ans:

In append first parent element is defined to which you can add new element.
In appendTo first child element is created then added to parent element.

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script src="../node_modules/jquery/dist/jquery.js"></script>
    <script>
        $(function(){
            $("p").append(``);
            $(``).appendTo("p");
        })
    </script>
</head>
<body>
    <p></p>
</body>
</html>

```

jQuery Iterations

`$.each()` : It is an iterator of jQuery that can read both properties and values.

Syntax:

```
$.each(collectionName, function(property, value){
```

```

    })

    $.each(collection, (prop, value)=>{

    })

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script src="../node_modules/jquery/dist/jquery.js"></script>
  <script>
    $(function(){
      var categories = ["Electronics", "Footwear", "Fashion"];
      $.each(categories,(property, value)=>{
        $('<li>[${property}] ${value}</li>').appendTo("ol");
        $("select").append('<option>${value}</option>');
      })
    })
  </script>
</head>
<body>
  <ol>

  </ol>
  <select></select>
</body>
</html>

```

jQuery Events

- jQuery separates the event from design.
- jQuery uses event listener technique.
- All JavaScript events are same, but the event args will change.

Syntax:

```

<button id="btnSubmit"> Submit </button>

```

```

$("#btnSubmit").click(function(){
  actions on click
})

```

onmouseover => mouseover
onclick => click
onchange => change

- jQuery contains only one default event argument, which can access both object and event properties.

```
$("#button").click(function(event){  
    event.EventProperty  
    event.target.ObjectProperty  
})
```

Ex:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <meta http-equiv="X-UA-Compatible" content="IE=edge">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>Document</title>  
    <script src="../node_modules/jquery/dist/jquery.js"></script>  
    <script>  
        $(function(){  
            $("#button").click((e)=>{  
                document.write(  
                    Button Id    : ${e.target.id} <br>  
                    Button Name  : ${e.target.name} <br>  
                    Button Class : ${e.target.className} <br>  
                    X Position   : ${e.clientX} <br>  
                    Ctrl Key     : ${e.ctrlKey}  
                );  
            })  
        })  
    </script>  
</head>  
<body>  
    <button id="btnInsert" name="Insert" class="btn btn-primary">Insert</button>  
</body>  
</html>
```

Ex: Form Input

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">
```

```

<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<script src="../node_modules/jquery/dist/jquery.js"></script>
<script>
    $(function(){
        $("#btnRegister").click(=>{
            $("#lblName").html($("#Name").val());
            $("#lblPrice").html($("#Price").val());
            $("#lblCity").html($("#City").val());

            var stock = ($("#Stock").prop("checked"))?"Available":"Out of Stock";
            $("#lblStock").html(stock);
        })
    })
</script>
</head>
<body>
    <h2>Register</h2>
    <dl>
        <dt>Name</dt>
        <dd><input type="text" id="Name"></dd>
        <dt>Price</dt>
        <dd><input type="text" id="Price"></dd>
        <dt>City</dt>
        <dd>
            <select id="City">
                <option>Delhi</option>
                <option>Hyd</option>
            </select>
        </dd>
        <dt>Stock</dt>
        <dd>
            <input type="checkbox" id="Stock"> Available
        </dd>
    </dl>
    <button id="btnRegister">Register</button>
    <div style="position:absolute; top:10px; left:300px">
        <h2>Product Details</h2>
        <dl>
            <dt>Name</dt>
            <dd id="lblName"></dd>
            <dt>Price</dt>
            <dd id="lblPrice"></dd>
            <dt>City</dt>
            <dd id="lblCity"></dd>
        </dl>
    </div>
</body>
</html>

```

```

        <dt>Stock</dt>
        <dd id="lblStock"></dd>
    </dl>
</div>
</body>
</html>

```

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script src="../node_modules/jquery/dist/jquery.js"></script>
    <script>
        $(function(){
            fetch("../data/products.json")
            .then(response=>response.json())
            .then(products=> {
                $.each(products,(property, product)=>{
                    $(
                        <tr>
                            <td>${product.Name}</td>
                            <td>${product.Price}</td>
                            <td><img src=${product.Photo} width="100" height="100"></td>
                            <td>${product.Stock}</td>
                            <td>${product.Rating.Rate} [${product.Rating.Count}]</td>
                        </tr>
                    `).appendTo("tbody");
                })
            })
        })
    </script>
</head>
<body>
    <table width="400" border="1">
        <thead>
            <tr>
                <th>Name</th>
                <th>Price</th>
                <th>Photo</th>
                <th>Stock</th>
                <th>Rating</th>
            </tr>

```

```
</thead>
<tbody>

</tbody>
</table>
</body>
</html>
```

jQuery Ajax

Sudhakar Sharma

8:43 AM

jQuery DOM methods

jQuery Events

jQuery Ajax

- Asynchronous JavaScript and XML
- Asynchronous is a technique where you can process any request without blocking other requests.
- JavaScript is used for handling functions.
- XML is used for transporting data.
- Ajax is used to handle "Partial Post" back in page.
- Without loading complete page new details can be added to page.

FAQ: Why we need Ajax?

Ans: Partial Post Back

- Ajax makes the page light weight
- Ajax makes the page perform better.
- jQuery provides various Ajax methods to handle interaction with API data.

\$.ajax() => can handle data from any file [html, json, xml etc..]

\$.getJSON() => can handle data only from JSON file

\$.ajaxStart()]

\$.ajaxStop()]

\$.ajaxSuccess()] Ajax life cycle methods.

\$.ajaxComplete()]

\$.ajaxError()]

Syntax:

```
$.ajax({  
  url: " ",  
  method: "get, post, put, delete",  
  success: function() {}  
  error: function() {}  
})
```

Features of AJAX

- It directly returns data in JSON.
- It doesn't require data conversions.
- It is good in error handling
- It can track the errors at every phase.
- It is more secured
- It can handle CORS issues. [Cross Origin Resource Sharing]
- It enable SPA environment for application. [Single Page Application]

Ex:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>SPA</title>  
  <link rel="stylesheet" href="../node_modules/bootstrap/dist/css/bootstrap.css">  
  <script src="../node_modules/jquery/dist/jquery.js"></script>  
  <script>  
    $(function(){  
      var now = new Date();  
      $("#time").html("Page Loaded At : " + now.toLocaleTimeString());  
  
      function LoadPage(pageName){  
        $.ajax({  
          method: "get",  
          url: pageName,  
          success: (data) => {  
            $("#main").html(data);  
          },  
        })  
      }  
    })  
  </script>  
</head>  
<body>  
  <div id="time"></div>  
  <div id="main"></div>  
</body>  
</html>
```

```

        error: (ex) => {
            alert(ex);
        }
    })
}

$("button").click(function(e){
    switch(e.target.id){
        case "emi":
            LoadPage("../javascript-examples/emi.html");
            break;
        case "kfc":
            LoadPage("../javascript-examples/kfconline.html");
            break;
        case "shopper":
            LoadPage("../javascript-examples/shopper-template.html");
            break;
    }
})
})
</script>
</head>
<body class="container-fluid">
    <nav class="mt-3">
        <ul class="list-unstyled d-flex">
            <li class="me-3">
                <button id="emi" class="btn btn-primary">EMI Calculator</button>
            </li>
            <li class="me-3">
                <button id="shopper" class="btn btn-primary">Shopper Template</button>
            </li>
            <li class="me-3">
                <button id="kfc" class="btn btn-primary">KFC Form</button>
            </li>
            <li class="me-3">
                <button class="btn btn-primary" id="time"></button>
            </li>
        </ul>
    </nav>
    <main class="mt-2">

```



```
</main>
</body>
</html>
```

FAQ: How to write event for any element, which is a part of its parent element?

Ans: by using "(document).on()"

Ex: Shopper

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="../node_modules/bootstrap/dist/css/bootstrap.css">
  <script src="../node_modules/jquery/dist/jquery.js"></script>
  <script>
    function LoadCategories(){
      $.ajax({
        method:"get",
        url: "http://fakestoreapi.com/products/categories",
        success: (data)=> {
          data.unshift("all");
          $.each(data,(property, value)=>{
            $(`<option
value=${value}>${value.toUpperCase()}</option>`).appendTo("#lstCategories");
          })
        }
      })
    }

    function LoadProducts(url) {
      $("main").html("");
      $.ajax({
        method:"get",
        url: url,
```

```

        success: (products)=> {
            $.each(products,(property, product)=>{
                $(
                    <div class="card m-2 p-2" style="width:200px">
                        <img src=${product.image} class="card-img-top" height="140">
                        <div class="card-header" style="height:130px">
                            <p>${product.title}</p>
                        </div>
                        <div class="card-footer">
                            <button id="btnAdd" value=${product.id} class="btn btn-danger">Add
to Cart</button>
                        </div>
                    </div>
                `).appendTo("main");
            })
        }
    })
}

$(function(){
    LoadCategories();
    LoadProducts("http://fakestoreapi.com/products");

    $("#lstCategories").change(function(e){
        if(e.target.value=="all") {
            LoadProducts("http://fakestoreapi.com/products");
        } else {
            LoadProducts(`http://fakestoreapi.com/products/category/${e.target.value}`);
        }
    })

    $(document).on("click", "#btnAdd", function(e){
        alert("Add Clicked : " + e.target.value);
    })
})
</script>
</head>
<body class="container-fluid">
    <header class="bg-dark text-white text-center p-2">

```

```

    Fakestore API
  </header>
  <section class="mt-3 row">
    <nav class="col-2">
      <div>
        <label class="form-label">Select Category</label>
        <div>
          <select id="lstCategories" class="form-select">

            </select>
          </div>
        </div>
      </nav>
      <main class="col-10 d-flex flex-wrap overflow-auto" style="height: 500px;" >

    </main>
  </section>
</body>
</html>

```

jQuery UI

Sudhakar Sharma

.

Yesterday

jQuery Ajax

- SPA
- API
- \$.ajax()

Ajax Life Cycle Methods

- Life Cycle methods can track the activity at various phases and report issues.
- It is easy to track the issues in Ajax request by using Ajax Life Cycle Methods.

.ajaxStart()

.ajaxStop()

```
.ajaxComplete()  
.ajaxSuccess()  
.ajaxError()
```

fetch(), \$.ajax() => XMLHttpRequest [It is the object that manages Ajax in browser]

Ex:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Document</title>  
  <script src="../node_modules/jquery/dist/jquery.js"></script>  
<script>  
  $(function(){  
    $("button").click(function(){  
      $.ajax({  
        method : "get",  
        url: "../data/product.json",  
        success: function(data){  
          $.each(data,(property,value)=>{  
            $('<li>${value.Name}</li>').appendTo("ol");  
          })  
        }  
      })  
    })  
  })  
  .ajaxStart(=> {  
    alert(`Your Request for Products Data Started`);  
  })  
  .ajaxSuccess(=> {  
    alert(`Your request proccessed successfully..`);  
  })  
  .ajaxComplete(=> {  
    alert(`Your request for products data completed.`);  
  })  
}
```

```

    })
    .ajaxStop(=> {
        alert(` Request Stopped and Memory Cleaned`);
    })
    .ajaxError((event, jqXHR)=>{
        alert(`${jqXHR.status} - ${jqXHR.statusText}`);
    })
</script>
</head>
<body>
    <button>Load</button>
    <br><br>
    <ol>

    </ol>
</body>
</html>

```

- XMLHttpRequest object provides various properties to track the error status

- a) status
- b) statusText
- c) header

- Status returns code:

2xx	Success
3xx	Redirection
4xx	Client Error
5xx	Server Error

- StatusText returns the code relative text

200	OK
302	Method
404	Not Found
502	Unauthorized

- Header returns information about content type which is requested

text/html	if you request HTML file
application/json	if you request JSON file
application/pdf	pdf file
text/plain	text file

jQuery Effects

slide()
 slideToggle()
 slideIn()
 fade()
 fadeToggle()
 fadeIn()
 show()
 hide()
 toggle()

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script src="../node_modules/jquery/dist/jquery.js"></script>
  <script>
    $(function(){
      $("button").click(function(){
        $("img").slideToggle(2000);
      })
    })
  </script>
</head>
<body>
  <button>Toggle</button>
  
```

```
<div>
  
</div>
</body>
</html>
```

jQuery UI

- It is a component library same like Bootstrap.
- It have simple methods to handle various interaction, but not easy to customize.

Download and Copy jQuery UI Library

1. Visit ["https://jqueryui.com/"](https://jqueryui.com/);
2. Download library
3. Extract the library
4. Copy all files and paste into your project "node_modules/jquery-ui" location

jQuery Interactions

- draggable()
- resizable()
- sortable() etc..

Ex:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="../node_modules/jquery-ui/jquery-ui.css">
  <script src="../node_modules/jquery/dist/jquery.js"></script>
  <script src="../node_modules/jquery-ui/jquery-ui.js"></script>
```

```

<script>
    $(function(){
        $("img").resizable();
        $("ol").sortable();
    })
</script>
</head>
<body>

    
    <div>
        <ol>
            <li>HTML</li>
            <li>JavaScript</li>
            <li>Bootstrap</li>
            <li>CSS</li>
        </ol>
    </div>

</body>
</html>

```

jQuery Widgets

- They are similar to bootstrap components like

button

accordion

dialog

nav

autocomplete etc...

Ex:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">

```



```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<link rel="stylesheet" href="../node_modules/jquery-ui/jquery-ui.css">
<script src="../node_modules/jquery/dist/jquery.js"></script>
<script src="../node_modules/jquery-ui/jquery-ui.js"></script>
<script>
    $(function(){
        $("#faq").dialog(); (or) $("#faq").accordion();
        $("#dep").datepicker();
    })
</script>
</head>
<body>
    <div>
        Departure : <input type="text" id="dep">
    </div>
    <div id="faq" title="Netflix FAQ">
        <h2>What is Netflix?</h2>
        <div>
            <p>something..</p>
        </div>
        <h2>How to use Netflix?</h2>
        <div>
            <p>something.. about..</p>
        </div>
    </div>
</body>
</html>

```

jQuery Plugins

- Plugin is software tool which is embedded into your project to add some additional functionality.
- It is just like an extension.

a) Barcode

b) Grid

c) Cookies

jQuery Plugin

Sudhakar Sharma

.

Yesterday

jQuery Ajax

jQuery Effects

jQuery UI

- Interactions

- Widgets

jQuery Plugin's

- Plugin is a software extension used for your application to handle specific interaction
- Plugin's are design by various 3rd party communities and developers, which you can implement and customize according to requirment.

"npmjs.com" => search for plugin's

Ex: Barcode

1. Install jQuery barcode library

```
> npm i jquery-ean13
```

2. Import the recommended library and implement using its functions

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Document</title>
```

```
  <script type="text/javascript" src="../node_modules/jquery/dist/jquery.js"></script>
```

```
  <script type="text/javascript" src="../node_modules/jquery-ean13/dist/jquery-ean13.min.js"></script>
```

```
  <script type="text/javascript">
```

```

    $(function(){
        $("#code").EAN13("9876112091213");
    })
</script>
</head>
<body>
    <figure>
        
        <figcaption>
            <canvas id="code" width="200" height="60">
                your browser doesn't support canvas.
            </canvas>
        </figcaption>
    </figure>
</body>
</html>

```

Ex: Grid

```
>npm install jsgrid --save
```

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Grid</title>
    <link rel="stylesheet" href="../node_modules/jsgrid/dist/jsgrid.min.css">
    <link rel="stylesheet" href="../node_modules/jsgrid/dist/jsgrid-theme.min.css">

    <script type="text/javascript" src="../node_modules/jquery/dist/jquery.js"></script>
    <script type="text/javascript" src="../node_modules/jsgrid/dist/jsgrid.js"></script>
    <script>
        var products = [
            {
                "Name": "Samsung TV",
                "Price": 35000.55,
                "Stock": true,

```

```

        "Rating": {"Rate":4.5, "Count":6000},
        "Photo": "../public/images/tv.jpg"
    },
    {
        "Name": "Nike Casuals",
        "Price": 5000.35,
        "Stock": true,
        "Rating":{"Rate":3.5, "Count":300},
        "Photo": "../public/images/shoe.jpg"
    },
    {
        "Name": "Speaker",
        "Price": 4000.22,
        "Stock": false,
        "Rating":{"Rate": 4.2, "Count": 5000},
        "Photo": "../public/images/speaker.jpg"
    }
];
$(function(){
    $("#grid").jsGrid({
        width: "100%",
        height: "300px",

        filtering: true,
        editing:true,
        sorting:true,
        paging: true,

        data : products,

        fields: [
            { name: "Name", type: "text", width:150},
            { name: "Price", type: "number", width:100},
            { name: "Stock", type: "checkbox", title: "In Stock" },
            { type: "control" }
        ]
    })
})
</script>
</head>

```

```
<body>
  <h2>Product Grid</h2>
  <div id="grid">

    </div>
</body>
</html>
```

Cookies

- You are developing web applications.
- Web applications use "HTTP" as protocol.
- HTTP [Hyper Text Transfer Protocol] - It set rules for handling communication in web application.

HTTP is used for => HTML, JSON, Images, Documents

SMTP is used for => Email(sending.receiving)

MSMQ is used for => Chatting

- HTTP is a state less protocol. It can't remember your information between requests.
- Your application requires various state management techniques to store data and transport across pages.

a) query string

b) cookies

jQuery Cookies

Sudhakar Sharma

.

Yesterday

Cookies

- Cookie is a simple text document.
- Client details can be stored in a cookie and appended into client memory.
- Client memory can be

a) Browser Memory [Temporary Cookie]

b) HDD [Persistent(permanently stored cookies) Cookie]

- jQuery provides a cookie plugin, which allows to create, access and remove cookies.
- Cookies allow to authenticate and authorize your application.

Ex:

1. Install Cookie library for jquery

```
>npm install jquery.cookie --save
```

2. Create a new site folder and add following files

```
index.html  
login.html  
shop.html
```

index.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Index</title>  
</head>  
<body>  
  <h1>Shopper | Online Shopping</h1>  
  <a href="shop.html">Shop</a> | <a href="login.html">Login</a>  
</body>  
</html>
```

login.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta http-equiv="X-UA-Compatible" content="IE=edge">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```

<title>Login</title>
<script src="../node_modules/jquery/dist/jquery.js"></script>
<script src="../node_modules/jquery.cookie/jquery.cookie.js"></script>
<script>
    $(function(){
        $("#error").html((navigator.cookieEnabled?"Cookies Enabled":"Please Allow
Cookies on your Browser - For our site");
        $("#btnLogin").click(function(){
            $.ajax({
                method: "get",
                url: "../data/users.json",
                success: (users) => {
                    for(var user of users){
                        if(user.UserName==$("#UserId").val() &&
user.Password==$("#Password").val()){
                            $.cookie("userid", $("#UserId").val(), {expires : 4});
                            location.href="shop.html";
                            break;
                        } else {
                            $("#error").html("Invalid UserId / Password");
                        }
                    }
                }
            })
        })
    })
</script>
</head>
<body>
    <a href="index.html">Shopping Index</a>
    <h2>User Login</h2>
    <dl>
        <dt>User Id</dt>
        <dd><input type="text" id="UserId"></dd>
        <dt>Password</dt>
        <dd><input type="password" id="Password"></dd>
    </dl>
    <button id="btnLogin">Login</button>

```

```
<p id="error" align="center"></p>
</body>
</html>
```

shop.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="../node_modules/bootstrap/dist/css/bootstrap.css">
  <script src="../node_modules/jquery/dist/jquery.js"></script>
  <script src="../node_modules/jquery.cookie/jquery.cookie.js"></script>
  <script>
    function LoadCategories(){
      $.ajax({
        method:"get",
        url: "http://fakestoreapi.com/products/categories&quot;,
        success: (data)=> {
          data.unshift("all");
          $.each(data,(property, value)=>{
            $(<option
value=${value}>${value.toUpperCase()}</option>`).appendTo("#lstCategories");
          })
        }
      })
    }

    function LoadProducts(url) {
      $("main").html("");
      $.ajax({
        method:"get",
        url: url,
        success: (products)=> {
```



```

$.each(products,(property, product)=>{
    $(
        <div class="card m-2 p-2" style="width:200px">
            <img src=${product.image} class="card-img-top" height="140">
            <div class="card-header" style="height:130px">
                <p>${product.title}</p>
            </div>
            <div class="card-footer">
                <button id="btnAdd" value=${product.id} class="btn btn-danger">Add
to Cart</button>
            </div>
        </div>
    `).appendTo("main");
})
}
})
}

```

```

$(function(){
    if($.cookie("userid")==undefined){
        location.href = "login.html";
    } else {
        $("#userid").html(` Hello ! ${$.cookie("userid")} `);
    }
    LoadCategories();
    LoadProducts("http://fakestoreapi.com/products");

    $("#lstCategories").change(function(e){
        if(e.target.value=="all") {
            LoadProducts("http://fakestoreapi.com/products");
        } else {
            LoadProducts(` http://fakestoreapi.com/products/category/$
{e.target.value}`);
        }
    })

    $(document).on("click", "#btnAdd", function(e){

```

```

        alert("Add Clicked :" + e.target.value);
    })

    $("#btnLogout").click(function(){
        $.removeCookie("userid");
        location.href="index.html";
    })
})
</script>
</head>
<body class="container-fluid">
    <header class="bg-dark text-white d-flex justify-content-between text-center p-2">
        <h2>Fakestore API</h2> <div id="userid"></div>
        <button id="btnLogout" class="btn bg-light btn-link">logout</button>
    </header>
    <section class="mt-3 row">
        <nav class="col-2">
            <div>
                <a href="index.html">Index</a>
                <label class="form-label">Select Category</label>
                <div>
                    <select id="lstCategories" class="form-select">

                        </select>
                    </div>
                </div>
            </nav>
            <main class="col-10 d-flex flex-wrap overflow-auto" style="height: 500px;" >

                </main>
            </section>
        </body>
    </html>

```

Back End for Front End Web Application

1. Database : MongoDB

- 2. Middleware : Express JS
- 3. Server Side : Node JS

<https://www.mongodb.com/try/download/community>

MongoDB compass

Back End for Front End Web Application

- 1. Database : MongoDB
- 2. Middleware : Express JS
- 3. Server Side : Node JS

<https://www.mongodb.com/try/download/community>

MongoDB compass

MongoDB

Sudhakar Sharma

.

8:57 AM

MongoDB

- It is document based Data Base.
- Database is used to store and manipulate data required for application.
- Database is used to keep related data, with data relations.
- Relational Database Management System. [RDMS]
- Easy to analyze data.
[Oracle, MySql, SqlServer, etc..]

What are the features of MongoDB?

- It is Non-Sql or No-Sql Database.
- It uses JavaScript as language and its methods for manipulation.
- It is document oriented.
- It is schema less. [Structure less]
- It supports Ad-hoc queries [Dynamic Queries]
- It supports data replications.
- It makes data high availability.

Issues with MongoDB:

- Structure less is not good for analytics and BI.
- It is not good for DS.

Setup MongoDB:

1. Download and Install MongoDB Community Server

<https://www.mongodb.com/try/download/community>

2. MongoDB provides a 2 application environments

- a) Server
- b) Client

3. MongoDB Server is the location where data is stored.

- You have to start MongoDB server after installing.
- Go to "Services" on your PC and start MongoDB server

[Run => Services.msc]

- Right Click on "MongoDB Server" => Select Start

4. MongoDB Compass is client tool used to connect with database server. All interactions are managed from client tool.

- Server Address is "mongodb://127.0.0.1:27017"
"mongodb://localhost:27017"

Demo Project [IEEE Project]

Project Document

Sudhakar Sharma

I

Apr 11

What is Database?

Various Types of Database Software

What is MongoDB?

- It is non-sql database. [No-Sql]
- It is document oriented database.
- It is schema less.
- It is JSON formatted


```
[ { Name:"", Price: 0 },
  { Name:"", Price: 0, Stock:true},
]
```
- It is easy for Browser to handle data.
- It supports indexing. You can query on any field.
- It supports Ad-hoc queries. It allows to build a query dynamically.
- It supports data replications.
- It can handle load balancing.
- High Availability

Features and Issues with MongoDB

- It is schema less, not good for analytics at ERP.
- It poor in relationships

Start MongoDB Server

Connect to Server using Client Tool : MongoDB Compass

IEEE Project

1. Visit IEEE website

ieeexplore.com

2. Search for "i-shop"

3. Copy DOI number of project

4. Open

<https://sci-hub.hkvisa.net/>

5. Paste DOI number and Open document

Project Documentation [Ms-Word]

1. Abstract
2. Existing System
3. Proposed System
4. Software and Hardware Requirements
5. Modules in Application

Data Models



shopping-online.pdf

Creating Database, Collections

Sudhakar Sharma

Yesterday

MongoDB Terminology

- 1. Database** : It is the location where data is maintained. It is a store for used for storing data.
- 2. Collection** : In database data is kept in the form of collection. [JSON]
In other database software it is kept in "Table".
- 3. Document** : Collection stores relative information is a document.

```
[  
  { key:value }    => document  
]
```

4. **Fields** : Field Names or Keys used for storing data in a document.

5. **Embedded Documents** : These are Joins in a database.

MongoDB CRUD Operations

C = Create

R = Read

U = Update

D = Delete

MongoDB Shell Commands

1. Creating a new Database

- Connect with MongoDB Compass

mongodb://127.0.0.1:27017

- Open MongoShell(below the app)

>use databaseName

- It creates a new database in mongodb server.

Ex:

>use shopping

2. To View list of databases

> show dbs

3. Create a new collection [Table to store data]

>db.createCollection("name", { options })

Ex:

```
>db.createCollection("vendors")
```

option	type	purpose
--------	------	---------

autoIndexId	boolean	It adds a new field in every document which is used as unique id for document.
-------------	---------	--

size	number	It defines space allocated for collection in memory. It is defined in bytes.
------	--------	--

capped	boolean	It allows to replace the old records with new when the max space is filled.
--------	---------	---

max	number	It defines the maximum number of documents allowed.
-----	--------	---

Syntax:

```
>db.createCollection("vendors", { autoIndexId:true, max:10, capped:true, size: 2097152 })
```

8 bits	= 1 byte
1024 bytes	= 1 KB
1024 kb	= 1 MB
1024 mb	= 1 GB
1024 gb	= 1 TB

1024 x 1024 = 1mb

```
> db.createCollection("customers")
```

4. To View the list of collections

```
>show collections
```

5. Inserting documents into collection

```
insertOne()  
insertMany()
```



```
>db.collection.insertOne({})
>db.collection.insertMany([{}, {}])
```

Ex:

```
>db.vendors.insertOne({VendorId:1, VendorName:"Reliance Digital", Cities:["Delhi",
"Hyd", "Mumbai"]});
```

```
>db.products.insertMany([ {ProductId:1, Name:"TV", Price: 45000.33}, {ProductId:2,
Name:"Nike Casuals", Price: 2400.45 } ])
```

Ex: Upload JSON data

- Open MongoDB Compass
- Open your collection in database from GUI tool
- Select collection name => Import Data
- Select "JSON" file in your PC and import

6. Reading Data from collection | Querying data from collection

`find()` It uses a query to read and return documents from collection.

```
> db.collection.find({query})
```

```
> db.vendors.find()      => returns all
```

query is written as below

<code>{fieldName:value}</code>	=> Equal to
<code>{fieldName:{\$gt:value}}</code>	=> Greater than
<code>{fieldName:{\$gte:value}}</code>	=> Greater than or Equal
<code>{fieldName:{\$lt:value}}</code>	=> Less than
<code>{fieldName:{\$lte:value}}</code>	=> Less than or equal
<code>{fieldName:{\$ne:value}}</code>	=> Not equal

`$and` => Logical AND

`$or` => Logical OR

EX:

```
> db.products.find({Price:{$lt:20000}})
```

```
> db.fakestore.find({$and:[{category:"jewelery"}, {price:{$gt:500}}]})  
> db.fakestore.find({$or:[{category:"jewelery"}, {price:{$gt:500}}]})
```

MongoDB

Sudhakar Sharma

.

8:52 AM

Creating Database

use

Adding Collection

createCollection()

Inserting data into collection

insertOne()

insertMany()

Query data from collection

find()

\$gt

\$lt

\$gte

\$lte

\$ne

\$and

\$or

Updating Documents

- updateOne()

- updateMany()

Syntax:

```
> db.collection.updateOne({findQuery}, {updateQuery})
```

\$set It can modify any field value

\$rename It can modify field name

\$unset It can remove field name

```
> db.collection.updateOne({Name:"Speaker"}, {$set:{Price:4500.55}})
```

```
> db.collection.updateMany({Category:"Electronics"}, {$set:{Stock:false}})
```

```
> db.products.updateMany({}, {$set:{Stock:true}})
```

Delete Query

- deleteOne()

- deleteMany()

```
> db.collection.deleteOne({findQuery})
```

```
> db.collection.deleteMany({findQuery})
```

```
> db.fakestore.deleteOne({id:1})
```

```
> db.fakestore.deleteMany({category:"electronics"})
```

Q) Delete All electronics products whose price is above 600

```
> db.fakestore.deleteMany({$and:[{category:"electronics"},{price:{$gt:600}}]})
```

```
> db.collection.drop()    => remove any collection
```

Ex:db.vendors.drop()—it will remove the vendors collection

```
> db.dropDatabase()    => removes the current Database.
```

Ex:use shopping

db.dropDatabase()---it will remove the shopping database

Ex: Adding Customer Data

```
> db.customers.insertOne({UserId:"john_nit", UserName:"John", Password:"john@123",  
Email:"john@gmail.com", Mobile:"+919876543210"})
```

Server Side Scripting

- Server Side scripting is a technique where scripts are employed on server in order to generate a response customized for every client request.

- There are various server side scripting technologies
CGI, JSP, ASP, PHP, ASP.NET, Node JS etc..

Node JS

- It is JavaScript based runs on google chrome "v8" compiler.
- It is single threaded.
- No Buffering
- Asynchronous
- Modular

Node and Mongo Connection

Sudhakar Sharma

.

Apr 14

1. Setup Library to work with Node, Express and MongoDB

Node : Server Side Script
Express : It is used to create API
MongoDB Library : It is used to connect with MongoDB database.
[MongoDB Driver]

In your project install

```
> npm install express --save  
> npm install mongodb --save  
> npm install cors --save
```

CORS [Cross Origin Resource Sharing] - Browser may block data from API.
CORS library will allow the API data.

2. Create a Server Side Application

- It will handle the request from client and process the request.
- If necessary it will communicate with database [CRUD]
- It sends response to client.

- Server side code is kept in JavaScript file ".js"

server.js

- You have to compile JavaScript file and run by using Node Compiler.

>node server.js

Note: Node JS uses JavaScript as language, you can't use DOM methods.

JS is used only as language:

variables, data types, operators, statements, functions, classes..
You can use "console" methods.

- JavaScript with HTML uses module system.
- Node JS use "require()" to import modules

```
var ref = require("moduleName");
```

```
import { functionName } from "ModuleName";
```

- Every Server Side application comprises of various requests to handles

GET	to fetch data
POST	to submit data
PUT	to modify all details
PATCH	to modify partial details
DELETE	to remove the data

Syntax:

```
app.get()  
app.post()  
app.put()  
app.delete()
```

- Every method uses a callback to 2 objects

a) request object	to fetch data from client
b) response object	to send data to client

Syntax:

```
app.get("/requestPath", function(request, response){
```

```
});
```

<http://127.0.0.1:port/requestPath>

- Request path

/ the request for application
/name the request for specific

Ex:

www.fakestoreapi.com/

www.fakestoreapi.com/products

- Response object can send any type of response

html, json, file, XML..

Ex:

1. Create a new folder

"server"

2. Add a new file

"server.js"

```
var express = require("express");
```

```
var app = express();
```

```
app.get("/", (req, res) => {  
  res.send("<h2>Welcome to My API</h2>");  
});
```

```
app.get("/products", (req, res) => {  
  var data = [{Name:"TV", Price:45000.33},{Name:"Mobile", Price:23000.33}];  
  res.send(data);  
});  
app.get("*", (req, res) => {  
  res.send("<code>Not Found : Page you requested not found.</code>");  
});
```

```
app.listen(4400);
```

```
console.log(`Server Started : http://127.0.0.1:4400`);
```

3. Open the file location in terminal

```
D:\Amazon\server>
```

4. compile and run

```
> node server.js
```

5. open any browser and request

<http://127.0.0.1:4400>

<http://localhost:4400/products>

Fetching Data from MongoDB and Sending to Client

1. server.js

```
var express = require("express");
var MongoClient = require("mongodb").MongoClient;
var cors = require("cors");

var app = express();
app.use(cors());

var conString = "mongodb://127.0.0.1:27017";

app.get("/", (req, res) => {
  res.send("<h2>Welcome to My API</h2>");
});

app.get("/products", (req, res) => {
  MongoClient.connect(conString)
    .then((clientObject) => {
      var database = clientObject.db("shopping");
      database.collection("products").find({}).toArray()
```

```

        .then(documents => {
            res.send(documents);
            res.end();
        })
    })
});

app.get("/customers", (req, res)=>{
    mongoClient.connect(conString)
    .then((clientObject)=>{
        var database = clientObject.db("shopping");
        database.collection("customers").find({}).toArray()
        .then(documents=>{
            res.send(documents);
            res.end();
        })

    })
});

app.get("*", (req, res)=>{
    res.send("<code>Not Found : Page you requested not found.</code>");
});

app.listen(4400);
console.log(`Server Started : http://127.0.0.1:4400`\);

> node server.js

```

2. home.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script src="../node_modules/jquery/dist/jquery.js"></script>
    <script>
        $(function(){

```



```

$.ajax({
  method:"get",
  url: "http://127.0.0.1:4400/products",
  success: (data)=>{
    $.each(data, (key, product)=>{
      $(
        <tr>
          <td>${product.Name}</td>
          <td>${product.Price}</td>
          <td><img src=${product.Photo} height="50" width="50"></td>
        </tr>
      `).appendTo("tbody");
    })
  }
})
</script>
</head>
<body>
<table border="1" width="300">
  <thead>
    <tr>
      <th>Name</th>
      <th>Price</th>
      <th>Photo</th>
    </tr>
  </thead>
  <tbody>

  </tbody>
</table>
</body>
</html>

```

End to End I-Shop APP

Sudhakar Sharma

.

Apr 15



ishop.zip

Compressed Archive

End to End CRUD Application

Sudhakar Sharma

.

Apr 16

Designing CRUD Operations

1. Create a new collection in "shopping" database by name "products"

2. Insert one document

```
{
  "ProductId": 1,
  "Name": "Samsung TV",
  "Price": 35000.33,
  "Stock": true
}
```

3. Create a server side API for CRUD Operations [Node JS & Express]

GET	/products	return all products
GET	/products/1	return specific product by id
POST	/addproduct	adds a new product into database
PUT	/updateproduct	update the data into database
DELETE	/deleteproduct	deletes the data from database collection

<http://localhost:4455/>

business-layer.js

```
var express = require("express");
var MongoClient = require("mongodb").MongoClient;
var cors = require("cors");
```

```

var connectionString = "mongodb://127.0.0.1:27017";

var app = express();
app.use(cors());

app.use(express.urlencoded({
  extended:true
}));

app.use(express.json());

app.get("/products", (req, res)=>{
  mongoClient.connect(connectionString).then(clientObject=>{
    var database = clientObject.db("shopping");
    database.collection("products").find({}).toArray()
    .then(documents=>{
      res.send(documents);
      res.end();
    })
  })
});

app.get("products/:id", (req, res)=>{
  var id = parseInt(req.params.id);
  mongoClient.connect(connectionString).then(clientObject=>{
    var database = clientObject.db("shopping");
    database.collection("products").find({ProductId:id}).toArray()
    .then(documents=>{
      res.send(documents);
      res.end();
    })
  })
});

app.post("/addproduct",(req, res)=>{
  var product = {
    "ProductId": parseInt(req.body.ProductId),
    "Name": req.body.Name,
    "Price": parseFloat(req.body.Price),
    "Stock": (req.body.Stock=="true"?true:false

```

```

    }
    mongoClient.connect(connectionString).then(clientObject=>{
        var database = clientObject.db("shopping");
        database.collection("products").insertOne(product).then((result)=>{
            console.log("Record Inserted");
            res.redirect("/products");
        })
    })
});

app.put("/updateproduct/:id", (req, res)=>{
    var id = parseInt(req.params.id);
    var product = {
        "ProductId": parseInt(req.body.ProductId),
        "Name": req.body.Name,
        "Price": parseFloat(req.body.Price),
        "Stock": (req.body.Stock=="true"?true:false
    }
    mongoClient.connect(connectionString).then(clientObject=>{
        var database = clientObject.db("shopping");
        database.collection("products").updateOne({ProductId:id},{ $set:product})
        .then(result=>{
            console.log("Record Updated");
            res.redirect("/products");
        })
    })
});

app.delete("/deleteproduct/:id", (req, res)=>{
    var id = parseInt(req.params.id);
    mongoClient.connect(connectionString).then(clientObject=>{
        var database = clientObject.db("shopping");
        database.collection("products").deleteOne({ProductId:id})
        .then(result=>{
            console.log("Record Deleted..");
            res.redirect("/products");
        })
    })
});

app.listen(4455);

```

```
console.log(`Server Started : http://localhost:4455`);
```

products.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Products</title>
  <link rel="stylesheet" href="../node_modules/bootstrap/dist/css/bootstrap.css">
  <link rel="stylesheet" href="../node_modules/bootstrap-icons/font/bootstrap-icons.css">
  <script src="../node_modules/jquery/dist/jquery.js"></script>
  <script>

    function LoadProducts(){
      $.ajax({
        method:"get",
        url:"http://localhost:4455/products",
        success: (products)=>{
          $.each(products, (key, product)=>{
            $(
              <tr>
                <td>${product.Name}</td>
                <td>
                  <button id="btnDetails" value=${product.ProductId} class="btn btn-primary">
                    <span class="bi bi-eye"> </span>
                  </button>
                  <button id="btnEdit" value=${product.ProductId} class="btn btn-warning">
                    <span class="bi bi-pen"> </span>
                  </button>
                  <button id="btnDelete" value=${product.ProductId} class="btn btn-danger">
                    <span class="bi bi-trash"> </span>
                  </button>
                </td>
              </tr>
            `).appendTo("tbody");
          })
        }
      })
    }
  </script>
</head>
<body>
  <table>
    <thead>
      <tr>
        <th>Product Name</th>
        <th>Product Details</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>Product 1</td>
        <td>
          <button id="btnDetails1" value="1" class="btn btn-primary">
            <span class="bi bi-eye"> </span>
          </button>
          <button id="btnEdit1" value="1" class="btn btn-warning">
            <span class="bi bi-pen"> </span>
          </button>
          <button id="btnDelete1" value="1" class="btn btn-danger">
            <span class="bi bi-trash"> </span>
          </button>
        </td>
      </tr>
    </tbody>
  </table>
</body>
</html>
```

```
    })  
}
```

```
$(function(){  
    LoadProducts();  
    $("#detailsContainer").hide();  
    $("#editContainer").hide();  
    $(document).on("click", "#btnDetails", (e)=>{  
        $("#detailsContainer").show(1000);  
        $("#editContainer").hide(1000);  
        $.ajax({  
            method:"get",  
            url: `http://localhost:4455/products/${e.currentTarget.value}`,  
            success: (products)=> {  
                var [product] = products;  
                $("#lblName").html(product.Name);  
                $("#lblPrice").html(product.Price);  
                $("#lblStock").html((product.Stock==true?"Available":"Out of Stock");  
            }  
        })  
    })  
})
```

```
$("#newProduct").click()==>{  
    location.href="newProduct.html";  
})
```

```
$(document).on("click", "#btnDelete", (e)=>{  
    var id = parseInt(e.currentTarget.value);  
    var flag = confirm("Are you sure\n Want to Delete?");  
    if(flag==true){  
        $.ajax({  
            method:"delete",  
            url: `http://localhost:4455/deleteproduct/${id}`  
        })  
        alert("Record Deleted");  
        location.reload();  
    }  
})  
$(document).on("click", "#btnEdit", (e)=>{  
    $("#detailsContainer").hide();  
    $("#editContainer").show(1000);
```

```

$.ajax({
  method:"get",
  url: `http://localhost:4455/products/${e.currentTarget.value}`,
  success: (products)=> {
    var [product] = products;
    $("#txtId").val(product.ProductId);
    $("#txtName").val(product.Name);
    $("#txtPrice").val(product.Price);
    $("#optStock").prop("checked", product.Stock);
  }
})
})
})
</script>
</head>
<body class="container-fluid">
  <h2>Products CRUD</h2>
  <button class="btn btn-primary" id="newProduct">New Product</button>
  <table class="table table-hover">
    <thead>
      <tr>
        <th>Name</th>
        <th>Actions</th>
      </tr>
    </thead>
    <tbody>

  </tbody>
</table>
<hr size="13" noshade color="gray">
<div id="detailsContainer">
  <h2>Product Details</h2>
  <dl>
    <dt>Name</dt>
    <dd id="lblName"></dd>
    <dt>Price</dt>
    <dd id="lblPrice"></dd>
    <dt>Stock</dt>
    <dd id="lblStock"></dd>
  </dl>
</div>

```

```

<div id="editContainer">
  <h2>Edit Product</h2>
  <dl>
    <dt>Product Id</dt>
    <dd><input type="number" id="txtId"></dd>
    <dt>Name</dt>
    <dd><input type="text" id="txtName"></dd>
    <dt>Price</dt>
    <dd><input type="text" id="txtPrice"></dd>
    <dt>Stock</dt>
    <dd><input type="checkbox" id="optStock"></dd>
  </dl>
  <button class="btn btn-success">Save</button>
</div>
</body>
</html>

```

newProduct.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script src="../node_modules/jquery/dist/jquery.js"></script>
  <script>
    $(function(){

      $("#btnAdd").click(=>{
        var product = {
          ProductId: $("#ProductId").val(),
          Name: $("#Name").val(),
          Price: $("#Price").val(),
          Stock: $("#Stock").prop("checked")
        };
        $.ajax({
          method:"post",
          url: "http://localhost:4455/addproduct&quot;;

```



```
        data: product
      })
      alert("Record Inserted");
      location.href="products.html";
    })
  })
</script>
</head>
<body>
  <h2>Add New Product</h2>
  <dl>
    <dt>Product Id</dt>
    <dd><input type="number" id="ProductId"></dd>
    <dt>Name</dt>
    <dd><input type="text" id="Name"></dd>
    <dt>Price</dt>
    <dd><input type="text" id="Price"></dd>
    <dt>Stock</dt>
    <dd>
      <input type="checkbox" id="Stock"> Available
    </dd>
  </dl>
  <button id="btnAdd">Add</button>
</body>
</html>
```

firebase - Google Cloud

1. visit
firebase.com
2. login with your google account
3. Go to console
4. Create a new project

ishop-ui

5. Install firebase-tools on your PC

```
C:\>npm install -g firebase-tools
```

6. Go to your project and open in terminal

```
> firebase login
```

```
> firebase init
```

```
? Which Firebase features do you want to set up for this directory? hosting
```

```
? Use Existing project
```

```
? Folder Name : public
```

```
? Rewrite Index : no
```

```
? public GIT : no
```

```
> firebase deploy
```

<https://ishop-ui.web.app>

Documents

Sudhakar Sharma

•

Apr 16

```
1. A document is a collection of data.
2. It is a JSON object that stores data in a key-value format.
3. It is a collection of key-value pairs.
4. It is a collection of key-value pairs that are stored in a document.
5. It is a collection of key-value pairs that are stored in a document.
6. It is a collection of key-value pairs that are stored in a document.
7. It is a collection of key-value pairs that are stored in a document.
8. It is a collection of key-value pairs that are stored in a document.
9. It is a collection of key-value pairs that are stored in a document.
10. It is a collection of key-value pairs that are stored in a document.
```

LESS.docx

Word

- introduction Mongo
 - Physical Model
 - Logical Model
 - Indexing Model
- MongoDB**
- Mongo is a NoSQL database
 - MongoDB is a database provided by Mongo Cloud
 - NoSQL
 - MongoDB is designed to meet the demands of modern applications
 - MongoDB is open source
 - MongoDB is cross platform
 - MongoDB is document based database
 - It is known as "Document Oriented MongoDB"
 - Every row is an object. Some rows are

MongoDB.docx

Word

- Server side templating is a technique used in web applications to render request from client and send a customized response for client's request
- Server side templating is a technique where a template is employed on server to render its generated a response customized for user's client request
- e.g. PHP, ASP.NET, JSP, Python, Ruby etc.
- Server Side Script is all about handling following things
 - Request Object
 - Response Object
 - Session Object
 - Cookie Object
 - Redirect Object

Node JS.docx

Word



UI_ResumeFormat.docx

Word

